

**O‘ZBEKISTON RESPUBLIKASI HARBIY XAVFSIZLIK VA
MUDOFAA UNIVERSITETINING**

**AXBOROT-KOMMUNIKATSIYA TEXNOLOGIYALARI VA HARBIY
ALOQA INSTITUTI**



**Qo‘shinlarning taktik qo‘mondonlik muhandisligi
(Axborot tizimlari va texnologiyalari) yo‘nalishi
bitiruvchilari uchun**

SAVOL-JAVOB TO‘PLAMI

Ushbu savol-javoblar to'plami "Axborot tizimlari va texnologiyalari" ixtisosligi bo'yicha bitiruvchi ofitserlar uchun mo'ljallangan bo'lib, mutaxassislikka oid 10 ta asosiy fan doirasida tuzilgan. To'plamda har bir fan bo'yicha asosiy tushunchalar, nazariy asoslar, amaliy masalalar hamda real texnologik jarayonlarda uchraydigan holatlar bo'yicha savollar va ularga qisqa, aniq hamda mazmunan to'liq javoblar keltirilgan.

Savol-javoblar mazmuni axborot texnologiyalari sohasining muhim yo'nalishlarini, jumladan hardware va software asoslari, dasturlash (Python), sun'iy intellekt, kompyuter tarmoqlari, axborot va kiberxavfsizlik, DevOps texnologiyalari, web texnologiyalar, videokonferensiya aloqa tizimlari, shuningdek kompyuter grafikasi va videomontaj kabi fanlarni qamrab oladi. Har bir fan bo'yicha savollar bilimlarni tizimlashtirish, asosiy termin va tushunchalarni mustahkamlash hamda amaliy faoliyatda qo'llash ko'nikmalarini shakllantirishga yo'naltirilgan.

Axborot tizimlari va texnologiyalari ixtisosligi bitiruvchilari uchun savol-javob to'plami O'zbekiston Respublikasi Harbiy xavfsizlik va Mudofaa universitetining Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti uslubiy kengashining 2026-yil 27-fevraldagi 5-sonli bayoni bilan muhokama qilingan va nashr qilishga ruxsat berilgan.

Axborot tizimlari va texnologiyalari ixtisosligi bitiruvchilari uchun savol-javob to'plami O'zbekiston Respublikasi Harbiy xavfsizlik va Mudofaa universiteti Axborot-kommunikatsiya texnologiyari va harbiy aloqa instituti "Axborot texnologiyalari va sun'iy intellekt" kafedrasining umumiy yig'ilishida muhokama qilingan va 2026-yil 27-fevraldagi 19-sonli bayoni bilan rasmiylashtirilgan.

Taqrizchilar:

- | | |
|--------------------------|--|
| Mulladjanov A.R.- | O'R QK BSH Aloqa, axborot texnologiyalari va axborotlarni himoyalash bosh boshqarmasi, Axborot-kommunikatsiya texnologiyalarini rivojlantirish bo'limi boshlig'i |
| Dexkonov O.R. - | PhD, dotsent. Axborot-kommunikatsiya texnologiyalari va harbiy aloqa instituti. "Raqamli texnologiyalar" kafedraasi "Infokommunikatsiya" sikli boshlig'i. |

MUNDARIJA

KIRISH	5
1-BOB. HARDWARE (MIKROPROTSESSORLAR VA MIKROKONTROLLERLAR)	7
ATAMALAR VA IZOHLAR.....	7
SAVOL-JAVOBLAR.....	8
1.1. Kompyuter arxitekturası qismi bo'yicha savol-javoblar:	9
1.2. Mikrokontrollerlar qismi bo'yicha savol-javoblar:	20
1.3. Mobil qurilmalar qismi bo'yicha savol-javoblar:	31
2. SOFTWARE.....	42
ATAMALAR VA IZOHLAR.....	42
SAVOL-JAVOBLAR	45
2.1. Dasturiy ta'minot asoslari.....	45
2.2. Algoritmik fikrlash va kod mantiqi.....	49
2.3. Dasturlash tillari va ularning qo'llanishi	53
2.4. IDE va dasturiy vositalar bilan ishlash.....	58
2.5. Algoritmilar va ma'lumotlar tuzilmalari.....	62
2.6. Kutubxonalar, API va integratsiya	65
2.7. Dasturiy xavfsizlik va xatoliklarni boshqarish	68
2.8. Testlash, debugging va sifat nazorati.....	71
2.9. Versiya boshqaruvi va jamoaviy ish	76
2.10. Dasturiy ta'minotni joriy etish va qo'llab-quvvatlash	79
3. PYTHON DASTURLASH TILI.....	82
ATAMALAR VA IZOHLAR.....	82
SAVOL-JAVOBLAR.....	83
4. SUN'IY INTELLEKT	106
ATAMALAR VA IZOHLAR.....	106
SAVOL-JAVOBLAR.....	108
4.1 Mashinali o'qitish va ma'lumotlar muhandisligi (ML & Data Engineering)	108
4.2. Chuqur o'rganish va neyron tarmoqlar (Deep Learning)	122
4.3. Tabiiy Tilni Qayta Ishlash (NLP) va LLM Texnologiyalari	133
4.4. Kompyuter Ko'rishi (Computer Vision) va MLOps.....	143
5. UCHUVCHISIZ UCHISH APPARATLARI (DRONLAR)	153
ATAMALAR VA IZOHLAR.....	153
SAVOL-JAVOBLAR	154
6. TARMOQ (NETWORKING) TEXNOLOGIYALARI.....	166
ATAMALAR VA IZOHLAR.....	166
SAVOL-JAVOBLAR	167
7. DEVOPS.....	183
ATAMALAR VA IZOHLAR.....	183
SAVOL-JAVOBLAR	184
8. WEB TEXNOLOGIYALARI.....	202

ATAMALAR VA IZOHLAR.....	202
SAVOL-JAVOBLAR	203
9. VIDEOKONFERENSIYA ALOQA TIZIMLARI.....	217
ATAMALAR VA IZOHLAR.....	217
SAVOL-JAVOBLAR	218
10. KOMPYUTER GRAFIKASI VA VIDEOMONTAJ.....	238
ATAMALAR VA IZOHLAR.....	238
SAVOL-JAVOBLAR	239
ALOQA TAYYORGARLIGI BO'YICHA SAVOLLAR VA JAVOBLAR.....	260

KIRISH

Zamonaviy raqamli transformatsiya sharoitida jamiyatning barcha sohalarida axborot texnologiyalarining jadal rivojlanishi malakali, keng qamrovli bilim va ko'nikmalarga ega mutaxassislar tayyorlashga bo'lgan ehtiyojni keskin oshirmoqda. Xususan, "Axborot tizimlari va texnologiyalari" yo'nalishida tahsil olayotgan bitiruvchilardan hardware va software asoslari, dasturlash tillari (jumladan Python), sun'iy intellekt, kompyuter tarmoqlari, DevOps texnologiyalari, web texnologiyalar, shuningdek amaliy faoliyatda tez-tez uchraydigan videokonferensiya aloqa tizimlari, kompyuter grafikasi va videomontaj bo'yicha puxta, tizimli va integrallashgan bilimlar talab etiladi. Ushbu yo'nalishlar bugungi kunda nafaqat axborot-kommunikatsiya sohasida, balki ta'lim, boshqaruv, harbiy va maxsus xizmatlar, sanoat hamda masofaviy hamkorlik tizimlarida muhim ahamiyat kasb etmoqda.

Shu bois mazkur savol-javoblar to'plami bitiruvchilarning nazariy bilimlarini mustahkamlash, asosiy tushunchalarni tezkor takrorlash, fanlararo bog'liqlikni anglash hamda yakuniy nazoratlar (imtihon, attestatsiya) jarayoniga maqsadli va tizimli tayyorgarlik ko'rishga xizmat qiladi. To'plam mazmuni zamonaviy o'quv jarayonida keng qo'llanilayotgan kompetensiyaviy yondashuv asosida ishlab chiqilgan bo'lib, unda bilimlarni faqat yodlash emas, balki ularni tahlil qilish, solishtirish va amaliy vaziyatlarda qo'llash ko'nikmalarini shakllantirishga alohida e'tibor qaratilgan.

Savol-javoblar to'plami fanlararo yondashuv asosida tuzilgan bo'lib, har bir bo'limda:

- mavzuga oid asosiy atamalar va ularning izohlari keltiriladi;
- o'rganilayotgan fan doirasida mantiqiy ketma-ketlikda tuzilgan savollar beriladi;
- savollarga qisqa, aniq va mazmunan boy javoblar taqdim etiladi;
- murakkab, tahlil va umumlashtirishni talab qiladigan savollar alohida belgilar yordamida ajratib ko'rsatiladi.

Mazkur to'plamdan samarali foydalanish uchun quyidagi tartib tavsiya etiladi:

1. bo'lim boshida berilgan atamalarni o'rganish orqali terminologiyani birxillashtirish;
2. savollarga dastlab mustaqil ravishda javob berishga harakat qilish;
3. keyinchalik berilgan namunaviy javoblar bilan solishtirib, bilimlardagi bo'shliqlarni to'ldirish;
4. har bir bo'lim yakunida asosiy tushunchalar bo'yicha qisqa konspekt yoki sxematik xulosa tuzish.

Natijada tinglovchi o'z bilimlarini tizimlashtiradi, tushunchalar o'rtasidagi mantiqiy va funksional bog'lanishlarni chuqurroq anglaydi hamda real amaliy vaziyatlarda — jumladan tizimlarni sozlash, tarmoq muammolarini tahlil qilish, dasturiy kod yozish, DevOps jarayonlarini tushunish, sun'iy intellekt asosidagi yechimlarni baholash kabi holatlarda to'g'ri va asoslangan qarorlar qabul qilish ko'nikmalarini rivojlantiradi. Ushbu savol-javoblar to'plami bitiruvchi ofitserlar uchun o'quv-uslubiy yordamchi manba bo'lib, mustaqil o'qish, bilimlarni baholash va amaliy tayyorgarlik jarayonida samarali foydalanish imkonini beradi.

1-bob. Hardware (Mikroprotessorlar va mikrokontrollerlar)

Atamalar va izohlar

Atamalar	Izohlar
ALU	Arifmetik va mantiqiy amallarni bajaruvchi protsessor qismi (Arithmetic Logic Unit)
Bus (Magistral)	Komponentlar o'rtasida ma'lumot uzatuvchi aloqa yo'li
Cache xotira	Tez-tez ishlatiladigan ma'lumotlar saqlanadigan tezkor xotira
CISC	Murakkab buyruqlar to'plamiga ega arxitektura (Complex Instruction Set Computer)
Clock Cycle	Protssorning bitta takt davri
CPU	Markaziy protsessor, buyruqlarni bajaruvchi asosiy qurilma (Central Processing Unit)
DMA	Protssorsiz to'g'ridan-to'g'ri xotiraga murojaat mexanizmi (Direct Memory Access)
GPU	Grafik va parallel hisoblashlar uchun protsessor (Graphics Processing Unit)
Hyper-Threading	Bitta yadroda bir nechta oqimni bajarish texnologiyasi
I/O	Ma'lumot kiritish va chiqarish tizimi (Input / Output)
Instruction Set	Protssor tushunadigan buyruqlar majmui
L1 Cache	Eng tezkor, protssorga eng yaqin kesh xotira (Level 1 Cache)
L2 Cache	O'rtacha tezlikdagi kesh xotira (Level 2 Cache)
L3 Cache	Bir nechta yadro uchun umumiy kesh xotira (Level 3 Cache)
Memory Hierarchy	Xotiralarning tezlik va hajm bo'yicha ierarxiyasi
Multicore	Bir nechta yadroga ega protsessor
Multithreading	Bir vaqtning o'zida bir nechta oqim bajarilishi
Pipelining	Buyruqlarni bosqichma-bosqich parallel bajarish
RAM	Tezkor, vaqtinchalik xotira (Random Access Memory)
RAID	Disklarni birlashtirib ishlash texnologiyasi (Redundant Array of Independent Disks)
RISC	Soddalashtirilgan buyruqlar arxitekturasi (Reduced Instruction Set Computer)
ROM	Doimiy xotira (Read Only Memory)
SIMD	Bitta buyruq bilan ko'p ma'lumotni qayta ishlash (Single Instruction Multiple Data)
Virtual Memory	Diskni xotira sifatida ishlatish mexanizmi
ADC	Analog signalni raqamli signalga aylantiruvchi modul (Analog to Digital Converter)
Arduino	Ochiq manbali mikrokontroller platformasi
Assembly	Past darajali dasturlash tili
Bootloader	Mikrokontroller ishga tushishida ishlaydigan dastur
EEPROM	Elektr bilan o'chiriladigan doimiy xotira (Electrically Erasable Programmable ROM)
GPIO	Universal kirish-chiqish pinlari (General Purpose Input/Output)
I2C	Ikki simli ketma-ket aloqa interfeysi (Inter-Integrated Circuit)

Interrupt	Dastur bajarilishini vaqtincha to'xtatuvchi signal
MCU	Mikrokontroller (Microcontroller Unit)
PWM	Impuls kengligini modulyatsiya qilish (Pulse Width Modulation)
RAM	Vaqtinchalik tezkor xotira (Random Access Memory)
Raspberry Pi	Bir platali mini-kompyuter
RTC	Real vaqt soati moduli (Real Time Clock)
Sensor	Fizik kattalikni o'lchovchi qurilma
SPI	Yuqori tezlikdagi ketma-ket interfeys (Serial Peripheral Interface)
UART	Ketma-ket aloqa interfeysi (Universal Asynchronous Receiver Transmitter)
Watchdog	Tizim osilib qolishini oldini oluvchi taymer
Accelerometer	Qurilma tezlanishini aniqlovchi sensor
AMOLED	Energiya tejamar ekran texnologiyasi (Active Matrix Organic Light Emitting Diode)
Android	Google tomonidan ishlab chiqilgan mobil operatsion tizim
Battery Management	Batareya zaryadini boshqarish tizimi
Bluetooth	Qisqa masofali simsiz aloqa texnologiyasi
Face ID	Yuz orqali autentifikatsiya qilish texnologiyasi
GPS	Global joylashuvni aniqlash tizimi (Global Positioning System)
Gyroscope	Qurilmaning burilish harakatini aniqlovchi sensor
iOS	Apple mobil operatsion tizimi (iPhone Operating System)
LTE	Yuqori tezlikdagi mobil aloqa (Long Term Evolution, 4G)
Multi-touch	Bir nechta barmoq bilan boshqarish texnologiyasi
NFC	Juda qisqa masofali aloqa (Near Field Communication)
OLED	Organik yorug'lik chiqaruvchi ekran (Organic Light Emitting Diode)
Power Saving Mode	Energiya tejash rejimi
SIM Card	Mobil tarmoqqa ulanish moduli (Subscriber Identity Module)
SoC	Bitta chip ichidagi butun tizim (System on a Chip)
Touchscreen	Sensorli ekran
USB-C	Zamonaviy universal port (Universal Serial Bus Type-C)
Wi-Fi	Simsiz tarmoq texnologiyasi (Wireless Fidelity)
5G	Mobil aloqaning beshinchi avlodi (Fifth Generation)

Savol-javoblar.

- **Savol belgisi**

Mazkur belgi savolni ifodalaydi va bilimni tekshirishga qaratilgan savollarni ko'rsatadi.

- **Javob (ochiq kitob) belgisi**

Mazkur belgi savolga berilgan izohli javobni bildiradi. Javoblar qisqa, tushunarli va mazmunan keng yoritilgan.

- **Murakkab savol belgisi**

Mazkur belgi nazariy jihatdan muhim, chuqur tahlil talab etuvchi yoki imtihon darajasidagi savollarni ajratib ko'rsatish uchun ishlatiladi.

1.1. Kompyuter arxitekturasini qismi bo'yicha savol-javoblar:

☆ 1. Kompyuter arxitekturasining asosiy komponentlari nimalardan iborat?

📖 Kompyuter arxitekturasini protsessor, xotira, saqlash qurilmalari, tizim magistrali va I/O qurilmalari kabi asosiy komponentlardan tashkil topadi.

? 2. Protsessor qanday ishlaydi?

📖 Protsessor buyruqlarni bajaradi, ma'lumotlarni qayta ishlaydi va kompyuter tizimi boshqaruvini amalga oshiradi.

☆ 3. RAM va ROM o'rtasidagi farqni tushuntiring.

📖 RAM (Random Access Memory) vaqtinchalik ma'lumotlarni saqlaydi, ROM (Read Only Memory) esa faqat o'qish uchun mo'ljallangan xotira bo'lib, operatsion tizimni yuklashda ishlatiladi.

? 4. Saqlash qurilmalari va ularning turlari qanday?

📖 HDD (Hard Disk Drive) mexanik saqlash qurilmasi bo'lsa, SSD (Solid-State Drive) elektron qurilma bo'lib, tezroq ishlaydi.

? 5. Cache xotiraning ishlash prinsipi qanday?

📖 Cache xotira protsessorga eng ko'p ishlatiladigan ma'lumotlarni tezda taqdim etish uchun ishlatiladi, bu tizimni tezlashtiradi.

☆ 6. ALU va CU o'rtasidagi farqni tushuntiring.

📖 ALU (Arithmetic and Logic Unit) arifmetik va mantiqiy amallarni bajaradi, CU (Control Unit) esa buyruqlarni boshqaradi va tizimning ishlash tartibini belgilaydi.

☆ 7. Kompyuter tizimida pipelining roli qanday?

📖 Pipelining – buyruqlarni bir nechta bosqichlarda bir vaqtning o'zida bajarish texnikasi, bu protsessorning samaradorligini oshiradi.

? 8. Kompyuter tizimi arxitekturasidagi I/O qurilmalari qanday ishlaydi?

📖 I/O qurilmalari kompyuterga ma'lumot kiritish va undan chiqarishni ta'minlaydi. Bunga klaviatura, sichqoncha, printer va ekran kiradi.

? 9. Protsessorning takt chastotasi qanday ta'sir qiladi?

📖 Protsessorning takt chastotasi soat signalining tezligi bo'lib, kompyuterning umumiy ishlash tezligini belgilaydi.

★ **10. Protsessorlar va GPUlar o'rtasidagi farqni tushuntiring.**

📖 Protsessorlar ko'proq umumiy maqsadlar uchun ishlatilsa, GPUlar (Graphics Processing Unit) asosan grafika va parallel hisoblash uchun mo'ljallangan.

★ **11. RISC va CISC arxitekturalari o'rtasidagi farqni tushuntiring.**

📖 RISC (Reduced Instruction Set Computing) kamroq va oddiy buyruqlarga ega arxitektura, CISC (Complex Instruction Set Computing) esa ko'proq va murakkab buyruqlarni bajaradi.

❓ **12. Kompyuter tizimida multitasking qanday amalga oshiriladi?**

📖 Multitasking bir nechta dasturlarni bir vaqtning o'zida bajarishga imkon beradi va operatsion tizim tomonidan boshqariladi.

❓ **13. Tizim magistrali nima va uning roli qanday?**

📖 Magistral kompyuter ichidagi ma'lumotlarni uzatish yo'li bo'lib, protsessor, xotira va I/O qurilmalari o'rtasida axborot almashinuvini ta'minlaydi.

★ **14. Kompyuter arxitekturasida va tizim dizaynida parallel hisoblashning roli qanday?**

📖 Parallel hisoblash bir nechta protsessorlar yordamida bir vaqtning o'zida ko'proq hisoblash operatsiyalarini bajarishga imkon beradi.

★ **15. Kompyuter tizimida xatolikni tuzatish mexanizmlari qanday ishlaydi?**

📖 Xatolikni tuzatish mexanizmlari, masalan, Hamming kodi yoki paritet bitlari yordamida ma'lumotlarning to'g'riligi tekshiriladi va tuzatiladi.

★ **16. Kompyuter tizimida virtualizatsiya qanday ishlaydi?**

📖 Virtualizatsiya apparat resurslarini bir nechta virtual tizimlarga ajratib, ularni mustaqil ishlatish imkonini beradi.

❓ **17. Protsessorni qanday optimallashtirish mumkin?**

📖 Protsessorni optimallashtirish uchun yuqori takt chastotasi, ko'p yadroli arxitektura va samarali quvvat boshqaruvi qo'llaniladi.

❓ **18. Cache xotira qanday ishlaydi va uning ahamiyati nima?**

📖 Cache xotira CPU uchun tez-tez ishlatiladigan ma'lumotlarni saqlaydi va tizim ish tezligini sezilarli darajada oshiradi.

❓ 19. Ko'p yadroli tizimlar qanday ishlaydi?

📖 Ko'p yadroli tizimlar bir nechta yadro orqali bir vaqtning o'zida bir nechta vazifalarni bajaradi va samaradorlikni oshiradi.

★ 20. Tizimda xotira va protsessor o'rtasidagi aloqani tushuntiring.

📖 Protsessor xotiradan ma'lumotlarni o'qish va yozish orqali ishlaydi, xotira tezligi esa tizim unumdorligiga bevosita ta'sir ko'rsatadi.

★ 21. Kompyuter arxitekturasini va dasturlash tillari o'rtasidagi aloqani tushuntiring.

📖 Dasturlash tillari kompyuter arxitekturasini bilan bog'liq bo'lib, ular protsessor va xotira bilan samarali ishlash uchun optimallashtirilgan.

❓ 22. Kompyuter tizimlarida samarali energiya boshqaruvi qanday amalga oshiriladi?

📖 Quvvatni tejash uchun quvvat boshqaruvi tizimlari, energiya tejovchi protsessorlar va past energiya sarfi bo'lgan komponentlar ishlatiladi.

❓ 23. Kompyuter tizimida tizimning ishlash samaradorligini qanday oshirish mumkin?

📖 Yangi protsessorlar, tezkor xotira, SSD disklar va ko'p yadroli tizimlar ishlatish orqali tizimning samaradorligini oshirish mumkin.

❓ 24. Kompyuter tizimida kommunikatsiya protokollari qanday ishlaydi?

📖 Kompyuter tizimlarida ma'lumotlarni uzatish uchun TCP/IP kabi protokollar ishlatiladi, bu qurilmalar o'rtasidagi ma'lumot almashishni ta'minlaydi.

★ 25. Birinchi darajali va ikkinchi darajali xotira o'rtasidagi farqni tushuntiring.

📖 Birinchi darajali xotira (L1) protsessorga eng yaqin joylashgan tezkor xotira bo'lib, ikkinchi darajali xotira (L2) protsessorga nisbatan sekinroq.

❓ 26. Kompyuter tizimida RAMning roli qanday?

📖 RAM dasturlar va ma'lumotlarni vaqtincha saqlash uchun ishlatiladi, bu protsessorning tezkor ishlashini ta'minlaydi.

❓ 27. Protsessorni ishlash tezligini qanday oshirish mumkin?

📖 Protsessorning soat chastotasini oshirish, ko‘p yadroli tizimlar va pipeliningni qo‘llash orqali uning tezligini oshirish mumkin.

☆ **28. Sistemaning xatolikni tuzatish mexanizmlari qanday ishlaydi?**

📖 Xatoliklarni aniqlash va tuzatish uchun Hamming kodi va boshqa xatolikni tuzatish mexanizmlari ishlatiladi.

☆ **29. Kompyuter tizimlarida CPU va GPU o‘rtasidagi farqni tushuntiring.**

📖 CPU umumiy maqsadlar uchun ishlatilsa, GPU asosan grafikalarini qayta ishlash va paralel hisoblash uchun mo‘ljallangan.

? **30. Kompyuter tizimida quvvat boshqaruvi qanday amalga oshiriladi?**

📖 Quvvat boshqaruvi tizimi kompyuterning energiya iste‘molini nazorat qiladi va tizimni energiya tejash rejimiga o‘tkazadi.

☆ **31. Kompyuter tizimida SIMD va MIMD o‘rtasidagi farqni tushuntiring.**

📖 SIMD (Single Instruction, Multiple Data) bir xil buyruqni bir nechta ma’lumotlar bilan bajaradi, MIMD (Multiple Instruction, Multiple Data) esa bir nechta buyruq va ma’lumotlar bilan parallel ishlashni ta’minlaydi.

☆ **32. Kompyuter tizimida RAID nima va qanday ishlaydi?**

📖 RAID (Redundant Array of Independent Disks) bir nechta qattiq diskni birlashtirib, ma’lumotlarni samarali saqlash va xavfsizlikni ta’minlashni nazarda tutadi.

? **33. Ko‘p yadroli protsessorlar qanday ishlaydi?**

📖 Ko‘p yadroli protsessorlar bir nechta yadroga ega bo‘lib, bir vaqtning o‘zida bir nechta vazifalarni bajaradi, bu samaradorlikni oshiradi.

? **34. Xotira haroratini nazorat qilish qanday amalga oshiriladi?**

📖 Xotira harorati xotira modulidagi sensorlar yordamida nazorat qilinadi va kerak bo‘lganda sovutish tizimlari ishlatiladi.

? **35. Protsessorni multi-tasking uchun qanday optimallashtirish mumkin?**

📖 Protsessorni multi-tasking uchun ko‘p yadroli tizimlar va samarali quvvat boshqaruv tizimlari ishlatiladi.

☆ **36. Virtualizatsiya va konteynerizatsiya o‘rtasidagi farq nima?**

📖 Virtualizatsiya bitta tizimda bir nechta virtual mashinalarni yaratishga imkon beradi, konteynerizatsiya esa tizim resurslarini yanada samarali ishlatish uchun ishlatiladi.

❓ **37. Kompyuter arxitekturasida bus (magistral) tizimining roli qanday?**

📖 Bus tizimi kompyuterning ichki qismlari o'rtasida ma'lumotlarni uzatish uchun ishlatiladi, masalan, protsessor va xotira o'rtasidagi aloqani ta'minlaydi.

❓ **38. Xotira keshining turlari nimalardan iborat?**

📖 L1, L2, L3 kesh xotiralari mavjud bo'lib, ular protsessor va xotira o'rtasida tezkor ma'lumot uzatishni ta'minlaydi.

★ **39. Kompyuter tizimida DMA (Direct Memory Access) qanday ishlaydi?**

📖 DMA tizimi ma'lumotlarni protsessordan to'g'ridan-to'g'ri xotiraga yoki saqlash qurilmalariga uzatadi, bu tizimning samaradorligini oshiradi.

★ **40. Kompyuter arxitekturasida parallel ishlashni qanday ta'minlash mumkin?**

📖 Parallel ishlash ko'p yadroli protsessorlar, ko'p protsessorli tizimlar va tarmoq orqali bir nechta tizimlarni o'zaro bog'lash orqali ta'minlanadi.

★ **41. Hyper-Threading texnologiyasi nima?**

📖 Hyper-Threading bir protsessor yadroda ikkita virtual yadro yaratishga imkon beradi, bu esa tizimning samaradorligini oshiradi.

★ **42. Xotira tarmog'i (Memory Hierarchy) nima?**

📖 Xotira tarmog'i kompyuter tizimida xotira resurslarini samarali taqsimlash uchun ishlatiladi, L1 keshdan boshlanib, saqlash qurilmalargacha bo'lgan turli xotira qatlamlarini o'z ichiga oladi.

❓ **43. Kompyuter tizimida IO (Input/Output) protsessor qanday ishlaydi?**

📖 IO protsessor kompyuterga tashqi qurilmalardan ma'lumot kiritish va chiqarishni nazorat qiladi, ma'lumotlarni xotiraga yoki saqlash qurilmalariga uzatadi.

❓ **44. CPU ning ishlash tezligini qanday oshirish mumkin?**

📖 CPU ning ishlash tezligini oshirish uchun protsessorni yuqori takt chastotasi bilan ishlatish, ko'p yadroli tizimlar va samarali quvvat boshqaruvi kerak.

❓ **45. Protsessorlar o'rtasidagi aloqalarni qanday ta'minlash mumkin?**

📖 Protsessorlar o'rtasidagi aloqalar tizim magistrali orqali amalga oshiriladi, bunda tezkor va samarali ma'lumot uzatish ta'minlanadi.

☆ **46. Kompyuter arxitekturasida va tarmoq xavfsizligi o'rtasidagi bog'liqlik qanday?**

📖 Kompyuter arxitekturasida va tarmoq xavfsizligi o'rtasida xavfsizlik protokollari, ma'lumotlarni shifrlash va himoya tizimlari mavjud bo'lib, ular tizimning xavfsiz ishlashini ta'minlaydi.

☆ **47. Protsessorlar va GPUlar o'rtasidagi o'xshashliklar va farqlar nimalardan iborat?**

📖 Protsessorlar ko'proq umumiy maqsadlar uchun ishlatiladi, GPUlar esa grafikalarini qayta ishlash va paralel hisoblash uchun mo'ljallangan.

☆ **48. Protsessor arxitekturasida o'zgarishlari qanday tizim samaradorligini ta'sir qiladi?**

📖 Protsessor arxitekturasida o'zgarishlari tizimning ishlash tezligini, energiya samaradorligini va paralel hisoblash qobiliyatini yaxshilaydi.

❓ **49. Tarmoq va saqlash tizimlari o'rtasidagi aloqani tushuntiring.**

📖 Tarmoq tizimi ma'lumotlarni saqlash tizimlaridan ma'lumotlarni uzatish va olishni ta'minlaydi, bu esa tezkor va samarali ishlashni ta'minlaydi.

☆ **50. Kompyuter arxitekturasida va hisoblash tizimlari o'rtasidagi aloqani tushuntiring.**

📖 Kompyuter arxitekturasida hisoblash tizimlarining ishlash prinsiplari va tuzilishini belgilaydi, hisoblash tizimlari esa ko'proq konkret vazifalarni bajarish uchun ishlatiladi.

☆ **51. Kompyuter arxitekturasida energo-tejamkorlikni ta'minlash uchun qanday texnologiyalar ishlatiladi?**

📖 Energiya tejash uchun quvvat boshqaruv tizimlari, tezkor xotira va protsessorlarni samarali ishlatish texnologiyalari qo'llaniladi.

❓ **52. Kompyuter tizimlarida zaxira nusxalarini yaratish qanday ishlaydi?**

📖 Zaxira nusxalarini yaratish uchun tizimda RAID kabi texnologiyalar va avtomatik zaxira olish mexanizmlari ishlatiladi.

☆ **53. Kompyuter tizimlarida fault tolerance qanday ishlaydi?**

📖 Fault tolerance tizimning bir qismining ishlamay qolganida, boshqa qismlari yordamida tizimning ishlashini davom ettirishga imkon beradi.

❓ **54. Xotira integratsiyasi qanday amalga oshiriladi?**

📖 Xotira integratsiyasi xotira qurilmalarining ishlashini samarali boshqarish, tezkor va ishonchli ma'lumot almashish uchun tizim arxitekturasini orqali amalga oshiriladi.

❓ **55. Kompyuter tizimida parallel ishlashni qanday ta'minlash mumkin?**

📖 Parallel ishlash ko'p yadroli protsessorlar va ko'p protsessorli tizimlar yordamida ta'minlanadi, bu esa ishlash tezligini oshiradi.

☆ **56. Kompyuter tizimlarida interleaving qanday ishlaydi?**

📖 Interleaving bir nechta ma'lumot bloklarini navbat bilan qayta ishlash orqali tizim samaradorligini oshiradi.

❓ **57. Tarmoq arxitekturasini nima va uning ishlash prinsipi qanday?**

📖 Tarmoq arxitekturasini ma'lumotlarni turli qurilmalarga uzatish va qabul qilish uchun ishlatiladigan qurilmalar va texnologiyalar majmuasidan iborat.

❓ **58. Kompyuter tizimlarida disklarni birlashtirish qanday amalga oshiriladi?**

📖 Disklarni birlashtirish RAID texnologiyalari yordamida amalga oshiriladi, bu tizimning ishlash samaradorligini oshiradi.

❓ **59. Kompyuter tizimlarida bir nechta protsessorni qanday o'zaro bog'lash mumkin?**

📖 Bir nechta protsessorlarni o'zaro bog'lash tizim magistrali va ko'p yadroli arxitektura yordamida amalga oshiriladi.

❓ **60. Tarmoq tarmog'ining ishlash samaradorligi qanday oshiriladi?**

📖 Tarmoq tarmog'ining samaradorligini oshirish uchun tarmoq protokollarini optimallashtirish, kengaytirilgan tarmoqlar va tezkor aloqa usullari qo'llaniladi.

❓ **61. Kompyuter tizimida ko'p protsessorli tizimlarning afzalliklari nimalardan iborat?**

📖 Ko'p protsessorli tizimlar bir vaqtning o'zida ko'proq vazifalarni bajarish imkonini beradi, bu samaradorlikni oshiradi.

? 62. Kompyuter tizimlarida I/O qurilmalarining ishlashini qanday boshqarish mumkin?

 I/O qurilmalari protsessor va operatsion tizim tomonidan boshqariladi, ular ma'lumotlarni uzatish va olishni ta'minlaydi.

☆ 63. Protsessor arxitekturasida poytaxtlar (clock cycles) qanday ishlaydi?

 Poytaxtlar (clock cycles) protsessorning ish tezligini o'lchaydi va uning bajarishi uchun zarur bo'lgan vaqtni belgilaydi.

? 64. Samarali ma'lumot uzatish uchun nima kerak?

 Ma'lumot uzatishda tizimning tezligi va resurslarni samarali taqsimlash zarur, buni ko'p yadroli tizimlar va yuqori tezlikdagi magistrallar ta'minlaydi.

☆ 65. Xotira arxitekturasida banklar qanday ishlaydi?

 Xotira banklari bir nechta xotira bloklarini o'z ichiga oladi, ular paralel ishlashni ta'minlab, tizimning samaradorligini oshiradi.

? 66. Protsessor va xotira o'rtasidagi aloqani qanday ta'minlash mumkin?

 Protsessor va xotira o'rtasidagi aloqani samarali ta'minlash uchun tizim magistrali va kesh xotira ishlatiladi.

☆ 67. Kompyuter arxitekturasidagi virtual xotira nima?

 Virtual xotira tizimi fizik xotira resurslari tugaganda diskni qo'shimcha xotira sifatida ishlatadi, bu esa tizimning ishlashini ta'minlaydi.

☆ 68. Tizim samaradorligini oshirishda multi-threadingning roli qanday?

 Multi-threading bir nechta vazifalarni parallel ravishda bajarish imkonini beradi, bu tizimning samaradorligini oshiradi.

? 69. Protsessorni qanday sinovdan o'tkazish mumkin?

 Protsessorni test qilish uchun maxsus dasturlar va diagnostika vositalari ishlatiladi, ularning yordamida protsessorning ishlashini tekshirish mumkin.

☆ 70. Kompyuter arxitekturasidagi SIMD va MIMD nima?

 SIMD (Single Instruction Multiple Data) — bitta buyruq bilan bir nechta ma'lumotlar ustida ishlash, MIMD (Multiple Instruction Multiple Data) esa bir nechta buyruq va ma'lumotlar bilan parallel ishlashdir.

? 71. Kompyuter tizimida ma'lumotlarni zaxira qilish qanday amalga oshiriladi?

 Ma'lumotlarni zaxira qilish uchun zaxira tizimlari, masalan, RAID yoki avtomatik zaxira olish mexanizmlari qo'llaniladi.

☆ 72. Protsessor arxitekturasidagi CISC va RISC o'rtasidagi farqni tushuntiring.

 CISC (Complex Instruction Set Computing) kompleks va ko'p buyruqlarga ega arxitektura, RISC (Reduced Instruction Set Computing) esa oddiy va kam buyruqlarga asoslangan arxitektura.

☆ 73. Protsessorning arxitekturasini optimallashtirishning qanday usullari mavjud?

 Protsessorni optimallashtirish uchun pipelining, superskalyar ishlov berish va kesh xotira tizimlari qo'llaniladi.

☆ 74. Kompyuter arxitekturasini va operatsion tizimlar o'rtasidagi aloqani tushuntiring.

 Operatsion tizimlar kompyuter arxitekturasini asosida ishlaydi, ular tizim resurslarini taqsimlash, protsessor va xotirani boshqarish vazifalarini bajaradi.

? 75. Kompyuter tizimlarida saqlash qurilmalarining o'zgarishi qanday samarlarni keltiradi?

 SSD kabi yangi saqlash texnologiyalari kompyuter tizimlarining tezligini oshiradi va umumiy ishlash samaradorligini ta'minlaydi.

? 76. Kompyuter tizimlarida xotira boshqaruvining roli qanday?

 Xotira boshqaruvi tizimning samarali ishlashi uchun xotira resurslarini taqsimlash va tezkor ma'lumot almashinuvini ta'minlaydi.

? 77. Protsessorlar va GPULar qanday birgalikda ishlaydi?

 Protsessorlar umumiy vazifalarni bajaradi, GPU esa grafik va parallel hisoblashni amalga oshiradi, ular birgalikda tizim unumdorligini oshiradi.

? 78. Kompyuter tizimlarida disklarni zaxira qilish qanday amalga oshiriladi?

 Disklarni zaxira qilish uchun RAID texnologiyalari, tarmoq saqlash tizimlari (NAS) yoki bulutli saqlash tizimlari ishlatiladi.

☆ **79. Kompyuter arxitekturasida xatolikni aniqlash va tuzatish qanday amalga oshiriladi?**

📖 Xatolikni aniqlash va tuzatish uchun xatolikni tuzatish kodlari (masalan, Hamming kodi) va paritet bitlari qo'llaniladi.

☆ **80. Kompyuter arxitekturasida va dasturlash tillari o'rtasidagi bog'liqlik qanday?**

📖 Dasturlash tillari kompyuter arxitekturasida asosida ishlaydi va protsessor hamda xotira bilan samarali ishlash uchun optimallashtirilgan.

? **81. Kompyuter tizimlarida CPU va I/O qurilmalari o'rtasidagi aloqani tushuntiring.**

📖 CPU va I/O qurilmalari tizim magistrali yoki boshqa bog'lanish vositalari orqali o'zaro ma'lumot almashadi.

? **82. Kompyuter arxitekturasida ko'p yadroli protsessorlarning afzalliklari qanday?**

📖 Ko'p yadroli protsessorlar bir vaqtning o'zida ko'proq vazifalarni bajarish imkonini beradi va tizimning samaradorligini oshiradi.

☆ **83. Kompyuter tizimida paritet bitlarining roli qanday?**

📖 Paritet bitlari ma'lumotlar to'g'riligi va yaxlitligini tekshirishda ishlatiladi, xatoliklarni aniqlash va tuzatishga yordam beradi.

☆ **84. Kompyuter tizimlarida saqlash qurilmalarining integratsiyasini qanday ta'minlash mumkin?**

📖 Saqlash qurilmalarining integratsiyasi RAID tizimlari va NAS (Network Attached Storage) yordamida amalga oshiriladi.

☆ **85. Kompyuter arxitekturasida virtual xotira qanday ishlaydi?**

📖 Virtual xotira tizimi xotiraning fizik chegaralaridan tashqarida ishlash imkoniyatini beradi va diskni qo'shimcha xotira sifatida ishlatadi.

☆ **86. Ko'p protsessorli tizimlarda resurslarni qanday taqsimlash mumkin?**

📖 Ko'p protsessorli tizimlarda resurslarni samarali taqsimlash uchun operatsion tizim, protsessorlar va xotira o'rtasidagi aloqalarni optimallashtirish zarur.

? **87. Kompyuter tizimlarida performansni oshirish uchun qanday texnologiyalar ishlatiladi?**

📖 Performansni oshirish uchun SSD diskleri, ko'p yadroli protsessorlar va parallel hisoblash texnologiyalari qo'llaniladi.

☆ 88. Kompyuter arxitekturasida hypervisor nima?

📖 Hypervisor — virtualizatsiya uchun ishlatiladigan dastur bo'lib, bitta fizik tizimda bir nechta virtual tizimlarni yaratishga imkon beradi.

? 89. Kompyuter tizimlarida I/O operatsiyalarini tezlashtirish uchun qanday usullar mavjud?

📖 I/O operatsiyalarini tezlashtirish uchun kesh xotira, disk kesh va tezkor tarmoq texnologiyalaridan foydalanish mumkin.

☆ 90. Kompyuter tizimlarida I/O va CPU o'rtasidagi aloqani qanday boshqarish mumkin?

📖 I/O va CPU o'rtasidagi aloqani tizim magistrali yoki DMA (Direct Memory Access) yordamida boshqarish mumkin.

? 91. Tarmoq tarmog'ida uzatish tezligi qanday ta'minlanadi?

📖 Tarmoq tarmog'ida uzatish tezligi yuqori tezlikdagi optik tolali kabellar, gigabit tarmoqlar va samarali protokollar yordamida ta'minlanadi.

? 92. Tizim samaradorligini oshirish uchun ma'lumotlarni qanday saqlash kerak?

📖 Ma'lumotlar tezkor saqlash uchun SSD disklar va ko'p qatlamli saqlash tizimlari ishlatiladi.

☆ 93. Kompyuter arxitekturasida ko'pgina qurilmalarning o'zaro ishlashini qanday ta'minlash mumkin?

📖 Tarmoq arxitekturasini va tizim magistralini optimallashtirish orqali bir nechta qurilmalarning o'zaro aloqasi ta'minlanadi.

? 94. Tarmoqda ma'lumotlarni uzatish tezligini oshirish uchun qanday protokollar ishlatiladi?

📖 TCP/IP, HTTP/2 va UDP kabi protokollar tarmoqdagi ma'lumotlar uzatishni samarali ta'minlaydi.

☆ 95. Kompyuter tizimlarida xatolikni tuzatish texnologiyalari qanday ishlaydi?

📖 Xatolikni tuzatish texnologiyalari, masalan, Hamming kodi yoki paritet bitlari yordamida ma'lumotlarning to'g'riligi tekshiriladi va tuzatiladi.

❓ **96. Protsessorni optimal ishlashga qanday tayyorlash mumkin?**

📖 Protsessorni optimal ishlash uchun samarali quvvat boshqaruvi, kesh xotira va yuqori takt chastotasi qo'llaniladi.

★ **97. Kompyuter tizimida tarmoq xavfsizligi qanday ta'minlanadi?**

📖 Tarmoq xavfsizligini ta'minlash uchun SSL/TLS, VPN va foydalanuvchi autentifikatsiyasi ishlatiladi.

★ **98. Kompyuter arxitekturasida energiya samaradorligi o'rtasidagi bog'liqlikni tushuntiring.**

📖 Kompyuter arxitekturasida energiya samaradorligini oshirish uchun quvvatni boshqarish, tezkor xotira va samarali protsessorlardan foydalaniladi.

★ **99. Tizimda ma'lumotlarni xavfsiz saqlash qanday ta'minlanadi?**

📖 Ma'lumotlarni shifrlash, xavfsiz saqlash tizimlari va foydalanuvchi autentifikatsiyasi yordamida saqlash xavfsizligi ta'minlanadi.

❓ **100. Kompyuter tizimlarida energiya boshqaruvining roli qanday?**

📖 Energiya boshqaruvi tizimlarning quvvat sarfini kamaytirish, batareyani tejash va tizimning samarali ishlashini ta'minlashga yordam beradi.

1.2. Mikrokontrollerlar qismi bo'yicha savol-javoblar:

❓ **1. Mikrokontroller nima?**

📖 Mikrokontroller – kichik o'lchamdagi elektron qurilma bo'lib, o'z ichiga protsessor, xotira va I/O qurilmalarini olgan tizimdir.

❓ **2. Mikrokontrollerning asosiy qismlari nimalardan iborat?**

📖 Mikrokontroller protsessor, RAM, ROM, I/O portlari va quvvat manbalarini o'z ichiga oladi.

❓ **3. Mikrokontrollerlar qanday ishlaydi?**

📖 Mikrokontrollerlar sensorlardan olingan ma'lumotlarni qayta ishlaydi va aktuatorlar yordamida natijalarni chiqaradi.

❓ **4. Mikrokontrollerlarni qanday dasturlash mumkin?**

📖 Mikrokontrollerlar C, C++, Python kabi dasturlash tillarida dasturlanadi, shuningdek Arduino va Raspberry Pi platformalarida ham ishlatiladi.

☆ 5. Mikrokontrollerlar va mikroprotsessorlar o'rtasidagi farq nima?

📖 Mikrokontrollerlar tizimning barcha qismlarini o'z ichiga oladi, mikroprotsessor esa faqat hisoblash yadrosidan iborat bo'ladi.

? 6. Arduino platformasining asosiy afzalliklari nimalardan iborat?

📖 Arduino ochiq manbali, foydalanish oson va keng kutubxonaga ega bo'lib, tezkor prototiplash imkonini beradi.

? 7. Raspberry Pi ning asosiy afzalliklari nimalardan iborat?

📖 Raspberry Pi to'liq kompyuter sifatida ishlaydi va yuqori hisoblash quvvatiga ega bo'lib, murakkab vazifalarni bajaradi.

☆ 8. ADC (Analog to Digital Converter) qanday ishlaydi?

📖 ADC analog signalni raqamli ko'rinishga o'tkazib, mikrokontroller tomonidan qayta ishlash imkonini yaratadi.

? 9. GPIO pinlari nima?

📖 GPIO (General Purpose Input/Output) pinlari tashqi qurilmalarni ulash, signal o'qish va boshqarish uchun ishlatiladi.

☆ 10. PWM (Pulse Width Modulation) qanday ishlaydi?

📖 PWM raqamli signalning impuls kengligini o'zgartirish orqali motor tezligi yoki yorug'lik intensivligini boshqaradi.

☆ 11. I2C va SPI interfeyslari o'rtasidagi farq nima?

📖 I2C kam simli va ko'p qurilmali aloqa uchun qulay, SPI esa yuqori tezlikda ma'lumot uzatishni ta'minlaydi.

☆ 12. Mikrokontrollerlarda interrupt qanday ishlaydi?

📖 Interrupt asosiy dastur oqimini to'xtatib, muhim hodisani zudlik bilan qayta ishlash imkonini beradi.

? 13. Mikrokontrollerlarni qanday optimallashtirish mumkin?

📖 Samarali kod yozish, quvvat tejash rejimlari va xotiradan oqilona foydalanish orqali optimallashtirish amalga oshiriladi.

? 14. Mikrokontrollerda qanday xotira turlari mavjud?

📖 ROM (Read-Only Memory), RAM (Random Access Memory) va EEPROM (Electrically Erasable Programmable Read-Only Memory) mavjud.

❓ **15. Mikrokontrollerda vaqtni qanday o'lchash mumkin?**

📖 Ichki taymerlar yoki tashqi vaqt modullari yordamida vaqtni aniqlash mumkin.

❓ **16. Mikrokontroller yordamida motorlarni qanday boshqarish mumkin?**

📖 PWM signallar yordamida motor tezligi va aylanish yo'nalishi boshqariladi.

❓ **17. Mikrokontroller yordamida senzordan ma'lumot olish qanday amalga oshiriladi?**

📖 Sensorlar analog yoki raqamli signal yuboradi, mikrokontroller esa ularni qayta ishlaydi.

☆ **18. Mikrokontrollerning energiya samaradorligini qanday oshirish mumkin?**

📖 Uyqu rejimlari, kam quvvatli komponentlar va energiya tejovchi dasturlar orqali samaradorlik oshiriladi.

❓ **19. Mikrokontrollerni qanday dasturlash mumkin?**

📖 Arduino IDE, MPLAB X, Keil va Raspberry Pi uchun Python muhiti orqali dasturlash amalga oshiriladi.

❓ **20. Mikrokontrollerlarni qanday test qilish mumkin?**

📖 Simulyatorlar, debugging vositalari va real apparat sinovlari orqali test o'tkaziladi.

❓ **20. Mikrokontrollerni qanday dasturlash mumkin?**

📖 Mikrokontrollerlarni dasturlash uchun Arduino IDE, MPLAB X, Keil va Raspberry Pi uchun Python muhiti ishlatiladi.

❓ **21. Mikrokontrollerlarni qanday test qilish mumkin?**

📖 Mikrokontrollerlarni test qilish uchun simulyatsiya dasturlari, debugging vositalari va real sharoitdagi sinovlar o'tkaziladi.

☆ **23. Mikrokontrollerlar yordamida haroratni qanday o'lchash mumkin?**

📖 Harorat sensorlarini mikrokontrollerga ulab, ularning analog yoki raqamli chiqishlarini o'qish orqali harorat aniqlanadi.

? 24. Mikrokontroller yordamida LED yoritgichlarini qanday boshqarish mumkin?

 PWM yordamida LED yoritgichlarining yorqinligi va rangini boshqarish mumkin.

? 25. Mikrokontrollerlar va mobil qurilmalar o'rtasidagi aloqani qanday tashkil qilish mumkin?

 Bluetooth, Wi-Fi yoki USB interfeyslari orqali mikrokontroller va mobil qurilmalar o'rtasida aloqa o'rnatiladi.

? 26. Mikrokontrollerlar yordamida ma'lumotlarni qanday uzatish mumkin?

 Serial interfeyslar, I2C, SPI yoki UART orqali ma'lumotlar uzatiladi.

☆ 27. Mikrokontrollerlarni ishlab chiqishda qanday dizayn shablonlari qo'llaniladi?

 Modular dizayn, moslashuvchan interfeyslar va samarali energiya boshqaruvi shablonlari qo'llaniladi.

☆ 28. Mikrokontroller yordamida aqlli uy tizimlarini qanday yaratish mumkin?

 Mikrokontrollerlar orqali harorat, yoritish va xavfsizlik tizimlarini avtomatik boshqarish mumkin.

☆ 29. Raspberry Pi va Arduino o'rtasidagi farq nima?

 Raspberry Pi to'liq kompyuter sifatida ishlaydi, Arduino esa mikrokontroller platformasi bo'lib, oddiy va real vaqtli tizimlar uchun mo'ljallangan.

? 30. Mikrokontrollerda qanday soat modullarini ishlatish mumkin?

 DS3231 yoki RTC (Real-Time Clock) modullari vaqtni aniq o'lchash uchun ishlatiladi.

? 31. Mikrokontrollerlarda RTC moduli qanday ishlaydi?

 RTC moduli real vaqtni mustaqil hisoblaydi va mikrokontrollerga aniq vaqt ma'lumotlarini uzatadi.

? 32. Mikrokontrollerlarni qanday sozlash va kalibrlash mumkin?

 Dasturiy sozlash, sensorlarni moslashtirish va tashqi qurilmalar bilan sinxronlash orqali kalibrlash amalga oshiriladi.

? 33. Mikrokontrollerlar uchun qaysi interfeyslar mavjud?

 UART, I2C, SPI va USB interfeyslari mikrokontrollerlarda keng qo'llaniladi.

? 34. Mikrokontroller yordamida ovozli tizimlarni qanday yaratish mumkin?

 Mikrofon va dinamiklardan foydalanib, signalni qayta ishlash orqali ovozli tizimlar yaratiladi.

? 35. Mikrokontroller yordamida haroratni o'lchash uchun qanday sensorlar ishlatiladi?

 TMP36, LM35 kabi harorat sensorlari mikrokontrollerlar bilan ishlash uchun keng qo'llaniladi.

☆ 36. Mikrokontrollerni uzatish va qabul qilish tizimlarini qanday yaratish mumkin?

 RF modullar, Bluetooth yoki Wi-Fi yordamida ma'lumot uzatish va qabul qilish tizimlari yaratiladi.

☆ 37. Mikrokontrollerlarni sinovdan o'tkazish uchun qanday metodlar mavjud?

 Logic analizatorlar, osiloskoplar, debuggerlar va simulyatsiya muhiti yordamida sinovlar o'tkaziladi.

? 38. Mikrokontrollerlarni tarmoq bilan qanday bog'lash mumkin?

 Ethernet, Wi-Fi yoki GSM modullari yordamida mikrokontrollerlar tarmoqqa ulanadi.

☆ 39. Mikrokontrollerda ma'lumotlarni shifrlash qanday amalga oshiriladi?

 AES yoki DES algoritmlari yordamida ma'lumotlar shifrlanadi va xavfsizlik ta'minlanadi.

? 40. Mikrokontrollerlarning xatolikni tuzatish metodlari qanday?

 Debugging vositalari, test skriptlari va simulyatorlar yordamida xatoliklar aniqlanadi va bartaraf etiladi.

☆ 41. Mikrokontrollerlarda analog va raqamli signalni qanday ishlatish mumkin?

 Mikrokontrollerlar analog signalni ADC (Analog to Digital Converter) orqali raqamli formatga aylantiradi, raqamli signalni esa to'g'ridan-to'g'ri ishlatadi.

☆ **42. Mikrokontroller yordamida aqlli uy tizimlarini qanday yaratish mumkin?**

📖 Mikrokontrollerlar yordamida harorat va yorug'likni boshqarish, xavfsizlik tizimlarini avtomatlashtirish va masofadan boshqarish imkoniyatlari yaratiladi.

? **43. I2C interfeysi qanday ishlaydi?**

📖 I2C ikki simli interfeys bo'lib, master–slave prinsipi asosida bir nechta qurilmalarni bir vaqtda bog'lash imkonini beradi.

? **44. SPI interfeysi qanday ishlaydi?**

📖 SPI (Serial Peripheral Interface) to'rtta sim asosida ishlaydi va yuqori tezlikda ma'lumot uzatishni ta'minlaydi.

☆ **45. Mikrokontrollerda PWM signalini qanday yaratish mumkin?**

📖 PWM (Pulse Width Modulation) signal orqali LED yorqinligi yoki motor tezligini boshqarish mumkin.

? **46. Mikrokontroller va mobil qurilmalar o'rtasida qanday aloqani o'rnatish mumkin?**

📖 Bluetooth, Wi-Fi yoki USB interfeyslari orqali mikrokontroller va mobil qurilmalar o'rtasida aloqa o'rnatiladi.

☆ **47. Mikrokontrollerlar uchun energiya tejash rejimlari qanday ishlaydi?**

📖 Mikrokontrollerlar uyqu rejimiga o'tish, qisqa faollashuv va minimal quvvat sarfi orqali energiyani tejaydi.

? **48. Raspberry Pi va Arduino platformalarining asosiy farqlari nima?**

📖 Raspberry Pi to'liq kompyuter bo'lsa, Arduino mikrokontroller bo'lib, real vaqtli va oddiy boshqaruv tizimlari uchun mo'ljallangan.

☆ **49. Mikrokontrollerda interrupt (pervaz) qanday ishlaydi?**

📖 Interrupt asosiy dastur oqimini vaqtincha to'xtatib, yuqori ustuvorlikka ega vazifani darhol bajarishni ta'minlaydi.

? **50. Mikrokontrollerda qanday soat (RTC) modullari ishlatiladi?**

📖 RTC (Real Time Clock) modullari mikrokontrollerga aniq vaqt ma'lumotlarini taqdim etadi va tizimni sinxronlaydi.

? **51. Mikrokontrollerda xotira qanday boshqariladi?**

📖 Xotira boshqaruvi RAM, ROM va EEPROM orqali amalga oshirilib, vaqtinchalik va doimiy ma'lumotlarni saqlashni ta'minlaydi.

❓ **52. Mikrokontroller yordamida ma'lumot uzatish va qabul qilishni qanday amalga oshirish mumkin?**

📖 UART, I2C va SPI kabi interfeyslar orqali ma'lumot uzatish va qabul qilish amalga oshiriladi.

❓ **53. Mikrokontrollerda masofadan boshqarish tizimlarini qanday yaratish mumkin?**

📖 IR (infraqizil) yoki RF modullar yordamida masofadan boshqarish tizimlari yaratiladi.

❓ **54. Mikrokontrollerda vaqtni aniqlash qanday amalga oshiriladi?**

📖 Ichki taymerlar yoki tashqi RTC modullari yordamida vaqt aniqlanadi.

☆ **55. Mikrokontrollerlar yordamida sensorlarni qanday integratsiya qilish mumkin?**

📖 Sensorlar analog yoki raqamli chiqishlar orqali ulanadi, analog signallar ADC orqali raqamlashtiriladi.

❓ **56. Mikrokontrollerlar yordamida motorlarni qanday boshqarish mumkin?**

📖 PWM signallari yordamida motor tezligi, yo'nalishi va ishlash holati boshqariladi.

❓ **57. Mikrokontroller yordamida haroratni qanday o'lchash mumkin?**

📖 LM35, DHT11 va DS18B20 kabi harorat sensorlari yordamida harorat aniqlanadi.

☆ **58. Mikrokontroller yordamida uzatish va qabul qilish tizimlarini qanday yaratish mumkin?**

📖 RF, Wi-Fi yoki Bluetooth modullari orqali uzatish va qabul qilish tizimlari yaratiladi.

☆ **59. Mikrokontrollerlar yordamida xavfsizlik tizimlarini qanday yaratish mumkin?**

📖 Harakat sensorlari, signalizatsiya va avtomatik eshik tizimlari orqali xavfsizlik tizimlari yaratiladi.

? 60. Mikrokontrollerlarni ishlab chiqishda qanday vositalar ishlatiladi?

 Arduino IDE, MPLAB X, Keil va PlatformIO kabi dasturiy vositalar mikrokontrollerlarni ishlab chiqishda qo'llaniladi.

? 61. Mikrokontrollerda GPIO pinlari qanday ishlaydi?

 GPIO pinlari mikrokontrollerga tashqi qurilmalarni ulash va ulardan ma'lumot olish uchun ishlatiladi. Ular kirish va chiqish sifatida sozlanishi mumkin.

? 62. Mikrokontrollerlar yordamida LCD ekranlarni qanday boshqarish mumkin?

 Mikrokontrollerlar LCD ekranga ma'lumot yuborish uchun I2C yoki parallel interfeyslardan foydalanadi.

☆ 63. Mikrokontrollerlarning ko'p turli tizimlar bilan integratsiyasi qanday amalga oshiriladi?

 Mikrokontrollerlar turli protokollar (I2C, SPI, UART) yordamida sensorlar, motorlar, ekranlar va boshqa qurilmalar bilan oson integratsiya qilinadi.

☆ 64. Mikrokontrollerlarda kodni qanday optimallashtirish mumkin?

 Kodni optimallashtirish uchun samarali algoritmlar, kamroq resurs ishlatadigan kutubxonalar va energiya tejash usullari qo'llaniladi.

☆ 65. Mikrokontrollerlarda energiya tejash qanday amalga oshiriladi?

 Mikrokontrollerlar uyqu rejimiga o'tkazilib, harakat sezgichlari yordamida faollashtirish va energiya tejash usullari qo'llaniladi.

? 66. Mikrokontrollerlarda ko'rsatkichlarni boshqarish qanday amalga oshiriladi?

 PWM signallari yordamida indikatorlar va ko'rsatkichlarning yorqinligi hamda holati boshqariladi.

☆ 67. Mikrokontroller yordamida motorlarni qanday teskari aylantirish mumkin?

 H-Bridge yoki maxsus motor haydovchilar yordamida motorlarning aylanish yo'nalishi o'zgartiriladi.

? 68. Mikrokontrollerlar yordamida sensorlardan olingan ma'lumotlarni qanday ko'rsatish mumkin?

📖 Sensorlardan olingan ma'lumotlar ekran, LED indikator yoki boshqa chiqish qurilmalari orqali ko'rsatiladi.

☆ **69. Mikrokontrollerlar yordamida ovozli tizimlarni qanday yaratish mumkin?**

📖 Mikrofon va dinamiklardan foydalanib, ovozli signalizatsiya va boshqaruv tizimlarini yaratish mumkin.

☆ **70. Mikrokontrollerlar yordamida signalni kuchaytirish qanday amalga oshiriladi?**

📖 Analog signalni kuchaytirish uchun tashqi kuchaytirgich (amplifikator) sxemalaridan foydalaniladi.

☆ **71. Mikrokontrollerlar uchun qanday xavfsizlik tizimlari ishlatiladi?**

📖 Ma'lumotlarni shifrlash, foydalanuvchini autentifikatsiya qilish va tasdiqlash mexanizmlari qo'llaniladi.

❓ **72. Mikrokontrollerlarda energiya tejash uchun qanday dasturlar qo'llaniladi?**

📖 Uyqu rejimlari, quvvatni kamaytirish algoritmlari va samarali dasturiy yechimlar orqali energiya tejash ta'minlanadi.

❓ **73. Mikrokontrollerlar yordamida dasturiy ta'minotni qanday yaratish mumkin?**

📖 Dasturiy ta'minot IDE (Integrated Development Environment) yordamida yoziladi va mikrokontrollerga yuklanadi.

❓ **74. Mikrokontrollerda turli qurilmalarni qanday boshqarish mumkin?**

📖 Tegishli interfeyslar orqali motorlar, sensorlar, ekranlar va LEDlar boshqariladi.

❓ **75. Mikrokontrollerlarning haroratni o'lchashda ishlatiladigan sensorlar nimalardan iborat?**

📖 LM35, DHT11 va DS18B20 kabi sensorlar haroratni o'lchashda keng qo'llaniladi.

☆ **76. Mikrokontrollerlar yordamida radio signallarini qanday uzatish mumkin?**

📖 RF modullari, masalan, NRF24L01 yordamida radioaloqa tashkil etiladi.

? 77. Mikrokontrollerlarni qanday sozlash va konfiguratsiya qilish mumkin?

 Dasturiy sozlash va apparat komponentlarini moslashtirish orqali mikrokontroller konfiguratsiya qilinadi.

? 78. Mikrokontrollerlar yordamida harakatni qanday aniqlash mumkin?

 Akselerometr va giroskop sensorlari yordamida harakat aniqlanadi.

? 79. Mikrokontrollerlarning foydalanuvchi interfeyslarini qanday yaratish mumkin?

 LCD ekranlar yoki sensorli (touch) panellar yordamida foydalanuvchi interfeysi yaratiladi.

☆ 80. Mikrokontrollerlarni qanday xavfsiz ishlatish kerak?

 To'g'ri quvvatlantirish, kodni himoyalash va elektr xavfsizligi qoidalariga rioya qilish zarur.

☆ 81. Mikrokontroller yordamida masofadan boshqarish tizimlarini qanday yaratish mumkin?

 Mikrokontroller yordamida IR (infraqizil) yoki RF (radio to'lqinlari) modullari orqali masofadan boshqarish tizimlarini yaratish mumkin.

? 82. Mikrokontrollerlarni dasturlashda qanday vositalar ishlatiladi?

 Mikrokontrollerlarni dasturlash uchun Arduino IDE, MPLAB X, Keil va PlatformIO kabi vositalar ishlatiladi.

? 83. Mikrokontrollerlar yordamida haroratni o'lchashni qanday amalga oshirish mumkin?

 LM35, DHT11 va DS18B20 kabi harorat sensorlari ulanib, o'lchov natijalari qayta ishlanadi va ekranda ko'rsatiladi.

☆ 84. Mikrokontrollerlar yordamida motor tezligini qanday boshqarish mumkin?

 PWM (Pulse Width Modulation) texnologiyasi yordamida motorning tezligi va aylanish yo'nalishi boshqariladi.

☆ 85. Mikrokontrollerlar yordamida real vaqt tizimlarini qanday yaratish mumkin?

 RTC (Real-Time Clock) moduli yordamida aniq vaqt nazorati ta'minlanib, real vaqt tizimlari yaratiladi.

? 86. Mikrokontroller yordamida yoritishni boshqarish uchun qanday usullar ishlatiladi?

 PWM yoki raqamli signallar orqali yoritish darajasi va holati boshqariladi.

? 87. Mikrokontrollerlarda qanday interfeyslar ishlatiladi?

 UART, I2C va SPI interfeyslari qurilmalar o'rtasida ma'lumot almashishni ta'minlaydi.

☆ 88. Mikrokontrollerlarni xavfsiz ishlatish uchun qanday choralar ko'rish kerak?

 Apparat himoyasi, kodni shifrlash va tashqi aloqa kanallarini himoyalash orqali xavfsizlik ta'minlanadi.

? 89. Mikrokontrollerlar yordamida ma'lumot uzatishning samarali usullari qanday?

 UART, I2C va SPI interfeyslari yordamida ishonchli va samarali ma'lumot uzatish amalga oshiriladi.

? 90. Mikrokontrollerlarni qanday test qilish mumkin?

 Simulyatsiya muhiti, real sinovlar, debugging vositalari va logik analizatorlar orqali test o'tkaziladi.

☆ 91. Mikrokontrollerlarda quvvatni boshqarish qanday amalga oshiriladi?

 Uyqu rejimlari, quvvatni kamaytirish mexanizmlari va samarali kod yozish orqali quvvat boshqariladi.

☆ 92. Mikrokontrollerlarda ko'p funksiyali tizimlarni qanday yaratish mumkin?

 Turli sensorlar, aktuatorlar, displeylar va modullarni integratsiya qilish orqali ko'p funksiyali tizimlar yaratiladi.

☆ 93. Mikrokontrollerlar yordamida IoT tizimlarini qanday yaratish mumkin?

 Wi-Fi, Bluetooth va Zigbee modullari yordamida IoT (Internet of Things) tizimlari tashkil etiladi.

? 94. Mikrokontroller yordamida yoritish tizimlarini qanday boshqarish mumkin?

📖 PWM signallari orqali LED yoritish darajasi sozlanadi va avtomatik yoritish tizimlari yaratiladi.

❓ **95. Mikrokontrollerlar yordamida ovozli tizimlarni qanday yaratish mumkin?**

📖 Mikrofon va dinamiklar ulanib, ovozli signalizatsiya va boshqaruv tizimlari yaratiladi.

☆ **96. Mikrokontrollerlarda ma'lumotlarni shifrlash qanday amalga oshiriladi?**

📖 AES yoki DES kabi shifrlash algoritmlari yordamida ma'lumotlar himoyalanaadi.

☆ **97. Mikrokontrollerlar yordamida xavfsizlik tizimlarini qanday yaratish mumkin?**

📖 Harakat sensorlari, kameralar va avtomatik boshqaruv tizimlari orqali xavfsizlik yechimlari yaratiladi.

❓ **98. Mikrokontrollerda analog signalni qanday raqamli signalga aylantirish mumkin?**

📖 ADC (Analog to Digital Converter) yordamida analog signal raqamli ko'rinishga o'tkaziladi.

❓ **99. Mikrokontrollerda xotira qanday boshqariladi?**

📖 RAM va ROM yordamida ma'lumotlar vaqtinchalik yoki doimiy tarzda saqlanadi.

❓ **100. Mikrokontrollerlarda dasturlashning asosiy tillari nimalardan iborat?**

📖 Mikrokontrollerlarni dasturlashda C, C++, Python va Assembly tillari keng qo'llaniladi.

1.3. Mobil qurilmalar qismi bo'yicha savol-javoblar:

❓ **1. Mobil qurilma nima?**

📖 Mobil qurilma harakatlanishga mo'ljallangan va mustaqil energiya manbaiga ega bo'lgan elektron qurilmadir.

❓ **2. Mobil qurilmalarning asosiy qismlari nimalardan iborat?**

📖 Mobil qurilmalar protsessor, xotira, ekran, batareya, sensorlar va I/O portlardan iborat.

? 3. Mobil qurilmalar qanday ishlaydi?

 Mobil qurilmalar protsessor va xotira asosida ishlaydi hamda sensorlar va boshqa qurilmalar bilan o‘zaro aloqada bo‘ladi.

? 4. Mobil qurilmada protsessor qanday ishlaydi?

 Protsessor buyruqlarni bajaradi, ma’lumotlarni qayta ishlaydi va foydalanuvchi bilan o‘zaro aloqani ta’minlaydi.

? 5. Mobil qurilmada RAM qanday ishlaydi?

 RAM dastur va ma’lumotlarni vaqtinchalik saqlaydi hamda qurilmaning tezkor ishlashini ta’minlaydi.

? 6. Mobil qurilmalarda ekran qanday ishlaydi?

 Ekran foydalanuvchi bilan interaktiv aloqa uchun xizmat qiladi, sensorli ekranlar boshqarishni osonlashtiradi.

? 7. Mobil qurilmalarda batareya qanday ishlaydi?

 Batareya qurilmani elektr energiyasi bilan ta’minlaydi va uning mustaqil ishlashiga imkon beradi.

? 8. Mobil qurilmada Wi-Fi qanday ishlaydi?

 Wi-Fi mobil qurilmani simsiz tarmoq orqali internetga ulash imkonini beradi.

? 9. Mobil qurilmada Bluetooth qanday ishlaydi?

 Bluetooth qisqa masofada boshqa qurilmalar bilan simsiz ma’lumot almashishni ta’minlaydi.

? 10. Mobil qurilmalarda GPS qanday ishlaydi?

 GPS qurilmaning joylashuvini sun’iy yo‘ldoshlar orqali aniqlaydi va navigatsiyada ishlatiladi.

? 11. Mobil qurilmalarda kameralar qanday ishlaydi?

 Kameralar tasvirni sensor orqali raqamli ma’lumotga aylantirib, foto va video olishni ta’minlaydi.

? 12. Mobil qurilmalarda akselerometr nima?

 Akselerometr qurilmaning harakati va tezlanishini aniqlash uchun ishlatiladi.

? 13. Mobil qurilmada giroskop nima?

 Giroskop qurilmaning burilish va yo‘nalish harakatlarini aniqlaydi.

14. Mobil qurilmada NFC qanday ishlaydi?

 NFC (Near Field Communication) juda qisqa masofada tezkor ma’lumot almashishni ta’minlaydi.

15. Mobil qurilmada xotira turlari qanday ishlaydi?

 RAM vaqtinchalik ma’lumotlarni saqlaydi, ROM esa operatsion tizim va dasturiy ta’minotni o‘z ichiga oladi.

16. Mobil qurilmalarda ma’lumotlarni qanday saqlash mumkin?

 Ma’lumotlar ichki xotirada yoki xotira kartalarida saqlanadi.

17. Mobil qurilmalarda sensorli ekran qanday ishlaydi?

 Sensorli ekran foydalanuvchining tegish va harakatlarini aniqlab, qurilmani boshqarish imkonini beradi.

18. Mobil qurilmalarda batareyani qanday tejash mumkin?

 Ekran yorqinligini kamaytirish, fon ilovalarni cheklash va energiya tejovchi rejimlardan foydalanish orqali batareya tejaladi.

19. Mobil qurilmalarda 5G texnologiyasi qanday ishlaydi?

 5G yuqori tezlik, kam kechikish va keng tarmoq imkoniyatlarini ta’minlaydi.

20. Mobil qurilmalarda SIM kartaning roli qanday?

 SIM karta mobil tarmoqqa ulanish va foydalanuvchini identifikatsiya qilish vazifasini bajaradi.

21. Mobil qurilmalarda operatsion tizimlar qanday ishlaydi?

 Mobil qurilmalarda operatsion tizimlar (Android, iOS) qurilmani boshqaradi, ilovalarni ishga tushiradi va tizim resurslarini taqsimlaydi.

22. Mobil qurilmalar uchun xotira kengaytirilishi mumkinmi?

 Ha, ayrim mobil qurilmalarda microSD kartalar yordamida xotirani kengaytirish mumkin.

23. Mobil qurilmalar orqali audio va video qanday ishlaydi?

 Audio va video ma’lumotlari protsessor tomonidan qayta ishlanib, ekran va dinamiklar orqali chiqariladi.

? 24. Mobil qurilmalarda ekranning yechimi qanday ta'minlanadi?

 Ekran yechimi piksel zichligi, rang aniqligi va displey texnologiyasiga bog'liq bo'ladi.

? 25. Mobil qurilmalarda saqlash joyi qanday boshqariladi?

 Saqlash joyi operatsion tizim tomonidan boshqariladi va foydalanuvchi fayllarni qo'lda tartibga solishi mumkin.

? 26. Mobil qurilmalar uchun dasturlarni qanday o'rnatish mumkin?

 Ilovalar Google Play Store yoki Apple App Store orqali o'rnatiladi.

☆ 27. Mobil qurilmada xavfsizlik qanday ta'minlanadi?

 Parollar, biometrik himoya (barmoq izi, yuzni aniqlash) va shifrlash texnologiyalari orqali xavfsizlik ta'minlanadi.

? 28. Mobil qurilmalarda akkumulyatorni qanday saqlash kerak?

 Akkumulyatorni 20–80% oralig'ida ishlatish va haddan tashqari qizib ketishdan saqlash tavsiya etiladi.

☆ 29. Mobil qurilmalar va mikrokontrollerlar o'rtasidagi farq nima?

 Mobil qurilmalar umumiy maqsadli bo'lsa, mikrokontrollerlar maxsus vazifalar uchun mo'ljallangan.

☆ 30. Mobil qurilmalar uchun ilovalar qanday ishlab chiqiladi?

 Android ilovalari Java yoki Kotlin, iOS ilovalari esa Swift dasturlash tillarida yaratiladi.

? 31. Mobil qurilmalar yordamida tarmoqqa ulanish qanday amalga oshiriladi?

 Wi-Fi, 4G va 5G tarmoqlari orqali mobil qurilmalar internetga ulanadi.

☆ 32. Mobil qurilmalarda xavfsizlik protokollari qanday ishlaydi?

 Xavfsizlik protokollari ma'lumotlarni shifrlash, autentifikatsiya va tarmoq himoyasini ta'minlaydi.

? 33. Mobil qurilmalarda NFC qanday ishlaydi?

 NFC (Near Field Communication) qisqa masofada tezkor va xavfsiz ma'lumot almashishni ta'minlaydi.

? 34. Mobil qurilmalarda batareya qanday ishlaydi?

 Batareya qurilmaga elektr energiyasini yetkazib beradi va uning mustaqil ishlashini ta'minlaydi.

? 35. Mobil qurilmalarda sensorlar qanday ishlaydi?

 Akselerometr, giroskop va boshqa sensorlar atrof-muhit va harakatni aniqlab, ma'lumotlarni qayta ishlaydi.

? 36. Mobil qurilmalarda GPS tizimi qanday ishlaydi?

 GPS sun'iy yo'ldoshlar orqali qurilmaning aniq joylashuvini aniqlaydi.

? 37. Mobil qurilmalarda ekran qanchalik samarali ishlaydi?

 Ekran samaradorligi yechim, sensor sezgirligi va display texnologiyasiga bog'liq.

? 38. Mobil qurilmalarda video qo'ng'iroqlarni qanday amalga oshirish mumkin?

 Video qo'ng'iroqlar Skype, WhatsApp, Zoom kabi ilovalar orqali internet yordamida amalga oshiriladi.

? 39. Mobil qurilmada video va audio fayllarni qanday oynatish mumkin?

 Media fayllar protsessor tomonidan dekodlanib, media pleerlar orqali ijro etiladi.

☆ 40. Mobil qurilmalar va mikrokontrollerlarning energiya sarfi qanday farq qiladi?

 Mobil qurilmalar yuqori hisoblash resurslari sabab ko'proq energiya sarflaydi, mikrokontrollerlar esa energiya tejamkor ishlashga moslashtirilgan.

? 41. Mobil qurilmalar uchun qanday simsiz ulanish texnologiyalari mavjud?

 Mobil qurilmalarda Wi-Fi, Bluetooth, NFC, 4G, 5G va Zigbee kabi simsiz ulanish texnologiyalari mavjud.

☆ 42. Mobil qurilmalar va kompyuterlarning xotira boshqaruvi o'rtasidagi farq nima?

 Mobil qurilmalar energiya samaradorligini hisobga olib xotirani optimallashtiradi, kompyuterlar esa yuqori ishlash tezligiga yo'naltirilgan.

? 43. Mobil qurilmalarda ekranning yoritilish darajasi qanday boshqariladi?

📖 Yoritilish darajasi foydalanuvchi tomonidan qoʻlda yoki avtomatik tarzda atrof-muhit yorugʻligiga qarab sozlanadi.

★ **44. Mobil qurilmalarda batareyaning quvvat saqlash muddatini qanday uzaytirish mumkin?**

📖 Energiya tejash rejimlaridan foydalanish, ekran yorqinligini kamaytirish va fon ilovalarni cheklash orqali batareya muddati uzaytiriladi.

★ **45. Mobil qurilmalarda maʼlumotlarni shifrlash qanday amalga oshiriladi?**

📖 Maʼlumotlar AES va RSA kabi shifrlash algoritmlari yordamida himoyalanaadi.

? **46. Mobil qurilmalarda USB-C porti qanday ishlaydi?**

📖 USB-C ikki tomonlama ulanishni taʼminlab, tezkor maʼlumot uzatish, zaryadlash va video chiqishini qoʻllab-quvvatlaydi.

★ **47. Mobil qurilmalarda Face ID qanday ishlaydi?**

📖 Face ID yuzni skanerlash orqali foydalanuvchini aniqlaydi va qurilmaga xavfsiz kirishni taʼminlaydi.

? **48. Mobil qurilmalar yordamida GPS tizimidan qanday foydalanish mumkin?**

📖 GPS joylashuvni aniqlash, navigatsiya va xarita ilovalari orqali yoʻnalish topishda ishlatiladi.

? **49. Mobil qurilmada qanday ilovalar oʻrnatilishi mumkin?**

📖 Ilovalar App Store (iOS) yoki Google Play Store (Android) orqali oʻrnatiladi.

? **50. Mobil qurilmada ekranni qanday himoya qilish mumkin?**

📖 Himoya oynalari (glass), gʻiloflar va toʻgʻri parvarish orqali ekran himoyalanaadi.

★ **51. Mobil qurilmalarda biometrik xavfsizlik tizimlari qanday ishlaydi?**

📖 Barmoq izi, yuzni aniqlash kabi biometrik usullar foydalanuvchini autentifikatsiya qiladi.

? **52. Mobil qurilmalarda RAM va saqlash oʻrtasidagi farqni tushuntiring.**

📖 RAM vaqtinchalik maʼlumotlar uchun, saqlash xotirasi esa doimiy maʼlumotlar uchun ishlatiladi.

? 53. Mobil qurilmalarda ma'lumot uzatish tezligi qanday o'lchanadi?

 Uzatish tezligi megabit yoki gigabitlarda o'lchanadi va tarmoq texnologiyasiga bog'liq.

☆ 54. Mobil qurilmalarda xavfsiz aloqalar qanday ta'minlanadi?

 SSL/TLS protokollari orqali internet aloqalari shifrlanib himoyalaniadi.

? 55. Mobil qurilmada Wi-Fi qanday ishlaydi?

 Wi-Fi simsiz tarmoq orqali tezkor va barqaror internet ulanishini ta'minlaydi.

? 56. Mobil qurilmada multi-touch texnologiyasi qanday ishlaydi?

 Multi-touch bir nechta barmoq bilan ekranni boshqarish imkonini beradi.

? 57. Mobil qurilmada SIM karta qanday ishlaydi?

 SIM karta foydalanuvchini mobil tarmoqda aniqlaydi va aloqa xizmatlarini ta'minlaydi.

☆ 58. Mobil qurilmalarda 5G texnologiyasining afzalliklari nimalardan iborat?

 5G yuqori tezlik, kam kechikish va ko'p qurilmalarni bir vaqtda qo'llab-quvvatlaydi.

? 59. Mobil qurilmalarda 4G texnologiyasi qanday ishlaydi?

 4G yuqori tezlikdagi mobil internetni ta'minlab, video va onlayn xizmatlarni tezkor bajaradi.

? 60. Mobil qurilmalarda video qo'ng'iroqlarni qanday amalga oshirish mumkin?

 Video qo'ng'iroqlar Skype, WhatsApp yoki Zoom kabi ilovalar orqali internet yordamida amalga oshiriladi.

? 61. Mobil qurilmalarda ilovalarni o'chirish qanday amalga oshiriladi?

 Ilovalarni sozlamalar bo'limi yoki asosiy menyuda ilova ustiga bosib turish orqali o'chirish mumkin.

? 62. Mobil qurilmalarda fon ilovalari qanday boshqariladi?

 Fon ilovalari operatsion tizim orqali avtomatik yoki qo'lda cheklanadi, bu batareya va xotirani tejaydi.

☆ **63. Mobil qurilmalarda energiya tejash rejimi qanday ishlaydi?**

📖 Energiya tejash rejimi fon jarayonlarini cheklab, ekran yorqinligini kamaytiradi va protsessor tezligini pasaytiradi.

? **64. Mobil qurilmalarda ma'lumotlarni bulutda saqlash nima?**

📖 Bulutli saqlash (Google Drive, iCloud) ma'lumotlarni internet orqali masofaviy serverlarda saqlash imkonini beradi.

? **65. Mobil qurilmalarda avtomatik aylantirish (auto-rotate) qanday ishlaydi?**

📖 Akselerometr va giroskop sensorlari yordamida ekran qurilma holatiga mos ravishda aylanadi.

☆ **66. Mobil qurilmalarda zararli ilovalardan qanday himoyalanish mumkin?**

📖 Faqat rasmiy do'konlardan ilova o'rnatish, antivirus va tizim yangilanishlaridan foydalanish zarur.

? **67. Mobil qurilmalarda hotspot (nuqta) nima?**

📖 Hotspot mobil qurilmani Wi-Fi tarqatuvchi nuqtaga aylantirib, boshqa qurilmalarni internetga ulaydi.

? **68. Mobil qurilmalarda ma'lumotlarni zaxiralash qanday amalga oshiriladi?**

📖 Ma'lumotlar avtomatik yoki qo'lda bulutli xotira yoki kompyuterga zaxiralanadi.

☆ **69. Mobil qurilmalarda biometrik autentifikatsiya nima?**

📖 Biometrik autentifikatsiya barmoq izi yoki yuz orqali foydalanuvchini aniqlash usulidir.

? **70. Mobil qurilmalarda ovozli yordamchilar qanday ishlaydi?**

📖 Ovozli yordamchilar (Google Assistant, Siri) ovozli buyruqlarni tanib, amallarni bajaradi.

? **71. Mobil qurilmalarda ekran texnologiyalari qanday turlarga bo'linadi?**

📖 LCD, OLED va AMOLED texnologiyalari keng qo'llaniladi.

? **72. Mobil qurilmalarda ilova ruxsatlari nima uchun kerak?**

📖 Ilova ruxsatlari kamera, mikrofon, joylashuv kabi funksiyalarga kirishni nazorat qiladi.

☆ **73. Mobil qurilmalarda maxfiylik rejimi qanday ishlaydi?**

📖 Maxfiylik rejimi foydalanuvchi ma'lumotlarini yashirish va kuzatuvni cheklashga xizmat qiladi.

❓ **74. Mobil qurilmalarda ekran qulfi nima vazifani bajaradi?**

📖 Ekran qulfi qurilmani ruxsatsiz foydalanishdan himoya qiladi.

❓ **75. Mobil qurilmalarda yangilanishlarni nega o'rnatish kerak?**

📖 Yangilanishlar xavfsizlikni oshiradi, xatolarni tuzatadi va yangi funksiyalar qo'shadi.

❓ **76. Mobil qurilmalarda ma'lumotlarni uzatish rejimlari qanday?**

📖 Wi-Fi, mobil internet, Bluetooth va USB orqali uzatish mumkin.

☆ **77. Mobil qurilmalarda zavod sozlamalariga qaytarish nima?**

📖 Zavod sozlamalariga qaytarish barcha ma'lumotlarni o'chirib, qurilmani boshlang'ich holatga keltiradi.

❓ **78. Mobil qurilmalarda skrinshot qanday olinadi?**

📖 Maxsus tugmalar kombinatsiyasi yoki tizim menyusi orqali olinadi.

❓ **79. Mobil qurilmalarda xotira to'lib qolsa nima qilish kerak?**

📖 Keraksiz ilovalar va fayllarni o'chirish yoki bulutga ko'chirish kerak.

❓ **80. Mobil qurilmalar kundalik hayotda qayerlarda qo'llaniladi?**

📖 Aloqa, internet, ta'lim, navigatsiya, bank xizmatlari va ko'ngilochar sohalarda keng qo'llaniladi.

❓ **81. Mobil qurilmada qanday ilovalar mavjud?**

📖 Mobil qurilmalarda ijtimoiy tarmoqlar, o'yinlar, navigatsiya, musiqa tinglash va video tomosha qilish ilovalari mavjud.

☆ **82. Mobil qurilmalarda va internetda xavfsiz aloqalar qanday ta'minlanadi?**

📖 Xavfsiz aloqalar SSL/TLS protokollari, VPN xizmatlari va foydalanuvchi autentifikatsiyasi orqali ta'minlanadi.

? 83. Mobil qurilmalarda zaruriy ilovalarni qanday yangilash mumkin?

 Ilovalar App Store yoki Google Play orqali avtomatik yoki qo'lda yangilanadi.

? 84. Mobil qurilmalarda qanday tarmoq texnologiyalari ishlatiladi?

 3G, 4G, 5G, Wi-Fi, Bluetooth va NFC texnologiyalari qo'llaniladi.

? 85. Mobil qurilmada ko'p vazifali ishlash qanday ta'minlanadi?

 Ko'p vazifali ishlash operatsion tizim tomonidan boshqarilib, bir nechta ilovani bir vaqtda ishlatishga imkon beradi.

? 86. Mobil qurilmalarda qanday ekranlar ishlatiladi?

 LCD, OLED va AMOLED ekranlar tasvir sifati va energiya samaradorligini ta'minlaydi.

? 87. Mobil qurilmalarda ma'lumotlarni qanday saqlash va uzatish mumkin?

 Ma'lumotlar ichki xotira va microSD orqali saqlanadi, Wi-Fi, Bluetooth, NFC orqali uzatiladi.

☆ 88. Mobil qurilmalarda ma'lumotlarni shifrlash qanday amalga oshiriladi?

 AES va RSA algoritmlari yordamida ma'lumotlar himoyalanaadi.

? 89. Mobil qurilmalarda video va audio qo'ng'iroqlarni qanday amalga oshirish mumkin?

 Skype, WhatsApp, Zoom kabi ilovalar orqali internet orqali amalga oshiriladi.

? 90. Mobil qurilmalarda batareyaning uzoq muddat ishlashini qanday ta'minlash mumkin?

 Energiya tejash rejimlari, ekran yorqinligini kamaytirish va fon ilovalarni cheklash orqali ta'minlanadi.

☆ 91. Mobil qurilmalar uchun xavfsizlikni qanday ta'minlash mumkin?

 Parollar, biometrik tizimlar va ikki bosqichli autentifikatsiya yordamida ta'minlanadi.

? 92. Mobil qurilmalarda tashqi qurilmalar bilan aloqani qanday ta'minlash mumkin?

 USB, Bluetooth, Wi-Fi yoki NFC orqali tashqi qurilmalar ulanadi.

? 93. Mobil qurilmada tarmoq ulanish tezligini qanday oshirish mumkin?

 Tezkor Wi-Fi, 4G/5G tarmoqlaridan foydalanish va signalni optimallashtirish orqali oshiriladi.

? 94. Mobil qurilmalarda ilovalarni qanday boshqarish mumkin?

 Ilovalar do‘konlar orqali o‘rnatiladi, yangilanadi va o‘chiriladi.

? 95. Mobil qurilmalarda joylashuvni aniqlash qanday ishlaydi?

 GPS, Wi-Fi va mobil tarmoq signallari asosida joylashuv aniqlanadi.

☆ 96. Mobil qurilmalar uchun tarmoq xavfsizligi qanday ta‘minlanadi?

 VPN, SSL/TLS protokollari va xavfsiz autentifikatsiya orqali ta‘minlanadi.

? 97. Mobil qurilmalarda internetga ulanishning afzalliklari nimalardan iborat?

 Tezkor aloqa, onlayn xizmatlar, ilovalar va qulay ma‘lumot almashish imkonini beradi.

? 98. Mobil qurilmalarda ko‘proq resurslar ishlatishning salbiy oqibatlari nimalardan iborat?

 Batareya tez tugaydi, qurilma sekinlashadi va tizim barqarorligi pasayadi.

? 99. Mobil qurilmada video o‘ynatishning afzalliklari nimalardan iborat?

 Yuqori sifatli tasvir va ovoz bilan qulay va interaktiv kontent tomosha qilish imkonini beradi.

☆ 100. Mobil qurilmada xavfsiz aloqa qanday ta‘minlanadi?

 Ma‘lumotlarni shifrlash, autentifikatsiya va xavfsizlik protokollari orqali ta‘minlanadi.

2. Software

Atamalar va izohlar

Atamalar	Izohlar
Artifact	Build natijasida hosil bo'ladigan tayyor "chiqarilma" (masalan, .exe, .jar, .apk, Docker image) va u deployment uchun ishlatiladi.
Build	Manba koddan (source code) ishga tushadigan paket/ilova yaratish jarayoni (kompilyatsiya, resurslarni yig'ish, paketlash).
Configuration	Dastur xulqini kodni o'zgartirmasdan boshqaradigan sozlamalar (masalan, server manzili, DB ulanishi, log darajasi).
Dependency	Dastur ishlashi uchun kerak bo'lgan tashqi kutubxona/modul; dependency versiyasi noto'g'ri bo'lsa "moslik muammosi" chiqadi.
Environment	Dastur ishlaydigan muhit: OS, versiyalar, sozlamalar, tarmoq, huquqlar va resurslar majmui (dev/test/prod).
Executable	Operatsion tizim ishga tushira oladigan fayl (Windows'da .exe, Linux'da binary).
Firmware	Qurilma ichiga yozilgan past darajadagi dastur; apparatni boshqaradi (masalan, router firmware).
Library	Dasturchi qayta ishlatishi mumkin bo'lgan tayyor funksiyalar to'plami; kodni tez yozish va standartlashtirishga yordam beradi.
License	Dasturdan foydalanish, tarqatish va o'zgartirish shartlarini belgilovchi huquqiy ruxsat (MIT, GPL va h.k.).
Platform	Dastur ishlaydigan asosiy ekotizim: OS + arxitektura + runtime (masalan, Android platformasi).
Process	Ishga tushirilgan dastur nusxasi; OS resurs ajratadi (CPU, RAM).
Runtime	Dastur ishlayotgan paytdagi bajarilish muhiti (masalan, JRE — Java Runtime Environment).
Source code	Inson o'qiy oladigan dastur kodi; build jarayonida ishchi ko'rinishga o'tadi.
Versioning	Dastur versiyalarini boshqarish; qaysi o'zgarish qachon chiqqani kuzatiladi (SemVer ko'p ishlatiladi).
Workflow	Ish jarayonining ketma-ketligi: task → branch → commit → PR → review → merge → deploy.
Backtracking	Variantlarni sinab ko'rib, mos kelmasa "orqaga qaytib" boshqa yo'lni tanlaydigan usul (kombinatorika, labirint).
Big-O	Algoritmning resurs talabini (vaqt/xotira) katta hajmda baholash usuli; optimal yechim tanlashda muhim.
Branching	Shart operatorlari orqali yo'nalish tanlash (if/else); mantiqni boshqaradi.
Complexity	Algoritmning tezlik (time) va xotira (space) bo'yicha murakkabligi; katta ma'lumotlarda hal qiluvchi omil.
Divide and Conquer	Muammoni kichiklarga bo'lib yechish (merge sort, quicksort); samarali yechimlar beradi.
Edge case	Kam uchraydigan, lekin tizimni buzishi mumkin bo'lgan holatlar (bo'sh qiymat, max/min, noto'g'ri format).
Greedy	Har qadamda "eng yaxshi ko'ringan" tanlovni qilish; ba'zi masalalarda optimal, ba'zilarida noto'g'ri bo'lishi mumkin.
Invariant	Sikl yoki algoritm davomida doim saqlanadigan shart; to'g'rilikni isbotlashda ishlatiladi.
Iteration	Takrorlash orqali yechim qurish; rekursiyaga alternativa sifatida barqaror bo'lishi mumkin.
Memoization	Oldin hisoblangan natijani saqlab, qayta hisoblamaslik (DP'da tezlikni oshiradi).
Recursion	Funksiyaning o'zini chaqirishi; daraxt, bo'linma masalalarda qulay, lekin stack limit bor.
Sorting	Ma'lumotni tartiblash; qidirish, filtr va analiz jarayonlarida asosiy rol o'ynaydi.
State	Algoritmning joriy holati (o'zgaruvchilar qiymati); state noto'g'ri boshqarilsa bug chiqadi.
Termination	Algoritmning to'xtash sharti; noto'g'ri bo'lsa "cheksiz sikl" paydo bo'ladi.

Time limit	Real tizimlarda algoritmlar belgilangan vaqtda natija berishi kerak; samaradorlik talabini bildiradi.
ABI	Application Binary Interface — binary darajada moslik qoidalari (chaqirish konvensiyasi, formatlar); kutubxona mosligiga ta'sir qiladi.
Bytecode	VM (Virtual Machine) bajaradigan oraliq kod; platformalararo ishlashga yordam beradi (masalan, Java bytecode).
Compiler	Kodni mashina tiliga yoki oraliq kodga o'giradi; tez ishlash va optimizatsiya beradi.
Dynamic typing	O'zgaruvchi turi runtime'da aniqlanadi; tez prototiplashga qulay, lekin xato ham kechroq chiqishi mumkin.
Garbage Collector	Xotirani avtomatik tozalovchi mexanizm; memory leak'ni kamaytiradi, ammo performancega ta'sir qilishi mumkin.
Interpreter	Kodni bosqichma-bosqich bajaradi; tez o'rganish va skriptlar uchun qulay.
JIT	Just-In-Time — runtime'da kompilyatsiya; VM tezligini oshiradi (JIT compiler).
Memory leak	Keraksiz xotira bo'shatilmay qolishi; serverda RAM to'lishi va yiqilishiga olib keladi.
Package	Kodni modul sifatida tarqatish birliklari; dependency boshqaruvining asosiy elementi.
Runtime error	Dastur ish jarayonida yuzaga keladigan xato; try/catch bilan boshqarilishi mumkin.
Scripting	Avtomatlashtirish va yordamchi ishlar uchun tez yoziladigan kod (bash, Python skript).
Standard library	Til bilan birga keladigan asosiy kutubxonalar; "tashqi dependency" siz ko'p vazifani bajaradi.
Static typing	O'zgaruvchi turi oldindan aniqlanadi; katta loyihalarda aniqlik va barqarorlik beradi.
Transpiler	Bir tilni boshqa tilga o'giradi (masalan, TypeScript → JavaScript).
VM	Virtual Machine — kodni bajaradigan virtual muhit; platformadan mustaqillik beradi.
Breakpoint	Debugger'da dastur bajarilishini vaqtincha to'xtatish nuqtasi; qiymatlarni tekshirish uchun.
Code completion	IDE kodni yozishda takliflar beradi; tezlik va xatoni kamaytiradi.
Debugger	Kodni qadam-baqadam bajarib, xatoni topish vositasi; stack/variable ko'rish imkonini beradi.
Formatter	Kodni yagona formatga keltiradi; jamoada o'qish va review'ni yengillashtiradi.
IDE	Integrated Development Environment — kod yozish, test, debug, build'ni bir joyda boshqaruvchi muhit.
Linter	Kod uslubi va potentsial xatolarni statik tekshiradi; sifatni oshiradi.
Package manager	Kutubxona o'rnatish va versiyalarni boshqarish vositasi (pip, npm).
Profiler	Qaysi funksiya ko'p vaqt/yoki xotira ishlatayotganini ko'rsatadi; optimizatsiya uchun.
Refactoring tools	Kodni xavfsiz o'zgartirish (rename, extract method) vositalari; bug chiqishini kamaytiradi.
Snippet	Tez yozish uchun tayyor kod bo'lagi; standart strukturalar uchun qulay.
Source control panel	IDE ichida Git operatsiyalarini boshqarish oynasi; commit/push/pull qulaylashadi.
Static analysis	Kodni ishlatmasdan tahlil qilish; xavfsizlik va xato riskini kamaytiradi.
Terminal	Buyruqlar orqali build/test/run ishlari; CI bilan mos ishlashni o'rgatadi.
Test runner	Testlarni ishga tushirish va natija ko'rsatish vositasi; tez tahlil beradi.
Workspace	IDE'ning loyiha muhiti; sozlamalar, pluginlar, path'lar shu yerda.
Array	Ketma-ket xotirada saqlanadigan tuzilma; indeks bo'yicha tez murojaat, lekin o'rta qo'shish qimmat.
Binary search	Tartiblangan ro'yxatda qidirish; log(n) tezlik beradi.
Graph	Tugun va qirralardan iborat; tarmoq, yo'l topish, bog'lanishlarda ishlatiladi.
Hash table	Kalit bo'yicha tez qidirish; collision boshqaruvi muhim.
Heap	Prioritet navbat uchun tuzilma; scheduling, top-k masalalarda ishlatiladi.
Linked list	Elementlar ko'rsatkich bilan ulangan; qo'shish oson, lekin indeks bo'yicha qidirish sekin.

Queue	FIFO (First In First Out) navbat; task queue, bufferlarda ishlatiladi.
Stack	LIFO (Last In First Out); call stack, undo, parsing'da muhim.
Trie	So'z/ prefiks qidirishda samarali daraxt tuzilma; autocomplete'da ishlatiladi.
Tree	Ierarxik tuzilma; file system, DOM, kategoriyalar.
Two pointers	Ikki indeks bilan optimallashtirish; massivlarda tez yechim beradi.
DFS	Depth-First Search — grafni chuqurlashib aylanib chiqish (DFS — Depth-First Search).
BFS	Breadth-First Search — qatlam-qatlam aylanib chiqish (BFS — Breadth-First Search).
DP	Dynamic Programming — submasalalar natijasini yig'ish orqali yechish (DP — Dynamic Programming).
Sorting stability	Tartiblashda teng elementlar tartibi saqlanishi; ba'zi biznes vazifalarda muhim.
API	Application Programming Interface — dasturlar o'zaro ishlashi uchun qoidalar to'plami.
Auth token	API chaqiruvida kimligini tasdiqlash uchun token; noto'g'ri saqlansa xavfsizlik buziladi.
Endpoint	API'dagi aniq resurs/amaliyot manzili (masalan, /users).
HTTP	HyperText Transfer Protocol — web muloqot protokoli (HTTP — HyperText Transfer Protocol).
HTTPS	HTTP Secure — shifrlangan kanal (HTTPS — HyperText Transfer Protocol Secure).
JSON	JavaScript Object Notation — eng ko'p ishlatiladigan data format.
OAuth	Ochib berilgan avtorizatsiya protokoli (OAuth — Open Authorization).
Rate limit	API'da so'rovlar sonini cheklash; DDoS va haddan tashqari yuklamadan himoya.
Request/Response	Klient so'rovi va server javobi; status code va body muhim.
REST	Representational State Transfer — resursga yo'naltirilgan API uslubi.
Retry	So'rov muvaffaqiyatsiz bo'lsa qayta urinish; lekin idempotency talab qiladi.
SDK	Software Development Kit — servis bilan ishlash uchun tayyor kutubxona to'plami.
Serialization	Obyektni uzatish formatiga o'tkazish (JSON, protobuf).
Timeout	Javob kutish vaqti limiti; tarmoq muammosida tizimni "osilib qolish"dan saqlaydi.
Webhook	Tashqi servis hodisa bo'lganda sizning endpoint'ingizga xabar yuboradi; real-time integratsiya.
Access control	Kim qaysi resursga kira olishini boshqarish; role-based yondashuv keng tarqalgan.
Authentication	Login orqali foydalanuvchi kimligini tekshirish; token/biometrik bo'lishi mumkin.
Authorization	Foydalanuvchining ruxsatlarini tekshirish; "admin" va "user" farqi.
Brute force	Parolni ko'p urinish bilan topishga urinish; rate limit va lockout bilan himoya qilinadi.
Encryption	Ma'lumotni shifrlash; uzatishda va saqlashda maxfiylik beradi.
Exception handling	Xatoni tutish va to'g'ri javob qaytarish; tizim yiqilib qolmasligini ta'minlaydi.
Input validation	Kiruvchi ma'lumotni tekshirish; SQLi/XSS xavfini kamaytiradi.
Least privilege	Minimal huquq prinsipi; har modul faqat kerakli huquqqa ega bo'lsin.
Logging security	Logda token/parol/PII chiqmasligi; audit uchun yetarli ma'lumot bo'lsin.
MFA	Multi-Factor Authentication — ikki yoki ko'p bosqichli autentifikatsiya.
OWASP	Web xavfsizlik bo'yicha tavsiyalar (OWASP — Open Worldwide Application Security Project).
Patch management	Xavfsizlik patch'larini vaqtida qo'llash; zaifliklarni yopadi.
Secure coding	Xavfsizlikka mos kod yozish standartlari; review va static scan bilan kuchaytiriladi.
Session management	Session'ni yaratish, yangilash, tugatish; hijacking'dan himoya.
Threat model	Xavf manbalari va ssenariylarini oldindan tahlil qilish; himoya strategiyasini beradi.
CI	Continuous Integration — har commitdan keyin avtomatik build/test ishlashi (CI — Continuous Integration).
Coverage	Test qamrovi foizi; lekin foizdan ko'ra riskli ssenariylar muhim.
Debugging	Xatoni topish va sababini aniqlash jarayoni; log + debugger + test bilan ishlaydi.
E2E test	End-to-End — foydalanuvchi ssenariysi bo'yicha to'liq sinov (E2E — End-to-End).

Fixture	Test uchun tayyorlangan ma'lumotlar to'plami; deterministik natija beradi.
Flaky test	Ba'zan o'tib, ba'zan yiqiladigan test; ishonchni yo'qotadi, sababi izolyatsiya yo'qligi bo'lishi mumkin.
Integration test	Modulning DB/API bilan birga ishlashini tekshiradi; unit'dan realroq.
Mock	Tashqi servisni "soxta" nusxa bilan almashtirish; unit test tezligini oshiradi.
Postmortem	Incidentdan keyin tahlil; sabab, ta'sir, tuzatish va profilaktika choralari yoziladi.
Profiling	Tezlik/xotira muammosini o'lchash va "bottleneck"ni topish.
Regression	Yangi o'zgarish eski funksiyani buzmagani tekshirish.
Test case	Shart + kutilgan natija; testlarning eng asosiy birligi.
Test pyramid	Unit ko'p, integration o'rtacha, E2E kam bo'lishi kerak degan amaliy konsepsiya.
Trace	So'rovning tizim bo'ylab yurish yo'lini ko'rsatadi; distributed tizimlarda muhim.
Triage	Bug'larni ustuvorlik bo'yicha saralash; qaysi birini birinchi tuzatishni belgilaydi.
Branch	Vazifa uchun alohida rivojlantirish yo'li; main'ni barqaror saqlashga yordam beradi.
Cherry-pick	Kerakli bitta commitni boshqa branch'ga ko'chirish; hotfix'larda foydali.
Code review	Kodni jamoa tekshirishi; xato, xavfsizlik, uslub va arxitektura nazorati.
Commit message	O'zgarish izohi; tarixni tushunarli qiladi, auditga yordam beradi.
Conflict	Bir xil joydagi o'zgarishlar to'qnashuvi; mantiqan yechish talab etiladi.
Fork	Repozitoriyaning alohida nusxasi; open-source'da keng ishlatiladi.
Git	Tarqatilgan versiya boshqaruvi; tarix va hamkorlikni boshqaradi.
Merge	Branch'larni birlashtirish; review'dan keyin bajariladi.
Pull	Remote'dagi o'zgarishni lokalga olish; sync uchun.
Pull Request	O'zgarishni merge qilishdan oldin review uchun yuborish (PR — Pull Request).
Push	Lokal commitlarni remote'ga yuborish; jamoaga ko'rsatadi.
Rebase	Commit tarixini tartibga keltirish; konfliktni oldindan hal qilishga yordam beradi.
Tag	Release nuqtasini belgilash; versiyalarni aniq ajratadi.
Trunk-based	Kichik branch, tez merge uslubi; CI kuchli bo'lsa juda samarali.
Workflow policy	Branch naming, review qoidalari, merge strategiyasi kabi kelishilgan jamoa qoidalari.
Blue-green	Ikkita muhitda (blue/green) navbat bilan release; downtime'ni kamaytiradi.
Canary release	Yangi versiyani avval kichik foydalanuvchilarga chiqarish; riskni pasaytiradi.
Deployment	Dastur versiyasini ishchi muhitga chiqarish; avtomatlashtirilsa xavf kamayadi.
Downtime	Tizim ishlamay qolgan vaqt; SLA bilan boshqariladi.
Health check	Servis "tirik"ligini tekshiruvchi endpoint; orchestratorlar ishlatadi.
Incident	Ishlashdagi jiddiy muammo; tezkor reaksiya va postmortem talab qiladi.
Log rotation	Loglarni bo'lib saqlash va eskisini arxivlash; disk to'lib ketmasligi uchun.
Monitoring	CPU/RAM/latency/error rate kuzatuvi; alerting bilan birga ishlaydi.
Patch	Kichik tuzatish yoki xavfsizlik yangilanishi; tez chiqariladi, lekin test shart.
Rollback	Muammo bo'lsa oldingi versiyaga qaytish; business uzilishining oldini oladi.
SLO	Service Level Objective — xizmat ko'rsatish maqsad ko'rsatkichlari (SLO — Service Level Objective).
SLA	Service Level Agreement — xizmat darajasi bo'yicha kelishuv (SLA — Service Level Agreement).
Staging	Production'ga o'xshash sinov muhiti; release oldidan tekshiriladi.
Version pinning	Dependency versiyasini "qotirib" qo'yish; kutilmagan yangilanishlardan himoya.
Zero-downtime	Servisni to'xtatmasdan yangilash; load balancer va strategiyalar bilan erishiladi.

Savol-javoblar

2.1. Dasturiy ta'minot asoslari

❓ **1. Kompyuterda dastur ishga tushganda operatsion tizim avval nimani bajaradi?**

📖 Operatsion tizim dastur faylini topadi, uni xotiraga (RAM) yuklaydi, kerakli kutubxonalarini ulaydi, dastur uchun jarayon (process) yaratadi va CPUga

bajariladigan buyruqlar oqimini beradi. Shu bosqichda ruxsatlar, foydalanuvchi huquqlari va resurslar (xotira, fayl, tarmoq) ajratiladi.

? 2. Dastur “o‘rnatilgan” bo‘lsa ham, nega ishga tushishda sekinlashishi mumkin?

📖 Sekinlashish odatda quyidagilardan keladi: disk sekinligi (HDD/SSD), antivirus tekshiruvlari, ko‘p kutubxonalar yuklanishi, internetga bog‘liq initial sozlamalar, ortiqcha “startup” servislari, yoki RAM yetishmasligi sabab “swap” (diskka vaqtinchalik yozish) ishlashi. Amaliy yechim: startup’ni qisqartirish, log/profiling, SSD, RAM, keraksiz servislarni o‘chirish.

? 3. Desktop, web va mobil dasturlarni tanlashda amaliy mezonlar qaysilar?

📖 Desktop — kuchli lokal resurs, offline ishlash, apparatga chuqur kirish (USB/serial) kerak bo‘lsa. Web — tez tarqatish, brauzer orqali kirish, platformadan mustaqillik muhim bo‘lsa. Mobil — sensorlar (GPS, kamera), push bildirishnoma, doimiy harakatdagi foydalanish bo‘lsa. Tanlov vazifaga bog‘liq: “foydalanuvchi qayerda va qanday ishlatadi?”

? 4. “Offline” ishlash talabi bo‘lsa, dastur arxitekturasida qanday o‘zgaradi?

📖 Offline rejimda dastur lokal saqlash (SQLite, fayl, cache)ga tayanadi, sinxronlash mexanizmi (online bo‘lganda serverga yuborish) bo‘ladi, konfliktlarni hal qilish qoidalari kerak (masalan, qaysi versiya “haqiqiy”). UI ham “internet yo‘q” holatini aniq ko‘rsatishi, so‘rovlarni navbatga qo‘yishi lozim.

? 5. Dastur “osilib qolishi” (freeze) ko‘pincha qaysi amaliy xatolardan kelib chiqadi?

📖 Ko‘p hollarda asosiy sabab: UI thread’da og‘ir ish bajarish (katta fayl o‘qish, tarmoq so‘rovi, katta hisoblash), cheksiz sikl, deadlock, yoki resurs tugashi. Yechim: og‘ir ishlarni background thread/async ga chiqarish, timeout qo‘yish, profiling qilish, xotira sarfini kuzatish.

? 6. Operatsion tizim dasturga qanday resurslarni beradi va ularni qanday cheklaydi?

📖 OS CPU vaqtini “rejalashtiradi”, RAM ajratadi, fayl tizimi va tarmoqdan foydalanishni ruxsat bilan boshqaradi. Cheklashlar: foydalanuvchi huquqlari, sandbox (mobil), file permission, portlarga ruxsat, memory limit. Shuning uchun dastur “admin”siz ham ishlashi, minimal huquq so‘rashi amaliy jihatdan muhim.

☆ 7. 32-bit va 64-bit muhit farqi dastur ishlashida amalda nimaga ta’sir qiladi?

📖 64-bit muhit ko‘proq xotiradan foydalanadi (katta RAM), katta ma’lumotlar bilan ishlashga qulay. 32-bitda xotira manzillash cheklangan, ayrim katta hajmli dasturlar crash bo‘lishi mumkin. Amaliyotda: katta dataset, AI modellar, video ishlov berish — 64-bit talab qiladi.

❓ **8. Dasturda “xotira to‘lib qolishi” muammosini amaliy aniqlash yo‘li qanday?**

📖 Belgilar: dastur sekinlashadi, tizim swap ishlata boshlaydi, “out of memory” xatolari, kutilmagan yopilish. Aniqlash: Task Manager/Activity Monitor’da RAM kuzatish, logging, profiler. Sabablar: katta obyektlarni bo‘shatmaslik, cache cheksiz o‘sishi, fayl/stream yopilmasligi.

❓ **9. Dasturiy ta’minotning “tizimli” va “amaliy” qismlari o‘zaro qanday hamkorlik qiladi?**

📖 Tizimli qism (OS, drayver, servislari) apparatni boshqaradi va standart xizmatlar beradi (fayl, tarmoq, xotira). Amaliy dastur esa shu xizmatlardan API orqali foydalanadi. Masalan, printer: drayver chop etishni ta’minlaydi, Word esa OS print servisiga buyruq beradi.

❓ **10. “Drayver” muammosi dastur ishlashiga qanday ta’sir ko‘rsatadi?**

📖 Drayver nosoz bo‘lsa qurilma tanilmaydi, tez-tez uzilib qoladi, performance pasayadi yoki tizim xato beradi. Amaliy holat: kamera/mikrofon ishlamasligi, grafikada artefakt, USB qurilma ko‘rinmasligi. Yechim: mos drayver, versiya mosligi, OS yangilanishi.

★ **11. Dastur barqarorligi (stability)ni oshirish uchun amaliy 3 ta shart nima?**

📖 1) Xatolarni boshqarish (exception handling) va foydalanuvchiga tushunarli xabar. 2) Resurslarni to‘g‘ri boshqarish (faylni yopish, xotirani nazorat qilish). 3) Real testlar: muhim “user flow”lar, turli qurilma/OS versiyalarida sinov.

❓ **12. Dastur tezligini oshirishda birinchi bo‘lib nimani tekshirish kerak?**

📖 Avvalo “qayeri sekin” ekanini o‘lchash: tarmoqmi, diskmi, DB so‘rovmi, CPUmi. Profiling va loglar bilan bottleneck topilmasa, optimizatsiya ko‘pincha samarasiz bo‘ladi. Keyin caching, so‘rovni optimallashtirish, asinxron bajarish kabi choralar tanlanadi.

❓ **13. “Kesh”dan noto‘g‘ri foydalanish qanday real muammolarga olib keladi?**

📖 Eski ma'lumot ko'rinib qolishi, foydalanuvchi “yangiladi” desa ham o'zgarmasligi, xotira ortib borishi. Amaliy yechim: cache TTL (vaqt), invalidation (yangilanganda o'chirish), cache limit.

★ **14. Dastur yangilanishi (update) noto'g'ri chiqsa, amaliy zararlar nimalar bo'ladi?**

📖 Dastur ochilmay qolishi, ma'lumotlar mos kelmasligi (migration xatolari), xavfsizlik teshiklari, foydalanuvchi ish jarayoni to'xtashi. Shuning uchun rollback, backup, versiyalash, release notes va staging sinovi muhim.

❓ **15. “Konfiguratsiya”ni koddan ajratish amaliy jihatdan nima beradi?**

📖 Bir xil dastur turli muhitda (test/staging/prod) har xil sozlamada ishlaydi: DB manzili, API kalitlari, log darajasi. Konfiguratsiya kodga kirib ketsa, xato tarqatish va xavfsizlik muammolari ko'payadi. Eng to'g'ri amaliyot: env variables yoki config fayl.

❓ **16. Dastur “portable” bo'lishi (ko'chma) uchun nimalarga e'tibor beriladi?**

📖 OSGa bog'liq yo'llar (path), kodlash (encoding), platformaga xos kutubxonalar, arxitektura (x86/ARM) farqlari, UI moslashuvi. Web'da — brauzer mosligi; mobil'da — ekran o'lchamlari va permission'lar.

★ **17. Litsenziya masalasi amaliyotda qachon muammo bo'lishi mumkin?**

📖 Korxona yoki davlat tashkilotida noqonuniy/litsenziyasiz dastur ishlatish tekshiruv, jarima va xavfsizlik riskini keltiradi. Open-source ham turli litsenziyada bo'ladi; ba'zilar tijoriy ishlatishda cheklov qo'yishi mumkin. Shuning uchun kutubxona/dastur tanlashda litsenziya tekshiriladi.

❓ **18. Dasturda “foydalanuvchi huquqlari” noto'g'ri sozlansa, qanday amaliy oqibat bo'ladi?**

📖 Oddiy foydalanuvchi bajarishi mumkin bo'lmagan vazifani ko'radi, yoki aksincha admin imkoniyatiga ruxsat berilib ketadi. Natija: ma'lumot sizishi, buzish, xizmat rad etilishi. Yechim: role-based access, minimal privilege, audit log.

❓ **19. Dasturiy ta'minotda “xizmat” (service) va “ilova” (app) o'rtasidagi farq amalda nimada seziladi?**

📖 Ilova odatda foydalanuvchi interfeysiga ega va foydalanuvchi ishga tushiradi. Service esa fon rejimida ishlaydi: avtomatik start, doimiy kuzatuv, boshqa dasturlarga xizmat ko'rsatish (masalan, update service, database service). Muammo chiqsa service log'larini tekshirish kerak bo'ladi.

☆ **20. SDLC bosqichlarini amaliy loyihada minimal qanday tartibda yuritish mumkin?**

📖 Minimal samarali tartib: talab → dizayn (oddiy diagramma) → implementatsiya → test → deploy → monitoring → qayta aloqa. Juda murakkab hujjat shart emas, lekin har bosqichdan “iz” qolishi kerak: talab yozuvi, commitlar, test natijalari, release notes.

❓ **21. “Monitoring” bo‘lmasa, real tizimda qanday muammo yuz beradi?**

📖 Tizim sekinlashgani, xato ko‘paygani, server resurslari tugayotgani kech bilinadi. Monitoring (CPU, RAM, disk, error rate, latency) bo‘lsa muammo erta aniqlanadi va avariya kamayadi.

❓ **22. Dastur “xavfsiz” bo‘lishi uchun minimal amaliy talablar qaysilar?**

📖 Shifrlangan aloqa (HTTPS), parollarni xesh bilan saqlash, input validation, yangilanishlarni o‘z vaqtida berish, ruxsatlarni to‘g‘ri sozlash, loglarda maxfiy ma’lumot qoldirmaslik.

☆ **23. Dasturda “ma’lumot yo‘qolishi”ning eng ko‘p uchraydigan amaliy sabablari nimalar?**

📖 Backup yo‘qligi, noto‘g‘ri update/migration, foydalanuvchi xatosi (delete), disk nosozligi, sinxronlash konfliktlari. Yechim: avtomatik backup, “soft delete”, audit, test migration, tranzaksiyalar.

❓ **24. Dasturiy ta’minotni tanlashda “TCO” (umumiy xarajat)ni qanday baholash kerak?**

📖 Faqat narx emas: o‘rnatish, yangilash, xizmat ko‘rsatish, o‘qitish, texnik yordam, server xarajati, xavfsizlik va integratsiya narxlari ham kiradi. Bepul dastur ham xizmat ko‘rsatish qimmat bo‘lishi mumkin.

☆ **25. Hardware bo‘limi bilan Software bo‘limi amalda qayerda “uchrashadi”?**

📖 Drayverlar, OS xizmatlari, qurilma interfeyslari (USB, Bluetooth, Wi-Fi), sensorlardan ma’lumot olish, real vaqt boshqaruvi, energiya boshqaruvi kabi joylarda uchrashadi. Masalan, mikrokontrollerdan kelgan serial ma’lumotni kompyuter dasturi qabul qiladi: bu yerda ham apparat protokoli, ham dasturiy parser va UI muhim.

2.2. Algoritmik fikrlash va kod mantiqi

❓ **1. Muammoni algoritmgaga aylantirishdan oldin qaysi savollarga javob berish kerak?**

📖 Avvalo muammoning **kirish ma'lumotlari** (input), **chiqish natijalari** (output) va **cheklovlari** aniqlanadi. Masalan: ma'lumotlar soni qancha, real vaqt talab qilinadimi, xotira cheklanganmi. Shundan keyin muammo kichik bosqichlarga bo'linadi. Bu bosqich keyingi kodlashda xatolarni keskin kamaytiradi.

❓ **2. Murakkab vazifani kichik qismlarga bo'lish amaliyotda qanday foyda beradi?**

📖 Kichik vazifalar oson testlanadi, qayta ishlatiladi va xatoni topish tezlashadi. Masalan, "hisobot yaratish" vazifasi: ma'lumot olish, tekshirish, qayta ishlash, chiqarish bosqichlariga ajratiladi. Bu yondashuv moduliylilikni ta'minlaydi.

❓ **3. Algoritmni yozishda qachon blok-sxema, qachon pseudokod qulayroq?**

📖 Blok-sxema vizual tushuntirish uchun qulay, ayniqsa boshlang'ich bosqichda. Pseudokod esa kodga yaqin bo'lib, murakkab mantiqni aniq ifodalashda samaraliroq. Amaliyotda murakkab algoritmlar uchun pseudokod ko'proq ishlatiladi.

❓ **4. Algoritmni tekshirishda "qo'lda bajarish" (dry run) nima beradi?**

📖 Dry run orqali algoritm har bir qadamda qanday ishlashini ko'rish mumkin. Bu mantiqiy xatolarni kod yozishdan oldin aniqlashga yordam beradi. Ayniqsa shartlar va sikllar ko'p bo'lsa, juda foydali.

❓ **5. Shart operatorlari noto'g'ri tuzilsa, amaliy muammo qanday ko'rinishda chiqadi?**

📖 Ba'zi holatlar umuman ishlanmay qoladi (edge case), noto'g'ri natija chiqadi yoki cheksiz sikl yuzaga keladi. Masalan, $\leq$ o'rniga $<$ ishlatilishi bitta muhim holatni tashlab ketishga olib kelishi mumkin.

❓ **6. Takrorlanish (loop) ishlatilganda eng ko'p uchraydigan amaliy xatolar qaysilar?**

📖 Cheksiz sikl, noto'g'ri boshlang'ich qiymat, noto'g'ri tugash sharti va indeks chegarasidan chiqish. Bu xatolar dastur "osilib qolishi" yoki crash bo'lishiga olib keladi.

❓ **7. Algoritmida "edge case"larni e'tiborsiz qoldirish nimaga olib keladi?**

📖 Bo'sh ro'yxat, bitta element, maksimal/minimal qiymatlar kabi holatlar ishlanmay qoladi. Real tizimlarda aynan shu holatlar ko'p muammo keltirib chiqaradi, shuning uchun alohida tekshiriladi.

★ **8. Algoritm samaradorligini baholashda Big-O'dan tashqari yana nimalar muhim?**

📖 Real ma'lumot hajmi, xotira sarfi, kiritish/chiqarish (I/O) kechikishi va konstant omillar muhim. Ba'zan nazariy jihatdan yomonroq algoritm amalda tezroq ishlashi mumkin.

❓ **9. Qidiruv vazifasida qachon oddiy, qachon optimallashtirilgan algoritm tanlanadi?**

📖 Ma'lumotlar kam bo'lsa oddiy qidiruv yetarli. Katta hajm va tezlik muhim bo'lsa, indekslash yoki maxsus tuzilmalar (masalan, hash) tanlanadi. Tanlov doimo real sharoitga bog'liq.

❓ **10. Tartiblash algoritmini tanlashda amaliy mezonlar qaysilar?**

📖 Ma'lumot hajmi, xotira cheklovi, barqarorlik (stable sort) va ma'lumot qisman tartiblanganmi yoki yo'q. Masalan, kichik massivlar uchun oddiy usullar yetarli bo'lishi mumkin.

❓ **11. Rekursiya qachon qulay, qachon xavfli bo'ladi?**

📖 Rekursiya mantiqni soddalashtiradi, lekin chuqurlik katta bo'lsa stack overflow yuz beradi. Real tizimlarda rekursiya o'rniga iteratsiya tanlanishi ko'pincha xavfsizroq.

❓ **12. Algoritmida xotirani tejash qachon tezlikdan ustun bo'ladi?**

📖 Mobil qurilmalar, mikrokontrollerlar va embedded tizimlarda xotira cheklangan. Bu holatda tezroq emas, kam xotira ishlatadigan algoritm tanlanadi.

❓ **13. Algoritmni qayta ishlatish (reusability) amalda qanday ta'minlanadi?**

📖 Umumiy funksiyalar ajratiladi, kirish-chiqish aniq belgilanadi, global o'zgaruvchilardan qochiladi. Bu kodni boshqa loyihalarda ham ishlatishga imkon beradi.

☆ **14. Real vaqtda ishlovchi tizimlarda algoritm tanlashda nimaga e'tibor beriladi?**

📖 Eng yomon holatda ham bajarilish vaqti oldindan ma'lum bo'lishi kerak. "O'rtacha tez" emas, **doimiy kafolatlangan vaqt** muhim bo'ladi.

❓ **15. Algoritmida noto'g'ri boshlang'ich qiymat qanday muammoga olib keladi?**

📖 Hisoblash noto'g'ri boshlanadi va oxirgi natija xato bo'ladi. Bu xato ko'pincha sezilmaydi, lekin tizim ishonchliligini pasaytiradi.

❓ **16. Murakkab algoritmni tushuntirishda izohlar (comment) qanday rol o'ynaydi?**

📖 Izohlar mantiqni tushunishga yordam beradi, ayniqsa jamoaviy ishda. Ammo izoh kodni takrorlamasligi, balki **nega shunday qilinganini** tushuntirishi kerak.

❓ 17. Algoritmni testlashda minimal test to‘plami qanday bo‘lishi kerak?

📖 Normal holat, chegaraviy holatlar (edge case), noto‘g‘ri kiritish va maksimal yuklama. Shu to‘plam algoritm barqarorligini tekshiradi.

★ 18. Bir xil natija beruvchi ikki algoritmdan qaysi biri tanlanadi?

📖 Soddaroq, tushunarliroq va kamroq xato keltiradigani tanlanadi. Haddan tashqari murakkab algoritm faqat zarurat bo‘lsa ishlatiladi.

❓ 19. Algoritmni optimallashtirishni qachon boshlash kerak?

📖 Avval ishlaydigan, to‘g‘ri algoritm yoziladi. Faqat sekinlik real muammo bo‘lsa, profilingdan keyin optimallashtiriladi. “Oldindan optimallashtirish” ko‘pincha zararli.

❓ 20. Algoritmik xatolarni koddan oldin aniqlashning eng samarali usuli qaysi?

📖 Dry run, pseudokod, test misollar bilan qo‘lda tekshirish. Bu usullar vaqtni tejaydi va murakkab xatolarni erta bosqichda topadi.

❓ 21. Algoritmni tanlashda foydalanuvchi tajribasi (UX) qanday rol o‘ynaydi?

📖 Juda sekin javob foydalanuvchini bezovta qiladi. Ba’zan biroz kam aniqlik evaziga tez javob UX’ni yaxshilaydi (masalan, real vaqtda qidiruv).

★ 22. Katta tizimlarda algoritmik qarorlar qayerda hujjatlashtiriladi?

📖 Texnik dizayn hujjatlarida: nima uchun aynan shu algoritm tanlangani, alternativalar va cheklovlar yoziladi. Bu keyinchalik tizimni tushunishni osonlashtiradi.

❓ 23. Algoritmni o‘zgartirishda regressiya xavfi qanday kamaytiriladi?

📖 Avval testlar yoziladi, keyin o‘zgartiriladi. Natijalar solishtiriladi. Test bo‘lmasa, eski xatolar qaytib kelishi mumkin.

❓ 24. Algoritmik fikrlash dasturchining qaysi ko‘nikmalarini rivojlantiradi?

📖 Mantiqiy tahlil, muammoni bo‘lish, sabr, aniqlik va optimallashtirish qobiliyatini rivojlantiradi. Bu ko‘nikmalar barcha IT sohalarda zarur.

☆ 25. Algoritmik fikrlash Hardware va AI bo'limlari bilan qayerda kesishadi?

📖 Hardware'da real vaqt va resurs cheklovi, AI'da ma'lumotni samarali qayta ishlash algoritmik fikrlashni talab qiladi. Algoritm — barcha yuqori texnologiyalarning asosi.

2.3. Dasturlash tillari va ularning qo'llanishi

❓ 1. Yangi loyiha boshlaganda dasturlash tilini “trend”ga qarab emas, qanday mezonlar asosida tanlash kerak?

📖 Til tanlashda eng amaliy mezonlar: (1) **maqsadli platforma** (web, mobil, desktop, embedded), (2) **jamoa tajribasi** (jamoa biladigan til tezroq natija beradi), (3) **ekotizim** (kutubxonalar, frameworklar, community), (4) **integratsiya** (DB, API, qurilmalar), (5) **performance va resurs** (CPU/RAM cheklovi bormi), (6) **xavfsizlik va support** (uzoq muddat yangilanish). Masalan, tez MVP kerak bo'lsa Python/JS tez; embedded bo'lsa C/C++; mobil native bo'lsa Kotlin/Swift.

❓ 2. Bir loyiha ichida bir nechta til ishlatish qachon o'zini oqlaydi, qachon zarar qiladi?

📖 Oqlaydi: komponentlar vazifasi keskin farq qilsa (masalan, backend — Python/Java, mobil — Kotlin/Swift, real-time modul — C++). Zarar qiladi: kichik jamoada har xil til sabab maintain qiyinlashsa, test va deploy murakkablashsa. Amaliy qoida: **yadro funksiyalarni bitta asosiy tilda** ushlang, faqat zarur joyda boshqa til kiriting.

❓ 3. Kompilyatsiya qilinadigan til va interpretatsiya qilinadigan til tanlovi amalda nimaga ta'sir qiladi?

📖 Kompilyatsiya (C/C++/Rust, ko'pincha Go): odatda yuqori tezlik, resursga samarali, binary tarqatish oson; lekin build jarayoni va platformaga moslash muhim. Interpretatsiya (Python, ko'p JS holatlari): tez yoziladi, prototiplash oson, lekin runtime performance va dependency boshqaruvi muhim bo'ladi. Real hayotda: **tez ishlab chiqish** kerak bo'lsa — Python/JS; **past kechikish, real-time, embedded** bo'lsa — C/C++/Rust.

☆ 4. Kompyuterda ishlaydigan dastur (desktop) uchun til tanlashda qaysi amaliy omillar hal qiluvchi?

📖 Desktop'da asosiy omillar: OS integratsiya (Windows/Mac/Linux), GUI framework mavjudligi, installer/packaging, qurilmalar bilan ishlash (USB, serial, kamera), offline rejim. Masalan: Windows ekotizimida C#/.NET (WPF/WinUI) qulay;

cross-platform uchun Python+Qt yoki Electron (JS) ishlaydi, lekin Electron resurs ko‘proq yeydi. Performance muhim bo‘lsa C++/Rust.

❓ 5. Web loyiha uchun til tanlashda front-end va back-endni qanday ajratish ma’qul?

📖 Amaliy yondashuv: front-end — odatda JavaScript/TypeScript (brauzer tili). Back-end: Python (Django/FastAPI), Java (Spring), C# (.NET), Node.js (JS/TS), PHP (Laravel). Tanlovda: mavjud jamoa tajribasi, trafik, xavfsizlik, hosting, kutubxonalar va integratsiya muhim. Ko‘p holatda front-end TS, back-end esa Python yoki Node bo‘lishi tez natija beradi.

❓ 6. Mobil ilova uchun “native” va “cross-platform” tanlovi amalda qanday qilinadi?

📖 Native (Android: Kotlin/Java, iOS: Swift) — maksimal tezlik, sensorlar bilan to‘liq integratsiya, UI silliq. Cross-platform (Flutter/Dart, React Native/JS) — bitta kod bazasi, tezroq ishlab chiqish, lekin ayrim nozik funksiyalar (kamera, Bluetooth, background servis)da native kod qo‘shimcha kerak bo‘lishi mumkin. Amaliy qoida: oddiy biznes ilovalar — cross-platform; yuqori performance, maxsus device integratsiya — native.

☆ 7. Mikrokontroller/embedded ishlarda nega C/C++ eng ko‘p ishlatiladi va qachon Python ishlashi mumkin?

📖 C/C++ bevosita temirga yaqin: xotira nazorati, real vaqt, driver, interrupt boshqaruvi. Embedded’da RAM/Flash kichik, shuning uchun C juda mos. Python (MicroPython/CircuitPython) prototiplash va o‘qitishda qulay, lekin performance va real-time cheklangan bo‘lishi mumkin. Amaliy qoida: final sanoat/harbiy embedded — C/C++; tez prototip yoki ta’lim — MicroPython.

❓ 8. Bir xil muammoni Python’da ham, C++da ham yechish mumkin bo‘lsa, qaysi holatda C++ tanlanadi?

📖 C++ tanlanadi: real-time talab bo‘lsa, latency juda past bo‘lsa, resurs cheklangan qurilma bo‘lsa, millionlab operatsiya/sekund kerak bo‘lsa (signal processing, video, robototexnika). Python tanlanadi: tez ishlab chiqish, AI/analitika, avtomatlashtirish, web servislari, prototiplash. Ko‘p real loyihada “gibrid” ishlaydi: tezkor core C++da, boshqaruv Python’da.

❓ 9. “Runtime error” va “compile-time error”ni kamaytirish uchun til tanlashda nimalar e’tiborli?

📖 Static typing (Java, C#, Rust, Go, TypeScript) ko‘p xatoni compile vaqtida ushlaydi. Dynamic typing (Python, JS) tez, lekin runtime‘da xato chiqishi ehtimoli yuqoriroq. Amaliy yechim: dynamic til ishlatilsa ham type hints (Python), TypeScript (JS o‘rniga), testlar va linterlar bilan risk kamaytiriladi.

☆ 10. Katta jamoada kodni yillar davomida ushlab turish (maintenance) uchun qaysi tillar ko‘proq qulay?

📖 Odatda Java/C# kabi kuchli ekotizimli, typing‘li, tooling‘i kuchli tillar katta jamoada qulay: codebase tartibli bo‘ladi, refactor xavfsizroq. Python ham katta tizimlarda ishlaydi, lekin standartlar (typing, lint, test, arxitektura) qat‘iy bo‘lishi kerak. JS/TS‘da ham TypeScript yirik loyihada ancha barqaror.

❓ 11. Til tanlashda “kutubxona va ekotizim”ni qanday amaliy tekshirish kerak?

📖 3 ta real tekshiruv:

1. Sizga kerak bo‘lgan modul bor-mi? (auth, DB, logging, testing, API client)
2. Kutubxona faolligi (oxirgi update, issue javobi, community)
3. Xavfsizlik va barqarorlik (patch tezligi, mashhurlik)
4. Keyin kichik POC (prototip) qilib ko‘riladi: 1–2 kunlik sinov ko‘p xatoni erta ko‘rsatadi.

❓ 12. “Back-end”da bir tilda yozilgan API‘ni boshqa tilda yozilgan ilova qanday ishlatadi?

📖 API odatda HTTP/HTTPS orqali JSON yuboradi. Klient tomoni (mobil, web, desktop) tildan qat‘i nazar HTTP so‘rov yuboradi va JSON‘ni parse qiladi. Muammo bo‘lishi mumkin: format mosligi, encoding, vaqt zonasi, versiyalash. Shuning uchun API contract (OpenAPI/Swagger) va testlar muhim.

☆ 13. Dasturlash tilini tanlash xavfsizlikka amalda qanday ta‘sir qiladi?

📖 Ba‘zi tillar xotira xatolariga (buffer overflow) ko‘proq moyil (C/C++), ba‘zilari memory safety beradi (Java, C#, Rust). Lekin xavfsizlik faqat til emas: input validation, auth, secret management, dependency update muhim. Amaliy xulosa: internetga ochiq servislar uchun xavfsizlik tooling‘i kuchli ekotizim va standartlar muhim.

❓ 14. “Script” yozish uchun (avtomatlashtirish) qaysi til ko‘proq amaliy va nima uchun?

📖 Python ko‘pincha birinchi tanlov: fayl, tarmoq, API, data ishlari uchun kutubxona ko‘p; sintaksis sodda. Bash/PowerShell tizim boshqaruvida kuchli. Real

ishda: server admin — bash/powershell; data/ETL — python; kichik utilita — python yoki go (binarni tarqatish oson).

? 15. Windows muhitida korxona dasturlarida C#/.NET ko‘p ishlatilishining amaliy sabablari?

📖 GUI (WPF/WinUI), Active Directory integratsiya, Office bilan integratsiya, kuchli IDE (Visual Studio), enterprise support. Bundan tashqari C# statik typing va tooling sabab katta tizimda barqaror.

? 16. Web front-end‘da JavaScript o‘rniga TypeScript tanlash amalda nimani yaxshilaydi?

📖 TypeScript katta loyihada xatolarni erta ushlaydi, refactor qilishni osonlashtiradi, IDE autocompletion kuchayadi. Amaliy natija: bug kamayadi, jamoada kod tushunish tezlashadi, vaqt tejaladi.

☆ 17. Bir tildagi “performance muammo”ni hal qilishning amaliy yo‘llari nimalar?

📖 1) Profiling: qayeri sekinligini topish. 2) Algoritmni yaxshilash (DSA). 3) I/O‘ni optimallashtirish (batch, cache). 4) Parallel/async. 5) Critical qismni tez tilga ko‘chirish (Python → C++ extension, Rust module). Avval o‘lchov, keyin optimizatsiya — asosiy qoida.

? 18. Dasturlash tili tanlashda “deploy” (tarqatish) masalasi qanday rol o‘ynaydi?

📖 Ba‘zi tillarda deploy oson: Go/Rust — bitta binary. Python/Node — dependency va virtual environment muhim. Mobil — store talablari bor. Korxona muhitida “o‘rnatish osonligi” ko‘p vaqtni tejaydi, shuning uchun packaging strategiyasi oldindan belgilanadi.

? 19. Dasturlash tili tanlashda “xodim topish” (kadrlar) omili qanday baholanadi?

📖 Real hayotda muhim: bozorda ko‘p mutaxassis bor til (Python, JS, Java) riskni kamaytiradi. Noyob til (Rust, Haskell) kuchli bo‘lsa ham, jamoani kengaytirish qiyin bo‘lishi mumkin. Shuning uchun “kadr + texnologiya” balansi tanlanadi.

☆ 20. AI/ML ishlari uchun nega Python ko‘p ishlatiladi va qachon boshqa til kerak bo‘ladi?

📖 Python‘da ML kutubxonalari juda boy (NumPy, PyTorch, TensorFlow). Tez eksperiment, notebook, vizualizatsiya qulay. Boshqa til kerak bo‘ladi: inference juda

tez bo‘lishi kerak bo‘lsa (C++/Rust), mobil qurilmada model ishlashi kerak bo‘lsa (Swift/Kotlin + native inference), yoki serverda yuqori throughput bo‘lsa (C++ backend).

❓ **21. Dasturda “kross-platform” talab bo‘lsa, texnologiya tanlovi qanday qilinadi?**

📖 Web-app yoki PWA ko‘pincha eng arzon yo‘l. Desktop cross-platform uchun Electron, Qt, .NET MAUI. Mobil cross-platform: Flutter/React Native. Tanlovda: performance, UI talablari, qurilma integratsiya, jamoa tajribasi va deploy jarayoni hisobga olinadi.

❓ **22. Katta tizimlarda “tilni almashtirish” (migratsiya) qachon o‘zini oqlaydi?**

📖 Faqat aniq sabab bo‘lsa: performance yetmayapti, xavfsizlik risk, vendor lock-in, maintenance juda qimmat. Migratsiya — qimmat jarayon: test, qayta yozish, kadr, vaqt. Amaliy yo‘l: to‘liq emas, **bosqichma-bosqich**: avval eng muhim modul.

☆ **23. Dasturlash tili tanlashda “kutladigan trafik/yuklama”ni qanday hisobga olish kerak?**

📖 Yuklama oshsa: parallel ishlash, memory, latency muhim bo‘ladi. Node.js I/O‘da kuchli, lekin CPU heavy ishda qiynaladi; Java/C# ko‘p yillik enterprise tajribaga ega; Go concurrency uchun qulay. Ammo yana asosiy qoida: avval arxitektura va DB optimizatsiya, keyin til.

❓ **24. “Legacy kod” (eski tizim) bilan integratsiyada til tanlash qanday bo‘ladi?**

📖 Eski tizim qaysi protokol va formatda ishlashiga qaraladi: SOAP, REST, message queue, DB access. Integratsiya oson bo‘lgan ekotizim tanlanadi. Ko‘p hollarda “adapter servis” yoziladi: eski tizimni o‘zgartirmasdan, yangi tizim bilan ulab beradi.

☆ **25. Bitiruvchi talaba uchun “universal start” texnologiya steki qanday bo‘lishi ma’qul?**

📖 Amaliy “ish bozoriga mos” start:

- Asosiy til: **Python** (avtomatlashtirish, data, backend asos)
- Web: **JavaScript/TypeScript** (front-end, API bilan ishlash)
- DB asos: PostgreSQL/MySQL tushunchalari
- Git va Linux asoslari

- Keyin yo‘nalishga qarab: mobil (Kotlin/Swift), embedded (C/C++), AI (PyTorch).
- Bu yo‘l talabning keyingi bo‘limlarga o‘tishini oson qiladi.

2.4. IDE va dasturiy vositalar bilan ishlash

? 1. IDE tanlashda “eng mashhur” emas, aynan qaysi omillar muhimroq hisoblanadi?

 IDE tanlashda quyidagilar hal qiluvchi: ishlatiladigan dasturlash tili va frameworklar, debugging sifati, autocompletion (kodni to‘ldirish), refactoring imkoniyatlari, pluginlar mavjudligi, tizim resurslariga ta’siri. Masalan, Python uchun PyCharm, Java uchun IntelliJ IDEA, C/C++ uchun CLion yoki VS Code + pluginlar amaliyroq bo‘ladi.

? 2. Yangi boshlovchi dasturchi uchun “og‘ir” IDE zarar qiladimi yoki foyda beradimi?

 Dastlab og‘ir IDE (IntelliJ, PyCharm) foydaliroq, chunki u xatolarni erta ko‘rsatadi, kodni to‘g‘ri yozishga o‘rgatadi. Lekin resursi kuchsiz kompyuterda sekinlashish motivatsiyani pasaytirishi mumkin. Bunday holatda VS Code kabi yengil, lekin kengaytiriladigan IDE ma’qul.

? 3. IDE’da avtomatik kod to‘ldirish (autocomplete)ga haddan ortiq tayanish qanday muammolarga olib keladi?

 Agar faqat autocomplete’ga suyanilsa, dasturchi API va mantiqni chuqur tushunmay qolishi mumkin. Amaliy yechim: autocomplete — tezlik uchun, lekin har bir chaqirilgan metod va kutubxonaning vazifasini tushunib ishlatish shart.

? 4. Debugger’dan foydalanishni bilmaslik real ishda qanday oqibatlarga olib keladi?

 Debugger bilmaslik xatoni “print” orqali qidirishga majbur qiladi, bu katta tizimlarda vaqtni juda ko‘p oladi. Debugger yordamida esa break-point, variable qiymatlari, call stack orqali xatoni tez va aniq topish mumkin. Real loyihalarda debugging bilimi majburiy ko‘nikma hisoblanadi.

☆ 5. Debugging jarayonida “breakpoint”larni noto‘g‘ri qo‘yish qanday muammolarga olib keladi?

 Keraksiz joyda breakpoint qo‘yish jarayonni sekinlashtiradi, mantiqni chalkashtiradi. Amaliy qoida: breakpoint faqat shubhali joylarda, ma’lumot

o'zgarayotgan nuqtalarda qo'yiladi. Katta loop ichida ko'p breakpoint qo'yishdan saqlanish kerak.

? 6. IDE'dagi "refactor" funksiyalaridan foydalanmaslik qanday xavf tug'diradi?

 Qo'lda refactor qilish (nom o'zgartirish, metod ko'chirish) xatoga olib kelishi mumkin. IDE refactor vositalari barcha bog'liq joylarni avtomatik va xavfsiz yangilaydi. Real loyihada refactor'ni IDE'siz qilish texnik qarz (technical debt)ni oshiradi.

? 7. Git IDE ichida ishlatilsa yaxshiroqmi yoki terminal orqali?

 IDE ichidagi Git vizual jihatdan qulay, boshlovchilar uchun tushunarli. Terminal esa kuchliroq va moslashuvchan. Amaliy tavsiya: boshlang'ichda IDE Git, keyin asta-sekin terminal Git'ni ham o'rganish — eng yaxshi yondashuv.

? 8. IDE'da bir nechta loyiha bilan ishlashda qanday tartib saqlash kerak?

 Har bir loyiha uchun alohida workspace, virtual environment (Python venv), alohida Git repository ishlatilishi kerak. Fayllarni aralashtirish, umumiy papkada bir nechta loyiha saqlash xatolarga olib keladi.

? 9. IDE'dagi lint va formatlash vositalari real loyihada nima beradi?

 Linter kod sifati va standartlarga mosligini tekshiradi, formatlash esa jamoada yagona uslubni ta'minlaydi. Natijada kod o'qilishi osonlashadi, code review tezlashadi va xatolar kamayadi.

☆ 10. IDE sekin ishlay boshlasa, avvalo nimani tekshirish kerak?

 Avvalo pluginlar soni va og'irligini tekshirish kerak. Keraksiz pluginlarni o'chirish, keshni tozalash, indekslash sozlamalarini ko'rish foyda beradi. Ko'p hollarda IDE emas, noto'g'ri sozlama muammo bo'ladi.

? 11. IDE'da testlarni avtomatik ishga tushirish nimani yengillashtiradi?

 Har bir o'zgarishdan keyin testlar avtomatik ishlasa, xatolar darhol aniqlanadi. Bu production'ga xato chiqish ehtimolini kamaytiradi. Real jamoalarda "test yozmasdan kod qabul qilinmaydi" qoidasi ko'p uchraydi.

? 12. Bir xil IDE'dan barcha dasturlash tillari uchun foydalanish qanchalik to'g'ri?

📖 Ba'zi IDE'lar universal (VS Code), ba'zilar maxsus (PyCharm, IntelliJ). Universal IDE qulay, lekin maxsus IDE ko'proq chuqur imkoniyat beradi. Amaliyotda ko'p mutaxassislar: bitta asosiy IDE + maxsus IDE kombinatsiyasidan foydalanadi.

❓ **13. IDE'da masofaviy server (remote) bilan ishlash real loyihada qanday foyda beradi?**

📖 Remote development orqali serverda ishlayotgan kodni lokal IDE'dan turib tahrirlash va debug qilish mumkin. Bu ayniqsa Linux serverlarda, Docker va cloud muhitida ishlaganda juda qulay.

☆ **14. IDE va Docker birgalikda ishlatilganda nimalarga e'tibor berish kerak?**

📖 IDE Docker konteyner ichidagi kodni ko'ra olishi, debug qila olishi muhim. Volume mapping, port forwarding va environment variables to'g'ri sozlanmasa, kod ishlamaydi. Bu ko'nikma zamonaviy backend ishlarida juda muhim.

❓ **15. IDE'da “run configuration”ni noto'g'ri sozlash qanday xatolarga olib keladi?**

📖 Noto'g'ri run config noto'g'ri environment'da kod ishga tushishiga, testlar noto'g'ri natija berishiga olib keladi. Amaliy qoida: development, test, production konfiguratsiyalari aniq ajratilgan bo'lishi kerak.

❓ **16. IDE'da klaviatura qisqa buyruqlarini bilish ish samaradorligiga qanchalik ta'sir qiladi?**

📖 Qisqa buyruqlar vaqtni sezilarli tejaydi. Masalan, kod qidirish, refactor, navigation bir necha barobar tezlashadi. Real ishda bu mayda ko'rinadi, lekin kuniga soatlar tejaladi.

❓ **17. IDE yangilanishlarini doimiy o'rnatish kerakmi yoki ehtiyot bo'lish kerakmi?**

📖 Yangilanishlar yangi imkoniyat va xavfsizlik patchlarini beradi, lekin ba'zida pluginlar mos kelmasligi mumkin. Amaliy yondashuv: asosiy ish muhitida barqaror versiya, yangi versiyani esa alohida sinab ko'rish.

☆ **18. IDE'da xotira (memory) sozlamalarini o'zgartirish qachon zarur bo'ladi?**

📖 Katta loyihalarda indekslash va tahlil ko'p RAM talab qiladi. IDE tez-tez “freeze” bo'lsa, memory limitni oshirish foyda beradi. Bu ayniqsa Java va Android loyihalarda muhim.

? 19. IDE yordamida kodni o‘qish (code reading) ko‘nikmasini qanday rivojlantirish mumkin?

 Navigation, definition’ga sakrash, call hierarchy, documentation preview kabi funksiyalar kodni tez tushunishga yordam beradi. Bu ko‘nikma real ishda yangi loyihaga tez moslashish uchun juda muhim.

? 20. IDE’da loglar bilan ishlashni bilmaslik qanday muammo tug‘diradi?

 Loglar tizim nima qilayotganini ko‘rsatadi. Loglarni filtrlash, qidirish va tahlil qilishni bilmaslik xatoni topishni qiyinlashtiradi. IDE’dagi log viewer bu jarayonni ancha osonlashtiradi.

? 21. IDE’da xavfsizlik (secret, token) bilan ishlashda qanday xatolardan saqlanish kerak?

 Parollarni kod ichida yozish katta xato. IDE environment variables, .env fayllar va secret manager’lardan foydalanish kerak. Bu real loyihalarda juda muhim xavfsizlik talabi.

☆ 22. IDE va CI/CD tizimlari o‘rtasidagi bog‘liqlik nimada?

 IDE lokal muhit, CI/CD esa avtomatik build va deploy. IDE’da test va build qanchalik to‘g‘ri sozlansa, CI/CD’da shuncha kam muammo chiqadi. Ular bir-birini to‘ldiradi.

? 23. IDE’da kod yozish tezligi bilan kod sifati o‘rtasida qanday muvozanat saqlanadi?

 Juda tez yozish sifatsiz kodga olib keladi, haddan ortiq tekshirish esa sekinlashtiradi. IDE vositalari (linter, test, refactor) yordamida tezlik va sifat balanslanadi.

? 24. Talaba yoki yangi bitiruvchi uchun IDE bo‘yicha minimal majburiy ko‘nikmalar qaysilar?

 Debugger, Git integratsiyasi, refactor, test ishga tushirish, environment sozlash. Bu ko‘nikmalarsiz real ishga moslashish qiyin bo‘ladi.

☆ 25. IDE’ni “oddiy muharrir”dan professional ish quroliga aylantirish uchun nima qilish kerak?

 Qisqa buyruqlarni o‘rganish, pluginlarni ongli tanlash, debugging va testlarni faol ishlatish, real loyiha bilan ishlash. IDE — faqat vosita, uni to‘g‘ri ishlata olgan dasturchi samarali bo‘ladi.

2.5. Algoritmlar va ma'lumotlar tuzilmalari

? 1. Algoritm tanlash dastur tezligiga real loyihada qanchalik ta'sir qiladi?

📖 Algoritm tanlash tizimning ishlash tezligiga bevosita ta'sir qiladi. Masalan, kichik ma'lumotlar uchun oddiy algoritm yetarli bo'lishi mumkin, ammo ma'lumotlar hajmi oshganda noto'g'ri algoritm tizimni sekinlashtiradi, server yukini oshiradi va foydalanuvchi tajribasini yomonlashtiradi.

? 2. Bir xil vazifani bajaruvchi ikki algoritmdan qaysi biri yaxshiroq ekanini qanday aniqlash mumkin?

📖 Ularning vaqt murakkabligi (time complexity), xotira sarfi (space complexity) va real ma'lumotlar ustida ishlash tezligi solishtiriladi. Nazariy jihatdan tez bo'lgan algoritm ham real sharoitda sekin ishlashi mumkin, shuning uchun test qilish muhim.

? 3. Katta ma'lumotlar bilan ishlaganda oddiy sikllar nega xavfli bo'lishi mumkin?

📖 Katta massiv yoki ro'yxatlarda ichma-ich sikllar ishlatilsa, dastur juda sekin ishlaydi. Bu serverda timeout, mobil qurilmada esa ilovaning "osilib qolishi"ga olib kelishi mumkin.

? 4. Qachon massiv (array), qachon ro'yxat (list) ishlatish ma'qul?

📖 Agar elementlarga indeks orqali tez murojaat kerak bo'lsa — massiv qulay. Agar elementlarni tez-tez qo'shish yoki o'chirish talab qilinsa — ro'yxat samaraliroq bo'ladi. Noto'g'ri tanlov ishlash tezligiga salbiy ta'sir qiladi.

? 5. Ma'lumotlar tuzilmasi noto'g'ri tanlansa, tizimda qanday muammolar yuzaga keladi?

📖 Xotira ortiqcha sarflanadi, ma'lumot izlash sekinlashadi, kod murakkablashadi va keyinchalik tizimni kengaytirish qiyinlashadi. Bu texnik qarzning oshishiga olib keladi.

☆ 6. Qidiruv algoritmlarining noto'g'ri tanlanishi foydalanuvchi tajribasiga qanday ta'sir qiladi?

📖 Qidiruv sekin bo'lsa, foydalanuvchi natijani kutib qoladi yoki ilovani tark etadi. Masalan, katta ro'yxatda oddiy ketma-ket qidiruv o'rniga binar qidiruv ishlatilsa, natija ancha tez olinadi.

? 7. Saralash (sorting) algoritmlarini bilish real loyihada qayerda kerak bo'ladi?

📖 Ma'lumotlarni jadvalda ko'rsatish, hisobotlar tuzish, reytinglar, narxlar yoki vaqt bo'yicha tartiblashda sorting algoritmlari bevosita ishlatiladi.

❓ 8. Har doim eng tez sorting algoritmini ishlatish to'g'rimi?

📖 Yo'q. Ma'lumotlar hajmi kichik bo'lsa, oddiy algoritmlar (masalan, insertion sort) amaliy jihatdan tezroq bo'lishi mumkin. Algoritm tanlash vaziyatga bog'liq.

❓ 9. Stack va queue tuzilmalari real dasturlarda qayerda ishlatiladi?

📖 Stack — funksiya chaqiriqlari, undo/redo, brauzer tarixida; queue — navbatlar, printer topshiriqlari, tarmoq paketlari va background vazifalarda ishlatiladi.

❓ 10. Rekursiya noto'g'ri ishlatilsa qanday xavf tug'iladi?

📖 Rekursiya chuqur bo'lsa, stack overflow xatosi yuzaga keladi. Bu server ilovalari yoki mikrokontrollerlarda tizimning ishdan chiqishiga olib kelishi mumkin.

☆ 11. Rekursiya va iteratsiya o'rtasida real loyihada qanday tanlov qilinadi?

📖 O'qilishi oson bo'lsa — rekursiya, samaradorlik va resurs nazorati muhim bo'lsa — iteratsiya tanlanadi. Ko'pincha iteratsiya xavfsizroq hisoblanadi.

❓ 12. Hash tuzilmalari (map/dictionary) ishlash tezligini qanday oshiradi?

📖 Hash tuzilmalari ma'lumotni kalit orqali deyarli doimiy vaqt ichida topishga imkon beradi. Bu login tizimlari, sozlamalar va konfiguratsiyalarda juda muhim.

❓ 13. Hash collision bo'lsa nima sodir bo'ladi va bu qanday hal qilinadi?

📖 Bir nechta kalit bitta joyga tushsa collision yuz beradi. Buni zanjirlash yoki qayta xeshlash orqali hal qilish mumkin, aks holda tezlik pasayadi.

❓ 14. Daraxt (tree) tuzilmalari real tizimlarda qayerda ishlatiladi?

📖 Fayl tizimi, menyular, XML/JSON strukturalari, tashkilot ierarxiyasi va ma'lumotlar bazasi indeksleri daraxt tuzilmasiga asoslanadi.

☆ 15. Balanslanmagan daraxtlar qanday muammo keltirib chiqaradi?

📖 Balans yo'qolsa, daraxt oddiy ro'yxatga o'xshab qoladi va qidiruv sekinlashadi. Shuning uchun AVL yoki Red-Black Tree kabi balanslangan daraxtlar ishlatiladi.

❓ 16. Graf algoritmlari real hayotda qayerda qo'llaniladi?

📖 Navigatsiya, ijtimoiy tarmoqlar, tarmoq topologiyasi, marshrutlash va tavsiya tizimlari graf algoritmlariga asoslanadi.

? 17. Eng qisqa yo‘lni topish algoritmlari qayerda muhim?

 GPS navigatsiya, logistika, tarmoq paketlarini uzatish va transport tizimlarida eng qisqa yo‘l algoritmlari ishlatiladi.

? 18. Algoritmni faqat ishlashi yetarlimi yoki boshqa omillar ham muhimmi?

 Ishlashi yetarli emas. O‘qilishi, kengaytirilishi, testlanishi va qo‘llab-quvvatlanishi ham muhim. Yomon yozilgan tez algoritm yomon kod hisoblanadi.

☆ 19. Algoritmni bilmaslik bitiruvchining ishga joylashishiga qanday ta’sir qiladi?

 Ko‘p kompaniyalar texnik suhbatda aynan DSA savollar beradi. Algoritmni bilmaslik bilim yetishmasligi sifatida baholanadi.

? 20. Algoritmik fikrlashni rivojlantirish uchun eng samarali usul qaysi?

 Real masalalarni algoritmik tarzda yechish, masalani bo‘laklash, vaqt va xotira tahlili qilish hamda ko‘p mashq bajarish eng samarali usuldir.

? 21. Kod ishlayapti, lekin sekin — bunday holatda nimadan boshlash kerak?

 Avvalo algoritm va ma’lumotlar tuzilmasini tahlil qilish kerak. Ko‘pincha muammo kodda emas, noto‘g‘ri tanlangan yondashuvda bo‘ladi.

? 22. Algoritmni optimallashtirish qachon zararli bo‘lishi mumkin?

 Juda erta optimallashtirish kodni murakkablashtiradi. Avval to‘g‘ri va tushunarli yechim, keyin zarur joylarda optimallashtirish qilinadi.

☆ 23. DSA bilimlari sun’iy intellekt va data science’da qanchalik muhim?

 Juda muhim. Ma’lumotlarni samarali saqlash, qayta ishlash va tez hisoblash algoritmlar va tuzilmalarsiz imkonsiz.

? 24. Mobil va mikrokontroller dasturlarida DSA yondashuvi nimasi bilan farq qiladi?

 Mobil va MCU’da xotira va energiya cheklangan, shuning uchun yengil va samarali algoritmlar tanlanadi. Har bir ortiqcha sikl muhim.

? 25. Algoritm va ma’lumotlar tuzilmasini bilish dasturchini qanday darajaga olib chiqadi?

 Bu bilim dasturchini oddiy kod yozuvchidan muammo yechuvchi muhandis darajasiga olib chiqadi. Aynan shu farq professional va havaskorni ajratadi.

2.6. Kutubxonalar, API va integratsiya

? 1. Tayyor kutubxonadan foydalanish dastur ishlab chiqish jarayonini qanday tezlashtiradi?

 Tayyor kutubxonalar oldindan sinovdan o'tgan funksiyalarni taqdim etadi. Bu dasturchini "noldan yozish"dan qutqarib, vaqtni tejaydi, xatolar sonini kamaytiradi va loyihani tezroq ishga tushirish imkonini beradi.

? 2. Har qanday tayyor kutubxonani ishlatish to'g'rimi yoki cheklovlar bormi?

 Yo'q, har doim ham to'g'ri emas. Kutubxonaning yangilanib turishi, xavfsizligi, litsenziyasi va loyihaga mosligi tekshirilishi kerak. Eskirgan yoki qo'llab-quvvatlanmaydigan kutubxona kelajakda jiddiy muammolarga olib keladi.

? 3. Kutubxona va framework o'rtasidagi farq real ishda nimada seziladi?

 Kutubxona — siz chaqirasiz, framework esa sizni boshqaradi. Framework loyihaning umumiy tuzilishini belgilab beradi. Noto'g'ri tanlov moslashuvchanlikni kamaytirishi yoki ortiqcha murakkablik keltirib chiqarishi mumkin.

? 4. API bilan ishlash ko'nikmasi nima uchun zamonaviy dasturchi uchun majburiy?

 Hozirgi ilovalarning aksariyati boshqa tizimlar bilan ma'lumot almashadi: to'lov, xarita, autentifikatsiya, AI servislar. API bilan ishlay olmagan dasturchi yopiq tizim doirasida qolib ketadi.

? 5. API integratsiyasida eng ko'p uchraydigan xatolar qaysilar?

 Noto'g'ri so'rov formati, autentifikatsiyani e'tiborsiz qoldirish, xatoliklarni qayta ishlamaslik va limitlarni hisobga olmaslik eng keng tarqalgan xatolardir.

☆ 6. API hujjatlarini o'qimasdan integratsiya qilish qanday oqibatlarga olib keladi?

 Bu noto'g'ri so'rovlar, xavfsizlik muammolari va serverdan bloklanishga olib keladi. API documentation — integratsiyaning asosiy yo'riqnomasi hisoblanadi.

? 7. REST API bilan ishlash jarayonida HTTP metodlarini to'g'ri tanlash nega muhim?

 GET, POST, PUT, DELETE metodlari noto'g'ri ishlatilsa, ma'lumotlar buzilishi yoki xavfsizlik muammolari yuzaga keladi. Bu backend va frontend o'rtasida kelishmovchilikka olib keladi.

? 8. API'dan kelayotgan xatoliklarni dasturda qanday to'g'ri qayta ishlash kerak?

 Har bir status kod (400, 401, 403, 500) alohida qayta ishlanishi kerak. Bu foydalanuvchiga tushunarli xabar berish va tizim barqarorligini saqlashga yordam beradi.

? 9. API integratsiyasida token va kalitlarni qayerda saqlash xavfsizroq?

 Tokenlar kod ichida emas, environment variables yoki maxsus secret managerlarda saqlanishi kerak. Aks holda ular sizib chiqib, tizimga ruxsatsiz kirish xavfi tug'iladi.

☆ 10. API orqali ishlaydigan tizim sekinlashsa, muammoni qayerdan izlash kerak?

 Avvalo tarmoq kechikishi, API javob vaqti, caching mavjudligi va so'rovlar soni tekshiriladi. Ko'pincha muammo API'da emas, noto'g'ri integratsiyada bo'ladi.

? 11. Kutubxonani yangilash (update) real loyihada qachon xavfli bo'ladi?

 Katta versiya o'zgarishlarida moslik buzilishi mumkin. Shu sababli yangilashdan oldin test muhiti va changelog diqqat bilan tekshiriladi.

? 12. Bir xil vazifa uchun bir nechta kutubxona mavjud bo'lsa, qaysi biri tanlanadi?

 Eng ko'p qo'llaniladigani, hujjatlari yaxshi yozilgani, faol jamoaga ega va xavfsizligi yuqori bo'lgan kutubxona tanlanadi.

? 13. API orqali tashqi to'lov tizimlarini ulashda qaysi jihatlar muhim?

 Xavfsizlik, tranzaksiya holatini tekshirish, xatoliklarni qayta ishlash va qayta so'rov (retry) mexanizmlari juda muhim.

☆ 14. API versiyalash (versioning) nima sababdan zarur bo'ladi?

 API yangilanganda eski mijozlar ishlamay qolmasligi uchun versiyalar ajratiladi. Bu yirik tizimlarda uzluksiz ishlashni ta'minlaydi.

? 15. Kutubxona ishlamay qolsa, loyiha to'xtab qolmasligi uchun nima qilish kerak?

 Muqobil kutubxona, abstraksiya qatlamlari va fallback mexanizmlar ishlab chiqilishi kerak. Bu tizim barqarorligini oshiradi.

? 16. API orqali ma'lumot almashishda format tanlovi (JSON, XML) nimaga ta'sir qiladi?

 JSON yengil va tezkor, XML esa murakkab va kengaytirilgan. Noto'g'ri tanlov tarmoq yukini oshirishi mumkin.

? 17. Kutubxonalar bilan ishlashda litsenziya masalasi nega muhim?

 Ba'zi litsenziyalar tijorat loyihalarda cheklov qo'yadi. Bu huquqiy muammolarga olib kelishi mumkin.

☆ 18. API'ni haddan ortiq chaqirish qanday muammoga olib keladi?

 Rate limit buziladi, servis bloklanadi yoki qo'shimcha xarajatlar yuzaga keladi. Shu sababli caching va so'rovlarni optimallashtirish muhim.

? 19. Frontend va backend o'rtasidagi API kelishuvlari qanday muammolarni kamaytiradi?

 Oldindan kelishilgan formatlar noto'g'ri ma'lumot uzatilishini kamaytiradi va ishlab chiqish jarayonini tezlashtiradi.

? 20. API test qilish real loyihada nega majburiy hisoblanadi?

 Test qilinmagan API xatoliklarni production'da chiqaradi. Bu foydalanuvchi ishonchini pasaytiradi.

? 21. API monitoring nima uchun kerak bo'ladi?

 Monitoring API ishlash holatini, sekinlashuv va xatoliklarni erta aniqlashga yordam beradi.

☆ 22. Bir nechta API bilan ishlaydigan tizimlarda muvofiqlik qanday ta'minlanadi?

 Oraliq servislar, adapterlar va standartlashtirilgan formatlar yordamida muvofiqlik ta'minlanadi.

? 23. API bilan ishlashda hujjatlashtirish qanchalik muhim?

 Hujjat yo'q bo'lsa, tizimni kengaytirish va qo'llab-quvvatlash qiyinlashadi. Documentation — jamoaviy ishning asosi.

? 24. Kutubxonalarni haddan ortiq ishlatish qanday salbiy ta'sir ko'rsatadi?

 Tizim og'irlashadi, bog'liqliklar ko'payadi va yangilash qiyinlashadi. Minimalizm muhim.

☆ **25. API va kutubxonalar bilan ishlashni bilgan dasturchi nimasi bilan ajralib turadi?**

📖 U noldan ham yozadi, tayyor vositani ham to'g'ri tanlaydi. Bu esa uni tezkor, moslashuvchan va professional mutaxassisga aylantiradi.

2.7. Dasturiy xavfsizlik va xatoliklarni boshqarish

? **1. Dastur ishlab chiqishda xavfsizlikni oxirida emas, boshida rejalash nega muhim?**

📖 Xavfsizlikni oxirida qo'shish qimmat va murakkab bo'ladi. Boshidan to'g'ri arxitektura qurilsa, ko'plab zaifliklar avtomatik yo'qoladi va tizim barqarorroq ishlaydi.

? **2. Oddiy dasturlarda ham xavfsizlik choralarini ko'rmaslik qanday xavf tug'diradi?**

📖 Hatto kichik dastur ham ma'lumot sizib chiqishiga, foydalanuvchi ishonchini yo'qotishga va huquqiy muammolarga olib kelishi mumkin.

? **3. Foydalanuvchi kiritgan ma'lumotni tekshirmaslik nimaga olib keladi?**

📖 Bu SQL injection, XSS kabi hujumlarga eshik ochadi. Har qanday input doimo tekshirilishi va tozalanishi shart.

? **4. Parollarni oddiy matn ko'rinishida saqlash qanday oqibatlarga olib keladi?**

📖 Baza buzilsa, barcha foydalanuvchi parollari ochilib ketadi. Hash va salt ishlatilmasa, bu jiddiy xavfsizlik muammosidir.

? **5. Xatolik xabarlarini foydalanuvchiga to'liq ko'rsatish nega xavfli?**

📖 Texnik tafsilotlar hujumchiga tizim tuzilishini ochib beradi. Foydalanuvchiga umumiy, loglarda esa batafsil xabar saqlanishi kerak.

☆ **6. Exception'larni "yutib yuborish" (try-catch ichida hech nima qilmaslik) qanday muammo tug'diradi?**

📖 Bu yashirin xatolarga olib keladi. Dastur noto'g'ri ishlaydi, lekin sababini topish juda qiyin bo'ladi.

? **7. Loglash tizimi bo'lmasa, real loyihada qanday qiyinchiliklar yuzaga keladi?**

📖 Xatolik qayerda chiqqanini aniqlab bo‘lmaydi. Bu support vaqtini oshiradi va tizim ishonchliligini pasaytiradi.

❓ 8. Loglarda maxfiy ma’lumotlarni saqlash qanday xavf tug‘diradi?

📖 Loglar ham sizib chiqishi mumkin. Agar parol yoki tokenlar bo‘lsa, bu to‘liq tizim buzilishiga olib keladi.

❓ 9. Autentifikatsiya va avtorizatsiyani aralashtirib yuborish nimaga olib keladi?

📖 Foydalanuvchi tizimga kirgan bo‘lsa ham, ruxsatsiz amallarni bajarib qo‘yishi mumkin. Bu rol va huquqlarni noto‘g‘ri boshqarishga olib keladi.

☆ 10. “Default sozlamalar bilan ishlaydi” degan yondashuv nega xavfli?

📖 Default sozlamalar ko‘pincha ochiq va zaif bo‘ladi. Server, framework va kutubxonalar xavfsiz sozlanishi shart.

❓ 11. Foydalanuvchi sessiyasini noto‘g‘ri boshqarish qanday oqibatlarga olib keladi?

📖 Sessiya o‘g‘irlanishi, boshqa foydalanuvchi nomidan tizimga kirish va ma’lumotlar buzilishi mumkin.

❓ 12. Xatoliklarni avtomatik qayta urish (retry) qachon foydali, qachon zararli?

📖 Tarmoq xatolarida foydali, lekin mantiqiy xatolarda zararli. Noto‘g‘ri retry serverga ortiqcha yuk tushiradi.

❓ 13. Dasturda ruxsatlarni minimal darajada berish nega muhim?

📖 Keraksiz ruxsatlar tizimni zaiflashtiradi. Minimal huquqlar prinsipi zarar ko‘lamini cheklaydi.

☆ 14. SQL injection hujumidan himoyalaniş uchun qaysi yondashuvlar muhim?

📖 Parametrli so‘rovlar, ORM ishlatish va input tekshirish asosiy himoya choralaridir.

❓ 15. XSS hujumlaridan himoyalaniş real frontend loyihalarda qanday amalga oshiriladi?

📖 Foydalanuvchi kiritgan matnni ekranga chiqarishda escape qilish, ishonchsiz skriptlarni bloklash orqali amalga oshiriladi.

? 16. Fayl yuklash funksiyasi qachon xavfli bo‘ladi?

 Agar fayl turi, hajmi va nomi tekshirilmasa, serverga zararli fayl yuklanishi mumkin.

? 17. Xatoliklarni markazlashtirilgan monitoring qilish nima beradi?

 Muammo foydalanuvchi sezishidan oldin aniqlanadi. Bu tizim ishonchliligini oshiradi.

☆ 18. Production va development muhitlarini aralashtirib yuborish qanday muammo keltiradi?

 Test ma’lumotlari haqiqiy foydalanuvchi ma’lumotlarini buzishi yoki sizib chiqishiga olib keladi.

? 19. Dasturda xavfsizlikni test qilmaslik nimaga olib keladi?

 Zaifliklar faqat real hujum paytida aniqlanadi. Bu esa obro‘ va moliyaviy zarar keltiradi.

? 20. Foydalanuvchi harakatlarini cheklash (rate limiting) nima uchun kerak?

 Brute force hujumlar va serverni ishdan chiqarishning oldini oladi.

? 21. Xatoliklar statistikasi nima uchun tahlil qilinadi?

 Qaysi joylar ko‘p muammo keltirayotganini aniqlash va ularni yaxshilash uchun.

☆ 22. Dasturiy xavfsizlik faqat backend masalasimi?

 Yo‘q. Frontend, mobil ilova va hatto UI dizayn ham xavfsizlikka ta’sir qiladi.

? 23. Xavfsizlik bo‘yicha bilimlar ishga joylashishda qanday baholanadi?

 Juda yuqori. Xavfsizlikni tushunadigan dasturchi ishonchli mutaxassis hisoblanadi.

? 24. Xatoliklarni to‘g‘ri boshqarish foydalanuvchi tajribasiga qanday ta’sir qiladi?

 Foydalanuvchi tushunarli xabar ko‘rsa, tizimga ishonchi oshadi.

☆ 25. Dasturiy xavfsizlik va xatoliklarni boshqarishni bilgan mutaxassis nimasi bilan ajralib turadi?

 U nafaqat ishlaydigan, balki barqaror, xavfsiz va professional tizim yaratadi.

2.8. Testlash, debugging va sifat nazorati

? 1. Ishlayotgan kodni ham testlash nega shart, “menda ishlayapti” yetarli emasmi?

📖 “Menda ishlayapti” — faqat sizning kompyuteringizdagi bir holatni bildiradi. Real loyihada esa: boshqa OS, boshqa versiyalar, boshqa ma’lumotlar, boshqa foydalanuvchi xatti-harakati mavjud bo’ladi. Testlar kodning turli sharoitlarda ham to’g’ri ishlashini isbotlaydi va eng muhimi: keyingi o’zgarishlar (refactor, yangi funksiya) eski funksiyalarni buzib yubormaganini avtomatik tekshiradi. Bu ishlab chiqish tezligini oshiradi va production’da xatolarni kamaytiradi.

? 2. Test yozishga vaqt ketadi, lekin real loyihada bu vaqt qanday “qaytadi”?

📖 Test yozish dastlab vaqt oladi, ammo keyin bir necha yo’l bilan vaqtni “qaytaradi”:

- xatoni erta topadi (arzon bosqichda), production’da topish esa qimmat;
- yangi developer kelganda kodni tushunishi osonlashadi (testlar — “tirik hujjat”);
- qo’lda tekshirish kamayadi;
- regressiya (eski funksiya buzilishi) keskin kamayadi.
- Natijada jamoa tezroq release qiladi va kamroq “o’t o’chirish”ga majbur bo’ladi.

? 3. Real loyihada testlashni qaysi joydan boshlash eng to’g’ri?

📖 Eng to’g’ri boshlanish — eng kritik biznes funksiyalardan: login/registratsiya, to’lov, buyurtma, hisob-kitob, ruxsatlar (role/permission), ma’lumot saqlash. Chunki bu joylarda xato chiqsa zarar katta bo’ladi. Keyin sekin-asta qolgan modullar qamrab olinadi. “Hammasiga birdan test yozaman” deyish ko’pincha tugallanmagan ishga aylanadi.

? 4. Unit test, integration test va end-to-end testni amaliy nuqtai nazardan qanday ajratish kerak?

📖 Ajratish mezonlari: “nima tekshirilayapti va qanchalik katta muhit kerak?”

- **Unit test:** bitta funksiya/klass mantiqini tekshiradi, tez ishlaydi, tashqi bog’liqliklar (DB, API) mock qilinadi.
- **Integration test:** modul DB, fayl tizimi yoki real servis bilan birga ishlaganda hammasi mos ekanini tekshiradi.
- **E2E:** foydalanuvchi ssenariysi (kirish → amal → natija) to’liq tekshiriladi, eng qimmat va sekin, lekin eng real.

❓ 5. Testlar “ko‘p” bo‘lsa ham nega baribir xato production’da chiqib qolishi mumkin?

📖 Sabablari: testlar noto‘g‘ri ssenariylarni qamrab olmagan, real ma’lumotlar boshqacha, tashqi servislar boshqacha javob bergan, concurrency (bir vaqtda ko‘p foydalanuvchi) muammosi bo‘lgan, konfiguratsiya farqi bor. Shuning uchun testlar bilan birga logging, monitoring, code review va staging muhiti ham zarur.

☆ 6. “Flaky test”lar (ba’zan o‘tib, ba’zan yiqiladigan) nega juda zararli?

📖 Flaky testlar jamoa ishonchini buzadi: developer test natijasiga ishonmay qo‘yadi va testlarni e’tiborsiz qoldiradi. Sabablari: vaqtga bog‘liq kod (sleep), tarmoq, umumiy resursdan foydalanish, tartibga bog‘liqlik, parallel testlar. Yechim: testni deterministik qilish (tasodifiylikni yo‘qotish), mock, izolyatsiya, aniq fixture, vaqtning “fake clock” bilan boshqarish.

❓ 7. Debugging’da “reproduce qila olish” nima uchun eng birinchi qadam?

📖 Reproduce qilinmagan xatoni tuzatish — taxmin bilan ishlashdir. Reproduce qilish: xatoni aniq sharoitda takrorlash, qadamlarni yozib olish, kerak bo‘lsa minimal misol yaratish. Shunda xato sababini topish osonlashadi va “tuzatdim” degan yechim haqiqatan ishlashini tekshirish mumkin bo‘ladi.

❓ 8. Debug qilishda loglar qachon yetarli bo‘lmaydi va debugger qachon kerak bo‘ladi?

📖 Loglar — “nima bo‘ldi?” ni ko‘rsatadi, debugger esa “nega bo‘ldi?” ni chuqurroq ochadi.

- Log yetarli: ketma-ket jarayon, server xatolari, prod monitoring.
- Debugger kerak: murakkab shartlar, noto‘g‘ri qiymatlar, step-by-step tekshirish, call stack.
- Ko‘p real holatda ikkalasi birga ishlatiladi: log xatoni ko‘rsatadi, debugger sababini topadi.

❓ 9. “Print debugging” bilan professional debugging o‘rtasidagi asosiy farq nima?

📖 Print debugging ko‘pincha tasodifiy va tartibsiz bo‘ladi, kodga vaqtinchalik “iflos” chiqishlar qo‘shadi. Professional debugging esa: breakpoints, watch variables, stack trace, profiling, log level’lar, replay, test bilan reproduksiya kabi usullarga tayangan holda tezroq va tizimli ishlaydi.

☆ 10. Xatoni tuzatgandan keyin “albatta” nima qilish kerak?

📖 Uchta majburiy ish:

1. **Root cause:** haqiqiy sababni aniqlash (faqat simptomni emas).

2. **Regression test:** aynan shu xato qaytmasligi uchun test yozish yoki mavjud testni kengaytirish.

3. **Refactor/cleanup:** vaqtinchalik workaround'larni olib tashlash, loglarni tartibga keltirish.

4. Shunda xato qaytalanish ehtimoli kamayadi.

❓ **11. Profiling qachon kerak bo'ladi va u debuggingdan nimasi bilan farq qiladi?**

📖 Debugging — xato va noto'g'ri mantiqni topish. Profiling — tezlik va resurs muammosini (CPU, RAM, I/O) aniqlash. Masalan, kod "to'g'ri", lekin sekin ishlayapti — profiling kerak. Profiling natijasida qaysi funksiya ko'p vaqt olayotgani, qayerda ortiqcha loop borligi aniqlanadi.

❓ **12. Performance test va load testni real tizimda qachon o'tkazish kerak?**

📖 Release oldidan va katta o'zgarishlardan keyin. Ayniqsa: yangi DB so'rovlari, caching o'zgarishi, autentifikatsiya, fayl yuklash, video stream kabi og'ir funksiyalar qo'shilganda. Maqsad: tizim ma'lum foydalanuvchi sonida ham barqarormi, response time normadami, server yiqilib qolmayaptimi — shuni bilish.

★ **13. Ma'lumotlar bazasi so'rovlari sekinlashsa, test va diagnostika qanday ketadi?**

📖 Amaliy tartib:

- sekin endpoint'ni topish (monitoring/log);
- DB query log'ni ko'rish;
- EXPLAIN plan tekshirish;
- indekslar yetarlimi tekshirish;
- N+1 muammosi bormi;
- caching kerakmi;
- so'rovni optimallashtirish;
- so'ngra regression/performance test bilan natijani tasdiqlash.
- Bu ishlar "tasodifiy" emas, tizimli ketma-ketlikda bajariladi.

❓ **14. Code review sifatni qanday oshiradi, testlar bor bo'lsa ham?**

📖 Testlar faqat kutilgan xatti-harakatni tekshiradi, lekin code review: arxitektura, xavfsizlik, o‘qilishi, naming, edge case, standartlarga moslikni tekshiradi. Jamoaviy tajriba kodga singadi. Ko‘pincha security xatolar (masalan, secret’ni logga yozish) test bilan ushlanmaydi, review bilan ushlanadi.

❓ **15. Static analysis (linter) nima uchun “sifat nazorati”ning asosiy qismi hisoblanadi?**

📖 Linter va static analysis kod ishlamasdan turib ham muammolarni ko‘rsatadi: noto‘g‘ri import, unreachable code, xavfli konstruktsiyalar, naming, format. Bu xatolarni erta bosqichda tozalab, code review’ni ham yengillashtiradi.

☆ **16. Test qamrovi (coverage) 90% bo‘lsa ham tizim ishonchsiz bo‘lishi mumkinmi?**

📖 Ha. Coverage raqam, lekin sifat — ssenariy. Agar testlar faqat “happy path”ni tekshirsa, edge case’lar (null, empty, katta data, noto‘g‘ri format, timeout) qamrab olinmasa, coverage yuqori bo‘lsa ham xatolar chiqadi. Shuning uchun coverage’ni emas, riskli ssenariylarni qamrab olishni maqsad qilish kerak.

❓ **17. Edge case’larni tizimli aniqlash uchun qanday yondashuv ishlatiladi?**

📖 Checklist yondashuvi:

- bo‘sh qiymatlar, minimal/maksimal chegaralar;
- noto‘g‘ri format (string o‘rniga number);
- vaqt/zonalar;
- tarmoq uzilishi;
- parallel so‘rovlar;
- ruxsatlar (permission).
- Shu asosda test ssenariylari tuziladi.

❓ **18. CI’da testlar o‘tadi, lekin production’da xato chiqadi — eng ko‘p uchraydigan sabablar qaysi?**

📖 Ko‘p uchraydiganlar: konfiguratsiya farqi (env), secret’lar, DB migratsiya muammosi, real trafik hajmi, tashqi servislar farqi, cache/proxy ta’siri, race condition. Shuning uchun staging muhiti production’ga maksimal o‘xshash bo‘lishi kerak.

☆ **19. “Race condition”ni test bilan ushlash qiyin, ammo amaliy yechim qanday?**

📖 Parallel testlar, stress test, lock/transaction testlari va log/trace yordam beradi. Shuningdek, kritik bo‘limlarda atomik amallar, mutex/lock, DB transaction va idempotent endpoint dizayni qo‘llaniladi. Bu muammo ko‘pincha “kam uchraydi”, lekin zarar katta bo‘ladi.

❓ 20. Idempotent so‘rovlar xatoliklarni boshqarishda qanday foyda beradi?

📖 Idempotent so‘rov qayta yuborilsa ham natija buzilmaydi. Masalan, to‘lov yoki buyurtma yaratishda retry bo‘lsa, ikki marta buyurtma ketib qolmasligi uchun idempotency key ishlatiladi. Bu real tizimlarda juda muhim.

❓ 21. Release oldidan “minimal sifat darajasi” (quality gate)ni qanday belgilash mumkin?

📖 Masalan: unit testlar o‘tishi shart, lint xatolari 0 bo‘lishi, kritik modul coverage minimal bo‘lishi, security scan’da critical topilmasligi, build muvaffaqiyatli bo‘lishi. Bu “subyektiv” emas, aniq mezon bilan boshqariladi.

☆ 22. Monitoring va alerting testlashni qanday to‘ldiradi?

📖 Testlar oldindan kutilgan holatni tekshiradi. Monitoring esa real vaqtda: sekinlashuv, error rate oshishi, DB connection muammosi, API timeout kabi holatlarni ushlaydi. Alerting bo‘lmasa, muammo foydalanuvchi shikoyatidan keyin bilinadi.

❓ 23. “Bug report” sifatli bo‘lishi uchun unda nimalar bo‘lishi kerak?

📖 Reproduce qadamlar, kutilgan natija vs real natija, muhit (OS, versiya), log yoki screenshot, vaqt (timestamp), foydalanuvchi roli. Shunda developer tezda xatoni topadi.

❓ 24. Test yozishda eng ko‘p qilinadigan xato va to‘g‘ri yondashuv nima?

📖 Eng ko‘p xato: test ichida juda ko‘p narsani tekshirish (bitta test — hammasi). To‘g‘ri yondashuv: testni kichik, aniq, nomi tushunarli qilish; bitta test bitta xulq-atvorni tekshirsin; fixture’lar tartibli bo‘lsin.

☆ 25. Testlash + debugging + sifat nazorati ko‘nikmalari bitiruvchini ishda qanday ajratib ko‘rsatadi?

📖 Bunday bitiruvchi faqat “kod yozadigan” emas, balki tizimni ishonchli ishlaydigan holatda ushlaydigan mutaxassis bo‘ladi. U xatoni tez topadi, qaytalanmasligini ta’minlaydi, jamoa ishini tezlashtiradi va production’dagi risklarni kamaytiradi. Bu ko‘nikmalar ko‘p kompaniyalarda juda yuqori baholanadi.

2.9. Versiya boshqaruvi va jamoaviy ish

❓ **1. Nega real loyihalarda kodni oddiygina fleshka yoki zip fayl bilan almashish qabul qilinmaydi?**

📖 Chunki bunday usulda kim nima o‘zgartirgani, qachon va nega o‘zgartirgani noma’lum bo‘lib qoladi. Xatolik chiqqanda orqaga qaytish imkoni bo‘lmaydi. Git esa barcha o‘zgarishlar tarixini saqlaydi va jamoada tartibni ta’minlaydi.

❓ **2. Git bilan ishlashni bilmaslik bitiruvchining ishga kirishida qanday muammo tug‘diradi?**

📖 Ko‘p kompaniyalarda Git — minimal talab. Git bilmagan bitiruvchi jamoa ishini sekinlashtiradi, mustaqil ishlay olmaydi va ko‘pincha sinov muddatidan o‘tolmaydi.

❓ **3. Nega real loyihalarda hech kim “to‘g‘ridan-to‘g‘ri main branch”ga kod yozmaydi?**

📖 Main branch — barqaror va ishlaydigan kod joyi. To‘g‘ridan-to‘g‘ri yozish xatolarni production’ga olib chiqadi. Branch’lar xavfsiz tajriba va mustaqil ishlash imkonini beradi.

❓ **4. Branch bilan ishlash jamoada qanday muammolarni kamaytiradi?**

📖 Har bir dasturchi o‘z vazifasini alohida branch’da bajaradi. Natijada kodlar bir-biriga aralashmaydi, testlash osonlashadi va xatolarni ajratish mumkin bo‘ladi.

❓ **5. Branch nomlash qoidalari nega jamoada muhim?**

📖 Tartibli nomlash (feature/, bugfix/, hotfix/) branch nima uchun yaratilganini darhol tushunishga yordam beradi. Bu code review va loyiha boshqaruvini yengillashtiradi.

☆ **6. Merge vaqtida conflict chiqishi nimadan dalolat beradi?**

📖 Conflict — ikki yoki undan ortiq dasturchi bir xil kod qatorini o‘zgartirganini bildiradi. Bu tabiiy holat, lekin jamoada kelishuv va to‘g‘ri ish oqimi bo‘lmasa, conflictlar ko‘payib ketadi.

❓ **7. Conflict’ni ko‘r-ko‘rona “accept all” qilish qanday xavf tug‘diradi?**

📖 Bu boshqa dasturchining muhim kodini o‘chirib yuborishi yoki yashirin xatolik kiritishi mumkin. Conflict har doim mantiqan tushunib hal qilinishi kerak.

❓ **8. Merge’dan oldin lokal testlarni ishga tushirish nega majburiy?**

📖 Chunki merge qilingan kod umumiy branch'ga tushadi. Agar test qilinmasa, butun jamoa ishiga xalaqit beruvchi xato kiritilishi mumkin.

❓ **9. Commit xabarlarini “fix”, “update” kabi yozish nima uchun yomon amaliyot?**

📖 Bunday xabarlar tarixni tushunarsiz qiladi. Yaxshi commit xabari *nima o'zgardi va nega* degan savolga javob berishi kerak. Bu keyinchalik xatoni topishda juda muhim.

★ **10. Kichik-kichik commit qilish katta commit'dan nimasi bilan afzal?**

📖 Kichik commitlar:

- xatoni tez topish
- kerakli joygacha “rollback” qilish
- code review'ni yengillashtirish
- imkonini beradi. Katta commit esa hammasini chalkashtiradi.

❓ **11. Code review faqat xato topish uchungina qilinadimi?**

📖 Yo'q. Code review — bilim almashish, kod sifatini oshirish, yagona uslubni saqlash va yosh dasturchilarni o'rgatish vositasi ham hisoblanadi.

❓ **12. Code review'ni “shaxsiy tanqid” sifatida qabul qilish qanday muammo tug'diradi?**

📖 Bu jamoada nizolar keltirib chiqaradi. Review kodga qaratilgan bo'lishi kerak, odamga emas. Professional jamoalarda bu madaniyat juda muhim.

★ **13. Review qilinmagan kodni merge qilish real loyihada nimaga olib keladi?**

📖 Xavfsizlik muammolari, yashirin bug'lar, yomon arxitektura qarorlari va texnik qarzning keskin oshishiga olib keladi.

❓ **14. Jamoada kod uslubi (coding style) bir xil bo'lmasa nima sodir bo'ladi?**

📖 Kodni o'qish qiyinlashadi, review va refactor vaqtini oshiradi. Shu sababli formatter va linter'lar jamoaviy ishda majburiy bo'ladi.

❓ **15. Git history'ni “toza” saqlash nimani anglatadi?**

📖 Keraksiz commitlar, tasodifiy o'zgarishlar va chalkash tarixdan qochish. Bu loyiha rivojini tushunishni osonlashtiradi.

☆ **16. Rebase va merge o‘rtasida real jamoalarda qanday tanlov qilinadi?**

📖 Merge — tarixni saqlaydi, rebase — tarixni tozalaydi. Ko‘p jamoalarda feature branch‘da rebase, main branch‘da esa merge ishlatiladi.

? **17. Jamoada “bitta odam hamma narsani biladi” holati nega xavfli?**

📖 Agar u ketib qolsa yoki mavjud bo‘lmasa, loyiha to‘xtab qoladi. Git, review va hujjatlashtirish bu xavfni kamaytiradi.

? **18. Issue tracker (GitHub Issues, Jira) bilan Git qanday bog‘liq ishlatiladi?**

📖 Har bir vazifa issue sifatida yaratiladi, branch va commitlar shu issue bilan bog‘lanadi. Bu ish jarayonini shaffof qiladi.

☆ **19. Jamoada “meniki ishlayapti” degan yondashuv nega qabul qilinmaydi?**

📖 Chunki jamoa uchun muhim narsa — umumiy tizimning ishlashi. Lokal muvaffaqiyat global muammo keltirmasligi kerak.

? **20. Konfliktlar soni ko‘payib ketayotgan bo‘lsa, bu nimadan darak beradi?**

📖 Branch‘lar juda uzoq yashayapti, vazifalar noto‘g‘ri bo‘lingan yoki jamoa ichida kommunikatsiya sust.

? **21. Jamoaviy ishda texnik bilimdan tashqari yana nimalar muhim?**

📖 Muloqot, mas‘uliyat, vaqtda xabar berish, hujjatlashtirish va kelishuvlarga rioya qilish.

☆ **22. Yangi bitiruvchi jamoaga tez moslashishi uchun nimaga e‘tibor berishi kerak?**

📖 Git workflow‘ni tushunish, review‘ga ochiq bo‘lish, savol berishdan uyalmay, lekin mustaqil o‘rganishga harakat qilish.

? **23. Code review‘da izoh qoldirishda qanday ohang professional hisoblanadi?**

📖 Hurmat bilan, aniq va asoslangan. “Bu yomon” emas, balki “bu yechimda mana bu xavf bor” tarzida.

? **24. Jamoada kelishilgan qoidalarga amal qilmaslik qanday oqibatlarga olib keladi?**

📖 Ish sifati tushadi, nizolar ko‘payadi, loyiha sekinlashadi. Ko‘pincha bunday xodimlar jamoada uzoq qolmaydi.

☆ 25. Ushbu qismni o‘zlashtirgan bitiruvchi nimasi bilan ajralib turadi?

📖 U faqat kod yozmaydi, balki jamoa bilan ishlay oladi, mas’uliyatni tushunadi, kodni muhokama qiladi va real ishlab chiqarish jarayoniga tayyor bo‘ladi.

2.10. Dasturiy ta’minotni joriy etish va qo‘llab-quvvatlash

? 1. Nega dastur ishga tushirilishi (deployment) loyihaning eng xavfli bosqichlaridan biri hisoblanadi?

📖 Chunki bu bosqichda kod real foydalanuvchilar, real ma’lumotlar va real yuklama bilan to‘qnashadi. Development’da sezilmagan xatolar aynan deployment vaqtida katta muammoga aylanadi. Shu sababli bu jarayon qat’iy qoidalar asosida amalga oshiriladi.

? 2. “Menda lokalda ishlayapti” degan holat production’da nega ishlamasligi mumkin?

📖 Chunki lokal muhit va production muhiti o‘rtasida OS, kutubxona versiyalari, konfiguratsiya, ruxsatlar, tarmoq va resurslar farq qiladi. Shuning uchun muhitlar maksimal darajada bir-biriga yaqinlashtirilishi kerak.

? 3. Deployment jarayonini qo‘lda bajarish qanday xavf tug‘diradi?

📖 Inson xatosi ehtimoli juda yuqori bo‘ladi: noto‘g‘ri fayl, noto‘g‘ri konfiguratsiya, eski versiyani o‘chirmaslik. Shu sababli real loyihalarda deployment maksimal darajada avtomatlashtiriladi.

? 4. Deployment oldidan bajariladigan minimal majburiy tekshiruvlar qaysilar?

📖 Testlar o‘tganmi, konfiguratsiya to‘g‘rimi, migratsiyalar tayyormi, zaxira (backup) mavjudmi, rollback rejasi bormi — bularsiz deployment qilish katta xato hisoblanadi.

? 5. Rollback rejasiz deployment nimasi bilan xavfli?

📖 Agar yangi versiya muammo keltirib chiqarsa, tezda oldingi barqaror holatga qaytish imkoni bo‘lmaydi. Bu esa tizimning uzoq vaqt ishlamasligiga olib keladi.

☆ 6. Yangilanish (update) va patchlarni nazoratsiz chiqarish qanday muammo tug‘diradi?

📖 Nazoratsiz yangilanishlar eski funksiyalarni buzishi, ma’lumotlar mos kelmasligi yoki xavfsizlik muammosi keltirib chiqarishi mumkin. Har bir update testdan o‘tishi shart.

? 7. Kichik patch'lar nega katta release'lardan xavfsizroq hisoblanadi?

 Kichik o'zgarishlar aniq muammoni hal qiladi va ta'sir doirasi kichik bo'ladi. Katta release'da esa xatoni qayerdan izlash qiyinlashadi.

? 8. Production muhitida test ma'lumotlaridan foydalanish nimaga olib keladi?

 Bu foydalanuvchi ishonchini yo'qotadi, noto'g'ri qarorlar qabul qilinishiga va hatto huquqiy muammolarga olib kelishi mumkin.

? 9. Monitoring tizimi bo'lmasa, muammo qachon aniqlanadi?

 Odatda foydalanuvchi shikoyat qilgandan keyin. Bu esa kechikkan reaksiya bo'lib, zarar ko'lamini oshiradi.

☆ 10. Monitoring real loyihada aynan nimalarni kuzatishi kerak?

 Server yuklamasi, xotira, disk, xatoliklar soni, javob vaqti, muhim endpoint'lar holati, foydalanuvchi faoliyati. Bu ma'lumotlar tizim sog'lomligini ko'rsatadi.

? 11. Loglar faqat xatolar uchunmi yoki boshqa vazifasi ham bormi?

 Loglar xatolikdan tashqari: foydalanuvchi harakati, tizim oqimi, xavfsizlik hodisalari va diagnostika uchun ishlatiladi. To'g'ri loglash — muammo topishning eng kuchli quroli.

? 12. Loglar haddan ortiq ko'p bo'lsa qanday muammo yuzaga keladi?

 Disk tez to'lib ketadi, muhim loglar orasida keraksizlari yo'qolib ketadi. Shu sababli log darajalari (info, warning, error) to'g'ri sozlanadi.

☆ 13. Loglarda maxfiy ma'lumot saqlanishi nega juda xavfli?

 Loglar ko'pincha ko'p odamlar tomonidan ko'riladi. Agar parol, token yoki shaxsiy ma'lumot bo'lsa, bu jiddiy xavfsizlik buzilishi hisoblanadi.

? 14. Texnik xizmat ko'rsatish doimiy jarayon ekanini talaba qanday tushunishi kerak?

 Dastur ishga tushgandan keyin foydalanuvchilar ko'payadi, muammolar chiqadi, talablar o'zgaradi. Shuning uchun tizim doimiy kuzatuv va qo'llab-quvvatlashni talab qiladi.

? 15. Texnik xizmat faqat xatoni tuzatishdan iboratmi?

 Yo'q. Unga optimallashtirish, xavfsizlikni yangilash, monitoringni yaxshilash, foydalanuvchi tajribasini oshirish ham kiradi.

☆ **16. Production’da “tez tuzatib qo‘ya qolish” nega xavfli yondashuv?**

📖 Shoshma-shosharlik bilan kiritilgan o‘zgarish yangi xatolar keltirib chiqarishi mumkin. Har qanday tuzatish ham test va nazorat bilan amalga oshirilishi kerak.

? **17. Foydalanuvchi shikoyatlari texnik rivojlanishda qanday rol o‘ynaydi?**

📖 Ular real foydalanishdagi muammolarni ko‘rsatadi. To‘g‘ri tahlil qilingan shikoyatlar tizimni yaxshilash uchun juda qimmatli manba hisoblanadi.

? **18. Dastur ishlamay qolganda birinchi qadam nima bo‘lishi kerak?**

📖 Avvalo muammoning ko‘lami aniqlanadi: kimlarga ta’sir qilyapti, qachondan beri, qaysi modul. Keyin vaqtinchalik barqarorlik ta’minlanadi, so‘ng sabab tahlil qilinadi.

☆ **19. Incident’lardan keyin “postmortem” qilish nimani beradi?**

📖 Xatodan saboq olinadi, kelajakda takrorlanmasligi uchun choralar belgilanadi. Aybdor izlash emas, tizimni yaxshilash maqsad qilinadi.

? **20. Zaxira nusxalarsiz (backup) ishlash qachon seziladi?**

📖 Odatda kech bo‘lganda. Ma’lumot yo‘qolganda backup yo‘qligi eng katta xato ekanini ko‘rsatadi.

? **21. Dastur barqaror ishlayotganda ham monitoringni davom ettirish nega kerak?**

📖 Chunki muammolar asta-sekin yig‘ilib boradi. Monitoring erta ogohlantirish beradi.

☆ **22. Dasturiy ta’minotni qo‘llab-quvvatlash jamoasi bilan ishlab chiquvchilar o‘rtasidagi aloqa nega muhim?**

📖 Qo‘llab-quvvatlash real muammolarni ko‘radi, developer esa yechimni biladi. Ularning aloqasi tizim sifatini oshiradi.

? **23. Talaba deployment va monitoringni bilsa, ishda nimasi bilan ajralib turadi?**

📖 U faqat kod yozuvchi emas, balki tizimning real ishlashiga mas’ul mutaxassis sifatida ko‘riladi.

? **24. Dastur hayot siklini tushunish bitiruvchiga qanday ustunlik beradi?**

📖 U har bir qarorning production’dagi oqibatini oldindan o‘ylaydi. Bu esa uni yetuk muhandisga aylantiradi.

3. Python dasturlash tili

Atamalar va izohlar

Atamalar	Izohlar
PVM	Python Virtual Machine (Python Virtual Mashinasi) — baytkodni mashina kodiga o'girib beruvchi qatlam.
GIL	Global Interpreter Lock (Global Interpretator Qulfi) — bir vaqtning o'zida faqat bitta oqim ishlashini ta'minlovchi mexanizm.
GC	Garbage Collector (Chiqindi yig'uvchi) — xotirani avtomatik tozalash tizimi.
MRO	Method Resolution Order (Metodlarni hal qilish tartibi) — klasslar ierarxiyasida metod qidirish ketma-ketligi.
ABC	Abstract Base Class (Abstrakt tayanch klass) — voris klasslar uchun shablon vazifasini o'tovchi klass.
ORM	Object-Relational Mapping (Obyekt-relyatsion xaritalash) — ma'lumotlar bazasini obyektlar orqali boshqarish.
WSGI	Web Server Gateway Interface (Veb-server shlyuz interfeysi) — Python ilovalari va veb-server o'rtasidagi standart.
ASGI	Asynchronous Server Gateway Interface (Asinxron server shlyuz interfeysi) — asinxron ilovalar uchun aloqa standarti.
REST	Representational State Transfer (Holatni ifodalovchi uzatish) — API yaratishning me'moriy uslubi.
JWT	JSON Web Token — xavfsiz autentifikatsiya uchun ishlatiladigan JSON formatidagi token.
CORS	Cross-Origin Resource Sharing (Resurslarni turli manbalardan almashish) — brauzer xavfsizlik cheklovlarini boshqarish.
CI/CD	Continuous Integration / Continuous Deployment (Uzluksiz integratsiya / Uzluksiz joylashtirish).
CSRF	Cross-Site Request Forgery (Saytlararo so'rovlarni soxtalashtirish) — foydalanuvchi nomidan yashirin so'rov yuborish.
XSS	Cross-Site Scripting (Saytlararo skript yozish) — veb-sahifaga zararli JS kodini kiritish.
SSH	Secure Shell (Xavfsiz qobiq) — masofaviy serverga xavfsiz ulanish protokoli.
GUI	Graphical User Interface (Grafik foydalanuvchi interfeysi) — dasturning vizual qismi.
SSL/TLS	Secure Sockets Layer / Transport Layer Security (Xavfsiz soketlar qatlami / Transport qatlami xavfsizligi).
TCP	Transmission Control Protocol (Uzatishni boshqarish protokoli) — ishonchli paket uzatish protokoli.
UDP	User Datagram Protocol (Foydalanuvchi datagrammalari protokoli) — tezkor, lekin kafolatlanmagan protokol.
API	Application Programming Interface (Ilovalarni dasturlash interfeysi) — dasturlararo aloqa interfeysi.
IDE	Integrated Development Environment (Integratsiyalashgan dasturlash muhiti) — kod yozish uchun majmuaviy dastur.
JSON	JavaScript Object Notation — ma'lumotlarni almashish uchun yengil matnli format.
SQL	Structured Query Language (Strukturalangan so'rovlar tili) — ma'lumotlar bazasi bilan ishlash tili.
URL	Uniform Resource Locator (Resursning bir xil joylashuvi) — internetdagi manzil.
CRUD	Create, Read, Update, Delete (Yaratish, O'qish, Yangilash, O'chirish) — asosiy baza amallari.

HTTP	HyperText Transfer Protocol (Gipermatn uzatish protokoli) — veb-brauzer va server aloqasi.
CPU	Central Processing Unit (Markaziy protsessor) — kompyuterning asosiy hisoblash qismi.
RAM	Random Access Memory (Operativ xotira) — vaqtinchalik tezkor xotira.
I/O-bound	Input/Output bound — kiritish-chiqarish amallariga (fayl, tarmoq) bog'liq bo'lgan vazifa.
CPU-bound	Central Processing Unit bound — hisob-kitob quvvatiga bog'liq bo'lgan vazifa.
MVC	Model-View-Controller — dastur arxitekturasini andozasi.
P2P	Peer-to-Peer (Tengdoshdan-tengdoshga) — markaziy serversiz tarmoq muloqoti.
DRY	Don't Repeat Yourself (O'zingni takrorlama) — kodni qisqartirish tamoyili.
SOLID	(Obyektga yo'naltirilgan dasturlashning 5 ta asosiy tamoyili qisqartmasi).
YAGNI	You Ain't Gonna Need It (Sizga bu kerak bo'lmaydi) — ortiqcha funksiyalardan qochish tamoyili.
MIME	Multipurpose Internet Mail Extensions (Ko'p maqsadli internet pochta kengaytmalari) — kontent turini aniqlash.
DIP	Dependency Inversion Principle (Bog'liqlikni teskari qilish tamoyili) — SOLID'ning "D" qismi.
FIFO	First In, First Out (Birinchi kirgan, birinchi chiqadi) — navbat algoritmi.
LIFO	Last In, First Out (Oxirgi kirgan, birinchi chiqadi) — stek algoritmi.
SDLC	Software Development Life Cycle (Dasturiy ta'minotni ishlab chiqish hayotiy sikli).

Savol-javoblar.

? 1. Python qanday dasturlash tili va uning interpretatsiya qilinishi (interpreted) nimani anglatadi?

 Python — bu yuqori darajali, interpretatsiya qilinadigan dasturlash tili. Bu shuni anglatadiki, kod yozilgandan so'ng u bevosita mashina kodiga emas, balki **Bytecode**ga o'giriladi va **PVM** (Python Virtual Machine) tomonidan qatorma-qator o'qib ijro etiladi. Bu jarayon dasturni platformaga bog'liq bo'lmagan (platform-independent) holatga keltiradi, ya'ni bir xil kod Windows, Linux va macOS'da o'zgarishsiz ishlaydi.

☆ 2. Python'da xotira boshqaruvi (Memory Management) va Garbage Collection qanday ishlaydi?

 Python'da xotira **Private Heap Space**'da saqlanadi va uni boshqarish uchun ikki asosiy mexanizm qo'llaniladi:

1. **Reference Counting (Havolalar sanog'i)**: Har bir ob'ektga nechta o'zgaruvchi murojaat qilayotgani sanab boriladi. Agar sanoq 0 ga tushsa, ob'ekt xotiradan o'chiriladi.

2. **Garbage Collector (GC)**: "Circular Reference" (aylanma havolalar)ni aniqlab, xotirani tozalaydi. Python buning uchun **Generational Garbage Collection** (avlodlarga bo'lingan tozalash) usulidan foydalanadi. Bunda ob'ektlar "yosh" va

"keksa" avlodlarga bo'linadi, chunki tajriba shuni ko'rsatadiki, yangi yaratilgan ob'ektlar tezroq yo'q bo'ladi.

❓ **3. List va Tuple farqini faqat "o'zgarmaslik" bilan emas, texnik jihatdan tushuntiring.**

📖 Texnik jihatdan farqlar quyidagicha:

- **Xotira (Overhead):** List o'zgaruvchan bo'lgani uchun Python unga zaxira joy ajratadi (over-allocation). Tuple esa yaratilishda qancha joy kerak bo'lsa, shuncha oladi. Shu sababli Tuple xotirani kamroq egallaydi.

- **Tezlik:** Tuple'ni iteratsiya qilish (aylanib chiqish) va unga kirish List'ga qaraganda tezroq ishlaydi, chunki u o'zgarmas struktura sifatida optimallashtirilgan.

- **Hashability:** Tuple o'zgarmas bo'lgani uchun u **hashable** hisoblanadi va lug'atda `key` (kalit) sifatida ishlatilishi mumkin. List'ni esa kalit qilib bo'lmaydi.

☆ **4. "Duck Typing" tushunchasi Python'da nimani anglatadi?** 📖 Bu tushuncha "Agar u o'rdak kabi yursa va o'rdak kabi qaqillasa, demak u o'rdakdir" tamoyiliga asoslanadi. Python'da ob'ektning qaysi klassga tegishli ekanligi emas, balki u ma'lum bir metod yoki atributga ega ekanligi muhim. Masalan, agar ob'ektda `.read()` metodi bo'lsa, u faylmi, socketmi yoki oddiy klassmi — Python uchun farqi yo'q, u bilan ishlashda davom etaveradi.

❓ **5. Python'da `None` nima va u `False` dan nimasi bilan farq qiladi?**

📖 `None` — bu Python'dagi maxsus doimiy (constant) ob'ekt bo'lib, "qiymatning yo'qligi"ni bildiradi. U o'ziga xos `NoneType` tipiga kiradi.

- `None` — qiymat umuman mavjud emasligini anglatadi.
- `False` — mantiqiy (boolean) "yolg'on" qiymatidir. Shart tekshirishda (`if` blokida) ikkalasi ham yolg'on deb qabul qilinadi, lekin `None is False` natijasi har doim `False` bo'ladi, chunki ular turli xil ob'ektlardir.

☆ **6. Python'da `is` va `==` operatorlarining xotira darajasidagi farqi.**



- `==` operatori — **Equality** (Tenglik). U ob'ektlarning **qiymati** bir xilligini tekshiradi.

- `is` operatori — **Identity** (Ayniyat). U ikki o'zgaruvchi xotiradagi **aynan bitta manzilda** turgan ob'ektga ishora qilayotganini tekshiradi (`id(a) == id(b)`).

❓ **7. `Mutable Default Arguments` muammosi nima?**

📖 Agar funksiya argumenti sifatida default qiymatga mutable obyekt (masalan, `def func(a=[])`) berilsa, bu list funksiya har gal chaqirilganda qayta yaratilmaydi. U funksiya ta'riflangan vaqtda bir marta yaratiladi va barcha chaqiriqlar uchun umumiy bo'lib qoladi.

- **Yechim:** Default qiymat sifatida `None` berish va funksiya ichida `if a is None: a = []` deb yozish tavsiya etiladi.

★ 8. Python'da Global Interpreter Lock (GIL) nima va u multithreading'ga qanday ta'sir qiladi?

📖 GIL — bu bir vaqtning o'zida faqat bitta oqim (thread) Python baytkodini bajarishini ta'minlaydigan mexanizmdir. Bu xotira boshqaruvini (reference counting) xavfsiz qilish uchun kerak. Natijada, Python'da CPU-bound (protsessorni ko'p talab qiladigan) vazifalarda multithreading samarali ishlamaydi. Bunday holatlarda multiprocessing modulidan foydalanish tavsiya etiladi.

? 9. `dir()` va `vars()` funksiyalarining farqi nimada?



- `dir()` — ob'ektning barcha atributlari va metodlari ro'yxatini (faqat nomlarini) qaytaradi, jumladan meros olinganlarini ham.
- `vars()` — ob'ektning `__dict__` atributini qaytaradi, ya'ni ob'ektga tegishli joriy atributlar va ularning qiymatlarini lug'at ko'rinishida ko'rsatadi.

★ 10. Python'da `interning` (string/integer `interning`) mexanizmi qanday ishlaydi?

📖 Bu xotirani optimallashtirish usuli bo'lib, Python kichik butun sonlar (-5 dan 256 gacha) va ba'zi satrlarni xotirada faqat bir marta saqlaydi. Agar siz turli joyda `x = 256` va `y = 256` deb yozsangiz, ular xotirada bitta manzilga ishora qiladi. Bu solishtirish tezligini oshiradi.

? 11. `list.append()` va `list.extend()` metodlarining farqi nimada?

📖 `append()` metodi argument sifatida berilgan ob'ektni (u list bo'lsa ham) ro'yxatning oxiriga bitta element sifatida qo'shadi. `extend()` metodi esa argument sifatida kelgan iteratsiya qilinadigan ob'ektning (list, tuple, set) har bir elementini bittalab ajratib, ro'yxatga qo'shadi. Masalan, `[1, 2].append([3, 4])` natijasi `[1, 2, [3, 4]]` bo'ladi, `extend`da esa `[1, 2, 3, 4]` bo'ladi.

★ 12. Python'da lug'at (dict) kalitlari sifatida nimalardan foydalanish mumkin va nima uchun?

📖 Lugʻat kaliti sifatida faqat **hashable** (xeshlanadigan) obʻektlardan foydalanish mumkin. Bularga immutable turlar: `int`, `float`, `str`, `tuple` (agar ichidagi elementlar ham immutable boʻlsa) va `frozenset` kiradi. Buning sababi, lugʻat elementni qidirishda xesh-jadvaldan foydalanadi. Agar obʻekt oʻzgarsa, uning xeshi ham oʻzgaradi va lugʻat uni topa olmay qoladi. List yoki Set mutable boʻlgani uchun ularni kalit sifatida ishlatib boʻlmaydi.

❓ **13. `shallow copy` va `deep copy` oʻrtasidagi farqni misol bilan tushuntiring.**

📖 `Shallow copy` (yuzaki nusxa) yangi obʻekt yaratadi, lekin uning ichidagi ichma-ich joylashgan elementlarga havolani (reference) saqlab qoladi. Agar ichki listni oʻzgartirsangiz, nusxada ham oʻzgaradi. `Deep copy` esa barcha ichki elementlarning ham toʻliq mustaqil nusxasini yaratadi.

★ **14. List Comprehension va Generator Expression farqi nimada?**

📖 List Comprehension kvadrat qavslar `[]` ichida yoziladi va natijani xotirada toʻliq list koʻrinishida saqlaydi. Generator Expression esa oddiy qavslar `()` ichida yoziladi va elementlarni faqat kerak boʻlganda (lazy) generatsiya qiladi. Agar maʼlumotlar hajmi millionlab boʻlsa, Generator Expression xotirani tejash uchun eng yaxshi tanlovdir.

❓ **15. Pythonʼda `lambda` funksiyalar nima va ularning asosiy cheklovi nimada?**

📖 `lambda` — bu nomlanmagan, bir qatorli anonim funksiya. U asosan qisqa muddatli operatsiyalar, masalan `map()`, `filter()` yoki `sort()` funksiyalariga argument sifatida berish uchun ishlatiladi. Asosiy cheklovi: uning ichida faqat bitta ifoda (expression) boʻlishi mumkin, murakkab mantiqiy bloklar (if-else, for, try-except) yozish imkonsiz.

★ **16. `map`, `filter` va `reduce` funksiyalari qanday vazifani bajaradi?**



- `map(func, seq)` — ketma-ketlikdagi har bir elementga funksiyani qoʻllaydi va natijalarni qaytaradi.

- `filter(func, seq)` — funksiya shartiga (True/False) mos keladigan elementlarni saralab oladi.

- `reduce(func, seq)` — `functools` modulidan olinadi. U ketma-ketlik elementlarini juftlab hisoblab, yakunda bitta qiymat qaytaradi (masalan, yig'indi yoki ko'paytmani hisoblashda).

❓ 17. `*args` va `**kwargs` operatorlarining funksiya chaqiruvidagi vazifasi?

📖 Bu operatorlar funksiyaga o'zgaruvchan miqdordagi argumentlarni uzatish imkonini beradi. `*args` barcha pozitsion argumentlarni **tuple**ga yig'adi, `**kwargs` esa kalit so'zli (keyword) argumentlarni **dictionary**ga yig'adi. Bu kutubxonalar yaratishda va dekoratorlar yozishda funksiyani universal qilish uchun ishlatiladi.

☆ 18. Python'da "Namespace" va "Scope" tushunchalari nimani anglatadi?

📖 **Namespace** — o'zgaruvchi nomlari va ularga tegishli ob'ektlar o'rtasidagi bog'lanish (lug'at ko'rinishida). **Scope** (ko'lam) — bu dasturning qaysi qismida ushbu o'zgaruvchini to'g'ridan-to'g'ri ko'rish mumkinligini belgilaydigan qoida. Python'da **LEGB** qoidasi amal qiladi: Local, Enclosing, Global, Built-in.

❓ 19. `pass`, `continue` va `break` operatorlarining farqi?



- `pass` — bu hech narsa qilmaydigan operator (placeholder). Kod strukturasi talab qilinsa-yu, lekin mantiq yozilmaganda ishlatiladi.

- `continue` — siklning joriy iteratsiyasini to'xtatib, keyingi iteratsiyaga o'tadi.
- `break` — butun siklni (loop) to'xtatib, undan chiqib ketadi.

☆ 20. `__name__ == "__main__"` ifodasi nima uchun ishlatiladi? 📖 Bu ifoda skriptni to'g'ridan-to'g'ri ishga tushirilganini yoki boshqa modulga import qilinganini aniqlash uchun kerak. Agar skript bevosita ishga tushirilsa, `__name__` o'zgaruvchisi `"__main__"` qiymatiga ega bo'ladi. Bu import paytida skript ichidagi asosiy kod bloklarini (masalan, testlarni) avtomatik ravishda ishga tushib ketishidan saqlaydi.

❓ 21. Python'da klass ichida `self` argumentining vazifasi nima?

📖 `self` — bu klassdan yaratilgan obyektning (instance) o'ziga ishora qiluvchi havoladir. U orqali obyektning atributlari va boshqa metodlariga murojaat qilinadi. Texnik jihatdan, metod chaqirilganda Python avtomatik ravishda obyektни birinchi argument sifatida uzatadi. Masalan, `obj.method(arg)` chaqiruvi aslida `Class.method(obj, arg)` ko'rinishida ishlaydi. `self` kalit so'z emas, uni

boshqacha nomlash ham mumkin, lekin standart bo'yicha `self` ishlatish majburiy hisoblanadi.

☆ 22. `__init__` va `__new__` metodlarining farqi nimada?

📖 Ko'pchilik `__init__` ni konstruktor deb o'ylaydi, lekin aslida:

- `__new__` — haqiqiy konstruktor. U obyektни xotirada yaratadi va qaytaradi. Bu metod asosan immutable tiplardan (masalan, `tuple` yoki `int`) meros olganda yoki **Singleton** patternini yaratishda ishlatiladi.

- `__init__` — initsializator. U `__new__` yaratib bergan tayyor obyektни qabul qiladi va unga boshlang'ich qiymatlarni (atributlarni) beradi.

? 23. Python'da inkapsulyatsiya (Encapsulation) qanday darajalarga bo'linadi?

📖 Python'da boshqa tillardagi kabi qat'iy `private` yoki `protected` cheklovlari yo'q. Buning o'rniga kelishuv (convention) ishlatiladi:

- **Public:** `self.name` — hamma uchun ochiq.
- **Protected:** `_self.name` — bitta pastki chiziq. Bu o'zgaruvchidan faqat klass va uning vorislari ichida foydalanish tavsiya etilishini bildiradi.
- **Private:** `__self.name` — ikkita pastki chiziq. Bu **Name Mangling**ni keltirib chiqaradi, ya'ni Python o'zgaruvchi nomini ichki tomondan `_ClassName__name`ga o'zgartiradi, bu esa unga to'g'ridan-to'g'ri tashqaridan kirishni qiyinlashtiradi.

☆ 24. "Multiple Inheritance" (Ko'p tomonlama merosxo'rlik) va MRO nima?

📖 Python bitta klassning bir nechta ota klasslardan meros olishini qo'llab-quvvatlaydi. Bunda **MRO (Method Resolution Order)** — metodlarni qidirish tartibi muhim rol o'ynaydi. Agar bir xil nomli metod bir nechta ota klassda bo'lsa, Python qaysi birini birinchi ishlatishni **C3 Linearization** algoritmi orqali belgilaydi. Siz buni `ClassName.mro()` yoki `ClassName.__mro__` orqali tekshirishingiz mumkin.

? 25. `@staticmethod` va `@classmethod` dekoratorlarining asosiy farqi nimada?



- `@classmethod` — birinchi argument sifatida klassning o'zini (`cls`) qabul qiladi. U klass atributlarini o'zgartira oladi va ko'pincha "Factory methods" (obyekt yaratishning muqobil usullari) uchun ishlatiladi.

- `@staticmethod` — na `self`ni, na `cls`ni qabul qiladi. U mantiqan klass ichida joylashgan, lekin uning holatiga (state) bog‘liq bo‘lmagan oddiy funksiyadir.

☆ 26. Python'da "Abstract Base Classes" (ABC) nima uchun kerak?

📖 ABC ma'lum bir klasslar guruhi uchun "shablon" yoki "shartnoma" vazifasini o‘taydi. `abc` modulidagi ABC klassidan meros olgan va `@abstractmethod` bilan belgilangan metodni voris klasslar albatta o‘zlarida yozishlari (implement) shart. Aks holda, voris klassdan obyekt yaratib bo‘lmaydi. Bu yirik loyihalarda arxitekturani tartibga solish uchun xizmat qiladi.

? 27. `super()` funksiyasi nima uchun ishlatiladi?

📖 `super()` — ota klass metodlariga murojaat qilish uchun ishlatiladi. Bu asosan voris klassda `__init__` metodini yozayotganda, ota klassning `__init__` metodini ham ishga tushirib, atributlarni meros qilib olish uchun kerak. U MRO tartibi bo‘yicha keyingi klassni chaqiradi, bu esa ko‘p tomonlama merosxo‘rlikda xatolarning oldini oladi.

☆ 28. "Magic Methods" (yoki Dunder Methods) nima?

📖 Bular ikki pastki chiziq bilan boshlanadigan va tugaydigan maxsus metodlar (masalan, `__str__`, `__repr__`, `__add__`). Ular Python'ning ichki operatorlari bilan bog‘lanish imkonini beradi. Masalan, ikkita obyektни + bilan qo‘shish uchun klass ichida `__add__` metodini yozish kifoya. Bu **Operator Overloading** (operatorlarni qayta yuklash) deb ataladi.

? 29. `__str__` va `__repr__` metodlarining farqi nimada?



- `__str__` — foydalanuvchi uchun tushunarli ko‘rinish (masalan, `print(obj)` qilinganda).
- `__repr__` — dasturchi uchun mo‘ljallangan, obyektни qayta yaratish uchun yetarli bo‘lgan texnik ma'lumot (debugging uchun). Standart bo‘yicha, agar `__str__` yozilmagan bo‘lsa, Python `__repr__` dan foydalanadi.

☆ 30. Python'da `@property` dekoratori nima uchun kerak?

📖 Bu dekorator klass metodini xuddi atribut (o‘zgaruvchi) kabi chaqirish imkonini beradi (ya'ni `obj.get_age()` emas, `obj.age` ko‘rinishida). U orqali **Getter**, **Setter** va **Deleter** mantiqlarini yaratish mumkin. Bu inkapsulyatsiyani saqlagan holda, atributlarga kirishda qo‘shimcha tekshiruvlar (masalan, yosh manfiy bo‘lmasligi kerak) qo‘shish uchun juda qulay.

? 31. Python'da Iterable va Iterator ob'ektlarining farqi nimada?

📖 **Iterable** — bu `__iter__` metodiga ega bo'lgan va elementlarini bittalab olish mumkin bo'lgan har qanday ob'ekt (masalan: `list`, `str`, `dict`).

Iterator — bu `__next__` metodiga ega bo'lgan ob'ekt bo'lib, u navbatdagi elementni qaytaradi va o'z holatini saqlab qoladi. Agar elementlar tugasa, `StopIteration` xatosini beradi. Har qanday iterator iterable hisoblanadi, lekin hamma iterable'lar iterator emas (masalan, `list`'dan iterator olish uchun `iter(list)` funksiyasini chaqirish kerak).

☆ 32. Generatorlar nima va ularning `return` ishlatadigan funksiyalardan afzalligi nimada?

📖 Generator — bu `yield` kalit so'zi yordamida yaratiladigan maxsus funksiya. Oddiy funksiya `return` qilinganda o'z ishini butunlay tugatsa, generator `yield` nuqtasida to'xtaydi va holatini saqlab qoladi.

Afzalligi: U elementlarni xotirada saqlamaydi, balki ularni so'ralgan vaqtda (on-the-fly) generatsiya qiladi. Bu juda katta ma'lumotlar bilan ishlashda RAMni tejaydi.

? 33. Python'da dekoratorlar (Decorators) qanday ishlaydi?

📖 Dekorator — bu boshqa bir funksiyani argument sifatida qabul qiluvchi va uning kodini o'zgartirmagan holda funksiyaga qo'shimcha imkoniyat qo'shuvchi funksiya. U "High-order function" konsepsiyasiga asoslangan. Masalan, funksiyaning qancha vaqt ishlashini o'lchash yoki foydalanuvchi tizimga kirganini tekshirish (auth) uchun ishlatiladi.

☆ 34. Bitta funksiyaga bir nechta dekorator qo'shilsa (Pie syntax), ular qaysi tartibda ishlaydi?

📖 Dekoratorlar pastdan yuqoriga qarab "o'raladi" (wrapping), lekin yuqoridan pastga qarab ishga tushadi. Ya'ni, funksiyaga eng yaqin turgan (pastdagi) dekorator birinchi bo'lib funksiyani o'raydi, lekin ijro vaqtida eng yuqoridagi dekorator birinchi bo'lib mantiqni boshlaydi.

? 35. `yield from` ifodasi nima uchun ishlatiladi?

📖 `yield from` — bu generator ichida boshqa bir generator yoki iterable ob'ektni (`list`, `tuple`) qisqaroq yo'l bilan qaytarish uchun ishlatiladi. U "sub-generator" yaratish imkonini beradi va qo'shimcha `for` sikli yozish zaruriyatini yo'qotadi.

☆ 36. Python'da Closures (Yopiq muhit) nima va u qachon ishlatiladi?

📖 Closure — bu ichki funksiya bo‘lib, u o‘zidan tashqaridagi (lekin global bo‘lmagan) o‘zgaruvchilarni eslab qoladi, hatto tashqi funksiya o‘z ishini yakunlagan bo‘lsa ham. Bu ma’lumotlarni yashirish (data hiding) va klass yaratmasdan holatni saqlab qolish (state persistence) uchun foydali.

? 37. Funksiya ichida `nonlocal` kalit so‘zi nima uchun kerak?

📖 `nonlocal` — ichki funksiya ichidan tashqi (lekin global bo‘lmagan) funksiya o‘zgaruvchisini o‘zgartirish uchun ishlatiladi. Agar `nonlocal` ishlatilmasa, Python ichki funksiyada yangi lokal o‘zgaruvchi yaratib yuboradi.

☆ 38. Kontekst menejerlari (`with` operatori) qanday ishlaydi va uning foydasi nima?

📖 Kontekst menejerlari resurslarni (fayllar, ma’lumotlar bazasi ulanishlari, lock’lar) xavfsiz boshqarish uchun ishlatiladi. U `__enter__` (resursni ochish) va `__exit__` (resursni yopish) metodlariga asoslanadi.

Foydasi: Agar kod ichida xatolik yuz bersa ham, `__exit__` metodi baribir ishga tushadi va resursni (faylni) yopib ketadi, bu esa xotira va tizim resurslari isrof bo‘lishining oldini oladi.

? 39. `contextlib` moduli yordamida dekorator ko‘rinishidagi kontekst menejeri qanday yaratiladi?

📖 Buning uchun `@contextmanager` dekoratori ishlatiladi. Funksiya ichida `yield` ishlatiladi: `yield`dan oldingi kod — `__enter__` vazifasini, `yield`dan keyingi kod esa — `__exit__` vazifasini bajaradi. Bu klass yaratmasdan turib kontekst menejerini yaratishning eng qulay usulidir.

☆ 40. Python’da Metaprogramming nima va `type()` funksiyasining roli?

📖 Metaprogramming — bu kodni yozadigan yoki o‘zgartiradigan kod yozishdir. Python’da `type()` funksiyasi nafaqat ob’ekt turini aniqlaydi, balki dinamik ravishda yangi klasslar yaratish uchun ham ishlatiladi: `type('KlassNomi', (OtaKlass,), {atributlar})`. Barcha klasslar `type` metaklassining namunasi hisoblanadi.

? 41. `try...except...else...finally` bloklarining har biri qachon ishlaydi?



- **try:** Xato berishi mumkin bo‘lgan kod yoziladi.

- **except:** Agar `try` ichida xato yuz bersa, shu blok ishlaydi.
- **else:** Agar `try` ichida hech qanday xato yuz bermasa, ishlaydi.
- **finally:** Xato bo'lishi yoki bo'lmasligidan qat'i nazar, har doim oxirida ishlaydi (odatda resurslarni tozalash uchun).

☆ 42. Nima uchun **except** **Exception:** yoki shunchaki **except:** deb yozish yomon amaliyat (anti-pattern) hisoblanadi?

📖 Chunki bu barcha turdagi xatolarni (hatto imlo xatolarini yoki `KeyboardInterrupt`ni ham) ushlab qoladi. Bu esa haqiqiy logik xatolarni topishni (debugging) qiyinlashtiradi. To'g'ri yo'li — faqat kutilayotgan aniq xato turini ko'rsatish (masalan: `except ValueError:`).

? 43. **raise** va **assert** operatorlarining farqi nimada?



- **raise:** Dastur davomida ma'lum bir shart asosida majburiy xatolik chiqarish uchun ishlatiladi (masalan: `if age < 0: raise ValueError`).
- **assert:** Debugging va testing uchun ishlatiladi. U berilgan shartni tekshiradi, agar shart `False` bo'lsa, `AssertionError` qaytaradi. `assert` odatda ishlab chiqarish (production) muhitida o'chirib qo'yiladi.

☆ 44. Python'da maxsus xatolik klassini (Custom Exception) qanday yaratish mumkin?

📖 Buning uchun Python'ning standart `Exception` klassidan meros olish kifoya. Masalan: `class MeningXatoim(Exception): pass`. Bu loyihada o'zimizga xos biznes-mantiq xatolarini ajratib ko'rsatish uchun kerak.

? 45. Faylni `open()` orqali ochish va `with open()` orqali ochishning farqi?

📖 Oddiy `open()` ishlatilganda, faylni ishlatib bo'lgach `file.close()` qilish dasturchining zimmasida. Agar xato chiqib dastur to'xtasa, fayl ochiq qolib ketishi mumkin. `with open()` esa kontekst menejeri bo'lib, ish tugashi bilan (yoki xato chiqsa ham) faylni avtomatik yopadi.

☆ 46. `json.dump()` va `json.dumps()` metodlarining farqi nimada?



- `json.dump(obj, file_obj):` Python ob'ektini JSON formatiga o'giradi va to'g'ridan-to'g'ri faylga yozadi.

- `json.dumps(obj)`: Python ob'ektini JSON formatidagi **satrga (string)** o'giradi. ("s" harfi "string" ma'nosini bildiradi).

❓ 47. Python'da **logging** moduli nima uchun `print()` dan ko'ra afzalroq?

📖 `logging` moduli xabarlarni darajalarga (DEBUG, INFO, WARNING, ERROR, CRITICAL) ajratish imkonini beradi. Shuningdek, xabarlarni vaqti, fayl nomi va qatori bilan birga faylga saqlashi mumkin. `print()` esa faqat konsolga chiqaradi va katta loyihalarda uni boshqarish imkonsiz bo'ladi.

☆ 48. Unit Testing nima va Python'da qaysi kutubxonalar mashhur?

📖 Unit testing — kodning eng kichik bo'laklarini (funksiya yoki metodlarni) alohida tekshirish jarayoni. Python'da standart `unittest` moduli mavjud. Biroq, amaliyotda qulayroq va imkoniyati kengroq bo'lgan `pytest` kutubxonasi eng ko'p ishlatiladi.

❓ 49. **mock** ob'ektlar testlashda nima uchun kerak?

📖 Test yozayotganda tashqi servislar (masalan, API yoki ma'lumotlar bazasi)ga bog'liq bo'lmaslik uchun `mock` ishlatiladi. U haqiqiy servisning javobini "simulatsiya" qilib beradi, natijada testlar tez va internetga bog'liq bo'lmagan holda ishlaydi.

☆ 50. Python modullarini import qilishda **Circular Import** muammosi nima?

📖 Bu ikki modul bir-birini import qilganda yuzaga keladi (A modul B ni, B modul esa A ni import qiladi). Python buni xato deb hisoblaydi. **Yechim:** Importni funksiya ichiga ko'chirish yoki kod arxitekturasini o'zgartirib, umumiy qismlarni uchinchi modulga chiqarish.

❓ 51. **Concurrency** (raqobatdoshlik) va **Parallelism** (parallellik) o'rtasidagi farq nimada?

📖 **Concurrency** — bu bir vaqtning o'zida bir nechta vazifa bilan shug'ullanish (lekin ularni bir vaqtda bajarish shart emas). Bu ko'proq vazifalar o'rtasida tez almashishga o'xshaydi (masalan, kofe ichib turib, xat yozish).

Parallelism — bu bir vaqtning o'zida bir nechta vazifani haqiqatda bajarish (masalan, ikki kishi ikki xil ishni qilishi). Python'da `multithreading` ko'proq `concurrency` beradi, `multiprocessing` esa haqiqiy `parallelism`ni ta'minlaydi.

☆ 52. Python'da **Threading** va **Multiprocessing** modullari qachon ishlatiladi?



- **Threading:** I/O-bound vazifalar (fayl o‘qish, tarmoqdan ma’lumot yuklash) uchun ishlatiladi. Chunki oqimlar bir-birini kutib turganda GIL (Global Interpreter Lock) boshqa oqimga yo‘l beradi.

- **Multiprocessing:** CPU-bound vazifalar (murakkab matematik hisob-kitoblar, videoga ishlov berish) uchun ishlatiladi. Har bir protsess o‘zining alohida Python interpretatori va xotira maydoniga ega bo‘ladi, bu esa GIL cheklovini chetlab o‘tishga imkon beradi.

? 53. AsyncIO nima va u qanday ishlaydi?



AsyncIO — bu bitta oqim (thread) ichida asinxron kod yozish uchun kutubxona. U **Event Loop** (hodisalar sikli) tushunchasiga asoslangan. Vazifa kutilayotgan vaqtda (masalan, API’dan javob kelishi), Event Loop boshqa vazifani bajarishga o‘tadi. Bu minglab bir vaqtda ishlovchi ulanishlarni (connections) bitta oqimda boshqarish imkonini beradi.

☆ 54. `async def` va `await` kalit so‘zlari nima vazifani bajaradi?



- **`async def`:** Funksiyani **Coroutines** (korutina)ga aylantiradi. Bunday funksiya chaqirilganda darhol ishlamaydi, balki korutina ob’ektini qaytaradi.

- **`await`:** Dastur ijrosini korutina natija qaytarguncha vaqtincha to‘xtatib turadi va Event Loop’ga boshqa ishlarni bajarish uchun nazoratni topshiradi.

? 55. Python’da **Race Condition** nima va uni qanday oldini olish mumkin?



Race Condition — bu ikki yoki undan ortiq oqim bir vaqtning o‘zida bitta umumiy ma’lumotni o‘zgartirmoqchi bo‘lganda yuzaga keladigan holat. Natijada ma’lumotlar kutilmagan ko‘rinishga kelib qoladi.

Yechim: `threading.Lock` (qulflash) mexanizmidan foydalanish. Bir oqim resursni ishlatayotganda uni qulflab qo‘yadi (`lock.acquire()`), ishini tugatgach esa ochadi (`lock.release()`).

☆ 56. **Thread-safe** tushunchasi nimani anglatadi?



Agar ma’lum bir ma’lumotlar tuzilmasi yoki funksiya bir vaqtning o‘zida bir nechta oqimlar tomonidan xavfsiz (ma’lumotlar buzilmasdan) ishlatila olsa, u **Thread-safe** hisoblanadi. Python’dagi `queue.Queue` klassi thread-safe hisoblanadi, lekin oddiy `list` yoki `dict` har doim ham unday emas.

? 57. **Deadlock** nima va u qanday sodir bo‘ladi?

📖 **Deadlock** (tupik) — bu ikki yoki undan ortiq oqim bir-birini cheksiz kutib qolishi. Masalan, 1-oqim A resursni band qilib, B ni kutmoqda; 2-oqim esa B ni band qilib, A ni kutmoqda. Natijada dastur qotib qoladi.

☆ 58. **Multiprocessing.Pool** va **Process** o‘rtasidagi farq?



- **Process**: Bitta alohida protsess yaratish uchun ishlatiladi. Siz uni qo‘lda boshqarishingiz (`start()`, `join()`) kerak.

- **Pool**: Bir guruh ishchi protsesslarni (worker processes) boshqarish uchun qulay interfeys. U vazifalarni protsesslar o‘rtasida avtomatik taqsimlaydi (masalan, `map()` metodi orqali).

? 59. **Python’da Daemon Thread** nima?

📖 Bu "fon"dagi oqimdir. Oddiy oqimlardan farqi shundaki, agar dasturning asosiy qismi (main thread) ishini tugatsa, barcha daemon oqimlar ham darhol to'xtatiladi. Ular odatda asosiy dasturga yordamchi xizmat ko'rsatish (masalan, xotirani tozalash yoki signal kutish) uchun ishlatiladi.

☆ 60. **ContextVar** asinxron dasturlashda nima uchun kerak?

📖 **ContextVar** — bu `threading.local()` ning asinxron analogidir. U har bir asinxron vazifa (task) uchun alohida o'zgaruvchi muhitini saqlashga imkon beradi. Bu asinxron muhitda global o'zgaruvchilar aralashib ketmasligi (masalan, har bir HTTP so'rov uchun alohida `request_id` saqlash) uchun juda muhim.

? 61. **Django** va **FastAPI/Flask** o‘rtasidagi asosiy falsafiy farq nimada?

📖 **Django** — bu "Batteries Included" framework. Unda barcha kerakli vositalar (Admin panel, ORM, Auth, Migratsiyalar) tayyor holda keladi. Bu katta loyihalarni tezroq boshlash imkonini beradi.

FastAPI va **Flask** — mikro-frameworklar hisoblanadi. Ular juda yengil va dasturchiga erkinlik beradi. Siz bazani, validatsiyani va boshqa kutubxonalarni o‘zingiz tanlab qo‘shasiz. FastAPI asinxronlik va avtomatik dokumentatsiya (Swagger) bilan ajralib turadi.

☆ 62. **ORM (Object-Relational Mapping)** nima va uning foydasi hamda zarari nimada?

📖 ORM — bu ma'lumotlar bazasidagi jadvallar bilan SQL yozmasdan, Python klasslari orqali ishlash imkonini beruvchi texnologiya.

Foydasi: Kod o'qilishi osonlashadi, SQL injection'dan himoya qiladi va bazani almashtirish (masalan, SQLite'dan PostgreSQL'ga) oson kechadi. **Zarari:** Juda murakkab SQL so'rovlarda ORM sekin ishlashi yoki ortiqcha so'rovlar (N+1 muammosi) yaratishi mumkin.

❓ 63. Django'da "N+1 query problem" nima va u qanday hal qilinadi?

📖 Bu muammo bitta so'rov o'rniga bazaga ko'plab mayda so'rovlar yuborilganda yuzaga keladi. Masalan, 10 ta kitobni va ularning mualliflarini chiqarishda, har bir kitob uchun alohida muallif so'rovi yuborilsa (1+10=11 so'rov).

Yechim: `select_related()` (ForeignKey va OneToOne uchun SQL JOIN ishlatadi) yoki `prefetch_related()` (ManyToMany uchun alohida so'rov yuborib Python'da birlashtiradi) metodlaridan foydalanish.

★ 64. WSGI va ASGI o'rtasidagi farq nimada?



- **WSGI (Web Server Gateway Interface):** Sinxron web ilovalar uchun standart (masalan, Django, Flask). U so'rovlarni bitta oqimda ketma-ket qayta ishlaydi.

- **ASGI (Asynchronous Server Gateway Interface):** Asinxron ilovalar uchun standart (masalan, FastAPI, Django Channels). U WebSockets va uzoq davom etuvchi HTTP ulanishlarni asinxron boshqarish imkonini beradi.

❓ 65. REST API nima va uning asosiy metodlari qaysilar?

📖 REST — bu mijoz (frontend) va server o'rtasida ma'lumot almashish arxitekturasi. Asosiy metodlari:

- **GET:** Ma'lumotni olish.
- **POST:** Yangi ma'lumot yaratish.
- **PUT:** Ma'lumotni to'liq yangilash.
- **PATCH:** Ma'lumotni qisman yangilash.
- **DELETE:** Ma'lumotni o'chirish.

★ 66. Django Middleware nima va u qanday ishlaydi?

📖 Middleware — bu so'rov (request) serverga kelishi bilan va javob (response) qaytib ketishidan oldin ishlovchi "filtr" yoki oraliq qatlamdir. Masalan,

foydalanuvchini identifikatsiya qilish, saytni xavfsizlikka tekshirish yoki barcha so'rovlarni log qilish uchun ishlatiladi.

? 67. SQL Injection nima va Python frameworklari undan qanday himoyalangan?

📖 SQL Injection — bu zararli SQL kodini inputlar orqali bazaga yuborib, ma'lumotlarni o'g'irlash yoki o'chirish hujumi. Python ORM'lari (Django ORM, SQLAlchemy) so'rovlarni avtomatik ravishda **parameterized queries** ko'rinishiga o'tkazadi, ya'ni foydalanuvchi kiritgan ma'lumot kod sifatida emas, shunchaki qiymat sifatida qabul qilinadi.

☆ 68. Migratsiyalar (Migrations) nima uchun kerak?

📖 Migratsiyalar — bu sizning Python klasslaringiz (modellaringiz)dagi o'zgarishlarni ma'lumotlar bazasi sxemasiga (jadvallariga) xavfsiz va izchil o'tkazish usulidir. Ular bazaning versiyalarini boshqarishga (Version Control for Database) o'xshaydi.

? 69. Redis nima va u Python loyihalarida nima uchun ishlatiladi?

📖 Redis — bu xotirada (RAM) ishlovchi tezkor ma'lumotlar ombori. U asosan:

1. **Caching (Kesh):** Tez-tez so'raladigan ma'lumotlarni saqlash.
2. **Session storage:** Foydalanuvchi sessiyalarini saqlash.
3. **Message Broker:** Celery kabi kutubxonalar uchun vazifalar navbatini (task queue) boshqarishda ishlatiladi.

☆ 70. Celery va vazifalar navbati (Task Queues) nima uchun kerak?

📖 Celery uzoq davom etadigan vazifalarni (masalan, email yuborish, rasmga ishlov berish yoki hisobot tayyorlash) asosiy dasturdan alohida, fonda (background) bajarish uchun ishlatiladi. Bu foydalanuvchiga server javobini kutib qolmasdan tezroq natija olish imkonini beradi.

? 71. Stateless va Stateful arxitekturalarning farqi nimada?

📖 **Stateless (holatsiz):** Server har bir so'rovni (request) bir-biriga bog'lanmagan, yangi so'rov deb qabul qiladi. Foydalanuvchi haqidagi ma'lumotlar (masalan, login holati) server xotirasida saqlanmaydi, balki har gal so'rov bilan birga (JWT token orqali) yuboriladi. Bu tizimni gorizontol kengaytirishni (scaling) osonlashtiradi.

Stateful (holatli): Server foydalanuvchi sessiyasi haqidagi ma'lumotlarni o'z xotirasida saqlaydi. Bu kichik loyihalarda qulay, lekin bir nechta server ishlatilganda ma'lumotlarni sinxronlash muammosini keltirib chiqaradi.

☆ 72. JWT (JSON Web Token) nima va u qanday tuzilgan?

📖 JWT — bu ikki tomon oʻrtasida maʼlumotlarni xavfsiz almashish uchun ishlatiladigan ixcham va oʻz-oʻzini tasdiqlovchi formatdir. U uch qismdan iborat:

1. **Header:** Algoritm turi va token tipi.
2. **Payload:** Foydalanuvchi maʼlumotlari (id, username, login vaqti).
3. **Signature:** Tokenning haqiqiylikini tekshirish uchun maxfiy kalit bilan imzolangan qism. JWT sessiyalarni maʼlumotlar bazasida saqlash zaruriyatini yoʻqotadi (Stateless).

❓ 73. API Throttling (yoki Rate Limiting) nima va u nima uchun kerak?

📖 Bu serverga maʼlum vaqt oraligʻida (masalan, 1 daqiqada) bitta foydalanuvchidan keladigan soʻrovlar sonini cheklashdir. **Maqsadi:** Serverni haddan tashqari yuklamadan (DoS hujumlaridan) himoya qilish, resurslarni barcha foydalanuvchilar oʻrtasida adolatli taqsimlash va botlar tomonidan maʼlumotlar oʻgʻirlanishining oldini olish.

☆ 74. Microservices (Mikroservislar) va Monolith arxitekturalarining farqi?

📖 **Monolith:** Butun loyiha (backend, frontend, baza) bitta yaxlit kod bazasida boʻladi. Uni yaratish va deploy qilish oson, lekin loyiha kattalashgani sari boshqarish va yangilash qiyinlashadi.

Microservices: Loyiha kichik, mustaqil servislar (masalan: Auth servis, Toʻlov servis, Buyurtma servis)ga boʻlinadi. Har bir servis oʻz bazasiga ega boʻlishi mumkin. Bu har bir qismni alohida texnologiyada yozish va kengaytirish imkonini beradi, lekin servislararo aloqani boshqarish (API calls, Message Brokers) murakkablashadi.

❓ 75. SQL-da "Database Indexing" nima va u qanday ishlaydi?

📖 Indeks — bu maʼlumotlar bazasidagi qidiruvni tezlashtirish uchun moʻljallangan maxsus maʼlumotlar tuzilmasi (koʻpincha B-Tree). U kitobning oxiridagi mundarijaga oʻxshaydi: bazadagi barcha qatorlarni koʻrib chiqish oʻrniga, indeks orqali kerakli qatorning manzilini tezda topish mumkin. **Eslatma:** Indekslar oʻqishni tezlashtiradi, lekin yozish (INSERT, UPDATE) amallarini biroz sekinlashtiradi, chunki indeksni ham yangilash kerak boʻladi.

☆ 76. Python'da "Signal"lar (masalan, Django Signals) nima uchun ishlatiladi?

📖 Signallar — bu dasturning bir qismida sodir boʻlgan hodisadan (event) boshqa qismlarini xabardor qilish mexanizmi. Masalan, "User" modelida yangi foydalanuvchi

yaratilishi bilan avtomatik ravishda unga "Profile" yaratish yoki tabrik xati yuborish uchun ishlatiladi. Bu kodning turli qismlarini bir-biriga qattiq bog'lamasdan (decoupling) ishlashga yordam beradi.

❓ **77. API-ni hujjatlashtirish (Documentation) nima uchun muhim va qanday vositalar bor?**

📖 API dokumentatsiyasi frontend dasturchilar yoki boshqa mijozlar server bilan qanday muloqot qilishini (qaysi URL'ga qanday ma'lumot yuborishni) tushunishi uchun kerak. **Vositalar:** * **Swagger (OpenAPI):** FastAPI-da avtomatik yaratiladi.

- **Redoc:** Chiroyli va o'qishga qulay format.
- **Postman:** API-ni testlash va hujjatlashtirish uchun eng mashhur dastur.

☆ **78. Database Sharding va Partitioning farqi nimada?**

📖 **Partitioning:** Bitta katta jadvalni mantiqiy qismlarga bo'lib, bitta serverning o'zida saqlash (masalan, yillar bo'yicha: 2023-yilgi ma'lumotlar bir faylda, 2024-yilniki boshqasida).

Sharding: Ma'lumotlarni gorizontal ravishda bo'lib, bir nechta **alohida serverlarga** joylashtirish. Bu juda ulkan ma'lumotlar (Big Data) bilan ishlashda bitta serverning resurslari yetmay qolganda qo'llaniladi.

❓ **79. CORS (Cross-Origin Resource Sharing) xatosi nima va u qanday hal qilinadi?**

📖 Bu brauzerning xavfsizlik cheklovi bo'lib, bitta domenda (masalan, `frontend.com`) turgan skriptga boshqa domendagi (`api.server.com`) resurslarga murojaat qilishni taqiqlaydi. **Yechim:** Server javobida (Response Headers) `Access-Control-Allow-Origin` sarlavhasini yuborishi va ruxsat berilgan domenlarni ko'rsatishi kerak.

☆ **80. Python loyihalarini Docker-da ishlashining qanday afzalliklari bor?**

📖 Docker — dasturni barcha kutubxonalari va muhiti (environment) bilan birga bitta "konteyner"ga joylaydi. **Afzalliklari:** 1. **"Mening kompyuterimda ishlayotgan edi"** degan muammoni yo'qotadi (hamma joyda bir xil muhit). 2. Servislarni osongina izolatsiya qiladi. 3. Deploy qilish va masshtablash jarayonini (Kubernetes bilan birga) sezilarli darajada tezlashtiradi va avtomatlashtiradi.

❓ **81. "Environment Variables" (Muhit o'zgaruvchilari) nima va nima uchun .env fayllari muhim?**

📖 Muhit o'zgaruvchilari — bu dasturning ishlashi uchun kerak bo'lgan, lekin kodning ichida yozilmasligi shart bo'lgan maxfiy ma'lumotlar (masalan, ma'lumotlar bazasi paroli, API kalitlar, maxfiy SECRET_KEY).

Nima uchun: Agar siz parollarni to'g'ridan-to'g'ri kod ichida yozsangiz (hardcode), kodni GitHub-ga yuklaganingizda barcha maxfiy ma'lumotlar ochiq qilib ketadi. `.env` fayli esa faqat serverning o'zida saqlanadi va `.gitignore` orqali begona ko'zlardan himoya qilinadi.

☆ 82. CSRF (Cross-Site Request Forgery) hujumi nima va Python web-frameworklari undan qanday himoyalangan?

📖 CSRF — bu foydalanuvchi nomidan uning ruxsatisiz so'rov yuborish hujumi. Masalan, siz bank saytiga login qilgan paytingizda, boshqa zararli saytga kirsangiz, u sayt sizning brauzeringizdagi cookie-fayllardan foydalanib bank saytiga "pul o'tkazish" so'rovini yuborishi mumkin.

Himoya: Django va boshqa frameworklar **CSRF Token** tizimidan foydalanadi. Har bir POST so'rov bilan birga server tomonidan generatsiya qilingan maxfiy token yuborilishi shart. Agar token bo'lmasa yoki noto'g'ri bo'lsa, server so'rovni rad etadi.

? 83. Linux-da "Gunicorn" va "Nginx" tandemining vazifasi nimada?



- **Gunicorn (WSGI Server):** Bu Python kodini ishga tushiruvchi va HTTP so'rovlarni Python ob'ektlariga aylantiruvchi serverdir. Lekin u static fayllar (rasm, CSS) bilan ishlashda va xavfsizlikda kuchsiz.

- **Nginx (Reverse Proxy):** Bu tashqi dunyodan kelayotgan barcha so'rovlarni birinchi bo'lib kutib oladi. U static fayllarni juda tez beradi, so'rovlarni keshlaydi va yuklamani bir nechta Gunicorn serverlariga taqsimlaydi (Load Balancing).

☆ 84. "Horizontal Scaling" va "Vertical Scaling" farqi nimada?



- **Vertical Scaling:** Bu mavjud serverning quvvatini oshirish (protsessorni kuchaytirish, RAM qo'shish). Bu usulning chegarasi bor (hardware limit).

- **Horizontal Scaling:** Bu bir xil quvvatdagi bir nechta serverlarni (node) parallel ravishda qo'shish. Bu usul cheksiz kengayish imkonini beradi va asosan Microservices arxitekturasida qo'llaniladi.

? 85. Python ilovalarini Linux-da `systemd` orqali boshqarishning afzalligi nimada?

 `systemd` — bu Linux tizimidagi servis menedjeri. U orqali Python dasturini "servis" (daemon) sifatida sozlash mumkin. **Afzalligi:** Agar server o'chib yonsa yoki dastur kutilmaganda xatolik bilan to'xtab qolsa, `systemd` uni avtomatik ravishda qayta ishga tushiradi (autorestart). Shuningdek, loglarni yig'ishni osonlashtiradi.

☆ **86. SSH kalitlari (SSH Keys) orqali serverga ulanish parolli ulanishdan ko'ra nega xavfsizroq?**

 Parol osonlikcha o'g'irlanishi yoki "brute-force" (taxmin qilish) orqali topilishi mumkin. SSH kalitlari esa murakkab matematik kriptografiyaga (Public va Private key) asoslangan. Private key faqat sizning kompyuteringizda bo'ladi va uni taxmin qilishning iloji yo'q. Bu serverni ruxsatsiz kirishlardan ishonchli himoya qiladi.

❓ **87. CI/CD (Continuous Integration / Continuous Deployment) nima?**

 Bu kodni avtomatik ravishda tekshirish va serverga joylash jarayoni.

- **CI:** Har safar yangi kod yozib GitHub-ga yuklaganingizda, testlar avtomatik ishga tushadi.
- **CD:** Agar testlar muvaffaqiyatli o'tsa, yangi kod avtomatik ravishda (dasturchi aralashuvisiz) real serverga (Production) joylanadi.

☆ **88. Cloud platformalarda (AWS, Google Cloud) "Serverless" (masalan, AWS Lambda) nima?**

 Serverless — bu dasturchi serverni boshqarishi (OS yangilash, xotira ajratish) shart bo'lmagan model. Siz faqat kodni (funksiyani) yuklaysiz, cloud provayder esa uni faqat kerak bo'lgan paytda (masalan, API so'rov kelganda) ishga tushiradi. Siz faqat kod ishlagan vaqt (millisekundlar) uchun pul to'laysiz.

❓ **89. "SQL Injection" va "XSS" (Cross-Site Scripting) farqi nimada?**



- **SQL Injection:** Ma'lumotlar bazasiga qaratilgan hujum. Maqsad — bazadagi ma'lumotlarni o'g'irlash yoki o'chirish.
- **XSS:** Foydalanuvchi brauzeriga qaratilgan hujum. Maqsad — saytga zararli JavaScript kodini kiritish va boshqa foydalanuvchilarning sessiya cookie-fayllarini yoki ma'lumotlarini o'g'irlash.

☆ **90. Docker-da "Image" va "Container" o'rtasidagi farq nimada?**



- **Image (Tasvir):** Bu dasturning ishlashi uchun kerakli bo'lgan barcha fayllar, kutubxonalar va sozlamalarning "muzlatilgan" nusxasi (shablon). U o'zgarmasdir.

- **Container (Konteyner):** Bu Image-ning jonli, ishlayotgan nusxasi. Bitta Image asosida bir vaqtning o'zida o'nlab bir-biridan mustaqil konteynerlarni ishga tushirish mumkin.

❓ **91. PyQt ilovalarida tarmoq bilan ishlashda (Socket) asosiy muammo nimada va u qanday hal qilinadi?**

📖 Asosiy muammo — **Bloklanish (Blocking)**. Standart `socket.recv()` yoki `socket.connect()` metodlari ma'lumot kelgunicha dastur ishini to'xtatib qo'yadi. PyQt'da esa asosiy oqim (Main Thread) doimiy ravishda interfeysni yangilab (render qilib) turishi kerak. Agar socket bloklansa, interfeys qotib qoladi.

Yechim: Socket amallarini alohida `QThread` oqimiga o'tkazish yoki PyQt'ning o'zini asinxron `QtNetwork` modulidan foydalanish.

★ **92. PyQt'da QThread va Socket birgalikda qanday ishlatiladi?**

📖 Socket amallari (masalan, serverdan xabar kutish) `QThread` ichidagi `run()` metodida bajariladi. Serverdan ma'lumot kelganda, oqim interfeysni to'g'ridan-to'g'ri o'zgartira olmaydi. Buning o'rniga, oqim maxsus **Signal** yuboradi, asosiy GUI oqimi esa bu signalni qabul qilib (Slot), ma'lumotni ekranga chiqaradi.

❓ **93. QtNetwork modulining an'anaviy Python socket kutubxonasidan afzalligi nimada?**

📖 `QtNetwork` (masalan, `QTcpSocket`) asinxron ishlaydi va PyQt'ning **Event Loop** (hodisalar sikli) bilan to'liq integratsiyalashgan. Bunda alohida oqimlar (threading) shart emas; ma'lumot kelganda socket o'zi `readyRead` signalini beradi. Bu kodni soddaroq va boshqarishni osonroq qiladi.

★ **94. PyQt ilovasida Socket orqali rasm yoki katta fayl yuborishda nimalarga e'tibor berish kerak?**

📖 Fayllar baytlar ko'rinishida uzatiladi. Eng muhimi — **Data Chunking** (ma'lumotni bo'laklash). Katta fayllarni birdaniga yuborib bo'lmaydi. Odatda, birinchi bo'lib fayl nomi va hajmi (header) yuboriladi, so'ngra qabul qiluvchi tomon `while` siklida barcha baytlar yetib kelgunicha ma'lumotni yig'ib boradi.

❓ **95. TCP va UDP socketlari o'rtasidagi farq PyQt dasturlari misolida qanday ko'rinadi?**



- **TCP (QTcpSocket):** Ishonchli aloqa. Masalan, chat dasturi yoki fayl almashish uchun. Ma'lumot yo'qolmasligi kafolatlanadi.

- **UDP (QUdpSocket):** Tezkor, lekin kafolatlanmagan. Masalan, video-striming yoki ovozli aloqa uchun. Ba'zi kadrlar yo'qolsa ham, aloqa davom etaverishi muhimroq.

☆ **96. PyQt ilovasida bir vaqtning o'zida ham Server, ham Client rejimida ishlash mumkinmi?**

📖 Ha. Buning uchun bitta dastur ichida ham `QTcpServer` (ulanishlarni kutish uchun), ham `QTcpSocket` (boshqa serverga ulanish uchun) obyektlarini yaratish kerak. Bu asosan **P2P (Peer-to-Peer)** tarmoqlarida yoki murakkab boshqaruv tizimlarida qo'llaniladi.

❓ **97. Socket orqali kelgan JSON ma'lumotni PyQt interfeysiga (masalan, `QTableWidget`) qanday chiqariladi?**

📖 Kelgan baytlar `json.loads()` yordamida lug'atga o'giriladi. Keyin signal orqali GUI oqimiga yuboriladi. GUI qismida esa `setItem()` metodlari yordamida jadval qatorlari dinamik ravishda to'ldiriladi. Har bir yangi ma'lumot uchun `insertRow()` qilish interfeysni jonli ko'rsatadi.

☆ **98. Socket ulanishi uzilib qolganda PyQt interfeysida buni qanday aniqlash mumkin?**

📖 `QTcpSocket` ob'ektining `disconnected` degan maxsus signali bor. Bu signalga ma'lum bir funksiyani bog'lab qo'yish kerak. Ulanish uzilishi bilan interfeysdagi holat belgisi (status icon) qizarishi yoki foydalanuvchiga "Tarmoq bilan aloqa yo'q" degan xabarnoma (`QMessageBox`) ko'rsatilishi kerak.

❓ **99. "Select" yoki "Selectors" moduli PyQt bilan ishlashda kerakmi?**

📖 Agar siz standart Python socketidan foydalansangiz va ko'p sonli ulanishlarni bitta oqimda boshqarmoqchi bo'lsangiz, `selectors` kerak bo'lishi mumkin. Biroq, PyQt muhitida `QTcpServer` ishlatish afzalroq, chunki u har bir yangi ulanish uchun avtomatik ravishda `newConnection` signalini beradi.

☆ **100. PyQt'da xavfsiz socket (SSL/TLS) muloqotini qanday tashkil qilish mumkin?**

📖 Buning uchun `QtNetwork` modulidagi `QSslSocket` klassidan foydalaniladi. U oddiy `QTcpSocket` kabi ishlaydi, lekin ma'lumotlarni shifrlash

uchun sertifikatlar bilan ishlay oladi. Bu bank ilovalari yoki shaxsiy ma'lumotlar uzatiladigan tizimlar uchun xavfsizlikni ta'minlaydi.

? 101. `QDataStream` nima va u socket dasturlashda qanday yordam beradi?

 `QDataStream` — bu ma'lumotlarni platformaga bog'liq bo'lmagan (binary) formatda ketma-ketlashtirish (serialization) uchun ishlatiladi. Socket orqali murakkab obyektlarni yuborayotganda, bitlar va baytlar tartibi (endianness) muammo tug'dirmasligi uchun `QDataStream`dan foydalanish eng ishonchli usuldir.

☆ 102. PyQt ilovasida "Heartbeat" yoki "Keep-alive" mexanizmi socket uchun nega kerak?

 Ba'zida tarmoq uzilib qolsa-da, socket ulanish ochiq deb o'ylashda davom etishi mumkin (zombi ulanish). Buni oldini olish uchun server va mijoz har 5-10 soniyada bir-biriga kichik signal yuborib turadi. Agar signal kelmasa, PyQt `QTimer` yordamida ulanishni yopiq deb e'lon qiladi va qayta ulanishga urinadi.

? 103. `QTcpServer` bir vaqtda 100 ta mijozni qabul qilishi uchun qanday yondashuv qo'llaniladi?

 `QTcpServer`da har bir yangi ulanish uchun `incomingConnection(socketDescriptor)` metodi chaqiriladi. Har bir yangi mijoz uchun alohida `QThread` yaratish va socketni o'sha oqimga `moveToThread()` orqali o'tkazish kerak. Bu bitta mijozning og'ir so'rovi boshqalarini kutib qolishiga yo'l qo'ymaydi.

☆ 104. PyQt'da `QNetworkAccessManager` va `QTcpSocket` farqi nimada?

 `QNetworkAccessManager` — yuqori darajali API bo'lib, asosan HTTP/HTTPS so'rovlar (REST API) bilan ishlash uchun. `QTcpSocket` esa past darajali (low-level) muloqot bo'lib, o'z xususiy protokollaringizni (masalan, binary data transfer) yaratish uchun ishlatiladi.

? 105. Socket orqali xabar yuborishda `write()` va `flush()` metodlarining farqi?

 `write()` xabarni tizimning ichki buferiga yozadi, lekin u darhol jo'natilmasligi mumkin. `flush()` esa buferni darhol bo'shatib, ma'lumotni tarmoq orqali yuborishga majbur qiladi. PyQt'da asinxron ishlayotganda `flush()` ishlatish xabar yetib borishini kafolatlaydi.

☆ 106. PyQt'da asinxron socketlar bilan ishlashda "Signal Overloading" qanday oldini olinadi?

📖 Agar server juda tez ma'lumot yuborsa, GUI oqimi signallarga ko'milib qolishi mumkin. Buni oldini olish uchun ma'lumotlarni buferga yig'ib, faqat ma'lum bir vaqt oralig'ida (QTimer yordamida) interfeysni yangilash yoki "Throttling" usulidan foydalanish kerak.

? 107. QUdpSocket yordamida "Broadcast" xabar yuborish PyQt'da qanday amalga oshiriladi?

📖 UDP orqali tarmoqdagi barcha kompyuterlarga xabar yuborish uchun `writeDatagram()` metodida manzil sifatida `QHostAddress.Broadcast` (255.255.255.255) ishlatiladi. Bu asosan mahalliy tarmoqdagi boshqa serverlarni avtomatik qidirib topish uchun qulay.

☆ 108. Socket dasturini debug qilish uchun qaysi tashqi vositalar tavsiya etiladi?

📖 Dastur ichidagi `print()` lardan tashqari, **Wireshark** (tarmoq paketlarini tahlil qilish uchun) va **Packet Sender** (soxta paketlar yuborib ko'rish uchun) vositalari socket dasturchilarining asosiy quroli hisoblanadi.

? 109. PyQt'da IP manzilni validatsiya qilish (to'g'riligini tekshirish) uchun qaysi klass bor?

📖 `QHostAddress` klassidan foydalanish tavsiya etiladi. U berilgan satrning haqiqiy IPv4 yoki IPv6 manzil ekanligini, yoki domen nomi ekanligini avtomatik tekshirib beradi.

☆ 110. Socket orqali yuborilayotgan ma'lumotlarni qanday siqish (compress) mumkin?

📖 Python'ning `zlib` yoki `gzip` kutubxonalari yordamida baytlarni siqish, keyin esa socketga yozish mumkin. Qabul qiluvchi tomon `decompress` qilib oladi. Bu ayniqsa katta hajmli tekstli ma'lumotlarni yuborishda trafikni 80-90% gacha tejaydi.

4. Sun'iy intellekt

Atamalar va izohlar

Atamalar	Izohlar
ML	Machine Learning (Mashinali o'rganish) — ma'lumotlar asosida o'zi o'rganuvchi algoritmlar to'plami.
EDA	Exploratory Data Analysis (Ma'lumotlarning dastlabki tahlili) — ma'lumotlar sifatini va qonuniyatlarini o'rganish jarayoni.
NaN	Not a Number (Raqam emas) — ma'lumotlar bazasida bo'sh qolgan yoki yetishmayotgan qiymatlarni ifodalovchi belgi.
OHE	One-Hot Encoding — kategorik (matnli) ma'lumotlarni 0 va 1 lardan iborat ustunlarga o'tkazish usuli.
SMOTE	Synthetic Minority Over-sampling Technique — nomutanosib ma'lumotlarda kam sonli klassni sun'iy ko'paytirish algoritmi.
PCA	Principal Component Analysis (Asosiy komponentlar tahlili) — ma'lumotlar ustunlari sonini ularning mazmunini saqlagan holda kamaytirish usuli.
SVM	Support Vector Machines (Tayanch vektorlar mashinasi) — ma'lumotlarni maksimal chegara (margin) bilan ajratuvchi klassifikatsiya algoritmi.
KNN	K-Nearest Neighbors (K-eng yaqin qo'shnilar) — yangi obyektning uning eng yaqin qo'shnilariga qarab toifalovchi algoritm.
MAE	Mean Absolute Error (O'rtacha mutloq xatolik) — bashorat va haqiqat orasidagi farqning o'rtachasi.
MSE	Mean Squared Error (O'rtacha kvadratik xatolik) — xatoliklarni kvadratga oshirib hisoblash usuli (katta xatolarni ko'proq jazolaydi).
R2 Score	R-squared — regressiya modeli ma'lumotlarga qanchalik mos tushganini ko'rsatuvchi koeffitsiyent.
Overfitting	Modelning ma'lumotlarni yodlab olishi va yangi ma'lumotlarda yomon natija ko'rsatishi.
Underfitting	Modelning juda sodda bo'lib, ma'lumotlardagi qonuniyatlarni umuman tushunmasligi.
Bias	Modelning haqiqiy javobdan qanchalik og'ishi (sodalik belgisi).
Variance	Modelning turli ma'lumotlar to'plamida natijasini keskin o'zgartirishi (murakkablik belgisi).
Regularization	Model murakkabligini jazolash orqali overfittingni oldini olish usuli
Hyperparameter	Model o'qishni boshlashidan oldin dasturchi tomonidan qo'lda beriladigan sozlamalar.
Fold	Cross-validation jarayonida ma'lumotlar bo'linadigan har bir bo'lak (segment).
Feature	Modelga kiruvchi ma'lumot (ustun), bashorat qilish uchun sababchi omil.
Label	Model topishi kerak bo'lgan yakuniy natija yoki maqsadli o'zgaruvchi.
DL	Deep Learning (Chuqur o'rganish) — ko'p qatlamli neyron tarmoqlari yordamida o'rganish sohasi.
ANN	Artificial Neural Network (Sun'iy neyron tarmog'i) — inson miyasidan ilhomlangan algoritmlar tizimi.
CNN	Convolutional Neural Network (Konvolyutsion neyron tarmog'i) — asosan rasmlarni tahlil qilish uchun mo'ljallangan tarmoq turi.
RNN	Recurrent Neural Network (Rekurrent neyron tarmog'i) — ketma-ket ma'lumotlar (matn, ovoz) bilan ishlovchi "xotirali" tarmoq.
LSTM	Long Short-Term Memory — RNN'ning uzoq muddatli xotirani saqlashga ixtisoslashgan turi.

ReLU	Rectified Linear Unit — manfiy sonlarni 0 ga tenglovchi eng mashhur aktivizatsiya funksiyasi.
GAN	Generative Adversarial Network — bir-biri bilan raqobatlashuvchi ikkita tarmoq (Generator va Discriminator) tizimi.
SGD	Stochastic Gradient Descent — vaznlarni har bir ma'lumot namunasi bo'yicha tezkor yangilash algoritmi.
Adam	Adaptive Moment Estimation — eng keng tarqalgan va samarali optimallashtirish algoritmi.
Epoch	Barcha o'quv ma'lumotlarining tarmoq orqali bir marta to'liq o'tib chiqishi.
Batch	Model vaznlarini yangilashdan oldin tarmoq ko'radigan ma'lumotlar bo'lagi (masalan, 32 ta rasm).
Backpropagation	Xatolikni chiqishdan kirishga qarab zanjir bo'ylab qaytarib, vaznlarni yangilash jarayoni.
Dropout	O'qitish davomida tasodifiy neyronlarni "o'chirib turish" orqali overfittingdan himoyalash usuli.
Pooling	Rasmdagi pikseller sonini kamaytirish (siqish) jarayoni (masalan, Max Pooling).
Activation Function	Neyron signalini keyingi qatlamga uzatish yoki uzatmaslikni belgilovchi matematik funksiya.
Weights	Vaznlar — kiruvchi ma'lumotning natijaga ta'sir kuchini belgilovchi raqamlar.
Bias (NN)	Og'ish — neyron chiqishini o'zgartirish imkonini beruvchi qo'shimcha parametr.
Gradient	Funksiya (xatolik) o'zgarishining yo'nalishi va tezligini ko'rsatuvchi hosila.
Learning Rate	Modelning xatolardan dars olish tezligi (qadam o'lchami).
GPU/TPU	Graphics/Tensor Processing Unit — neyron tarmoqlarni tezkor hisoblash uchun mo'ljallangan qurilmalar.
NLP	Natural Language Processing (Tabiiy tilni qayta ishlash) — matn va ovozni tahlil qilish sohasi.
LLM	Large Language Model (Katta til modeli) — milliardlab parametrlarga ega matn generatsiya qiluvchi model (masalan, GPT-4).
GPT	Generative Pre-trained Transformer — keyingi so'zni bashorat qilishga asoslangan generativ model.
BERT	Bidirectional Encoder Representations from Transformers — matnni ikki tomonlama tushunuvchi model.
TF-IDF	Term Frequency-Inverse Document Frequency — so'zning matn ichidagi ahamiyatini o'lchovchi statistika.
BPE	Byte Pair Encoding — so'zlarni sub-so'z (bo'lak)larga bo'lish orqali tokenizatsiya qilish usuli.
RAG	Retrieval-Augmented Generation — AI javob berishdan oldin tashqi bazadan ma'lumot qidiruvchi texnologiya.
RLHF	Reinforcement Learning from Human Feedback — inson baholari asosida AI'ni odobli va foydali qilish usuli.
NER	Named Entity Recognition — matn ichidan ismlar, joylar va sanalarni ajratib olish.
BLEU	Bilingual Evaluation Understudy — mashina tarjimasini baholash metriki.
Prompt	Foydalanuvchi tomonidan AI'ga beriladigan buyruq yoki so'rov matni.
Context Window	Model bir vaqtda "xotirasida" saqlab tura oladigan tokenlar (matn) hajmi.
Token	Matnning AI uchun bo'lingan eng kichik birligi (so'z yoki bo'g'in).
Embedding	So'z yoki gapning raqamli ko'p o'lchamli vektor ko'rinishidagi ifodasi.
Self-Attention	Matndagi har bir so'zning boshqa so'zlar bilan bog'liqligini hisoblash mexanizmi.
Fine-tuning	Tayyor modelni maxsus soha ma'lumotlarida (masalan, tibbiyot) qo'shimcha o'qitish.
Hallucination	Modelning mavjud bo'lmagan, yolg'on ma'lumotni to'qib chiqarishi.

Stemming	Soʻzning qoʻshimchasini shunchaki kesib tashlash orqali asosini topish.
Lemmatization	Soʻzning lugʻatdagi grammatik asosiy shaklini (lemma) topish.
Zero-shot	Modelga hech qanday misol koʻrsatmasdan buyruq berish orqali natija olish.
CV	Computer Vision (Kompyuter koʻrishi) — tasvirlarni tahlil qilish va tushunish texnologiyasi.
YOLO	You Only Look Once — obyektlarni real vaqt rejimida oʻta tez aniqlovchi algoritmi.
IoU	Intersection over Union — model bashorati va haqiqiy obyekt oʻrni qanchalik ustma-ust tushishini oʻlchovchi koeffitsiyent.
OCR	Optical Character Recognition — rasmdagi yozuvlarni matn formatiga oʻtkazish.
NMS	Non-Maximum Suppression — bitta obyekt ustidagi takroriy toʻrtburchaklarni oʻchirib, eng yaxshisini qoldirish.
MLOps	Machine Learning Operations — ML modellarini ishlab chiqarishga (production) joylash va boshqarish madaniyati.
CI/CD/CT	Continuous Integration / Deployment / Training — avtomatik testlash, yuklash va qayta oʻqitish zanjiri.
DVC	Data Version Control — maʼlumotlar va modellarning versiyalarini boshqarish tizimi.
API	Application Programming Interface — modelning tashqi dunyo (sayt, ilova) bilan muloqot qilish interfeysi.
Edge AI	AI modelini bulutli serverda emas, balki qurilmaning (telefon, kamera) oʻzida ishlatish.
Data Drift	Vaqt oʻtishi bilan kiruvchi maʼlumotlar xarakterining oʻzgarishi (model eskirishi).
ONNX	Open Neural Network Exchange — modellarni turli framework'lar orasida oʻtkazish uchun umumiy format.
Anchor Box	Obyektlarni aniqlashda ishlatiladigan oldindan tayyorlangan shakl shablonlari.
Segmentation	Tasvirdagi har bir pikselning qaysi obyektga tegishli ekanini aniqlash.
Pose Estimation	Inson tanasidagi boʻgʻinlar nuqtasini topish orqali uning holatini aniqlash.
Deployment	Tayyor modelni serverga yuklash va foydalanuvchilar uchun ochiq qilish.
Containerization	Modelni (masalan, Docker orqali) barcha kutubxonalari bilan birga mustaqil paketga aylantirish.
Model Decay	Modelning aniqligi vaqt oʻtishi bilan tabiiy ravishda pasayib borishi.
Inference	Tayyor oʻqitilgan modelning yangi maʼlumot boʻyicha bashorat qilish (ishlash) jarayoni.
XAI	Explainable AI (Tushuntirilishi mumkin boʻlgan AI) — model qarorlarini inson tushunadigan tilda izohlash texnikasi.

Savol-javoblar.

4.1 Mashinali oʻqitish va maʼlumotlar muhandisligi (ML & Data Engineering)

? 1. Sunʼiy Intellekt loyihasining muvaffaqiyati koʻproq algoritimga bogʻliqmi yoki maʼlumotlar sifatiga?



Zamonaviy AI olamida "Data-centric AI" (Maʼlumotga yoʻnaltirilgan AI) tamoyili ustuvor hisoblanadi. Bu shuni anglatadiki, hatto eng murakkab va zamonaviy neyron

tarmoqlari ham, agar ularga "shovqinli" (xato, noto'g'ri yoki yetarli bo'lmagan) ma'lumot berilsa, juda yomon natija ko'rsatadi.

Masalan, loyihaning 80% vaqti ma'lumotlarni yig'ish, tozalash va tayyorlashga sarflanadi. Sifatli va toza ma'lumotlar to'plami bo'lsa, hatto oddiy mantiqiy algoritmlar ham yuqori aniqlik berishi mumkin. Shuning uchun loyihani boshlashda modelni tanlashdan ko'ra, ma'lumotlarning reprezentativligiga (haqiqatni aks ettirishiga) e'tibor qaratish shart.

☆ 2. "Correlation" (Korrelyatsiya) va "Causation" (Sababiyat) o'rtasidagi farqni real biznes misolida tushuntiring.



Korrelyatsiya — bu ikki o'zgaruvchining bir vaqtda o'zgarishi. Sababiyat — birining o'zgarishi ikkinchisining o'zgarishiga bevosita sabab bo'lishidir.

Real misol: Ma'lumotlar shuni ko'rsatishi mumkinki, odamlar ko'p muzqaymoq sotib olganda, quyoshdan himoyalovchi ko'zoynaklar sotuvi ham oshadi. Bu yerda korrelyatsiya bor. Lekin muzqaymoq yeyish ko'zoynak sotib olishga sabab bo'lmaydi. Ularning ikkalasiga ham uchinchi omil — **issiq havo** sababchidir. Agar model sababiyatni tushunmasa, u noto'g'ri mantiqiy bog'liqliklarni o'rganib oladi va bashorat qilishda jiddiy xatolarga yo'l qo'yadi.

❓ 3. Ma'lumotlar bazasidagi "Missing Values" (Yetishmayotgan qiymatlar)ni to'ldirishda qanday strategiyalar mavjud?



Ma'lumotlar bo'sh qolgan bo'lsa, ularni to'g'ri to'ldirish modelning "xolisligini" saqlash uchun juda muhim:

1. **O'chirish:** Agar ma'lumotlar juda ulkan bo'lsa va bo'sh qatorlar soni kam bo'lsa (5% dan kam), ularni shunchaki o'chirib yuborish mumkin.

2. **O'rtacha (Mean) yoki Mediana (Median) bilan to'ldirish:** Sonli ustunlar uchun ishlatiladi. Agar ma'lumotlarda o'ta katta farqli sonlar (outliers) bo'lsa, mediana ishonchliroq hisoblanadi.

3. **Moda (Mode):** Kategorik (so'zli) ustunlar uchun eng ko'p takrorlangan qiymatni qo'yish.

4. **Bashorat qilish:** Bo'sh qolgan joyni to'ldirish uchun kichik bir ML modelidan foydalanish (masalan, KNN imputer). Bu eng aniq, lekin hisoblash jihatidan qimmat usuldir.

☆ 4. "Data Leakage" (Ma'lumotlar sizib chiqishi) muammosi nima va u intervyularda nega ko'p so'raladi?



Data Leakage — bu modelni o'qitish (training) jarayonida, aslida test vaqtida mavjud bo'lmisligi kerak bo'lgan ma'lumotlarning "sizib" kirib qolishi. Bu modelning real hayotda ishlamaydigan, "soxta" va hayratlanarli darajada yuqori aniqlik berishiga sabab bo'ladi.

Misol: Kasallikni aniqlovchi modelga bemorning shifoxonadagi ID raqamini ham berish. Agar shifoxona og'ir bemorlarga kichikroq ID raqam bersa, model kasallikni emas, raqamni taniy boshlaydi.

Oldini olish: Ma'lumotlarni o'qitish va test to'plamlariga bo'lishni hamma narsadan oldin bajarish kerak. Hech qachon butun bazani birga qayta ishlash (scaling, normalization) mumkin emas.

❓ 5. "One-Hot Encoding" va "Label Encoding"ni qaysi holatlarda tanlash kerak?



AI faqat raqamlar bilan ishlagani uchun so'zlarni raqamga o'tkazish kerak:

- **Label Encoding:** Agar so'zlar orasida tabiiy tartib yoki daraja bo'lsa. Masalan: ["Past", "O'rta", "Yuqori"] -> [1, 2, 3]. Bu yerda 3 ning 1 dan kattaligi modelga mantiqiy ma'no beradi.

- **One-Hot Encoding:** Agar tartib bo'lmasa. Masalan: ["Toshkent", "London", "Nyu-York"]. Agar bularga 1, 2, 3 bersak, model "Nyu-York Toshkentdan 3 baravar katta yoki yaxshi" deb o'ylashi mumkin. One-Hot har bir shahar uchun alohida ustun yaratib, 0 yoki 1 qo'yadi (masalan, London uchun: Toshkent: 0, London: 1, NY: 0).

☆ 6. "Imbalanced Data" (Nomutanosib ma'lumotlar) muammosi va uni hal qilish usullari.



Bu muammo bir sinf vakillari (masalan, sog'lom odamlar) 99% ni, ikkinchisi (kasallar) 1% ni tashkil qilganda yuz beradi. Model shunchaki "hamma sog'lom" desa ham 99% aniqlikni qo'lga kiritadi, lekin aslida u foydasizdir.

Yechimlar:

1. **Oversampling (SMOTE):** Kam sonli ma'lumotlarni sun'iy ko'paytirish.

2. **Undersampling:** Ko'p sonli ma'lumotlarni kamaytirish.

3. **Class Weights:** Modelga kam uchraydigan klassni topa olmagan uchun kattaroq "jarima" belgilash. Bu modelni "kamchilikka" ko'proq diqqat qilishga majbur qiladi.

🔗 7. ML loyihalarida "Feature Selection" nima uchun majburiy hisoblanadi?



Feature Selection — bu modelga beriladigan ma'lumotlar ichidan eng muhimlarini tanlab olish.

Nega kerak:

1. **Model tezligi:** Kamroq ma'lumot — tezroq o'qitish.

2. **Overfittingni kamaytirish:** Keraksiz "shovqinli" ustunlar (masalan, mijozning ismi) modelni adashtiradi.

3. **Interpretatsiya:** Kamroq o'zgaruvchi bilan ishlash inson uchun model natijasini tushunishni osonlashtiradi.

☆ 8. "Regularization" (L_1 va L_2) qanday qilib modelning "hadidan oshib ketishi"ni (overfitting) to'xtatadi?



Regularizatsiya — bu modelning murakkabligi uchun unga "soliq" solishdir.

- **L_2 Regularization (Ridge):** Vaznlarni (weights) nolga yaqinlashtiradi, lekin nol qilmaydi. Bu barcha ustunlarni biroz bo'lsa-da hisobga olishni ta'minlaydi.

- **L_1 Regularization (Lasso):** Keraksiz deb topilgan ustunlarning vaznlarini mutlaqo nolga tenglashtiradi. Bu usul avtomatik ravishda eng muhim xususiyatlarni tanlab beradi va keraksizlarini "o'chiradi". Bu modelning umumlashtirish qobiliyatini oshiradi.

🔗 9. "Curse of Dimensionality" (O'lchovlar la'nati) tushunchasini sodda tilda tushuntiring.



Tasavvur qiling, siz 1 metrlik ipda (1D) biror nuqtani qidirayapsiz — bu oson. Keyin sizga 1 kvadrat metrlik maydonda (2D) o'sha nuqtani topish buyurildi — qiyinroq. Agar 1 kub metrli xonada (3D) bo'lsa — yanada qiyin.

Ma'lumotlar bazasida ustunlar (dimensions) soni oshib borsa, nuqtalar (ma'lumotlar) orasidagi masofa shunchalik uzoqlashib ketadiki, model ular orasida qonuniyat topa olmay qoladi. Buning yechimi — PCA (O'lchovlarni qisqartirish).

☆ 10. PCA (Principal Component Analysis) algoritmi qanday mantiq asosida ishlaydi?



PCA — bu ma'lumotlar to'plamidagi eng muhim o'zgarishlarni (variance) saqlab qolgan holda, ustunlar sonini kamaytirish usuli.

Misol: Sizda mashinaning balandligi, eni, g'ildirak o'lchami va rangi bor. PCA bu ma'lumotlarni birlashtirib, mashinaning "umumiy hajmi" degan bitta yangi ustunga aylantirishi mumkin. Bu jarayonda biz ba'zi mayda tafsilotlarni (rang kabi) yo'qotamiz, lekin mashinaning eng asosiy xususiyatini saqlab qolamiz. Bu hisoblashni bir necha barobar tezlashtiradi.

🔗 11. "Random Forest" modelida `n_estimators` parametrini qanday tanlash kerak?



`n_estimators` — bu ansamblidagi qarorlar daraxtlari soni.

1. Daraxtlar soni ko'paygani sari model barqarorroq bo'ladi va xatolik kamayadi.
2. Lekin ma'lum bir nuqtadan keyin (masalan, 500 ta daraxt) aniqlik oshishi to'xtaydi va faqat hisoblash vaqti (CPU sarfi) ko'paya boshlaydi.
3. Shuning uchun "oltin o'rtaliq"ni topish uchun `GridSearch` kabi usullardan foydalaniladi.

☆ 12. "Support Vector Machines" (SVM) algoritmidagi "Kernel Trick" nima uchun kerak?



Ba'zan ma'lumotlar chiziqli emas (masalan, bir guruh nuqtalar markazda, ikkinchisi esa atrofida aylana bo'lib turibdi). Ularni to'g'ri chiziqli bilan ajratish imkonsiz.

Kernel Trick: Bu matematik usul bo'lib, ma'lumotlarni sun'iy ravishda yuqoriroq o'lchamli fazoga (masalan, 2D dan 3D ga) "ko'taradi". U yerda nuqtalar tekislik bilan osongina ajraladi. Keyin natija yana pastga qaytariladi. Bu murakkab chegarali ma'lumotlarni aniq klassifikatsiya qilish imkonini beradi.

🔗 13. Nega "Logistic Regression" klassifikatsiya uchun ishlatilsa-da, nomi "Regressiya"?



Chunki uning asosi "Linear Regression" (chiziqli regressiya)ga tayanadi. U avval sonli qiymatni (z) hisoblaydi, so'ngra olingan natijani Sigmoid funksiyasidan o'tkazadi. Sigmoid bu raqamni 0 va 1 oralig'idagi ehtimollikka aylantiradi (masalan, 0.85 yoki 85%). Agar ehtimollik 0.5 dan katta bo'lsa — 1 (musbat), kichik bo'lsa — 0 (manfiy) deb qabul qilinadi.

☆ 14. "Hyperparameter Tuning"da Grid Search va Random Search o'rtasidagi amaliy farq nimada?



- **Grid Search:** Siz bergan barcha parametrlar kombinatsiyasini bittalab, qat'iy tartibda tekshiradi. Agar 10 ta variant bersangiz, 100% eng yaxshisini topadi, lekin juda uzoq vaqt oladi.

- **Random Search:** Parametrlar fazosidan tasodifiy nuqtalarni tanlab tekshiradi.

Xulosa: Katta loyihalarda Random Search afzal, chunki u 90% holatlarda eng yaxshi natijaga Grid Search-dan 10 baravar tezroq erishadi.

? 15. "K-Means Clustering" algoritmi yangi guruhlarini qanday aniqlaydi?



U bosqichma-bosqich ishlaydi:

1. **Initsializatsiya:** Tasodifiy K ta markaz (centroids) tanlanadi.

2. **Biriktirish:** Har bir nuqta o'ziga eng yaqin markazga biriktiriladi.

3. **Yangilash:** Har bir guruhning o'rtacha markazi hisoblanadi va markaz o'sha yerga ko'chadi.

4. **Takrorlash:** Markazlar o'rnidan jilmay qolguncha yoki sikllar soni tugaguncha bu jarayon davom etadi. Natijada ma'lumotlar tabiiy guruhlariga bo'linadi.

? 16. Sun'iy intellekt (AI) va oddiy kompyuter dasturi (Algorithm) o'rtasidagi asosiy farq nimada?



Oddiy dastur — bu "Agar A bo'lsa, B ni bajar" degan qat'iy qoidalar to'plami. Dasturchi barcha shartlarni oldindan yozib chiqadi. Agar kutilmagan vaziyat bo'lsa, dastur xato beradi.

Sun'iy intellekt esa ma'lumotlar ichidan o'zi qonuniyatlarni qidirib topadi. Unga qoidalar berilmaydi, unga misollar beriladi.

- **Misol:** Spam-filtri. Oddiy dasturda "pul", "yutuq" soʻzlari boʻlsa oʻchir deb yoziladi. AI esa minglab spam xatlarni koʻrib, qaysi soʻzlar kombinatsiyasi shubhali ekanini oʻzi oʻrganib oladi va vaqt oʻtishi bilan "aqliroq" boʻlib boradi.

☆ **17. Mashinali oʻrganish (Machine Learning)da "Model" deganda nimani tushunish kerak?**



Model — bu algoritmni maʼlumotlar bilan oʻqitish natijasida hosil boʻlgan "bilim" yoki "formula"dir.

Buni shunday tasavvur qiling: Algoritm — bu boʻsh miya (oʻqish qobiliyati), Model — bu oʻqib boʻlgan, tajribaga ega boʻlgan miya. Modelga yangi maʼlumot berilsa, u oʻzining oʻtmishdagi tajribasiga tayanib bashorat qiladi. Matematik jihatdan bu juda murakkab $y = f(x)$ funksiyasidir, bu yerda x — kiruvchi maʼlumot, y — natija.

? **18. "Train set" (Oʻqitish toʻplami) va "Test set" (Sinov toʻplami) nega alohida boʻlishi shart?**



Bu talabaning imtihonga tayyorlanishiga oʻxshaydi.

- **Train set** — bu darslikdagi misollar. Talaba (model) ularni yechib oʻrganadi.
- **Test set** — bu imtihon savollari.

Agar biz modelni oʻzi koʻrgan (oʻqigan) maʼlumotlari bilan tekshirsak, u shunchaki javoblarni yodlab olgan boʻlishi mumkin. Haqiqiy bilimni tekshirish uchun unga oʻzi ilgari koʻrmagan maʼlumotlarni berish shart. Odatda maʼlumotlar 80% (oʻqish) va 20% (test) nisbatida boʻlinadi.

☆ **19. "Feature" (Xususiyat) va "Label" (Belgi/Javob) nima?**



Bular maʼlumotlar jadvalidagi ustunlardir:

- **Feature (X):** Bu bashorat qilish uchun ishlatiladigan maʼlumotlar (sabablar). Masalan, uy narxini aniqlashda: kvadrat metr, xonalar soni, manzili.
- **Label (Y):** Bu biz topmoqchi boʻlgan yakuniy natija. Yuqoridagi misolda: uyning narxi.

AI'ning vazifasi — Features (X) va Label (Y) oʻrtasidagi yashirin bogʻliqlikni topishdir.

🔗 20. Chiziqli regressiya (Linear Regression) nima va u qachon ishlatiladi?



Bu eng oddiy va fundamental AI algoritmidir. U ikki narsa o'rtasidagi chiziqli bog'liqlikni qidiradi.

Qachon ishlatiladi: Natija aniq bir son bo'lganda.

- Misollar: Ertaga havo harorati necha daraja bo'ladi? Aksiyalar narxi qanchaga ko'tariladi? Reklamaga 100 dollar tiksak, qancha daromad olamiz?

U ma'lumotlar orasidan eng to'g'ri o'tuvchi chiziqni chizishga harakat qiladi.

☆ 21. Klassifikatsiya (Classification) va Regressiya (Regression) farqi nimada?



Buni natija (output) turiga qarab ajratamiz:

- **Regressiya:** Natija sonli qiymat (uzluksiz). Masalan: narx, harorat, vazn.
- **Klassifikatsiya:** Natija guruh/kategoriya bo'lganda. Masalan: Bu rasmda itmi yoki mushuk? Bu xat spammi yoki yo'q? Mijoz bizdan ketadimi yoki qoladimi? (Ha/Yo'q).

🔗 22. "Normalization" (Normalizatsiya) va "Scaling" (Masshtablash) nega kerak?



Algoritmlar katta raqamlarni "muhimroq" deb o'ylashadi.

- Misol: Sizda "Yosh" (20 dan 80 gacha) va "Oylik daromad" (1 000 000 dan 50 000 000 gacha) degan ustunlar bor. AI daromad soni kattaligi uchun faqat unga e'tibor berishi mumkin.

Scaling barcha sonlarni bir xil oralikka (masalan, 0 dan 1 gacha) keltiradi. Natijada yosh ham, daromad ham algoritm uchun bir xil "vaznga" ega bo'ladi.

☆ 23. "Standard Deviation" (Standart og'ish) AI'da nimani bildiradi?



Bu ma'lumotlarning tarqoqligini ko'rsatadi.

- Agar standart og'ish kichik bo'lsa, barcha ma'lumotlar o'rtacha qiymatga yaqin (zich) joylashgan.
- Agar katta bo'lsa, ma'lumotlar juda turlicha (tarqoq).

AI uchun bu juda muhim, chunki u ma'lumotlar ichidagi "shovqin" yoki kutilmagan og'ishlarni (Outliers) aniqlashga yordam beradi.

❓ **24. "Outlier" (Ekstremal qiymat) nima va u modelga qanday zarar yetkazadi?**



Outlier — bu qolgan ma'lumlardan juda keskin farq qiluvchi nuqta.

- Misol: 10 nafar talabaning oylik xarajati 100 dollar atrofida, lekin bittasini 100 000 dollar.

Agar bu ma'lumotni tozalab tashlamasak, chiziqli regressiya o'sha bitta nuqtaga yetishaman deb o'z chizig'ini qiyshaytirib yuboradi va qolgan 10 kishi uchun bashorati noto'g'ri chiqadi.

★ **25. "K-Nearest Neighbors" (KNN) algoritmi qanday ishlaydi?**



Bu "Menga do'stingni ko'rsat, men senga kimgaligini aytaman" tamoyilida ishlaydi.

Yangi ma'lumot kelganda, algoritm unga eng yaqin turgan K ta qo'shnisini topadi. Agar 5 ta qo'shnidan 4 tasi "it" bo'lsa, demak bu yangi nuqta ham "it" deb xulosa qiladi. Bu juda sodda, lekin xotirani ko'p talab qiladigan algoritmdir.

❓ **26. "Decision Tree" (Qarorlar daraxti) algoritmi qanday qaror qabul qiladi?**



Bu algoritm xuddi "Ha/Yo'q" o'yiniga o'xshaydi. U ma'lumotlarni eng muhim xususiyatiga qarab bo'laklarga bo'lib boradi.

- Misol: Bank kredit berishni hal qilyapti. Birinchi savol: "Mijozning oyligi 5 mln dan ko'pmi?". Agar "Ha" bo'lsa, keyingi savol: "Mijozning qarzi bormi?".

Shu tariqa daraxt "barg"lariga (yakuniy qarorga) yetib boradi. Bu algoritm inson mantiqiga juda yaqin va tushunish oson, lekin u ma'lumotlarni haddan tashqari yodlab olishga (overfitting) moyil.

★ **27. Daraxtlarda "Entropy" va "Information Gain" nima uchun kerak?**



Daraxt ma'lumotni qaysi ustun bo'yicha bo'lishni (masalan, yoshi bo'yichami yoki jinsi?) qanday tanlaydi?

- **Entropy:** Bu "tartibsizlik" o'lchovi. Agar guruh ichida hamma narsa aralash bo'lsa (itlar va mushuklar teng), entropy yuqori bo'ladi.

- **Information Gain:** Bu ma'lum bir ustun bo'yicha bo'lgandan keyin entropy qanchalik kamayganini ko'rsatadi. Algoritm har doim eng ko'p "ma'lumot foydasi" (Information Gain) beradigan ustunni birinchi bo'lib tanlaydi.

🔗 28. "Ensemble Learning" (Ansambl o'rganish) nima?



Bu "bir boshdan ikki bosh yaxshi" tamoyili. Bitta kuchli modelni yaratish o'rniga, biz bir nechta modelni birlashtiramiz.

- **Misol:** Random Forest. U yuzlab kichik qarorlar daraxtlarini yaratadi va ularning umumiy "ovoziga" qarab yakuniy javobni chiqaradi. Agar bitta daraxt adashsa, qolganlari to'g'ri yo'lni ko'rsatadi. Bu modelning aniqligini va barqarorligini keskin oshiradi.

★ 29. "Bagging" va "Boosting" o'rtasidagi farq nimada?



Ikkalasi ham modellarni birlashtiradi, lekin usuli turlicha:

- **Bagging (Parallel):** Bir nechta mustaqil modellar bir vaqtda o'qitiladi va ularning o'rtacha natijasi olinadi (Masalan: Random Forest).

- **Boosting (Ketma-ket):** Modellar birin-ketin o'qitiladi. Har bir keyingi model oldingi model qilgan xatolarga ko'proq e'tibor beradi va ularni tuzatishga harakat qiladi (Masalan: XGBoost, CatBoost).

🔗 30. "Support Vector Machines" (SVM) dagi "Margin" (Chegara) nima?



SVM klassifikatsiya qilishda shunchaki chiziq chizmaydi. U ikki guruh nuqtalari orasidan eng keng "yo'l" (bo'shliq) qoldirishga harakat qiladi.

- **Margin:** Bu ikki guruhga eng yaqin turgan nuqtalar va ajratuvchi chiziq orasidagi masofa. Margin qanchalik keng bo'lsa, model shunchalik ishonchli bo'ladi, chunki u yangi keladigan ma'lumotlar uchun ko'proq "joy" qoldiradi.

★ 31. Neyron tarmoqlaridan oldin "SVM" nega eng kuchli algoritm hisoblangan?



Chunki u "Kernel Trick" yordamida chiziqli bo'lmagan murakkab ma'lumotlarni juda aniq ajrata olgan. Masalan, rasmda nuqtalar aylana shaklida bo'lsa, SVM ularni matematik yo'l bilan 3D fazoga "ko'tarib", tekislik bilan bo'la olgan. Bu o'sha davrda (90-yillar) kashfiyot edi.

🔗 32. "Naive Bayes" algoritmi nega "sodda" (naive) deb ataladi?



Chunki u ma'lumotlar ichidagi barcha xususiyatlar bir-biridan mutlaqo mustaqil deb hisoblaydi.

- **Misol:** Spam filtrida u "pul" so'zi bilan "yutuq" so'zi bir-biriga bog'liq emas deb o'ylaydi. Haqiqiy hayotda esa ular bog'liq bo'lishi mumkin. Shunga qaramay, bu algoritm juda tez va matnlarni klassifikatsiya qilishda (masalan, xatning mazmunini aniqlashda) hayratlanarli darajada yaxshi ishlaydi.

☆ 33. "Cross-Validation" (K-fold) nima va u nega oddiy bo'lishdan yaxshiroq?



Agar biz ma'lumotni faqat bir marta (80/20) bo'lsak, test qismiga "omadli" yoki "omadsiz" ma'lumotlar tushib qolishi mumkin.

K-fold: Ma'lumotni 5 qismga bo'lamiz. Birinchi marta 1-qismni test, qolganini o'qitish uchun olamiz. Ikkinchi marta 2-qismni test qilamiz va hokazo. Yakunda 5 ta natijaning o'rtachasini olamiz. Bu modelning har qanday ma'lumotda qanchalik barqaror ishlashini kafolatlaydi.

🔗 34. "Hyperparameters" va "Parameters" o'rtasidagi farq nimada?



- **Parameters:** Bu model o'qish jarayonida o'zi o'rganadigan qiymatlar (vaznlar, nishabliklar). Buni biz qo'lda o'zgartirmaymiz.

- **Hyperparameters:** Bu model o'qishni boshlashidan oldin dasturchi tomonidan belgilanadigan sozlamalar. Masalan: daraxtning chuqurligi, o'rganish tezligi (learning rate), qo'shnilar soni (\$K\$). Bularni to'g'ri sozlash model sifatini hal qiladi.

☆ 35. "Stochastic Gradient Descent" (SGD) oddiy Gradient Descentdan nimasi bilan farq qiladi?



- **Gradient Descent:** Har bir qadamda (vaznlarni yangilashda) bazadagi barcha 1 millionta ma'lumotni ko'rib chiqadi. Bu juda sekin.

- **SGD:** Har bir qadamda faqat bitta tasodifiy ma'lumotni ko'radi va shunga qarab vaznni o'zgartiradi. Bu juda tez, lekin yo'nalishi biroz "tartibsiz" bo'ladi. Zamonaviy neyron tarmoqlarida ikkalasining o'rtasi — "Mini-batch Gradient Descent" ishlatiladi.

🔗 36. "Confusion Matrix" (Chalkashlik matritsasi) dagi 4 ta asosiy qiymatni tushuntiring.



Tasavvur qiling, biz kasallikni aniqlayapmiz:

1. **True Positive (TP):** Kasalni — kasal dedi (To'g'ri).
2. **True Negative (TN):** Sog'lomni — sog'lom dedi (To'g'ri).
3. **False Positive (FP):** Sog'lomni — kasal dedi (Xato — soxta xavf).
4. **False Negative (FN):** Kasalni — sog'lom dedi (Xato — eng xavflisi, kasallik qolib ketdi).

☆ 37. "Precision" va "Recall" — qachon qaysi biri muhimroq?



- **Precision (Aniqlik):** "Men kasal deganlarimning nechitasi haqiqatda kasal?". Bu spam-filtrda muhim (muhim xat spamga tushib ketmasligi kerak).
- **Recall (Eslab qolish):** "Jami kasallarning nechtasini topa oldim?". Bu saratonni aniqlashda muhim (bitta ham kasal e'tibordan chetda qolmasligi kerak).

🔗 38. "F1-Score" nima va u nega kerak?



Agar bizda Precision 90% bo'lib, Recall 10% bo'lsa, modelimiz yomon. Bizga ikkalasi ham yuqori bo'lishi kerak. F1-Score — bu Precision va Recall-ning muvozanatlashgan (garmonik) o'rtachasidir. Agar ma'lumotlarimizda bir sinf vakillari juda kam bo'lsa (imbalanced data), F1-Score eng to'g'ri o'lchov hisoblanadi.

☆ 39. "Underfitting" qanday belgilar bilan namoyon bo'ladi va uni qanday tuzatish mumkin?



Underfitting — model darsni umuman o'qimagan talabaga o'xshaydi. U ham o'qitish to'plamida, ham test to'plamida yomon natija beradi.

Tuzatish: 1. Murakkabroq modelni tanlash (masalan, Chiziqlidan ko'ra Random Forest).

2. Ko'proq "feature" (xususiyatlar) qo'shish.

3. Regularizatsiyani (jarimani) kamaytirish.

🔗 40. "Clustering" (Klasterlash) algoritmlari qachon ishlatiladi?



Bu "Unsupervised" (nazorat qilinmaydigan) o'rganishdir. Bizda javoblar (label) yo'q, biz faqat o'xshashliklarni topmoqchimiz.

Ishlatilishi: * Mijozlarni guruhlariga bo'lish (marketing).

- O'xshash yangiliklarni guruhlash.
- Tasvirdagi ranglarni kamaytirish.

☆ 41. "Elbow Method" (Tirsak usuli) K-Means algoritmi uchun nega kerak?



K-Means-da biz K (guruhlar soni) ni oldindan berishimiz kerak. Uni qanday tanlaymiz?

Biz $K=1, 2, 3...$ qilib modelni o'qitamiz va xatolik (inertia) grafigini chizamiz. Grafik pastga tushib boradi va ma'lum bir nuqtada tirsak kabi bukiladi. Shu bukilingan nuqtadagi K qiymati eng optimal guruhlar soni hisoblanadi.

🔗 42. "Dimensionality Reduction" (O'lchovni qisqartirish) faqat PCA'mi?



Yo'q, boshqa usullar ham bor:

- **t-SNE:** Ma'lumotlarni 2D yoki 3D da vizualizatsiya qilish uchun juda kuchli usul.
- **LDA:** Klassifikatsiya uchun eng mos o'lchovlarni topadi.

Bu usullar 100 ta ustunli murakkab jadvalni 2 ta ustunga tushirib, grafikda ko'rish imkonini beradi.

☆ 43. "A/B Testing" ML modelini ishga tushirishda qanday rol o'ynaydi?



Yangi model laboratoriyada (test setda) yaxshi natija berishi mumkin, lekin real foydalanuvchilar bilan qanday ishlaydi?

Biz foydalanuvchilarning 5% iga yangi modelni, 95% iga eskisini ko'rsatamiz. Agar yangi model haqiqatda savdoni yoki foydalanishni oshirsa, keyin uni hamma uchun yoqamiz.

❓ 44. "Bias" (Og'ish) va "Variance" (O'zgaruvchanlik) o'rtasidagi farqni "Nishonga otish" misolida tushuntiring.



- **High Bias:** O'qlarning hammasi nishondan uzoqda, lekin bir joyda to'plangan (Model underfitting, juda sodda).
- **High Variance:** O'qlar nishon atrofida har tomonga tarqalib ketgan (Model overfitting, juda sezgir).
- **Ideal:** Ikkalasi ham past bo'lishi — hamma o'qlar markazda bo'lishi kerak.

☆ 45. "StandardScaler" va "MinMaxScaler" — qaysi birini tanlash kerak?



- **MinMaxScaler:** Ma'lumotlarni 0 va 1 oralig'iga tushiradi. Agar siz neyron tarmoq ishlatayotgan bo'lsangiz, bu yaxshi.
- **StandardScaler:** Ma'lumotlarni o'rtacha qiymati 0 va standart og'ishi 1 bo'lgan holatga keltiradi (Z-score). Agar ma'lumotlaringizda o'ta katta farqli sonlar (outliers) bo'lsa, bu usul barqarorroq ishlaydi.

❓ 46. "Imputation" nima va u nega xavfli bo'lishi mumkin?



Imputation — bu bo'shliqlarni (NaN) to'ldirish. Xavfi shundaki, biz mavjud bo'lmagan ma'lumotni "to'qib" chiqaramiz. Agar biz xato to'ldirsak, model noto'g'ri mantiqni o'rganadi.

- **Masalan:** Kambag'al odamlarning daromadi yashirilgan bo'lsa-yu, biz ularni "o'rtacha" boylik bilan to'ldirsak, model kambag'allik belgilarini taniy olmay qoladi.

☆ 47. "Overfitting"ni to'xtatishning 3 ta asosiy usuli.



1. **Ma'lumotni ko'paytirish:** Model qancha ko'p misol ko'rsa, shuncha yaxshi umumlashtiradi.
2. **Regularizatsiya:** Vaznlarni jazolash (L_1 , L_2).
3. **Modelni soddalashtirish:** Masalan, daraxt chuqurligini cheklash yoki neyronlar sonini kamaytirish.

❓ 48. "Label Encoding"ni "Jins" (Erkak/Ayol) ustuni uchun ishlatib bo'ladimi?



Ha, chunki bu yerda faqat 2 ta qiymat bor (0 va 1). Model uchun 1 ning 0 dan "kattaligi" muammo tug'dirmaydi. Lekin 3 tadan ko'p va tartibi yo'q ustunlar uchun "One-Hot Encoding" tavsiya etiladi.

★ 49. **"Correlation Matrix" (Korrelyatsiya matritsasi) dagi 1.0 va -1.0 nimani bildiradi?**



- **1.0:** Ikki o'zgaruvchi bir-biriga mukammal bog'liq (Biri oshsa, ikkinchisi ham xuddi shunday oshadi).
- **-1.0:** Teskari bog'liqlik (Biri oshsa, ikkinchisi xuddi shunday kamayadi).
- **0:** Ular orasida hech qanday bog'liqlik yo'q.

Agar ikki Feature orasida 1.0 bo'lsa, ulardan birini o'chirib yuborsak ham bo'ladi (Multicollinearity).

❓ 50. **Loyihani boshlashda nega har doim "Baseline Model" (Sodda model) yaratish kerak?**



Masalan, biz murakkab neyron tarmog'ini yaratmoqchimiz. Avval eng oddiy "Chiziqli regressiya"ni tekshiramiz. Agar u 80% aniqlik bersa, bizning neyron tarmog'imiz kamida 85-90% berishi shart. Agar neyron tarmog'i ham 80% bersa, demak ortiqcha vaqt va resurs sarflashning keragi yo'q — muammo modelda emas, ma'lumotda.

4.2. Chuqur o'rganish va neyron tarmoqlar (Deep Learning)

❓ 51. **Chuqur o'rganish (Deep Learning) oddiy Mashinali o'rganishdan (ML) nimasi bilan farq qiladi?**



Asosiy farq — ma'lumotlarni qayta ishlash usulida. Oddiy ML-da dasturchi modelga qaysi xususiyatlar (features) muhimligini o'zi ko'rsatishi kerak (Feature Engineering).

Deep Learning (Neyron tarmoqlar) esa ma'lumotlar ichidan eng muhim belgilarni o'zi avtomatik ravishda qidirib topadi. Masalan, rasmda mushukni tanish uchun ML-ga "quloq shakli", "mo'ylov uzunligi"ni kiritish kerak bo'lsa, Deep Learning minglab mushuk rasmlarini ko'rib, o'zi bu belgilarni o'rganib oladi. "Chuqur" (Deep) so'zi tarmoqdagi qatlamlar soni ko'pligini anglatadi.

☆ 52. Sun'iy neyron nima va u biologik neyronga qanchalik o'xshash?



Sun'iy neyron — bu matematik funksiya. U biologik neyrondan ilhomlangan:

1. **Kirish (Dendritlar):** Ma'lumotlarni qabul qiladi ($x_1, x_2, \dots$).
2. **Vaznlar (Sinapslar):** Har bir ma'lumotning muhimlik darajasi ($w_1, w_2, \dots$).
3. Yig'indi va Aktivizatsiya (Hujayra tanasi): Ma'lumotlar yig'iladi va ma'lum bir "bo'sag'a"dan o'tsa, signal keyingi neyronga uzatiladi.

Biologik neyron murakkabroq, lekin sun'iy neyron ham xuddi shunday elektr signali o'rniga raqamlarni uzatish orqali muloqot qiladi.

? 53. "Perceptron" nima?



Perceptron — bu eng oddiy va birinchi yaratilgan neyron tarmog'i modelidir. U faqat bitta neyrondan iborat bo'lib, ma'lumotlarni faqat ikki guruhga (masalan, "Ha" yoki "Yo'q") ajrata oladi. Perceptron faqat chiziqli bog'liqliklarni tushuna oladi. Zamonaviy tarmoqlar esa millionlab shunday perceptronlarning murakkab birikmasidan iborat.

☆ 54. Neyron tarmoqdagi "Hidden Layers" (Yashirin qatlamlar) vazifasi nima?



Neyron tarmog'i 3 qismdan iborat:

1. **Input Layer (Kirish):** Ma'lumotni qabul qiladi.
2. **Hidden Layers (Yashirin):** Ma'lumotdagi murakkab qonuniyatlarni o'rganadi. Birinchi yashirin qatlam oddiy chiziqlarni, keyingisi shakllarni, oxirgisi esa butun bir obyektни taniydi.
3. **Output Layer (Chiqish):** Yakuniy natijani beradi.

Yashirin qatlamlar qancha ko'p bo'lsa, tarmoq shunchalik "chuqur" va "aqlli" bo'ladi.

? 55. "Weights" (Vaznlar) va "Bias" (Og'ish) nima?



Bular neyron tarmog'ining o'zgaruvchan qismlaridir:

- **Weights (w):** Har bir kiruvchi ma'lumotning natijaga ta'sir kuchi. Masalan, uy narxini aniqlashda "kvadrat metr" ustunining vazni "rangi" ustuniga qaraganda ancha yuqori bo'ladi.

- **Bias (b):** Bu neyronga moslashuvchanlik beradi. U natijani ma'lum bir tomonga siljitish imkonini beradi, hattoki barcha kiruvchi ma'lumotlar nolga teng bo'lsa ham.

Modelni o'qitish — bu aynan mana shu w va b qiymatlarini eng to'g'ri holatga keltirishdir.

☆ 56. "Activation Function" (Aktivizatsiya funksiyasi) nega kerak?



Aktivizatsiya funksiyasi neyron chiqishiga "nolinearlik" kiritadi. Agar u bo'lmasa, neyron tarmoq qanchalik chuqur bo'lmasin, u faqat bitta to'g'ri chiziqni chiza oladigan oddiy formula bo'lib qoladi.

U neyronga: "Agar sendagi signallar yig'indisi yetarli bo'lsa, keyingi qatlamga xabar yubor, aks holda jim tur" degan buyruqni beradi.

? 57. Eng ko'p ishlatiladigan 3 ta aktivizatsiya funksiyasini ayting.



1. **Sigmoid:** Natijani 0 va 1 oralig'iga tushiradi. Ehtimollikni hisoblash uchun qulay.

2. **ReLU (Rectified Linear Unit):** Eng mashhuri. Agar son manfiy bo'lsa 0 beradi, musbat bo'lsa o'zini qoldiradi. Hisoblash uchun juda tez va samarali.

3. **Softmax:** Oxirgi qatlamda ishlatiladi. Bir nechta klasslar orasidan (it, mushuk, qush) eng ehtimoli yuqorisini foizlarda chiqarib beradi.

☆ 58. "Forward Propagation" nima?



Bu ma'lumotning neyron tarmog'i orqali kirishdan chiqishga qarab harakatlanishi. Ma'lumot har bir qatlamdagi vaznlarga ko'paytirilib, aktivizatsiya funksiyasidan o'tib boradi va yakunda model o'z taxminini (bashoratini) chiqaradi. Bu jarayon xuddi ma'lumotni "filtrlar" orqali o'tkazib borishga o'xshaydi.

? 59. "Loss Function" (Xatolik funksiyasi) vazifasi nima?



Loss function — bu model qanchalik adashganini ko'rsatuvchi raqamdir.

Masalan, model rasmga "it" dedi (bashorat), lekin rasmda aslida "mushuk" (haqiqat). Xatolik funksiyasi bu ikki qiymat orasidagi masofani o'lchaydi. O'qitishdan maqsad — mana shu Loss (xatolik) raqamini iloji boricha nolga yaqinlashtirishdir.

☆ 60. "Backpropagation" (Xatolikni orqaga qaytarish) qanday ishlaydi?



Bu Deep Learning-ning "yuragi". Model bashorat qilgandan keyin xatoligini ko'radi va orqaga qarab zanjir bo'ylab yuradi.

U har bir neyronga: "Sening vazning (weight) xatolikka mana shunchalik sababchi bo'ldi, shuning uchun vazningni biroz kamaytir/ko'paytir" deb buyruq beradi. Bu jarayon millionlab marta takrorlanadi va model saboq olib, xatosini tuzatib boradi.

? 61. "Gradient Descent" (Gradiyent tushishi) nima?



Bu xatolikni kamaytirish strategiyasi. Tasavvur qiling, siz tumanda tog' tepasidasiz va pastga (xatolik eng kam nuqtaga) tushishingiz kerak. Siz oyoqlaringiz bilan nishablikni (gradiyentni) tekshirasiz va eng dik pastga qarab kichik qadam tashlaysiz.

Neyron tarmoqda bu "nishablik" — matematik hosila, "qadam" — learning rate'dir.

☆ 62. "Learning Rate" (O'rganish tezligi) juda katta bo'lsa nima bo'ladi?



Learning rate — bu model vaznlarni qanchalik tez o'zgartirishini belgilovchi raqamdir.

- **Agar u juda katta bo'lsa:** Model eng pastki (optimal) nuqtadan "sakrab" o'tib ketadi va hech qachon aniqlikka erisha olmaydi (divergence).
- **Agar juda kichik bo'lsa:** Model juda sekin o'rganadi va hisoblash resurslari isrof bo'ladi.

Shuning uchun optimal o'rganish tezligini tanlash muhim.

? 63. "Epoch" (Davr) nima?



Bitta Epoch — bu barcha o'qitish ma'lumotlarining neyron tarmog'i orqali bir marta to'liq (oldinga va orqaga) o'tib chiqishi. Odatda modelni yaxshi o'qitish uchun 10, 100 yoki 1000 ta epoch kerak bo'ladi. Lekin epoch haddan tashqari ko'p bo'lsa, model ma'lumotlarni yodlab oladi (Overfitting).

☆ 64. "Batch Size" nima?



O'qitishda biz barcha ma'lumotlarni (masalan, 1 mln rasmni) birdaniga xotiraga yuklay olmaymiz. Buning o'rniga ma'lumotlarni kichik bo'laklarga — "Batch"larga bo'lamiz.

Masalan, `batch_size=32` bo'lsa, tarmoq 32 ta rasmni ko'radi, xatoligini hisoblaydi va vaznlarini yangilaydi. Keyingi 32 taga o'tadi. Bu xotirani tejashga va o'rganishni barqaror qilishga yordam beradi.

🔗 65. "Optimizer" (Optimallashtiruvchi) nima va eng mashhurini ayting?



Optimizer — bu Gradient Descent-ni qanday amalda bajarish usuli. U vaznlarni yangilash tezligini aqlli ravishda boshqaradi.

Eng mashhuri: Adam. U momentum (tezlikni saqlash) va moslashuvchan o'rganish tezligini birlashtiradi, shuning uchun ko'p holatlarda eng yaxshi natija beradi.

★ 66. "Vanishing Gradient" (Yo'qolayotgan gradiyent) muammosi nima?



Neyron tarmoq juda chuqur (qatlamlari ko'p) bo'lsa, orqaga qaytayotgan xatolik signali (gradiyent) birinchi qatlamlarga yetib borguncha shunchalik kichrayib ketadiki (0 ga yaqinlashadi), birinchi qatlamlar o'rganishni to'xtatib qo'yadi. Bu modelning qotib qolishiga olib keladi.

Yechim: ReLU funksiyasini ishlatish yoki maxsus "Residual connections" (ResNet) dan foydalanish.

🔗 67. "Exploding Gradient" nima?



Vanishing Gradient-ning teskarisi. Xatolik signali zanjir bo'ylab shunchalik kattalashib ketadiki, vaznlar astronomik darajadagi katta sonlarga aylanib ketadi va model "buziladi" (NaN natija bera boshlaydi).

Yechim: "Gradient Clipping" — gradiyent ma'lum bir qiymatdan (masalan, 1.0) oshib ketsa, uni majburan kichraytirish.

★ 68. CNN (Convolutional Neural Network) nima uchun ishlatiladi?



CNN asosan tasvirlar (rasmlar) bilan ishlash uchun yaratilgan.

Oddiy neyron tarmoq rasmdagi piksellarning joylashuvini "tushunmaydi". CNN esa "Filtrlar" (Convolution qatlamlari) yordamida rasmdagi qirralarni, shakllarni va teksturalarni o'rganadi. Bu xuddi inson ko'zining ishlashiga o'xshaydi.

? 69. CNN qatlamlaridan biri "Pooling" (masalan, Max Pooling) nima vazifani bajaradi?



Pooling — rasmdagi eng muhim ma'lumotlarni qoldirib, uning hajmini kichraytirish (siqish) jarayonidir.

Masalan, 2x2 pikseli maydondan faqat eng katta raqamni tanlab olish (Max Pooling). Bu modelga obyekt rasmning qayerida joylashganidan qat'i nazar (sal chaproqdami yoki o'ngroqdami) uni tanib olish imkonini beradi va hisoblashni tezlashtiradi.

☆ 70. RNN (Recurrent Neural Network) nima va uning asosiy farqi nimada?



RNN — bu "xotirasi bor" neyron tarmog'idir. U ketma-ketliklar (matn, ovoz, vaqtga bog'liq ma'lumotlar) bilan ishlash uchun mo'ljallangan.

Oddiy tarmoqlar har bir ma'lumotni alohida ko'radi. RNN esa gapdagi oldingi so'zlarni eslab qoladi va keyingi so'zni shunga qarab bashorat qiladi. Masalan: "Men O'zbekistonda yashayman, mening tilim..." — bu yerda RNN "O'zbekiston" so'zini eslab qolgani uchun "o'zbek" so'zini bashorat qiladi.

? 71. "LSTM" (Long Short-Term Memory) nima uchun kerak?



Oddiy RNN-ning xotirasi juda qisqa — u uzun gaplarning boshini tezda esdan chiqarib qo'yadi. LSTM — bu takomillashtirilgan RNN bo'lib, unda maxsus "gate" (darvozalar) bor. Ular qaysi ma'lumotni uzoq vaqt eslab qolish va qaysinisini o'chirib tashlashni hal qiladi. Bu unga uzun matnlarni tushunish imkonini beradi.

☆ 72. "Overfitting" neyron tarmoqlarda qanday namoyon bo'ladi?



Model o'qitish ma'lumotlarida 99% aniqlik beradi, lekin yangi test ma'lumotlarida 60% beradi. Bu model qonuniyatlarni emas, ma'lumotlardagi "shovqin" va tafsilotlarni yodlab olganini bildiradi.

Belgisi: O'qitish xatoligi (train loss) kamayishda davom etadi, lekin test xatoligi (val loss) o'sa boshlaydi.

? 73. "Dropout" qatlami overfittingni qanday to'xtatadi?



O'qitish vaqtida biz har bir epoch'da tasodifiy neyronlarning (masalan, 20% qismini) "o'chirib" qo'yamiz. Bu tarmoqni faqat ma'lum bir "kuchli" neyronlarga suyanib qolmasdan, barcha neyronlarni ishlashga majbur qiladi. Bu jamoada bitta odam hamma ishni qilmasligi uchun uni vaqtincha chetlatishga o'xshaydi — shunda qolganlar ham mas'uliyatni o'rganadi.

☆ 74. "Data Augmentation" nima?



Bu mavjud rasmlarni sun'iy ravishda ko'paytirish. Masalan, mushuk rasmimi biroz qiyshaytiramiz, teskari o'giramiz yoki rangini o'zgartiramiz.

Model uchun bular yangi rasmlar hisoblanadi. Bu modelga mushuk har qanday burchakda va yorug'likda mushuk ekanini o'rganishga yordam beradi va ma'lumotlar kamligida qo'l keladi.

? 75. "Transfer Learning" (Bilimlarni ko'chirish) nima?



Siz noldan boshlamaysiz. Siz allaqachon millionlab rasmlarda o'qitilgan tayyor modelni (masalan, Google yaratgan VGG yoki ResNet) olasiz. Bu model allaqachon chiziqlarni va shakllarni taniy oladi. Siz faqat oxirgi qatlamini o'zgartirib, o'zingizning maxsus rasmlaringizga (masalan, rentgen rasmlari) moslab o'qitasiz. Bu vaqtni va hisoblash resurslarini 100 baravar tejaydi.

? 76. "Autoencoder" (Avto-kodlovchi) nima va u qachon ishlatiladi?



Autoencoder — bu o'z kirish ma'lumotlarini chiqishda qayta tiklashga harakat qiladigan neyron tarmog'idir. U ikki qismdan iborat:

1. **Encoder (Kodlovchi):** Ma'lumotni siqadi va uning eng muhim xususiyatlarini (latent space) ajratib oladi.

2. **Decoder (Dekodlovchi):** Siqilgan ma'lumotdan foydalanib, asl tasvirni qayta tiklaydi. **Ishlatilishi:** Rasmlardagi "shovqin"larni tozalash (denoising), ma'lumotlarni siqish va noodatiy holatlarni (anomaly detection) aniqlash.

☆ 77. GAN (Generative Adversarial Network) qanday ishlaydi?



GAN — bu bir-biri bilan raqobatlashadigan ikkita neyron tarmog'i:

1. **Generator:** Soxta ma'lumot (masalan, rasm) yaratadi va uni haqiqiy deb ko'rsatishga harakat qiladi.

2. **Discriminator:** Unga berilgan rasm haqiqiy yoki soxta (Generator yaratgan) ekanini aniqlashga harakat qiladi. Bu "poyga" natijasida Generator shunchalik mukammal rasmlar yaratishni o'rganadiki, hatto inson ham ularni haqiqiysidan ajrata olmay qoladi.

🔗 78. "Batch Normalization" qatlami nima uchun kerak?

📖 Neyron tarmoq qatlamlari orasida ma'lumotlar raqamli qiymatlari juda katta yoki juda kichik bo'lib ketishi mumkin. Bu o'qitishni sekinlashtiradi. **Batch Normalization** har bir qatlamdan chiqayotgan ma'lumotlarni o'rtacha qiymati 0 va og'ishi 1 bo'lgan holatga (normalizatsiya) keltirib turadi. Bu o'qitish tezligini oshiradi va modelni barqaror qiladi.

☆ 79. "Early Stopping" strategiyasi overfittingni qanday oldini oladi?

📖 Modelni o'qitish davomida biz doimiy ravishda "Validation Loss" (sinov xatoligi)ni kuzatib boramiz.

- Ma'lum bir nuqtadan keyin o'qitish xatoligi (train loss) kamayishda davom etsa-da, sinov xatoligi (val loss) o'sa boshlaydi. Bu model yodlashni boshlaganini bildiradi. **Early Stopping** sinov xatoligi oshishni boshlagan zahoti o'qitishni to'xtatadi va eng yaxshi natija bergan modelni saqlab qoladi.

🔗 80. "Softmax" funksiyasi nega faqat oxirgi qatlamda ishlatiladi?

📖 Chunki Softmax barcha chiqish neyronlaridagi qiymatlarni shunday o'zgartiradiki, ularning yig'indisi aniq **1.0 (ya'ni 100%)** bo'ladi. Bu bizga natijani ehtimollik ko'rinishida olish imkonini beradi. Masalan: "Bu rasm 80% ehtimol bilan it, 15% mushuk va 5% qush".

☆ 81. "Weights Initialization" (Vaznlarni boshlang'ichlash) nega muhim?

📖 Agar o'qitish boshida barcha vaznlarga 0 yoki bir xil son bersangiz, barcha neyronlar bir xil xatoni hisoblaydi va bir xil o'rganadi. Natijada tarmoq rivojlanmaydi. **Yechim:** "Xavier" yoki "He" initialization kabi usullar yordamida vaznlarga kichik tasodifiy raqamlar beriladi. Bu tarmoqning har bir qismi har xil xususiyatlarni o'rganishini ta'minlaydi.

🔗 82. "1x1 Convolution" nima va u nega kerak?

📖 Bu rasmning balandligi va enini o'zgartirmagan holda, faqat uning **chuqurligini (kanallar sonini)** kamaytirish yoki ko'paytirish uchun ishlatiladi. Bu

hisoblash resurslarini sezilarli darajada tejash imkonini beradi (masalan, GoogleNet arxitekturasida).

☆ 83. "Residual Connections" (Skip connections) nima?

📖 Bu ResNet arxitekturasining asosi. Unda ma'lumot faqat qatlamlar orqali o'tib qolmay, balki ba'zi qatlamlarni "sakrab o'tib" keyingi qatlamga ham qo'shiladi. **Nima uchun:** Bu "Vanishing Gradient" muammosini hal qiladi, chunki xatolik signali (gradiyent) o'sha sakrash yo'li orqali tarmoqning boshiga qiyinchiliksiz yetib boradi.

🔗 84. "Hyperparameter"larni optimallashtirishda "Bayesian Optimization" nima?

📖 Bu shunchaki tasodifiy qidirish emas, balki "aqlli" qidirishdir. Algoritm o'tmishdagi sinovlar natijasini tahlil qiladi va qaysi parametrlar yaxshi natija berishini taxmin qilib, keyingi sinovni aynan o'sha sohada o'tkazadi. Bu eng yaxshi model sozlamalarini topishni tezlashtiradi.

☆ 85. "Leaky ReLU" oddiy ReLU dan nimasi bilan afzal?

📖 Oddiy ReLU manfiy sonlarni mutlaqo 0 qiladi, bu esa "Dying ReLU" muammosiga (neyron o'lib, hech qachon o'rganmay qolishiga) olib kelishi mumkin. **Leaky ReLU** manfiy sonlar uchun kichik bir nishablik (masalan, 0.01) qoldiradi. Natijada signal 0 bo'lsa ham biroz o'tadi va o'sha neyron o'rganishda davom etishi mumkin.

🔗 86. "Fine-tuning" va "Feature Extraction" (Transfer Learning'da) farqi nimada?



1. **Feature Extraction:** Tayyor modelning (masalan, ImageNet'da o'qitilgan) barcha vaznlarini muzlatib (freeze), faqat oxirgi yangi qatlamingizni o'qitasiz.

2. **Fine-tuning:** Tayyor modelning oxirgi bir necha qatlamlarini muzlatishdan chiqarib, juda kichik learning rate bilan o'z ma'lumotlaringizda qayta o'qitasiz. Bu modelni sizning ma'lumotlaringizga yanada moslashtiradi.

☆ 87. "Gradient Clipping" nima?

📖 Bu "Exploding Gradient" muammosiga qarshi kurashish usuli. Agar gradiyent qiymati ma'lum bir chegaradan (masalan, 1.0) oshib ketsa, u majburan o'sha chegaraga tenglashtiriladi. Bu modelning "aqldan ozib" ketishidan saqlaydi.

🔗 88. "U-Net" arxitekturasida nega tibbiy tasvirlarda mashhur?

📖 U-Net tasvirni nafaqat klassifikatsiya qiladi, balki uni **segmentatsiya** ham qiladi (pikselbay aniqlaydi). Uning shakli "U" harfiga o'xshaydi va rasmning mayda tafsilotlarini yuqori aniqlikda saqlab qoladi. Bu rentgen rasmidagi o'simtani aniq chizib berish uchun juda qulay.

☆ 89. "Momentum" (Gradiyent tushishida) nima vazifani bajaradi?

📖 Tasavvur qiling, shar tepalikdan pastga dumalayapti. U faqat pastga qaramaydi, balki o'zining oldingi tezligini ham saqlab qoladi. **Momentum** xatolikni kamaytirishda modelga kichik "tepalik"larni oson o'tib ketishga va optimal nuqtani (global minimum) tezroq topishga yordam beradi.

? 90. "Binary Cross-Entropy" va "Categorical Cross-Entropy" farqi?



- **Binary Cross-Entropy:** Faqat 2 ta klass bo'lganda ishlatiladi (it yoki mushuk).
- **Categorical Cross-Entropy:** 3 ta va undan ko'p klasslar bo'lganda ishlatiladi (it, mushuk, qush va h.k.).

☆ 91. Neyron tarmoqlarda "Flatten" qatlami nima qiladi?

📖 U ko'p o'lchamli ma'lumotni (masalan, 28x28 pikselli rasm kvadratini) bitta uzun qator (784 ta neyron) holatiga keltirib beradi. Bu rasm qatlamlaridan oddiy (fully connected) qatlamlarga o'tish uchun kerak.

? 92. "Multi-head Attention" nima? (Transformers asoslari)

📖 Bu transformer modellarning (masalan, GPT) "ko'zi"dir. U matndagi so'zlarga bir emas, bir nechta "burchak"dan qaraydi. Masalan, bitta "boshi" grammatikaga e'tibor bersa, ikkinchisi so'z ma'nosiga, uchinchisi mantiqiy bog'liqlikka qaraydi.

☆ 93. "PyTorch" va "TensorFlow" — qaysi birini tanlash kerak?



- **PyTorch:** Dasturchilar uchun qulay ("Pythonic"), dinamik grafikali. Tadqiqotlar va yangi modellar yaratishda ko'proq ishlatiladi.
- **TensorFlow:** Google tomonidan qo'llab-quvvatlanadi, mobil va sanoat darajasidagi katta tizimlarga joylash (deployment) uchun juda ko'p tayyor vositalarga ega. Amalda ikkalasi ham bir xil imkoniyatga ega, tanlov ko'proq jamoaning tajribasiga bog'liq.

? 94. "Embedding Layer" nima vazifani bajaradi?

📖 U soʻzlarni yoki kategoriyalarni raqamli vektorlarga aylantiradi. Oddiy raqamlashdan (1, 2, 3) farqi shundaki, oʻxshash soʻzlar (masalan, "choy" va "qahva") fazoda bir-biriga yaqin vektorlarga ega boʻladi.

★ 95. "Style Transfer" (Uslubni koʻchirish) qanday ishlaydi?

📖 U bitta rasmning **mazmunini** (content) va ikkinchi rasmning (masalan, Van Gog asari) **uslubini** (style) birlashtiradi. Bu neyron tarmogʻining turli qatlamlaridan olingan xususiyatlarni bir-biriga moslashtirish orqali amalga oshiriladi.

❓ 96. "Inception Module" nima?

📖 Bu bitta qatlamda bir vaqtning oʻzida turli oʻlchamdagi (1x1, 3x3, 5x5) filtrlar bilan ishlash usuli. Model rasmdagi obyekt katta boʻlsa ham, kichik boʻlsa ham uni taniy olish imkonini beradi.

★ 97. "Learning Rate Decay" (Oʻrganish tezligini kamaytirish) nega kerak?

📖 Oʻqitish boshida biz katta qadamlar bilan optimal nuqtaga yaqinlashamiz. Lekin nishonga yaqinlashganda qadamni kichraytirmasak, biz undan oʻtib ketamiz. Shuning uchun epochlar oʻtishi bilan learning rate avtomatik kamaytirib boriladi.

❓ 98. "Collaborative Filtering" neyron tarmoqlarda qanday ishlatiladi?

📖 Bu tavsiya tizimlari (masalan, Netflix yoki Youtube) uchun ishlatiladi. Foydalanuvchi va kontent (video) orasidagi bogʻliqlik "Matrix Factorization" yoki neyron tarmoq orqali oʻrganiladi va sizga oʻxshash foydalanuvchilar yoqtirgan narsalar tavsiya etiladi.

★ 99. "Hardware Acceleration" (GPU va TPU) nega Deep Learning'da shart?

📖 Neyron tarmoqlar trillionlab **matritsa koʻpaytirish** amallaridan iborat. Oddiy protsessor (CPU) bu ishlarni ketma-ket (bittalab) bajaradi. GPU (Grafik protsessor) esa minglab yadrolari bilan bu ishlarni parallel (bir vaqtda) bajaradi. TPU (Google yaratgan) esa faqat AI amallari uchun ixtisoslashgan va yanada tezroq.

❓ 100. "The Black Box Problem" (Qora quti muammosi) nima?

📖 Bu neyron tarmoqlarining asosiy kamchiligi. Biz modelni oʻqitamiz va u 99% aniqlik beradi, lekin u nega aynan shu qarorni qabul qilganini aniq tushuntirib bera olmaydi (chunki ichkarida milliardlab matematik vaznlar bor). Bu tibbiyot yoki sud tizimida AI ishlatishda xavfsizlik va etika muammolarini keltirib chiqaradi.

4.3. Tabiiy Tilni Qayta Ishlash (NLP) va LLM Texnologiyalari

? 101. NLP (Natural Language Processing) o'zi nima va uning asosiy qiyinchiligi nimada?

📖 NLP — bu kompyuterlarga inson tilini o'qish, tushunish va generatsiya qilishni o'rgatuvchi sohadir. Asosiy qiyinchilik — **noaniqlik (ambiguity)**. Inson tilida bitta so'z vaziyatga qarab turli ma'nolarni berishi mumkin. Masalan, "Olmang" so'zi "mevani olmang" yoki "olma" (meva)ning egalik shakli bo'lishi mumkin. AI so'zning atrofidagi gapga (kontekstga) qarab ma'noni ajratishni o'rganishi kerak.

☆ 102. "Tokenization" jarayoni qanday ishlaydi?

📖 Kompyuter matnni butunligicha qabul qilolmaydi. **Tokenization** — matnni kichik bo'laklarga (tokenlarga) bo'lishdir.

- **Word-level:** Har bir so'z bitta token (masalan: "Men", "maktabga", "bordim").
- **Subword-level (BPE/WordPiece):** So'zlar bo'laklarga bo'linadi (masalan: "Dasturlash" -> "Dastur", "lash"). Zamonaviy LLMlar (GPT kabi) subword-leveldan foydalanadi, chunki bu lug'at hajmini kamaytiradi va yangi (lug'atda yo'q) so'zlarni ham tushunish imkonini beradi.

? 103. "Stop Words" (To'xtovchi so'zlar) nima va nega ular ko'pincha o'chirib tashlanadi?

📖 Stop words — bu tilda juda ko'p takrorlanadigan, lekin gapning asosiy ma'nosini tashimaydigan so'zlar (masalan: "va", "bilan", "uchun", "ham"). An'anaviy NLP tizimlarida (masalan, qidiruv tizimlarida) bu so'zlar o'chiriladi. Bu hisoblashni tezlashtiradi va modelga faqat muhim so'zlarga (masalan: "Python", "intervyu") e'tibor qaratish imkonini beradi. Biroq, zamonaviy Transformer modellarda ular ko'pincha qoldiriladi, chunki ular grammatik bog'liqlikni saqlaydi.

☆ 104. "Stemming" va "Lemmatization" o'rtasidagi farq nimada?

📖 Ikkalasi ham so'zni asosiga qaytaradi, lekin usuli turlicha:

- **Stemming:** So'zning qo'shimchasini shunchaki kesib tashlaydi (masalan: "olmalar" -> "olma", "ishlamoqda" -> "ishla"). Ba'zan mantiqsiz natija berishi mumkin.
- **Lemmatization:** So'zning lug'atdagi asosiy shaklini (lemma) topadi (masalan: "bordim" -> "bormoq", "yaxshiroq" -> "yaxshi"). Bu til qoidalarini hisobga oladi va aniqroq natija beradi.

❓ 105. "Bag of Words" (BoW) modeli qanday ishlaydi?

📖 Bu NLP-ning eng sodda modeli. U gapdagi soʻzlarning tartibiga eʼtibor bermay, faqat qaysi soʻz necha marta takrorlanganini sanaydi. Masalan: "Men olmani sevaman va olma shirin" -> {"Men": 1, "olma": 2, "sevaman": 1, "shirin": 1}. Bu usul matnlarni klassifikatsiya qilish (masalan, xat mavzusini aniqlash) uchun ishlatiladi, lekin gapning chuqur maʼnosini tushunolmaydi.

★ 106. "TF-IDF" (Term Frequency-Inverse Document Frequency) nima uchun kerak?

📖 Bu soʻzning matn ichida qanchalik "muhim" ekanini oʻlchovchi statistik raqam.

- **TF:** Soʻz shu matnda necha marta takrorlangan (koʻp boʻlsa yaxshi).
- **IDF:** Bu soʻz barcha boshqa matnlarda necha marta uchraydi (kam boʻlsa yaxshi). Agar soʻz bitta matnda koʻp uchrab, boshqa hamma matnlarda kam uchrasa (masalan: "kvant fizikasi"), demak bu soʻz shu matn uchun juda muhim hisoblanadi.

❓ 107. "Word Embeddings" (Word2Vec) qanday qilib soʻzlar maʼnosini tushunadi?

📖 Bu soʻzlarni koʻp oʻlchamli fazodagi vektorlar (raqamlar toʻplami) koʻrinishida ifodalashdir. Moʻjizasi shundaki, oʻxshash soʻzlar fazoda bir-biriga yaqin joylashadi. **Real misol:** Vektorlar yordamida $Qiro\text{ l} - Er\text{ kak} + Ayol = Qiro\text{ l}i\text{ cha}$ natijasini olish mumkin. Bu model soʻzning shunchaki harf emas, mazmun ekanini matematik tushunayotganini koʻrsatadi.

★ 108. "Cosine Similarity" (Kosinus oʻxshashligi) nima?

📖 Bu ikki matn yoki soʻzning vektorlari orasidagi burchakni oʻlchash usuli. Agar burchak 0 daraja boʻlsa, vektorlar bir xil yoʻnalishda va matnlar oʻta oʻxshash. Agar 90 daraja boʻlsa, ular mutlaqo aloqasiz. Bu qidiruv tizimlarida sizning soʻrovingizga eng mos maqolani topish uchun ishlatiladi.

❓ 109. RNN (Recurrent Neural Network) matnlar bilan ishlashda nega sekin?

📖 Chunki RNN matnni soʻzma-soʻz, ketma-ket qayta ishlaydi. U gapning 1-soʻzini oʻqimasdan turib, 2-soʻzini oʻqiy olmaydi. Bu esa zamonaviy GPU-larning parallel hisoblash kuchidan foydalanishga toʻsqinlik qiladi va uzun matnlarni oʻqitishni juda sekinlashtiradi.

☆ 110. "Attention Mechanism" (Diqqat mexanizmi) NLP-da qanday inqilob qildi?

📖 Bu mexanizm AI-ga gapdagi qaysi soʻz boshqa qaysi soʻzlarga bogʻliqligini aniqlash imkonini beradi. **Misol:** "Bankka bordim, u yopiq ekan". AI "u" soʻzi "bank"ga tegishli ekanini "diqqat" (attention) yordamida tushunadi. Transformerlardan oldin modellar bunday uzoq bogʻliqliklarni (long-range dependencies) tezda yoʻqotib qoʻyar edi.

❓ 111. "Self-Attention" nima?

📖 Bu Transformer modelining ichki qismi boʻlib, unda gapdagi har bir soʻz boshqa barcha soʻzlar bilan "muloqot" qiladi. Har bir soʻz boshqa soʻzlarga ball beradi: "Men uchun bu soʻz qanchalik muhim?". Natijada model gapning butun kontekstini bir vaqtning oʻzida tushunadi.

☆ 112. "Transformer" arxitekturasini nima va uning asosi nimadan iborat?

📖 Transformer — bu zamonaviy AI (GPT, BERT, Gemini)ning asosi. U 2017-yilda Google tomonidan "Attention is All You Need" maqolasida eʼlon qilingan. Uning asosi **Encoder** (matnni tushunish qismi) va **Decoder** (yangi matn yaratish qismi)dan iborat. RNN-lardan farqi — u parallel ishlaydi va matn uzunligidan qatʼi nazar maʼnoni yaxshi saqlaydi.

❓ 113. BERT (Bidirectional Encoder Representations from Transformers) nima?

📖 BERT — bu Google tomonidan yaratilgan, matnni "ikki tomonlama" (oʻngdan chapga va chapdan oʻngga) oʻqiydigan model. Bu unga soʻzning maʼnosini uning atrofidagi gapga qarab juda aniq tushunish imkonini beradi. BERT asosan matnni tahlil qilish, savol-javob tizimlari va qidiruvni yaxshilashda ishlatiladi.

☆ 114. GPT (Generative Pre-trained Transformer) modeli qanday ishlaydi?

📖 GPT — bu faqat **Decoder** qismidan iborat boʻlgan model. Uning asosiy vazifasi: Berilgan matndan keyin keladigan **keyingi soʻzni (tokenni) bashorat qilish**. U internetdagi trillionlab matnlarda oʻqitilgani uchun, keyingi soʻzni shunchalik aniq topadiki, natijada u mantiqiy gaplar va hatto kod yozayotgandek koʻrinadi.

❓ 115. LLM (Large Language Model) nima uchun "Katta" deb ataladi?

📖 Ikki sababga koʻra:

1. **Parametrlar soni:** Ularda milliardlab (masalan, GPT-3 da 175 milliard) oʻzgaruvchan vaznlar (vaznlar) mavjud.

2. **Ma'lumotlar hajmi:** Ular deyarli butun internet (Vikipediya, kitoblar, GitHub, maqolalar) ma'lumotlarida o'qitiladi.

☆ 116. "Pre-training" va "Fine-tuning" farqi?



- **Pre-training:** Modelni ulkan, umumiy ma'lumotlarda o'qitish (masalan, GPT-ni internetdagi hamma matnlarda o'qitish). Bu juda qimmat va oylar davom etadi.

- **Fine-tuning:** Tayyor modelni kichikroq, maxsus ma'lumotlarda o'qitish (masalan, GPT-ni tibbiy maqolalarda o'qitish). Bu modelni maxsus soha mutaxassisiga aylantiradi.

? 117. "Hallucination" (Gallyutsinatsiya) nima va nega u sodir bo'ladi?



Bu LLM-ning haqiqatga to'g'ri kelmaydigan ma'lumotni juda ishonchli ohangda to'qib chiqarishi. Nega: Chunki model faktlarni "bilmaydi", u faqat keyingi so'zni statistika yordamida topadi. Agar u o'z bazasida javobni topolmasa, u "eng ehtimolli ko'ringan" yolg'onni yozib yuborishi mumkin.

☆ 118. RLHF (Reinforcement Learning from Human Feedback) nima uchun kerak?



Bu ChatGPT-ni aqlli va "odobli" qilgan texnologiya. Model o'qitilgandan keyin, insonlar uning javoblarini baholaydi: "Bu javob yaxshi", "Bu javob zararli". Model bu fikrlar asosida qayta o'qitiladi. Natijada AI insonlar kutganidek foydali va xavfsiz javob berishni o'rganadi.

? 119. "Prompt Engineering" nima?



Bu modeldan eng yaxshi natijani olish uchun savolni (promptni) to'g'ri shakllantirish san'ati. Masalan, shunchaki "Dastur yoz" deyish o'rniga, "Sen tajribali Python dasturchisidan, menga xavfsiz va tushunarli kod yozib ber" deyish natijani keskin yaxshilaydi.

☆ 120. "Context Window" (Kontekst oynasi) nima?



Bu model bir vaqtning o'zida o'z "xotirasida" saqlab tura oladigan matn hajmi. Masalan, agar kontekst oynasi 8000 token bo'lsa, siz kitobning 9000-tokenini kiritganingizda, model birinchi 1000 ta tokeni "esdan chiqaradi". Zamonaviy modellarda bu oyna 128 mingdan 1 milliongacha yetib bormoqda.

? 121. "Zero-shot" vs "Few-shot" o'rganish nima?



- **Zero-shot:** Modelga misol ko'rsatmasdan buyruq berish (masalan: "Bu gapni inglizchaga tarjima qil").

- **Few-shot:** Modelga vazifani tushuntirish uchun 2-3 ta misol ko'rsatish (masalan: "Olma -> Apple, Anor -> Pomegranate, Uzum -> ?"). Bu modelga yangi vazifalarni tezroq tushunishga yordam beradi.

☆ 122. RAG (Retrieval-Augmented Generation) tizimi qanday ishlaydi?

📖 Bu AI gallyutsinatsiyasini to'xtatishning eng yaxshi usuli. Model savolga o'z miyasidan javob qidirishdan oldin, tashqi bazadan (masalan, sizning PDF fayllaringizdan) tegishli ma'lumotni qidirib topadi. Keyin o'sha ma'lumotga tayanib javob beradi. Bu AI-ni "kitobga qarab javob beradigan" o'quvchiga aylantiradi.

❓ 123. "Vector Database" (Vektorli baza) nima uchun LLM-larga kerak?

📖 LLMlar matnlarni vektorlar (embeddings) sifatida saqlaydi. Oddiy SQL bazalarda vektorlar bo'yicha qidirish qiyin. **Vector Database** (masalan: Pinecone, Chroma) millionlab vektorlar orasidan ma'nosi yaqin bo'lgan matnlarni millisekundlarda topib berish uchun ixtisoslashgan.

☆ 124. "Chain-of-Thought" (Fikrlar zanjiri) prompting nima?

📖 Modelga "Qadamba-qadam o'yla" (Let's think step by step) deb buyruq berish. Bu modelga murakkab matematik yoki mantiqiy savollarni yechishda yordam beradi. Model yakuniy javobni aytishdan oldin, oraliq mantiqiy bosqichlarni yozib chiqadi va xatoga yo'l qo'ymaydi.

❓ 125. Kelajakda LLMlar o'rnini "World Models" egallashi haqidagi nazariya nima?

📖 Hozirgi LLMlar faqat matnli statistika mashinalari. **World Models** (Dunyo modellari) esa dunyo qanday ishlashini (fizika qonuniyatlari, sabab-oqibat) tushunadigan AI bo'ladi. Bu AI-ga nafaqat gapirishni, balki haqiqiy "ong" va rejalashtirish qobiliyatini berishi taxmin qilinadi.

❓ 126. "Encoder-only", "Decoder-only" va "Encoder-Decoder" modellari o'rtasidagi farq nima?



Transformer arxitekturasini turli maqsadlar uchun bo'lib yuborilgan:

- **Encoder-only (masalan, BERT):** Matnni tushunish va tahlil qilish uchun zo'r. U gapning ikki tomoniga (o'ng va chap) qaray oladi. Klassifikatsiya va so'z ma'nosini aniqlashda ishlatiladi.

- **Decoder-only (masalan, GPT):** Yangi matn yaratish (generatsiya) uchun mo'ljallangan. U faqat o'tmishdagi (chapdagi) so'zlarga qarab, keyingisini bashorat qiladi.

- **Encoder-Decoder (masalan, T5, BART):** Bir matnni ikkinchisiga o'tkazish uchun. Masalan, tarjima qilish yoki matnni xulosa (summarization) qilishda ishlatiladi.

★ 127. "Masked Language Modeling" (MLM) nima?



Bu BERT kabi modellarni o'qitish usuli. Gap ichidagi ba'zi so'zlar yashirib (mask qilinib) qo'yiladi (masalan: "Men bugun [MASK]ga bordim"). Model atrofidagi so'zlarga qarab, yashirilgan so'z "maktab" yoki "bozor" ekanini topishi kerak. Bu modelga tilning grammatikasi va mantiqiy bog'liqligini chuqur o'rgatadi.

? 128. "Sentiment Analysis" (Hissiyotlar tahlili) qanday amalga oshiriladi?



Bu matnning kayfiyatini (ijobiy, salbiy yoki neytral) aniqlash jarayonidir.

Model matnni vektorga aylantiradi va oxirgi qatlamda klassifikatsiya qiladi.

Amaliy misol: Kompaniyalar ijtimoiy tarmoqlardagi izohlarni tahlil qilib, odamlar ularning mahsuloti haqida nima deb o'ylayotganini avtomatik bilib oladi.

★ 129. "Named Entity Recognition" (NER) nima vazifani bajaradi?



NER — matn ichidan maxsus nomlarni (shaxslar, tashkilotlar, joy nomlari, sanalar) topish va toifalash jarayoni.

Masalan: "Alisher Navoiy 1441-yilda Hirotida tug'ilgan" gapidan model Alisher Navoiy (Person), 1441-yil (Date) va Hirot (Location) ekanini ajratib beradi.

? 130. "BLEU Score" nima va u tarjimani qanday baholaydi?



BLEU (Bilingual Evaluation Understudy) — bu mashina tarjimasining sifatini o'lchovchi metrika.

U AI tomonidan qilingan tarjimani inson tomonidan qilingan professional tarjima bilan solishtiradi. Ular orasidagi so'zlar mosligi (n-grams) qanchalik yuqori bo'lsa, ball shunchalik baland bo'ladi (0 dan 1 gacha).

★ 131. "Perplexity" metriksi LLMlar uchun nimani anglatadi?



Perplexity — bu modelning keyingi soʻzni topishda qanchalik "hayron" qolishi yoki ikkilanishi oʻlchovi.

Raqam qanchalik past boʻlsa, model shunchalik ishonchli va aniq bashorat qilmoqda deb hisoblanadi. Bu modelning matnni qanchalik yaxshi oʻrganganini koʻrsatuvchi asosiy metrikadir.

? 132. "N-gram" modeli nima?



Bu matnni ketma-ket keluvchi n ta soʻz boʻlaklariga boʻlib tahlil qilishdir.

- **Unigram:** bittalab ("men", "maktabga").
- **Bigram:** juftlab ("men maktabga", "maktabga bordim").

Bu usul qidiruv tizimlarida keyingi soʻzni taxmin qilish (autocomplete) uchun eng sodda va tezkor yoʻldir.

☆ 133. "WordSense Disambiguation" (Ma'noni ajratish) muammosi nima?



Bitta soʻzning turli maʼnolari orasidan kontekstga mosini tanlash.

Masalan: "Yoz keldi" (fasl) va "Daftarga yoz" (fe'l). Model soʻzning atrofidagi "keldi" yoki "daftarga" soʻzlariga qarab, bu soʻz qaysi turkumga tegishli ekanini tushunishi kerak.

? 134. "Sequence-to-Sequence" (Seq2Seq) modellari qanday ishlaydi?



Bu arxitektura oʻzgaruvchan uzunlikdagi kirishni (masalan, 5 soʻzli gap) boshqa oʻzgaruvchan uzunlikdagi chiqishga (masalan, 7 soʻzli tarjima) oʻtkazadi.

U Encoder (maʼnoni siqib "context vector"ga aylantiradi) va Decoder (vektordan yangi gap hosil qiladi) qismlaridan iborat. Google Translate'ning eski versiyalari shu asosda ishlagan.

☆ 135. "Positional Encoding" Transformerda nima uchun kerak?



Transformer modellari RNN-lardan farqli oʻlaroq, barcha soʻzlarni parallel oʻqiydi. Shuning uchun u qaysi soʻz gapning boshida, qaysisi oxirida ekanini "bilmaydi".

Positional Encoding har bir soʻzning vektoriga uning gapdagi oʻrni haqidagi matematik maʼlumotni qoʻshib yuboradi. Bu modelga soʻzlar tartibini (tartib esa maʼnoni oʻzgartiradi) tushunish imkonini beradi.

? 136. "Coreference Resolution" nima?



Matndagi olmoshlar (u, bu, ular) aynan qaysi otga tegishli ekanini aniqlash.

Masalan: "Ali doʻstini koʻrdi. U xursand edi." Bu yerda "u" kim? Ali-mi yoki doʻstim? AI bu mantiqiy bogʻliqlikni yechishi kerak.

☆ 137. "Parameter-Efficient Fine-Tuning" (PEFT) nima?



Katta modellarni (LLM) toʻliq qayta oʻqitish juda qimmat.

PEFT usuli modelning 99% vaznlarini muzlatib qoʻyadi va faqat juda kichik bir qismini (yoki yangi qoʻshilgan qatlamlarni) oʻqitadi. Bu xotirani 10-50 baravar tejaydi va oddiy videokartalarda ham LLMlarni sozlash imkonini beradi.

? 138. "LoRA" (Low-Rank Adaptation) texnologiyasi qanday ishlaydi?



Bu PEFT-ning eng mashhur turi. Modelning ulkan matritsalarini toʻliq oʻzgartirmasdan, ularga ikkita juda kichik (past darajali) matritsani qoʻshadi. Faqat shu kichik matritsalar oʻqitiladi. Bu GPT-4 kabi modellarni shaxsiy kompyuterda "sozlash" imkonini bergan texnologiyadir.

☆ 139. "Beam Search" va "Greedy Search" farqi nimada?



Model keyingi soʻzni generatsiya qilayotganda:

- **Greedy Search:** Har doim faqat bitta — eng yuqori ehtimolli soʻzni tanlaydi. Bu tez, lekin baʼzan mantiqsiz gapga olib keladi.
- **Beam Search:** Bir vaqtning oʻzida bir nechta (masalan, top 5 ta) variantni koʻrib boradi va yakunda eng mantiqiy gap hosil boʻladigan yoʻlni tanlaydi. Bu sifatni oshiradi.

? 140. "ROUGE" metriksi nima uchun ishlatiladi?



BLEU tarjima uchun bo'lsa, ROUGE (Recall-Oriented Understudy for Gisting Evaluation) matnni xulosa qilish (summarization) sifatini o'lchaydi. U model yaratgan qisqa xulosada original matndagi muhim ma'lumotlar qanchalik saqlanib qolganini tekshiradi.

☆ 141. "Zero-shot Classification" qanday qilib hech qanday o'qitishsiz ishlaydi?



Modelga matn va bir nechta variantlar beriladi (masalan: "sport", "siyosat"). Model o'zining ulkan bilimi (NLP embeddings) yordamida matnning vektori qaysi variantning vektoriga yaqinroq ekanini hisoblaydi. Bu modelni oldindan o'qitmagan sohagizda ham ishlatish imkonini beradi.

? 142. "Contextual Embeddings" va "Static Embeddings" farqi?



- **Static (Word2Vec):** "Bank" so'zi har doim bir xil vektorga ega (sohil yoki moliya muassasasi bo'lishidan qat'i nazar).
- **Contextual (BERT/GPT):** So'zning vektori uning atrofidagi gapga qarab o'zgaradi. "Daryo bo'yidagi **bank**" va "Markaziy **bank**" gaplarida "bank" so'zi turlicha vektorlarga ega bo'ladi.

☆ 143. "Long Context Window" muammosi nimada?



Matn qanchalik uzun bo'lsa, Self-Attention mexanizmi hisoblashi shunchalik kvadratik ($O(N^2)$) oshib ketadi. Ya'ni matn 2 barobar uzun bo'lsa, hisoblash 4 barobar qiyinlashadi. Zamonaviy tadqiqotlar (masalan, Flash Attention) buni chiziqli holatga keltirishga harakat qilmoqda.

? 144. "Tokenizer Bias" nima?



Agar tokenizator asosan ingliz tilida o'qitilgan bo'lsa, u boshqa tillarni (masalan, o'zbekcha) noto'g'ri yoki juda mayda bo'laklarga bo'lib tashlaydi. Bu o'sha tilda modelning sekin ishlashiga va ma'noni yomon tushunishiga olib keladi.

☆ 145. "Instruction Fine-tuning" nima?



Bu shunchaki matn davomini yozadigan modelni (base model) inson buyruqlarini bajaradigan AI-ga (instruction model) aylantirish jarayoni. Modelga "Savol: ... Javob: ..." formatidagi minglab misollar beriladi. Shundan keyingina u ChatGPT kabi muloqot qilishni o'rganadi.

🔗 146. "Knowledge Distillation" AI-da nima uchun kerak?



Katta modelni (Teacher) kichikroq modelga (Student) o'rgatish. Kichik model kattasining javoblarini kuzatib, uning bilimini nusxalaydi. Natijada biz telefonlarda tez ishlaydigan, lekin aqlli modelga ega bo'lamiz.

☆ 147. "Automatic Speech Recognition" (ASR) va NLP bog'liqligi?



ASR ovozni matnga o'giradi. Keyin NLP o'sha matnni tushunadi. Masalan, Siri yoki Google Assistant avval ovozni taniydi (ASR), keyin buyruqni bajaradi (NLP).

🔗 148. "Chatbot" va "Conversational AI" farqi?



- **Chatbot:** Ko'pincha qat'iy qoidalar yoki oddiy "tugmalar" asosida ishlaydi.
- **Conversational AI:** LLM kabi chuqur texnologiyalarga asoslangan bo'lib, inson kabi erkin muloqot qila oladi, kontekstni saqlaydi va murakkab savollarga javob beradi.

☆ 149. "Vector Search" (Semantic Search) nega kalit so'zli qidiruvdan yaxshi?



Kalit so'zli qidiruv faqat bir xil so'zlarni qidiradi. Vektorli qidiruv esa ma'noni qidiradi. Agar siz "tezkor ulov" deb qidirsangiz, u matn ichidan "avtomobil" yoki "mashina" so'zlari bor gaplarni ham topib beradi, chunki ularning vektorlari yaqin.

🔗 150. AI sohasida "AGI" (Artificial General Intelligence) tushunchasi nimani anglatadi?



Bu inson kabi har qanday aqliy vazifani bajara oladigan, o'z-o'zini anglaydigan va yangi sohalarni mustaqil o'rganadigan nazariy AI darajasi. Hozirgi NLP modellar

qanchalik "aqlli" ko'rinmasin, ular hali AGI darajasiga yetmagan, ular faqat maxsus o'qitilgan sohalarida kuchli.

Ajoyib! Yakuniy **4-qism: Kompyuter ko'rishi (Computer Vision) va MLOps** bo'limiga o'tamiz. Bu bo'limda biz AI qanday qilib rasmlarni "ko'rishi", videolarni tahlil qilishi va tayyor modellarni qanday qilib real mahsulot darajasiga olib chiqishni (deployment) ko'rib chiqamiz.

4.4. Kompyuter Ko'rishi (Computer Vision) va MLOps

🔗 **151. Kompyuter ko'rishi (Computer Vision) sohasining asosiy maqsadi nima?**

📖 Kompyuter ko'rishi — bu mashinalarga raqamli tasvirlar (rasm va videolar) orqali atrofdagi dunyoni "ko'rish" va tushunishni o'rgatuvchi AI sohasidir. U piksellar to'plamini inson tushunadigan obyektlar, yuzlar yoki harakatlar darajasiga olib chiqadi. **Amaliy misol:** Sanoat robotlarining detallarni saralashi, tibbiy skanerlarning kasallikni topishi yoki FaceID orqali telefonni ochish.

★ **152. Rasm raqamli ko'rinishda kompyuter xotirasida qanday saqlanadi?**

📖 Kompyuter uchun rasm — bu shunchaki sonlardan iborat **matritsa**.

- **Oq-qora rasm:** Har bir piksel 0 (qora) va 255 (oq) oralig'idagi bitta son bilan ifodalanadi.

- **Rangli rasm (RGB):** Har bir piksel uchun 3 ta qiymat bor: Qizil (Red), Yashil (Green) va Ko'k (Blue). Natijada rasm 3 qavatli matritsa (tensor) ko'rinishida saqlanadi. AI algoritmlari aynan mana shu sonlar o'rtasidagi bog'liqlikni qidiradi.

🔗 **153. CNN (Convolutional Neural Network) dagi "Kernel" yoki "Filter" nima?**

📖 Filtr — bu kichik o'lchamli (masalan, 3x3) sonli matritsa bo'lib, u rasm ustidan sirpanib o'tadi. Uning vazifasi rasmdagi ma'lum bir belgilarni (masalan, gorizontall chiziqlar, burchaklar yoki rang o'zgarishi) aniqlashdir. Tarmoq o'qish jarayonida bu filtr ichidagi sonlarni shunday o'zgartiradiki, natijada u rasmdagi eng muhim xususiyatlarni ajratib olishni o'rganadi.

★ **154. "Max Pooling" qatlami nega rasm tanish aniqligini oshiradi?**

📖 Max Pooling rasmini kichik bo'laklarga bo'lib, har bir bo'lakdan faqat eng katta qiymatni tanlab oladi.

1. **Hajmni kamaytiradi:** Hisoblashni tezlashtiradi.

2. Translation Invariance: Obyekt rasmning qayerida (chapdami, o'ngdami) joylashganidan qat'i nazar, model uni taniy olishiga yordam beradi. Chunki eng muhim signal (eng katta raqam) baribir saqlanib qoladi.

? **155. "Image Classification" va "Object Detection" o'rtasidagi farq nimada?**



- **Image Classification:** Butun rasmga bitta nom (label) beradi. Masalan: "Bu rasmda mushuk bor".

- **Object Detection:** Rasm ichidagi bir yoki bir nechta obyektни topadi, ularning atrofini to'rtburchak (bounding box) bilan o'raydi va har biriga alohida nom beradi. Masalan: "Bu yerda 2 ta mashina va 1 ta odam bor".

☆ **156. YOLO (You Only Look Once) algoritmi nega shunchalik mashhur?**

YOLO inqilobiy algoritm hisoblanadi, chunki u rasmni bir marta neyron tarmog'idan o'tkazishda barcha obyektlarni aniqlaydi. Eski algoritmlar rasmni bo'laklab, har birini alohida tekshirgan va juda sekin ishlagan. YOLO soniyasiga 45 tadan 150 tagacha kadrni qayta ishlay oladi, bu esa uni real vaqtdagi video-kuzatuvlar uchun tengsiz qiladi.

? **157. "Face Recognition" (Yuzni tanish) qanday bosqichlardan iborat?**



1. **Face Detection:** Rasmda inson yuzi borligini aniqlash.

2. **Alignment:** Yuzni to'g'ri burchakka keltirish (ko'zlar va burun joylashuvini tekislash).

3. **Feature Extraction:** Yuzning takrorlanmas "kodini" (embedding vektorini) yaratish.

4. **Verification:** Olingan kodni bazadagi mavjud kodlar bilan solishtirish.

☆ **158. "Semantic Segmentation" qachon ishlatiladi?**

Bu usul rasmning har bir pikseliga klass beradi. Masalan, rasmning bir qismini "yo'l", ikkinchi qismini "osmon", uchinchisini "daraxt" deb ajratib chiqadi. **Ishlatilishi:** Avtopilot mashinalarda yo'lining har bir santimetrini aniqlashda va tibbiyotda rentgen rasmidagi organlar chegarasini aniq chizishda.

? **159. "Keypoint Detection" nima?**

📖 Bu obyektning muhim nuqtalarini (bo‘g‘inlarini) topishdir. Masalan, inson tanasidagi 17 ta nuqtani (tirsak, tizza, ko‘zlar) topish orqali uning harakatini (Pose Estimation) tahlil qilish mumkin. Bu fitness ilovalarida yoki sportchilar harakatini o‘rganishda ishlatiladi.

☆ 160. "Optical Character Recognition" (OCR) qanday ishlaydi?

📖 OCR — rasmdagi matnlarni kompyuter tushunadigan matn formatiga o‘giradi. AI avval harflarning shaklini (patterns) taniydi, keyin ularni so‘zlarga birlashtiradi. Zamonaviy OCR tizimlari qo‘lyozmalarni ham o‘qiy oladi.

❓ 161. "Image Augmentation" — real misol keltiring.

📖 Sizda 100 ta rasm bor, bu AI uchun juda kam. **Augmentation:** Har bir rasmni 10 darajaga qiyshaytirasiz, rangini biroz xiralashtirasiz yoki chapdan o‘ngga akslantirasiz. Natijada sizda 500 ta "yangi" rasm hosil bo‘ladi. Bu modelga obyektни har qanday sharoitda taniy olishni o‘rgatadi.

☆ 162. "Pre-trained Models" (ResNet, VGG, Inception) nima uchun kerak?

📖 Bu dunyo miqyosidagi ulkan ma’lumotlarda o‘qitilgan tayyor "miyalar". Ularni o‘qitish uchun minglab videokartalar kerak bo‘lgan. Siz ularni olib, o‘z loyihangizga (Transfer Learning orqali) moslashtirib ishlatishingiz mumkin. Bu noldan boshlashga qaraganda 100 baravar tezroq.

❓ 163. "IoU" (Intersection over Union) metriksi nimani o‘lchaydi?

📖 U Object Detection modellarining aniqligini o‘lchaydi. Model chizgan to‘rtburchak haqiqiy obyekt turgan joy bilan qanchalik mos kelishini (ustma-ust tushishini) ko‘rsatadi. Ball 1.0 ga yaqin bo‘lsa — model obyektни ideal topgan bo‘ladi.

☆ 164. "Anchor Boxes" Object Detection'da nima uchun ishlatiladi?

📖 Rasmdagi obyektlar turli shaklda bo‘ladi (masalan, odam — uzunchoq, mashina — enli). Algoritmga yordam berish uchun oldindan tayyorlangan shakl shablonlari (Anchor Boxes) beriladi. Algoritm obyektни aynan shu shablonlarga qiyoslab tezroq topadi.

❓ 165. "NMS" (Non-Maximum Suppression) algoritmi nima?

📖 Ba’zan model bitta obyekt ustida 10 ta to‘rtburchak chizib yuboradi. NMS ularning ichidan ehtimolligi eng yuqorisini qoldirib, qolgan ustma-ust tushgan "soxta" to‘rtburchaklarni o‘chirib tashlaydi.

☆ 166. MLOps (Machine Learning Operations) tushunchasi nimani anglatadi?

📖 MLOps — bu AI modellarini yaratish va ularni serverda doimiy ishlatish madaniyatidir. Oddiy dasturdan farqli o'laroq, AI modelini bir marta yaratib qo'yish yetarli emas. Uni doimiy monitoring qilish, yangi ma'lumotlar bilan qayta o'qitish va xatosiz ishlashini ta'minlash kerak.

❓ 167. "Model Versioning" (Model versiyalash) nega muhim?

📖 Sizda yangi model (v2) bor, lekin u eskisidan (v1) yaxshiroq ekaniga ishonchingiz komil emas. Versiyalash sizga istalgan vaqtda orqaga qaytish (rollback) yoki ikki modelni solishtirish imkonini beradi. Buning uchun DVC (Data Version Control) kabi vositalar ishlatiladi.

★ 168. "Data Drift" (Ma'lumotlar siljishi) muammosini tushuntiring.

📖 Dastur kodida xato bo'lmasa-da, model vaqt o'tishi bilan noto'g'ri ishlay boshlaydi. **Sabab:** Atrof-muhit o'zgaradi. Masalan, modalar o'zgargani uchun "kiyim tanish" modeli eski kiyimlarni taniy olmay qoladi. MLOps bu "siljish"ni aniqlab, modelni yangilash haqida xabar beradi.

❓ 169. AI modelini serverga yuklashda "API"ning roli qanday?

📖 Model serverda bir chekkada turadi. Tashqaridan (mobil ilova yoki saytdan) rasm kelganda, **API** (FastAPI yoki Flask yordamida) bu rasmni modelga uzatadi va modeldan kelgan javobni (masalan: "Bu olma") foydalanuvchiga qaytaradi.

★ 170. "Edge AI" nima va uning foydasi nimada?

📖 Bu AI modelini ulkan serverlarda emas, balki qurilmaning o'zida (telefonda, kamerada, datchikda) ishlatishdir. **Foydasi:** Internet shart emas, ma'lumotlar maxfiy qoladi va javob juda tez qaytadi (masalan, yuzni tanish orqali qulfni ochish).

❓ 171. "Model Monitoring" nima uchun kerak?

📖 Model serverda ishlayotganda u qancha RAM sarflayotgani, so'rovlarga necha millisekundda javob berayotgani va xatolari ko'payib ketmaganini kuzatib borishdir. Bu tizimning "sog'lig'ini" bildiradi.

★ 172. "A/B Testing" AI modellarini solishtirishda qanday qo'llaniladi?

📖 Foydalanuvchilarning bir qismiga eski model, ikkinchi qismiga yangi model natijalari ko'rsatiladi. Agar yangi model haqiqatda ko'proq foyda keltirsa (masalan, ko'proq mahsulot sotilsa), u hamma uchun yoqiladi.

❓ 173. "ONNX" (Open Neural Network Exchange) formati nima uchun kerak?

📖 Siz modelni PyTorch'da yozgan bo'lishingiz mumkin, lekin serveringiz faqat boshqa muhitda ishlaydi. **ONNX** — modelni istalgan framework'dan (PyTorch, TensorFlow) boshqasiga xatosiz o'tkazish uchun umumiy "tarmog'aro" formatdir.

☆ 174. "Cloud AI" (AWS, Google Cloud, Azure) platformalarining afzalligi?

📖 Ular tayyor infratuzilmani beradi. Sizga o'z serveringizni sotib olish shart emas. Siz bir necha tugmani bosish orqali modelni millionlab foydalanuvchilarga xizmat qiladigan darajada kengaytirishingiz (scaling) mumkin.

❓ 175. "Fairness in AI" (Aida adolatlilik) va Etika nima uchun muhim?

📖 Model o'qitilgan ma'lumotlar tufayli biror guruhga nisbatan adolatsiz bo'lishi mumkin (masalan, yuzni tanish tizimi faqat bir xil irqdagi odamlarni yaxshi tanishi). MLOps mutaxassislari modelni nafaqat aniqlikka, balki uning hamma uchun teng va adolatli ishlashiga ham tekshirishi shart.

Ajoyib! **4-qism: Kompyuter ko'rishi (Computer Vision) va MLOps** bo'limini yakunlovchi so'nggi 25 ta (176–200) savol-javobga o'tamiz. Bu qismda biz modelni ishlab chiqarishga (Production) tayyorlash, uning xavfsizligi va arxitektura nozikliklarini ko'rib chiqamiz.

❓ 176. "One-Shot Learning" nima va u yuzni tanish tizimlarida qanday qo'llaniladi?



An'anaviy neyron tarmoqlar bitta obyektни tanishi uchun minglab rasmlar kerak bo'ladi. Lekin FaceID tizimida siz telefoningizga yuzingizni faqat bir marta ko'rsatasiz.

One-Shot Learning (Siam tarmoqlari yordamida) rasmni klassifikatsiya qilmaydi, balki ikkita rasm orasidagi o'xshashlik masofasini o'lchaydi. Model "Bu kim?" deb emas, "Mana bu yangi rasm bazadagi egasining rasmiga qanchalik o'xshash?" deb savol beradi.

☆ 177. "Instance Segmentation" va "Semantic Segmentation" o'rtasidagi farqni real hayotiy misolda tushuntiring.



- **Semantic Segmentation:** Rasmdagi bir xil turdagi barcha obyektlarni bitta rangga bo'yaydi. Masalan, ko'chada ketayotgan barcha 5 ta odamni yaxlit "odamlar to'dasi" (ko'k rang) deb tushunadi.

- **Instance Segmentation:** Har bir obyektни alohida shaxs deb taniydi. Masalan, 5 ta odamning har birini alohida rang (qizil, yashil, sariq va h.k.) bilan bo'yaydi. Bu robot-jarrohlikda yoki omborxona robotlarida har bir detalni alohida ushlab uchun o'ta muhimdir.

🔗 **178. "Object Tracking" (Obyektни kuzatish) nima va u "Detection"dan nimasi bilan farq qiladi?**



Detection har bir kadrda (frameda) obyektни qaytadan qidiradi. Tracking esa kadrlar o'rtasidagi bog'liqlikни tushunadi. Masalan, futbol o'yinida 10-kadrda aniqlangan to'p 11-kadrda biroz siljigan bo'lsa, Tracking bu o'sha-o'sha to'p ekanini tushunadi va unga bitta ID raqamini biriktiradi. Bu videoni tahlil qilishda hisoblash resurslarini tejaydi.

★ **179. "Inception Module" (GoogLeNet) nima uchun yaratilgan?**



Bu modulning asosiy g'oyasi — "Parallel Convolution". Bir qatlamda faqat 3x3 filtr ishlatmasdan, bir vaqtning o'zida 1x1, 3x3 va 5x5 filtrlarni ishlatadi.

Nima uchun: Rasmdagi obyekt juda katta (masalan, butun rasmda bitta yuz) yoki juda kichik bo'lishi mumkin. Inception turli o'lchamdagi filtrlarni birlashtirib, obyekt o'lchamidan qat'i nazar uni aniq taniy olishga yordam beradi.

🔗 **180. "Bounding Box Regression" nima?**



Object Detection modellarida neyron tarmoq ikkita narsani bashorat qiladi:

1. Obyektning turi (Klassifikatsiya).
2. Obyekt turgan to'rtburchakning koordinatalari (x , y , width, height).

Regression qismi aynan shu to'rtburchakning chegaralarini pikseligacha to'g'ri joylashtirish (ya'ni haqiqiy obyektни iloji boricha qisib ushlab uchun xizmat qiladi).

★ **181. "Data Pipeline" nima va u MLOps loyihalarida qanday rol o'ynaydi?**



Ma'lumotlar xom ko'rinishdan modelga tayyor holatga kelgunicha bir nechta bosqichdan o'tadi (tozalash, normalizatsiya, formatni o'zgartirish).

Pipeline — bu jarayonning avtomatlashgan konveyeridir. Dasturchi har gal qoʻlda kod yozib oʻtirmaydi; yangi maʼlumot kelishi bilan Pipeline uni avtomatik qayta ishlab, modelni oʻqitishga yuboradi.

? 182. "CI/CD/CT" tushunchasi AI loyihalarida nimani anglatadi?



- **CI (Continuous Integration):** Kod va maʼlumotlarni muntazam testlash.
- **CD (Continuous Deployment):** Modelni avtomatik serverga yuklash.
- **CT (Continuous Training):** MLOps'ga xos tushuncha. Agar modelning aniqligi tushib ketsa, tizim yangi maʼlumotlar bilan modelni **avtomatik qayta oʻqitishni** boshlaydi.

☆ 183. "Model Compression" (Modelni siqish) usullari qaysilar?



Agar model juda ogʻir boʻlib, telefonda ishlamasa, quyidagilar qoʻllaniladi:

1. **Pruning:** Tarmoqdagi eng kuchsiz, natijaga taʼsir qilmaydigan vaznlarni (weights) oʻchirib tashlash.
2. **Quantization:** Raqamlarning aniqligini kamaytirish (masalan, 32 bitli sonlarni 8 bitga oʻtkazish).
3. **Knowledge Distillation:** Katta modelni "oʻqituvchi" qilib, uning bilimni kichik modelga oʻrgatish.

? 184. "Hyperparameter Tuning"da "Grid Search"dan koʻra "Random Search" nega afzal?



Grid Search barcha variantlarni tekshirib juda koʻp vaqt yoʻqotadi. Random Search esa tasodifiy nuqtalarni tekshirib, muhim boʻlmagan parametrlarga vaqt sarflamaydi. Tadqiqotlar shuni koʻrsatadiki, Random Search kamroq hisoblash bilan deyarli bir xil sifatda eng yaxshi parametrlarni topa oladi.

☆ 185. "Model Serving" nima va u orqali qanday qilib millionlab foydalanuvchiga xizmat koʻrsatiladi?



Bu modelni API (masalan, TensorFlow Serving, TorchServe yoki NVIDIA Triton) koʻrinishida ishga tushirishdir. U kelayotgan soʻrovlarni navbatga qoʻyadi, yuklamani

bir nechta videokartalarga taqsimlaydi va javoblarni tezkor qaytaradi. Bu modelning "production" muhitida barqaror ishlashini ta'minlaydi.

🔗 186. "Feature Store" nima uchun kerak?



Katta kompaniyalarda o'nlab jamoalar turli AI loyihalari ustida ishlaydi. Feature Store — bu tayyorlangan, tozalangan ma'lumotlar (features) omboridir. Masalan, "mijozning oxirgi bir oydagi xarajatlari" feature-sini bir marta hisoblab, omborga qo'yishadi, shunda boshqa jamoalar uni noldan hisoblab o'tirmaydi.

★ 187. "Kubeflow" yoki "MLflow" kabi vositalarning vazifasi nima?



Bular MLOps platformalaridir. Ular model qachon o'qitilgani, qaysi ma'lumotlar ishlatilgani va qanday natija (accuracy) berganini yozib boradi. Bu dasturchiga o'zining 100 ta sinovini tartibli saqlashga va eng yaxshisini osongina tanlab olishga yordam beradi.

🔗 188. "A/B Testing" AI loyihalarida qanday xavfni kamaytiradi?



Yangi model laboratoriyada yaxshi natija bergani bilan, real foydalanuvchilarga yoqmasligi mumkin. A/B test yangi modelni faqat 5% foydalanuvchida sinab ko'rib, biznes uchun (masalan, foyda kamayishi kabi) jiddiy xavflarning oldini oladi.

★ 189. "Concept Drift" nima?



Bu Data Drift'dan murakkabroq. Ma'lumotlar o'zgarmaydi, lekin ma'no o'zgaradi.

- **Misol:** Firibgarlikni aniqlovchi model. Firibgarlar o'z uslubini mutlaqo o'zgartirsa, eski belgilar endi ishlamay qoladi. Modelning mantiqiy fundamenti eskirishi Concept Drift deyiladi.

🔗 190. "Explainability" (XAI) da LIME va SHAP vositalari qanday ishlaydi?



Ular "Qora kuti" ichini ko'rsatib beradi. Masalan, AI bemorga tashxis qo'ysa, SHAP har bir simptom (isitma, yo'tal) qarorga necha foiz ta'sir qilganini ko'rsatib beradi. Bu insonning AI qaroriga ishonchini oshiradi.

★ 191. "Docker" va "Kubernetes" MLOps'da nega shart?



- **Docker:** Model va uning barcha kutubxonalarini (Python versiyasi, keros, pytorch) bitta konteynerga o‘raydi. Bu modelning har qanday kompyuterda bir xil ishlashini kafolatlaydi.

- **Kubernetes:** Agar foydalanuvchilar ko‘paysa, model konteynerlarini avtomatik ravishda ko‘paytirib, bir nechta serverga tarqatadi (Auto-scaling).

🔗 192. "Model Decay" (Model eskirishi) qanday aniqlanadi?



Muntazam ravishda modelning bashoratlarini haqiqiy natijalar bilan solishtirish orqali. Agar vaqt o‘tishi bilan "Accuracy" yoki "F1-score" grafigi pastga qarab keta boshlasa, demak model eskirgan va uni qayta o‘qitish vaqti kelgan.

★ 193. AI loyihalarida "Security" (Xavfsizlik) — Adversarial Attacks nima?



Xaker rasmga ko‘zga ko‘rinmas kichik shovqin (noise) qo‘shishi mumkin. Inson uchun bu hali ham "Panda" bo‘lib ko‘rinadi, lekin AI bu shovqin tufayli rasmni "Gvintovka" deb taniy boshlaydi. Bu avtopilot mashinalar uchun juda xavfli. MLOps buni testlashni o‘z ichiga oladi.

🔗 194. "Cloud-native AI" nima?



Bu modelni noldan yaratish o‘rniga, AWS, Google Cloud yoki Azure taqdim etayotgan tayyor xizmatlardan (masalan, Amazon Rekognition) foydalanish. Bu kichik startaplar uchun o‘z AI xizmatini bir necha soatda ishga tushirish imkonini beradi.

★ 195. "Cold Start Problem" tavsiya tizimlarida nima?



Yangi foydalanuvchi ro‘yxatdan o‘tsa, tizim unga nima tavsiya qilishni bilmaydi (chunki ma’lumot yo‘q). Bu Cold Start.

Yechim: Foydalanuvchidan qiziqishlarini so‘rash yoki eng mashhur (top) kontentlarni ko‘rsatish.

🔗 196. "Transfer Learning" da "Freezing" bosqichi nega kerak?



Tayyor modelning boshlang'ich qatlamlari umumiy belgilarni (chiziqlar, burchaklar) taniy oladi. Biz ularni "muzlatib" (vaznlarini o'zgartirmay), faqat oxirgi qatlamlarni o'z ma'lumotlarimizga moslaymiz. Bu o'qitish jarayonini tezlashtiradi va overfittingni oldini oladi.

☆ 197. "NVIDIA TensorRT" nima uchun ishlatiladi?



Bu modelni NVIDIA videokartalarida eng yuqori tezlikda ishlashi uchun optimallashtiruvchi vosita. U modelning matematik amallarini videokarta arxitekturasiga moslab qayta tartiblaydi (Inference optimization).

? 198. "Human-in-the-loop" nima?



AI model har doim ham 100% to'g'ri javob bermaydi. Shubhali holatlarda qaror qabul qilishni insonga (moderatorga) topshiradigan tizim Human-in-the-loop deyiladi. Insonning tuzatishlari modelni yanada aqlliroq qilish uchun qayta o'qitishga yuboriladi.

☆ 199. "Ethical Bias" AI modeliga qayerdan kirib qoladi?



Model yomon bo'lgani uchun emas, u o'qigan ma'lumotlar adolatsiz bo'lgani uchun. Masalan, agar tarixda faqat erkaklar rahbar bo'lgan bo'lsa, AI "ayol kishi rahbar bo'lolmaydi" degan noto'g'ri xulosani o'rganib oladi. Buni tuzatish uchun ma'lumotlar balansini sun'iy to'g'rilash shart.

? 200. Kelajakdagi "Autonomous AI Agents" nima?



Hozirgi AI faqat sizning savolingizga javob beradi. Kelajakdagi Agentlar esa maqsad qo'yilsa (masalan: "Menga eng arzon aviachipta top va mehmonxona band qil"), mustaqil ravishda internetga kiradi, saytlarni ko'radi, narxlarni solishtiradi va hamma ishni oxirigacha bitiradi. Bu AI rivojlanishining keyingi bosqichidir.

5. Uchuvchisiz uchish apparatlari (dronlar)

Atamalar va izohlar

Atamalar	Izohlar
UUA (UAV)	Uchuvchisiz Uchish Apparati (Unmanned Aerial Vehicle) — bortida uchuvchisi bo'lmagan har qanday uchish qurilmasi.
UAS	Unmanned Aircraft System — dron, pult va aloqa tizimini o'z ichiga olgan butun boshli kompleks.
VTOL	Vertical Take-Off and Landing — tikka ko'tariluvchi va qo'nuvchi qurilma (gibrid dronlar).
FC	Flight Controller (Parvoz nazoratchisi) — dronning "miyasi", barcha sensorlarni boshqaruvchi mikrokontroller.
ESC	Electronic Speed Controller — motorlarning aylanish tezligini elektr quvvati orqali boshqarib beruvchi qurilma.
IMU	Inertial Measurement Unit — dronning havoda muvozanat saqlashi uchun giro va akselerometr datchiklari jamlanmasi.
P-I-D	Proportional-Integral-Derivative — dronning havoda silliq va qimirlamay turishini ta'minlovchi matematik algoritim.
FPV	First Person View — birinchi shaxs ko'rinishi (dron kamerasidagi tasvirni ko'zoynak orqali real vaqtda ko'rish).
OSD	On-Screen Display — uchish ma'lumotlarini (tezlik, batareya) jonli video tasvir ustiga yozma chiqarish qatlami.
RTH	Return to Home — aloqa uzilganda yoki batareya tugaganda dronning avtomatik uchgan nuqtasiga qaytish funksiyasi.
GCS	Ground Control Station — yerdagi boshqaruv stansiyasi (dronni kuzatish va unga vazifa berish uchun kompyuter dasturi).
MAVLink	Micro Air Vehicle Link — dron va yerdagi stansiya orasidagi ma'lumot almashish protokoli (tili).
SLAM	Simultaneous Localization and Mapping — bir vaqtning o'zida ham o'zi turgan joyni aniqlash, ham hudud xaritasini chizish.
LIDAR	Light Detection and Ranging — lazer nurlari yordamida masofani o'lchash va atrofning 3D modelini yaratish tizimi.
RTK	Real-Time Kinematic — GPS aniqligini 1-3 santimetrgacha oshiruvchi yuqori aniqlikdagi navigatsiya tizimi.
BVLOS	Beyond Visual Line of Sight — dronni ko'rish masofasidan tashqarida (juda uzoqda) boshqarish.
GSD	Ground Sampling Distance — dron xaritasidagi bitta piksel yerda necha santimetrga tengligini ko'rsatuvchi aniqlik balli.
NDVI	Normalized Difference Vegetation Index — o'simliklar salomatligini aniqlash uchun ishlatiladigan spektral ko'rsatkich.
Payload	Foydali yuk — dron ko'tarib yuradigan qurilmalar (kamera, raketa, oziq-ovqat va h.k.).
Jammer	Signal bo'g'uvchi — dron va pult orasidagi radioaloqani to'sib qo'yuvchi elektron qurol.
UCAV	Unmanned Combat Aerial Vehicle — zarba beruvchi yoki jangovar harbiy dron.
ISR	Intelligence, Surveillance, Reconnaissance — razvedka, kuzatuv va ma'lumot yig'ish vazifasi.
Loitering Munition	Daydi o'q-dori — nishonni qidirib havoda uzoq aylanib yuruvchi kamikaze droni.

MALE / HALE	Medium/High-Altitude Long-Endurance — oʻrta yoki yuqori balandlikda uzoq muddat uchuvchi dronlar sinfi.
SATCOM	Satellite Communication — dronni sunʼiy yoʻldosh orqali dunyoning istalgan nuqtasidan boshqarish tizimi.
SIGINT	Signals Intelligence — dushmanning radio va telefon signallarini havoda tutib oluvchi razvedka turi.
EW	Electronic Warfare (Elektron urush) — radio va GPS signallarini boʻgʻish yoki aldash orqali jang qilish.
Geofencing	Virtual devor — dronni maʼlum bir hududdan tashqariga chiqarmaslik yoki taqiqlangan joyga kiritmaslik tizimi.
Drone Swarm	Dronlar toʻdasi — yuzlab dronlarning yagona jamoa boʻlib, oʻzaro kelishilgan holda harakatlanishi.
Remote ID	Masofaviy identifikatsiya — dronning raqamli pasporti (joylashuvi va egasi haqidagi maʼlumotni tarqatib turadi).

Savol-javoblar

? 1. UUA (UAV) va Dron tushunchalari oʻrtasida farq bormi?

 **UUA (UAV - Unmanned Aerial Vehicle)** — bu bortida uchuvchisi boʻlmagan har qanday uchish apparatining texnik nomi. **Dron** — bu soʻz koʻproq xalq orasida va ommaviy axborot vositalarida qoʻllaniladigan umumiy nomdir (inglizcha "drone" - arining gʻoʻngʻillashi degan maʼnoni anglatadi). Shuningdek, **UAS (Unmanned Aircraft System)** tushunchasi ham bor, u nafaqat dronni, balki uni boshqaradigan yerdagi pult va aloqa kanallarini ham oʻz ichiga olgan butun bir tizimni anglatadi.

☆ 2. Dronlar necha turga boʻlinadi?

 Dronlar asosan konstruksiyasi va uchish mexanizmiga koʻra 3 turga boʻlinadi:

1. **Multi-rotor (Multirotorlar):** Eng keng tarqalgan (kvadrokopter, geksakopter). Tikka koʻtarila oladi va havoda muallaq tura oladi.

2. **Fixed-wing (Qanotli dronlar):** Samolyotga oʻxshaydi. Koʻp energiya tejaydi, uzoq masofaga uchadi, lekin havoda toʻxtab tura olmaydi.

3. **VTOL (Vertical Take-Off and Landing):** Gibril tur. Tikka koʻtariladi, lekin havoda samolyot kabi gorizontal uchadi.

? 3. Kvadrokopter qanday qilib yoʻnalishini oʻzgartiradi?

 Kvadrokopterda 4 ta motor bor. Ularning ikkitasi soat mili boʻylab, ikkitasi esa unga teskari aylanadi (bu dronni oʻz oʻqi atrofida aylanib ketishdan saqlaydi).

- **Koʻtarilish:** 4 ta motorning tezligi baravar oshiriladi.
- **Burilish (Yaw):** Qarama-qarshi turgan ikki motor tezlatiladi, qolgan ikkitasi sekinlatiladi.

- **Oldinga/Orqaga (Pitch):** Orqa motorlar tezlatilib, oldingilari sekinlatiladi (dron oldinga egiladi).

☆ 4. Dronning "miyasi" — **Flight Controller (Parvoz nazoratchisi)** nima?

📖 **Flight Controller (FC)** — bu dronning barcha datchiklaridan (sensorlaridan) ma'lumot olib, motorlar tezligini soniyasiga yuzlab marta hisoblab, boshqarib turuvchi mikrokontroller (kichik kompyuter). U drosel, giro va akselerometr ma'lumotlarini qayta ishlab, dronni havoda barqaror ushlab turadi.

❓ 5. **IMU (Inertial Measurement Unit) datchigi nima vazifani bajaradi?**

📖 **IMU** — bu dronning havoda qanday holatda turganini aniqlaydigan eng muhim datchiklar jamlanmasi. U odatda ikki qismdan iborat:

1. **Gyroscope (Giroscop):** Dronning qaysi burchakka og'ganini aniqlaydi.
2. **Accelerometer:** Dronning tezlanishi va og'irlik kuchini o'lchaydi. Busiz dron havoda muvozanatni saqlay olmaydi va darhol ag'darilib tushadi.

☆ 6. **ESC (Electronic Speed Controller) nima uchun kerak?**

📖 **ESC** — bu parvoz nazoratchisidan (FC) kelgan buyruqni qabul qilib, motorlarga kerakli miqdorda elektr quvvati yuboruvchi qurilma. U akkumulyatordagi to'g'ridan-to'g'ri tokni motor aylanishi uchun kerakli uch fazali tokka aylantirib beradi. Har bir motorning o'z ESC bo'lishi shart.

❓ 7. **Dronlarda Lipo (Lithium Polymer) akkumulyatorlar ishlatilishining sababi nima?**

📖 Lipo akkumulyatorlar boshqa turdagi (masalan, Li-ion) batareyalarga qaraganda "**Power-to-Weight**" (quvvatning vaznga nisbati) bo'yicha eng yuqori ko'rsatkichga ega. Ular juda yengil va qisqa vaqt ichida juda katta tok (C-rating) bera oladi, bu esa dron motorlarini tez aylantirish uchun zarur.

☆ 8. **GPS datchigi drona qanday imkoniyatlar beradi?**

📖 **GPS (Global Positioning System)** dronga o'zining geografik koordinatalarini bilish imkonini beradi. Buning yordamida:

- **Position Hold:** Dron shamol bo'lsa ham bir nuqtada qimirilamay tura oladi.
- **Return to Home (RTH):** Aloqa uzilsa, dron avtomatik ravishda uchgan nuqtasiga qaytib keladi.
- **Waypoint Mission:** Oldindan chizilgan xarita bo'ylab avtonom uchadi.

❓ 9. **Dronlarda Barometr (Barometer) nima uchun ishlatiladi?**

📖 Barometr havo bosimini o'lchaydi. Balandlik oshgani sari bosim o'zgaradi. GPS balandlikni aniqlashda juda katta xatoga (5-10 metr) yo'l qo'yishi mumkin, Barometr esa dronning balandligini santimetr gacha aniqlikda ushlab turishga yordam beradi (Altitude Hold).

☆ 10. "Telemetry" (Telemetriya) nima?

📖 Telemetriya — bu drondan yerdagi boshqaruv pultiga yoki kompyuterga keladigan real vaqtdagi ma'lumotlar oqimi. Bunga batareya quvvati, balandlik, tezlik, GPS koordinatalari va motorlar holati kiradi. Bu ma'lumotlar orqali uchuvchi dronning holatini masofadan kuzatib turadi.

? 11. FPV (First Person View) tizimi qanday ishlaydi?

📖 FPV — bu "birinchi shaxs ko'rinishi" degan ma'noni anglatadi. Dronga o'rnatilgan kamera tasvirni maxsus video-uzatkich (VTX) orqali radioto'lqinlar yordamida uchuvchining ko'zoynagiga (goggles) yoki monitoriga yuboradi. Bu uchuvchiga xuddi dronning "kokpiti"da o'tirgandek uchish hissini beradi.

☆ 12. Dron pultlarida 2.4 GHz va 5.8 GHz chastotalarining farqi nimada?



- **2.4 GHz:** Asosan boshqaruv signali (pult va dron aloqasi) uchun ishlatiladi. U uzoq masofaga yetib boradi va to'siqlardan yaxshi o'tadi.
- **5.8 GHz:** Asosan video uzatish (FPV) uchun ishlatiladi. U tasvirni juda tez (kechikishsiz) uzatadi, lekin masofasi qisqaroq va to'siqlarga (devor, daraxt) sezgir.

? 13. "Latentlik" (Latency) nima va u dronlar uchun nega muhim?

📖 Latentlik — bu drondagi kameraning tasvirni olib, ko'zoynakka yetkazib berishigacha ketgan vaqt (kechikish). Agar latentlik yuqori bo'lsa, uchuvchi dron allaqachon to'qnashib ketganidan keyin rasmda to'siqni ko'radi. Poyga dronlarida latentlik 20-30 millisekunddan oshmasligi kerak.

☆ 14. OSD (On-Screen Display) nima?

📖 OSD — bu dron kamerasidan kelayotgan jonli tasvir ustiga dronga tegishli ma'lumotlarni (masalan, batareya kuchlanishi, parvoz vaqti) yozma ravishda chiqarib beruvchi qatlamdır. Bu uchuvchiga tasvirdan ko'z uzmaganda holda muhim ko'rsatkichlarni bilish imkonini beradi.

? 15. Dronlarda "Magnetometr" (Compass) nima uchun kerak?

📖 Magnetometr dronning dunyo tomonlariga (Shimol, Janub) nisbatan qayerga qarab turganini aniqlaydi. Bu GPS bilan birgalikda dronning to'g'ri yo'nalishda (heading) uchishini ta'minlaydi. Yaqin atrofdagi temir konstruksiyalar yoki elektr liniyalari kompas ishiga xalaqit berishi mumkin.

★ 16. "Loss of Signal" (LoS) sodir bo'lganda "Fail-safe" tizimi qanday ishlaydi?

📖 **Fail-safe** — bu xavfsizlik protokoli. Agar dron pult bilan aloqani yo'qotsa, u avtomatik ravishda oldindan belgilangan biror amalni bajaradi:

1. **Drop:** Motorlarni o'chiradi va pastga tushadi.
2. **Land:** Turgan joyiga asta-sekin qo'nadi.
3. **RTH (Return to Home):** Avtomatik uchgan joyiga qaytib keladi.

❓ 17. PID Controller (Proportional-Integral-Derivative) nima?

📖 Bu dronning havoda silliq va qimirlamay turishini ta'minlaydigan matematik algoritm.

- **P (Proportional):** Xatolikni hozirgi holatda tuzatadi.
 - **I (Integral):** O'tmishdagi kichik og'ishlarni yig'ib, doimiy xatolarni tuzatadi.
 - **D (Derivative):** Kelajakdagi tebranishlarni bashorat qilib, dronni tormozlaydi.
- Dronni "sozlash" (tuning) aynan mana shu 3 ta koeffitsiyentni to'g'ri tanlashdir.

★ 18. Dron motorlaridagi "KV" ko'rsatkichi nimani anglatadi?

📖 **KV** — bu motorga 1 volt kuchlanish berilganda, u daqiqasiga necha marta aylanishini ko'rsatadi (RPM per volt).

- **Yuqori KV (masalan, 2500KV):** Kichik parraklar bilan tez uchish uchun.
- **Past KV (masalan, 400KV):** Katta parraklar bilan og'ir yuk ko'tarish uchun.

❓ 19. "Brushless" (Cho'tkasiz) motorlarning oddiy motorlardan afzalligi nimada?

📖 Brushless motorlarda ishqalanish juda kam, chunki unda fizik aloqa qiluvchi cho'tkalar yo'q. **Afzalliklari:** Yuqori samaradorlik (kam quvvat sarfi), juda uzoq xizmat muddati va o'ta yuqori aylanish tezligi. Zamonaviy dronlarning 99% qismi aynan shu motorlarda ishlaydi.

★ 20. Gimbal (Kamera stabilizatori) nima vazifani bajaradi?

 Gimbal — bu kamerani dronning tebranishlari va egilishlaridan mustaqil ravishda bir tekisda ushlab turuvchi mexanizm (odatda 3 oʻqli). Dron havoda shamolda qiyshayib tursa ham, gimbal tufayli olingan video xuddi shtativda turgandek silliq chiqadi.

❓ 21. Dronlar uchun eng mashhur "Open Source" parvoz dasturlari (Firmware) qaysilar?

 Dronning "miyasi" ishlashi uchun maxsus operatsion tizim kerak. Eng mashhurlari:

1. **ArduPilot:** Juda kuchli va ko'p qirrali. Professional va sanoat dronlari, avtonom kemalar va mashinalar uchun ishlatiladi.
2. **PX4 (Autopilot):** Akademik tadqiqotlar va sanoatda standart hisoblanadi. Juda xavfsiz va modulli tuzilishga ega.
3. **Betaflight:** Asosan poyga (FPV) dronlari uchun mo'ljallangan. Tezlik va boshqaruv sezgirlikiga (latency) urg'u beradi.

☆ 22. "Ground Control Station" (GCS) nima?

 GCS — bu yerda turib dronning parvozini kuzatish, unga yangi vazifalar yuklash va sozlash uchun ishlatiladigan dasturiy ta'minot (ko'pincha noutbuk yoki planshetda).

- **Mission Planner:** ArduPilot uchun asosiy dastur.
- **QGroundControl:** PX4 va ArduPilot uchun zamonaviy, mobil qurilmalarga mos interfeys.

❓ 23. MAVLink protokoli nima?

 MAVLink (Micro Air Vehicle Link) — bu dron va yerdagi stansiya (GCS) o'rtasida ma'lumot almashish uchun ishlatiladigan "til" (protokol). Bu protokol juda yengil bo'lib, batareya quvvati, GPS koordinatalari va buyruqlarni xavfsiz paketlarda uzatadi. Deyarli barcha zamonaviy avtopilotlar shu tilda gaplashadi.

☆ 24. "Autonomous Flight" (Avtonom parvoz) va "Automated Flight" farqi nimada?



- **Automated (Avtomatlashgan):** Dron oldindan belgilangan aniq yo'riqnomani bajaradi (masalan, "A nuqtadan B nuqtaga uch"). Agar yo'lda daraxt chiqsa, u urilib ketadi.

- **Autonomous (Avtonom):** Dron o'z datchiklari orqali qaror qabul qiladi. U to'siqlarni ko'radi, ularni aylanib o'tish yo'lini o'zi hisoblaydi va o'zgaruvchan vaziyatga moslashadi.

❓ 25. "SLAM" (Simultaneous Localization and Mapping) texnologiyasi dronga nima beradi?

📖 SLAM — bu dron bir vaqtning o'zida ham o'zi turgan joyni aniqlashi (Localization), ham atrofdagi noma'lum joyning xaritasini chizishi (Mapping) jarayonidir. Bu texnologiya dronlarga GPS ishlamaydigan joylarda (masalan, binolarning ichida yoki g'orlarda) xavfsiz uchish imkonini beradi.

☆ 26. "LIDAR" datchigi dronlarda nima uchun ishlatiladi?

📖 LIDAR (Light Detection and Ranging) — lazer nurlari yordamida masofani o'lchovchi datchik. Dron lazer yuboradi va uning qaytish vaqtiga qarab atrofning 3D modelini yaratadi. Bu to'siqlardan qochish va hududning o'ta aniq topografik xaritasini chizish (Mapping) uchun eng zo'r vositadir.

❓ 27. "Optical Flow" sensori qanday ishlaydi?

📖 Bu sensor dronning ostiga o'rnatilgan kichik kamera bo'lib, u yerning yuzasidagi piksellar harakatini kuzatadi. Agar GPS signali yo'qolsa (masalan, bino ichida), dron yerga qarab o'zining qaysi tomonga qancha siljiganini hisoblaydi va havoda bir nuqtada muallaq tura oladi.

☆ 28. Dronlarda "Computer Vision" (Kompyuter ko'rishi) qayerda qo'llaniladi?

📖 Kamera orqali kelayotgan tasvirni AI tahlil qiladi:

1. **Object Tracking:** Masalan, velosipedchini belgilasangiz, dron uning ortidan avtomatik ergashadi.

2. **Obstacle Avoidance:** Daraxtlar yoki simlarni tanib, ulardan qochadi.

3. **Precision Landing:** Yerdagi maxsus belgini (H harfi kabi) tanib, aynan o'sha yerga millimetr aniqlikda qo'nadi.

❓ 29. "Obstacle Avoidance" (To'siqlardan qochish) tizimi qanday sensorlarga tayanadi?

📖 Dron atrofni ko'rishi uchun quyidagi datchiklardan foydalanadi:

- **Ultrasonic (Ultratovush):** Yaqin masofadagi to'siqlarni aniqlash uchun (ko'pincha pastda).

- **Stereo Cameras:** Ikkita kamera yordamida (inson ko'zi kabi) masofani va chuqurlikni hisoblaydi.

- **Infrared (IQ):** Qorong'uda yoki shaffof to'siqlarni aniqlashda yordam beradi.

☆ 30. "Geofencing" nima?

📖 **Geofencing** — bu dron dasturiy ta'minotida chizilgan vertikal va gorizontal virtual devorlardir. Masalan, aeroportlar atrofida "No-Fly Zone" (Uchish taqiqlangan hudud) o'rnatiladi. Dron bu hududga yaqinlashsa, dastur uni to'xtatadi yoki ichkarida bo'lsa, motorini o'chiradi.

? 31. Dronlarda "RTK GPS" (Real-Time Kinematic) nima va u oddiy GPSdan nimasi bilan farq qiladi?

📖 Oddiy GPS 2-5 metr xatolikka yo'l qo'yadi. **RTK** tizimida drondan tashqari yerda qo'zg'almas "Base Station" (tayanch stansiya) bo'ladi. Ular o'zaro ma'lumot almashib, sun'iy yo'ldosh xatolarini tuzatadi. Natijada dron o'z joyini **1-3 santimetr** aniqlikda biladi. Bu qishloq xo'jaligi va qurilishda juda muhim.

☆ 32. "Collision Avoidance" algoritmlari qanday ishlaydi?

📖 Eng keng tarqalgan usul — **Vector Field Histogram**. Dron atrofini 360 darajali sektorlarga bo'ladi. Har bir datchikdan kelgan masofani o'lchaydi. Qaysi tomonda to'siq bo'lsa, o'sha tomonga "itarish kuchi" (repulsive force), borish kerak bo'lgan tomonga "tortish kuchi" (attractive force) qo'llaniladi.

? 33. "Companion Computer" (Masalan, Raspberry Pi yoki Jetson Nano) dronga nega kerak?

📖 Flight Controller (FC) faqat parvozni barqaror ushlab bilan band. Murakkab AI vazifalarni, masalan, yuzni tanish yoki 3D xarita chizishni FC bajara olmaydi. Shuning uchun dronga ikkinchi kuchli kompyuter qo'shiladi. U hisob-kitoblarni bajaradi va FC'ga "chapgga buril" yoki "to'xta" degan buyruqlarni yuboradi.

☆ 34. "Waypoints" nima?

📖 Waypoints — bu xaritada belgilangan nuqtalar ketma-ketligi. Dastur orqali dron qaysi nuqtaga qaysi balandlikda borishi, u yerda necha soniya kutishi yoki rasmga tushirishi kerakligi rejalashtiriladi. Bu jarayon "Mission Planning" deb ataladi.

? 35. Dronlarda "ROS" (Robot Operating System) nima uchun ishlatiladi?

📖 **ROS** — bu robotlar (shu jumladan dronlar) uchun yozilgan tayyor kodlar va kutubxonalar to'plamidir. Dasturchi har safar "kameradan rasm olish" yoki "to'siqni chetlab o'tish" kodini noldan yozmaydi, balki ROS'dagi tayyor modullarni birlashtiradi. Bu murakkab avtonom tizimlarni yaratishni tezlashtiradi.

☆ 36. "Fail-safe" turlari va ularning mantiqi.

📖 Dron kutilmagan vaziyatda qanday qaror qabul qilishi kerak?

- **Low Battery Fail-safe:** Quvvat kam qolsa, dron darhol uyiga qaytadi yoki turgan joyiga qo'nadi.

- **RC Loss Fail-safe:** Pult aloqasi uzilsa, 5 soniya kutadi va aloqa tiklanmasa RTH (Return to Home) rejimini yoqadi.

🔗 37. "Telemetry" ma'lumotlarini tahlil qilish (Log Analysis) nega muhim?

📖 Dron halokatga uchrasa (crash), uning ichidagi "Black Box" (qora quti) fayllari tahlil qilinadi. Dastur orqali motorlar vibratsiyasi, sensorlardagi xatoliklar yoki uchuvchining xatosi aniqlanadi. Bu kelgusidagi parvozlarni xavfsiz qilishga yordam beradi.

★ 38. Dronlarda "5G" texnologiyasi aloqani qanday o'zgartiradi?

📖 An'anaviy radioaloqa masofasi 5-10 km bilan cheklangan. 5G orqali dron internet tarmog'iga ulanadi. Bu degani, siz Toshkentda turib, masofaviy internet orqali Samarqanddagi dronni boshqarishingiz va undan 4K tasvirni kechikishsiz olishingiz mumkin bo'ladi.

🔗 39. "Swarm Intelligence" (To'da intellekti) nima?

📖 Bu o'nlab yoki yuzlab dronlarning xuddi qushlar yoki arilar kabi bitta yaxlit jamoa bo'lib uchishi. Dronlar o'zaro muloqot qiladi va bir-biriga urilib ketmasdan murakkab shakllarni hosil qiladi yoki katta maydonlarda qidiruv-qutqaruv ishlarini birgalikda bajaradi.

★ 40. Dronlarda "Edge Computing" afzalligi.

📖 Ma'lumotlarni serverga yuborib, u yerdan javob kutish vaqt oladi. **Edge Computing** — bu barcha AI hisob-kitoblarni dronning o'zida bajarish. Bu dronga to'siqlarga bir necha millisekund ichida reaksiya qaytarish imkonini beradi.

🔗 41. "Precision Agriculture" (Aniq dehqonchilik)da dronlarning roli qanday?

📖 Dronlar qishloq xo'jaligida inqilob qildi. Ular orqali:

1. **NDVI Mapping:** Maxsus multispektral kameralar yordamida o'simliklarning sog'lig'ini aniqlash (qaysi joyda o'g'it kam, qayerda kasallik boshlangan).

2. **Crop Spraying:** Dronlar dori va o'g'itlarni faqat kerakli nuqtalarga (spot spraying) sepa oladi. Bu xarajatlarni 30-50% gacha tejaydi.

3. **Inventory:** Ekinlar sonini va rivojlanish darajasini avtomatik hisoblash.

☆ 42. "NDVI" (Normalized Difference Vegetation Index) nima va u qanday o'lchanadi?

📖 NDVI — bu o'simliklarning fotosintez qilish darajasini ko'rsatuvchi indeks. Sog'lom o'simliklar infraqizil nurlarni kuchli qaytaradi va qizil nurlarni yutadi. Dron kamerasidagi datchiklar shu nurlarni o'lchab, dalaning "salomatlik xaritasi"ni chizadi. Qizil rangli joylar — o'simlik quriyotganini, yashil esa yaxshi rivojlanayotganini bildiradi.

❓ 43. Qurilish sohasida dronlar qanday foyda keltiradi?

📖 Qurilishda dronlar "Monitoring" vazifasini bajaradi:

- **Progress Tracking:** Har kuni bir xil balandlikdan rasmga olib, qurilish reja asosida ketayotganini tekshirish.
- **Volume Calculation:** Dron yordamida tuproq yoki qum uyumlarining hajmini bir necha daqiqada hisoblash mumkin (qo'lda bu soatlab vaqt oladi).
- **Safety:** Inson chiqishi xavfli bo'lgan balandliklarni (kranlar, tom) ko'zdan kechirish.

☆ 44. "Photogrammetry" (Fotogrammetriya) nima?

📖 Bu dron yordamida olingan yuzlab 2D rasmlarni birlashtirib, hududning o'ta aniq **3D modelini** yoki **Orthomosaic** (katta o'lchamli va aniq xarita) yaratish jarayonidir. Maxsus dasturlar (Pix4D, Agisoft Metashape) rasmlardagi o'xshash nuqtalarni topib, ularni fazoviy bog'laydi.

❓ 45. Elektr liniyalarini tekshirishda dronlarning afzalligi nimada?

📖 Liniyalarda minglab volt kuchlanish bor. Dronlar:

1. **Thermal Imaging:** Issiqlik kamerasidan foydalanib, qizib ketgan (nosoz) izolyatorlarni yoki transformatorlarni aniqlaydi.
2. **Zoom:** Kuchli optik yaqinlashtirish orqali liniyadagi kichik yoriqlarni masofadan ko'ra oladi. Bu jarayon vertolyot ijarasidan 10 baravar arzon va inson hayoti uchun mutlaqo xavfsiz.

☆ 46. "Payload" tushunchasi nimani anglatadi?

📖 **Payload** (Foydali yuk) — bu dronga o'rnatilgan, uning parvozi uchun shart bo'lmagan, lekin vazifa uchun kerakli qurilmalar. Masalan: kamera, dori sepish idishi, yetkazib berish paketi yoki maxsus sensorlar. Dron tanlashda uning "Max Payload Capacity" (maksimal ko'tara oladigan yuki) eng asosiy ko'rsatkichlardan biridir.

❓ **47. Yetkazib berish (Delivery) dronlari oldidagi asosiy texnik to'siqlar nima?**



1. **Battery Life:** Aksariyat dronlar yuk bilan 20-30 daqiqadan ko'p ucha olmaydi.
2. **Noise:** O'nlab dronlarning shahardagi shovqini aholiga noqulaylik tug'diradi.
3. **Safety:** Aholi zich joylarda parraklar odamlarga shikast yetkazishi xavfi bor.
4. **Beyond Visual Line of Sight (BVLOS):** Dronni ko'zdan uzoq masofada boshqarish qiyinchiliklari.

☆ **48. "BVLOS" (Beyond Visual Line of Sight) nima va u nega muhim?**

📖 BVLOS — bu uchuvchining ko'ziga ko'rinmaydigan masofada (masalan, 50 km uzoqlikda) dronni boshqarishdir. Sanoat uchun bu o'ta muhim (masalan, gaz quvurlarini tekshirish). Buning uchun dronda ishonchli sun'iy yo'ldosh aloqasi yoki 4G/5G tarmog'i va to'siqlardan avtomat qochish tizimi bo'lishi shart.

❓ **49. Dronlarni ro'yxatdan o'tkazish nega majburiy (ICAO va mahalliy qoidalar)?**

📖 Dronlar havo hududidan foydalanadi, demak ular samolyotlar va vertolyotlar uchun xavf tug'dirishi mumkin. Ro'yxatdan o'tkazish (masalan, O'zbekistonda ham maxsus qoidalar bor) orqali:

1. Noqonuniy parvozlarni to'xtatish.
2. Baxtsiz hodisa yuz berganda egasini topish.
3. Havo hududi xavfsizligini ta'minlash ko'zlanadi.

☆ **50. "Remote ID" texnologiyasi nima?**

📖 Remote ID — bu dronning "raqamli pasporti" (davlat raqami kabi). Dron uchayotganida o'zining ID raqami, joylashuvi va balandligini maxsus radioto'lqin (Bluetooth yoki Wi-Fi) orqali tarqatib turadi. Politsiya yoki havo harakatini nazorat qiluvchilar maxsus skaner orqali dron kimga tegishli ekanini darhol aniqlay oladi.

❓ **51. Dronlarda "Sense and Avoid" texnologiyasi nima?**

📖 Bu dronning boshqa uchish apparatlari bilan to'qnashuvining oldini oluvchi tizim. U **ADSB-In** datchigidan foydalanib, yaqin atrofdagi samolyotlarning signallarini qabul qiladi. Agar samolyot yaqinlashsa, dron avtomatik ravishda balandligini pasaytiradi yoki chetga chiqadi.

☆ 52. Qidiruv-qutqaruv ishlarida (SAR - Search and Rescue) dronlarning foydasi.

📖 Tog'larda yoki o'rmonlarda yo'qolgan odamlarni topishda dronlar:

- **Thermal Camera:** Inson tanasining issiqligini qorong'uda ham ko'ra oladi.
- **Speaker:** Odamga ovozli xabar berishi mumkin.
- **Payload Drop:** Birinchi yordam to'plami yoki suvni odamga yetkazib beradi.

❓ 53. "Drone Swarm" (Dronlar galasi) shousi qanday ishlaydi?

📖 Yuzlab dronlar bitta markaziy kompyuterga ulanadi. Har bir dronning aniq koordinatalari (RTK GPS orqali) va bajarishi kerak bo'lgan animatsiyasi oldindan hisoblanadi. Kompyuter ularga sinxron buyruq beradi va natijada osmonda ulkan 3D tasvirlar hosil bo'ladi.

☆ 54. Dronda "Vibration Isolation" (Vibratsiyani so'ndirish) nima uchun kerak?

📖 Motorlar soniyasiga minglab marta aylanadi va kuchli titrash hosil qiladi. Bu titrash:

1. Flight Controller datchiklarini (Giroscop) "aqlan ozdiradi".
2. Videoda "Jello effect" (tasvirning chayqalishi)ni keltirib chiqaradi. Buning oldini olish uchun "Gimbal" va "Anti-vibration dampeners" (rezina mahkamlagichlar) ishlatiladi.

❓ 55. "Ground Sampling Distance" (GSD) nima?

📖 GSD — bu dron yordamida olingan xaritada bitta piksel yerda necha santimetr to'g'ri kelishini bildiradi. Masalan, $GSD = 2$ cm bo'lsa, siz xaritada 2 santimetrdan katta bo'lgan har qanday detalni ko'ra olasiz. Bu xaritaning aniqlik darajasini belgilovchi bosh ko'rsatkichdir.

☆ 56. "Anti-Drone" tizimlari qanday ishlaydi?

📖 Taqiqlangan hududlarda (qamoqxona, harbiy baza) dronlarni to'xtatish uchun:

- **Jamming:** Dron va pult orasidagi signalni (2.4/5.8 GHz) kuchli shovqin bilan to'sib qo'yish.
- **Spoofing:** Dronning GPS signalini aldaydigan soxta koordinatalar yuborish (dron "men boshqa joydaman" deb o'ylaydi).
- **Physical capture:** To'r bilan tutish yoki lazer bilan urib tushirish.

❓ 57. Dron akkumulyatorlaridagi "C-rating" nimani bildiradi?

📖 "C" belgisi batareyaning o'zidan bir vaqtda qancha tok chiqara olish qobiliyatini ko'rsatadi. Masalan, 1500mAh batareyada 100C bo'lsa, u qisqa vaqtda motorlarga juda ulkan energiya bera oladi. Bu ayniqsa keskin manevr qiladigan poyga dronlari uchun muhim.

☆ 58. Dronlarning "Center of Gravity" (Og'irlik markazi) nega o'rtada bo'lishi kerak?

📖 Agar og'irlik markazi bir tomonga siljigan bo'lsa (masalan, batareya noto'g'ri qo'yilgan), o'sha tomondagi motorlar muvozanatni saqlash uchun qolganlaridan ko'ra ko'proq ishlashga majbur bo'ladi. Bu batareyaning tez tugashiga va motorlarning qizib ketishiga olib keladi.

❓ 59. "Wind Resistance" (Shamolga chidamlilik) nima?

📖 Har bir dronning maksimal bardosh bera oladigan shamol tezligi (masalan, 10 m/s) bo'ladi. Agar shamol kuchli bo'lsa, dron motorlari qarshi kuch bera olmaydi va shamol uni uchirib ketadi. Katta va og'ir dronlar shamolga chidamliroq bo'ladi.

☆ 60. "Drone as a Service" (DaaS) biznes modeli nima?

📖 Bu kompaniyalar dron sotib olmasdan, dron xizmatlarini (masalan, dalani dori sepish yoki xaritaga olish) ijaraga olishidir. Mutaxassis (Drone Pilot) o'z qurilmalari bilan keladi, ishni bajaradi va faqat olingan ma'lumotlar yoki natija uchun haq oladi.

Harbiy yo'nalishdagi dronlar (UUA) bugungi kunda zamonaviy urushlarning taktikasini butunlay o'zgartirib yubordi. Quyida harbiy dronlarning turlari, qo'llanilishi, texnik xususiyatlari va strategik ahamiyatiga oid keyingi 20 ta (61–80) savol-javobni taqdim etaman.

6. Tarmoq (Networking) texnologiyalari

Atamalar va izohlar

Atama	To'liq nomi / Kengaytmasi	Tavsif va Vazifasi
LAN	Local Area Network	Kichik hududiy (uy, ofis) ichki tarmog'i.
WAN	Wide Area Network	Katta geografik (shahar, global) tarmoq.
OSI	Open Systems Interconnection	Tarmoq muloqotining 7 qatlamli nazariy modeli.
TCP/IP	Transmission Control Protocol/IP	Internetning 4 qatlamli amaliy ishchi modeli.
IP Address	Internet Protocol Address	Qurilmaning tarmoqdagi mantiqiy raqamli manzili.
MAC Address	Media Access Control	Qurilma tarmoq kartasining fizik, o'zgarmas manzili.
DNS	Domain Name System	Sayt nomlarini IP manzilga o'giruvchi "telefon kitobi".
NAT	Network Address Translation	Ichki IPni global IPga o'zgartirib internetga chiqish.
DHCP	Dynamic Host Configuration Protocol	Qurilmalarga avtomatik IP manzil tarqatuvchi xizmat.
TCP	Transmission Control Protocol	Ma'lumot yetib borishini kafolatlovchi ishonchli protokol.
UDP	User Datagram Protocol	Tezkor, lekin yetib borishini kafolatlamaydigan protokol.
HTTP	HyperText Transfer Protocol	Veb-sahifalarni uzatishning asosiy qoidalar to'plami.
HTTPS	HTTP Secure	HTTPning shifrlangan, xavfsiz versiyasi.
SSL/TLS	Secure Sockets Layer / TLS	Ma'lumotlarni shifrlash va xavfsizlikni ta'minlash.
VPN	Virtual Private Network	Internet ustida qurilgan xavfsiz va maxfiy tunnel.
Router	Yo'naltiruvchi	Turli tarmoqlar orasida paketlarni yo'naltiruvchi qurilma.
Switch	Kommutator	MAC manzil orqali ichki tarmoq qurilmalarini bog'laydi.
Firewall	Tarmoqlararo ekran	Kelayotgan trafikni xavfsizlik uchun filtrlash tizimi.
Gateway	Asosiy shlyuz	Mahalliy tarmoqdan tashqi dunyoga chiqish nuqtasi.
Subnet Mask	Tarmoq osti maskasi	IPning tarmoq va xost qismini ajratuvchi raqam.
Atama	To'liq nomi / Kengaytmasi	Tavsif va Vazifasi
CIDR	Classless Inter-Domain Routing	IP manzillarni slash (/) orqali moslashuvchan yozish.
VLAN	Virtual Local Area Network	Bitta fizik switchda bir nechta mantiqiy tarmoq yaratish.
Packet	Paket	Tarmoq qatlamidagi (L3) ma'lumot birligi.
Frame	Freym	Kanal qatlamidagi (L2) ma'lumot birligi.
Port	Port	Ilovalarning "eshigi" (masalan: 80-HTTP, 443-HTTPS).
SMTP	Simple Mail Transfer Protocol	Elektron pochta yuborish protokoli.

IMAP	Internet Message Access Protocol	Pochtalarni serverda saqlab boshqarish usuli.
SSH	Secure Shell	Serverlarni masofadan shifrlangan boshqarish.
FTP	File Transfer Protocol	Fayllarni uzatishning klassik protokoli.
ICMP	Internet Control Message Protocol	Xatolarni xabar berish (Ping shu protokolda ishlaydi).
ARP	Address Resolution Protocol	IP manzilni MAC manzilga o'girib beruvchi protokol.
BGP	Border Gateway Protocol	Internet provayderlar orasidagi asosiy marshrutizator.
Latency	Kechikish	Ma'lumot borib-qaytish vaqti (Ping ko'rsatkichi).
Bandwidth	O'tkazuvchanlik	Tarmoq kanalining maksimal ma'lumot uzatish hajmi.
DDoS	Distributed Denial of Service	Saytni soxta so'rovlar bilan to'ldirib "yiqitish" hujumi.
Proxy	Proksi	Mijoz va internet orasidagi vositachi server.
Load Balancer	Yuklama taqsimlagichi	Trafikni bir nechta serverlarga teng bo'lib beruvchi.
WLAN	Wireless LAN	Wi-Fi texnologiyasi asosidagi simsiz tarmoq.
SSID	Service Set Identifier	Wi-Fi tarmog'ining hammaga ko'rinadigan nomi.
Access Point	Kirish nuqtasi	Kabeldagi internetni Wi-Fi signaliga aylantiruvchi.
Atama	To'liq nomi / Kengaytmasi	Tavsif va Vazifasi
Handshake	Salomlashish	Aloqa o'rnatishda qurilmalarning o'zaro kelishuvi.
CDN	Content Delivery Network	Kontentni eng yaqin serverdan yetkazish tarmog'i.
Socket	Soket	IP + Port kombinatsiyasi, ulanish nuqtasi.
WebSocket	WebSocket	Ikki tomonlama real-vaqt aloqasi (masalan, chat).
MTU	Maximum Transmission Unit	Bitta paketda yuborilishi mumkin bo'lgan maksimal bayt.
Loopback	127.0.0.1	Kompyuterning o'z-o'ziga murojaat qilish manzili.
SDN	Software Defined Networking	Dastur orqali markaziy boshqariladigan aqlli tarmoq.
PoE	Power over Ethernet	Bir kabelda ham ma'lumot, ham elektr toki uzatish.
Traceroute	Marshrut kuzatuv	Paket qaysi routerlardan o'tganini ko'rsatuvchi buyruq.
Ping	Packet Internet Groper	Aloqa bor-yo'qligini tekshiruvchi asosiy vosita.

Savol-javoblar

1. Kompyuter tarmog'i o'zi nima?

 Kompyuter tarmog‘i — bu ma’lumot almashish va resurslardan (printer, internet, fayllar) birgalikda foydalanish uchun bir-biriga bog‘langan kompyuterlar va qurilmalar to‘plamidir.

☆ 2. LAN va WAN o‘rtasidagi asosiy farq nimada?



- **LAN (Local Area Network):** Kichik hududni (uy, ofis, maktab) qamrab oluvchi tarmoq. Tezligi yuqori, kechikish kam.

- **WAN (Wide Area Network):** Katta geografik hududni (shahar, mamlakat, butun dunyo) qamrab oladi. Eng katta WAN — internetdir.

? 3. OSI modeli nima va u nega kerak?

 **OSI (Open Systems Interconnection)** modeli — bu tarmoq muloqotini tushuntirib beruvchi 7 qatlamli standartdir. U turli ishlab chiqaruvchilar yaratgan qurilmalarning bir-biri bilan muammosiz ishlashini ta’minlash uchun xizmat qiladi.

☆ 4. OSI modelining 7 ta qatlamini tartib bilan ayting.



1. Physical (Fizik)
2. Data Link (Kanal)
3. Network (Tarmoq)
4. Transport (Transport)
5. Session (Seans)
6. Presentation (Taqdimot)
7. Application (Ilova)

? 5. IP-manzil nima va u nega kerak?

 **IP (Internet Protocol)** manzil — bu tarmoqqa ulangan har bir qurilmaning takrorlanmas "raqamli manzili". U xuddi uy manzili kabi ma’lumotlar paketi aynan qaysi qurilmaga borishi kerakligini aniqlash uchun kerak.

☆ 6. TCP/IP modeli nima va u OSI modelidan nimasi bilan farq qiladi?

 TCP/IP — internetning real ishchi modeli bo‘lib, u 4 ta qatlamdan iborat (Application, Transport, Internet, Network Access). OSI ko‘proq nazariy qo‘llanma bo‘lsa, TCP/IP amaliy standartdir.

? 7. Protokol nima?

 Protokol — bu tarmoqdagi ikki qurilma qanday muloqot qilishi kerakligini belgilaydigan qoidalar to‘plami. Masalan, odamlar muloqotida "salomlashish" bir protokol bo‘lsa, internetda **HTTP** yoki **FTP** shunday vazifani bajaradi.

☆ 8. TCP va UDP protokollarining farqi nimada?



- **TCP (Transmission Control Protocol):** "Ishonchli" protokol. Ma’lumot yetib borishini kafolatlaydi, xatolarni tekshiradi. Fayl yuklash va veb-saytlar uchun ishlatiladi.

- **UDP (User Datagram Protocol):** "Tezkor" protokol. Ma’lumot yetib borishini kafolatlamaydi. Video qo‘ng‘iroqlar, onlayn o‘yinlar va striming uchun ishlatiladi.

? 9. HTTP va HTTPS farqi nimada?

 HTTPS — bu HTTPning shifrlangan versiyasidir. U **SSL/TLS** sertifikatlari yordamida foydalanuvchi va server orasidagi ma’lumotlarni begonalardan himoya qiladi. "S" harfi "Secure" (Xavfsiz) ma’nosini bildiradi.

☆ 10. DNS (Domain Name System) qanday ishlaydi?

 DNS — internetning "telefon kitobi"dir. Biz brauzerga `google.com` deb yozganimizda, DNS server uni kompyuter tushunadigan IP-manzilga (masalan, `142.250.190.46`) o‘girib beradi.

? 11. Router (Router) va Switch (Switch) farqi nimada?



- **Switch:** Bir tarmoq ichidagi (masalan, ofis ichidagi) qurilmalarni bir-biriga bog‘laydi.

- **Router:** Turli tarmoqlarni (masalan, uyingizdagi LANni internet bilan) bog‘laydi.

☆ 12. NAT (Network Address Translation) nima uchun kerak?

 NAT — bu bitta ochiq (public) IP-manzil orqali uyingizdagi o‘nlab qurilmalarni internetga chiqarish texnologiyasi. U IPv4 manzillar yetishmovchiligi muammosini hal qiladi va xavfsizlikni oshiradi.

? 13. VPN (Virtual Private Network) qanday vazifani bajaradi?

 VPN — ochiq internet ustidan xavfsiz va shifrlangan "tunnel" hosil qiladi. U foydalanuvchining haqiqiy IP-manzilini yashiradi va ma’lumotlarni xakerlardan himoya qiladi.

☆ 14. MAC-manzil nima va u IP-manzildan nimasi bilan farq qiladi?



- **MAC-manzil:** Qurilmaning tarmoq kartasiga ishlab chiqaruvchi tomonidan yozilgan o'zgarmas fizik raqam.
- **IP-manzil:** Tarmoq admini yoki provayder tomonidan beriladigan va o'zgarishi mumkin bo'lgan mantiqiy manzil.

? 15. DHCP (Dynamic Host Configuration Protocol) nima?



DHCP — bu tarmoqqa yangi ulangan qurilmalarga avtomatik ravishda IP-manzil, maska va shlyuzlarni tarqatuvchi xizmatdir. Busiz har bir telefonga IPni qo'lda yozib chiqish kerak bo'lar edi.

☆ 16. Firewall (Tarmoqlararo ekran) nima?



Firewall — bu tarmoqqa kirayotgan va undan chiqayotgan trafikni nazorat qiluvchi "filtr". U ruxsat berilmagan ulanishlarni to'sib, tarmoqni hujumlardan himoya qiladi.

? 17. Ping buyrug'i nima uchun ishlatiladi?



Ping — bu tarmoqdagi boshqa qurilma bilan aloqa bor-yo'qligini va ma'lumot borib-qaytish vaqtini (latency) tekshirish uchun ishlatiladigan vositadir.

☆ 18. Default Gateway (Asosiy shlyuz) nima?



Bu qurilmaning o'z tarmog'idan tashqariga (internetga) chiqish uchun o'tadigan "darvozasi"dir. Odatda bu sizning routingizning IP-manzili bo'ladi.

? 19. IPv4 va IPv6 farqi nimada?



IPv4 manzillar soni (4 milliarddan ortiq) tugab borayotgani uchun IPv6 yaratilgan. IPv4 32-bitli, IPv6 esa 128-bitli bo'lib, u deyarli cheksiz miqdordagi qurilmalarni ulash imkonini beradi.

☆ 20. Port (Port) nima?



Port — bu kompyuter ichidagi ma'lum bir xizmatning (programmaning) "eshigi". Masalan, 80-port veb-saytlar (HTTP) uchun, 443-port xavfsiz veb (HTTPS) uchun ishlatiladi. Bitta IPda minglab portlar bo'lishi mumkin.

? 21. "Subnet Mask" (Tarmoq osti maskasi) nima?

 Bu IP-manzilning qaysi qismi **Tarmoqqa** tegishli, qaysi qismi esa **Xostga** (qurilmaga) tegishli ekanini ko'rsatuvchi raqamdir. Masalan, 255.255.255.0 maskasi IP-manzilning birinchi uchta qismi tarmoq nomi ekanini bildiradi.

☆ 22. Public IP va Private IP farqi nimada?



- **Public IP:** Butun internet olamida ko'rinadigan va global ravishda takrorlanmas manzil.
- **Private IP:** Faqat mahalliy tarmoq (LAN) ichida ishlatiladigan manzil (masalan, 192.168.x.x). Ular internetda to'g'ridan-to'g'ri ishlamaydi (NAT orqali chiqadi).

? 23. "Loopback Address" (127.0.0.1) nima uchun kerak?

 Bu kompyuterning "o'z-o'ziga" murojaat qilishi uchun ishlatiladigan manzil. U tarmoq kartasining ishlashini tekshirish yoki kompyuterning o'zida o'rnatilgan serverga ulanish uchun ishlatiladi (Localhost).

☆ 24. "Static IP" va "Dynamic IP" farqi nimada?



- **Static IP:** Qurilmaga qo'lda berilgan va o'zgarmaydigan doimiy manzil (serverlar uchun muhim).
- **Dynamic IP:** DHCP server tomonidan vaqtincha beriladigan va har gal ulanishda o'zgarishi mumkin bo'lgan manzil.

? 25. CIDR (Classless Inter-Domain Routing) nima?

 Bu IP manzillarni klasslarga bo'lmasdan (A, B, C), yanada moslashuvchan taqsimlash usuli. U IP-manzil oxiriga slash (/) va raqam qo'yish orqali yoziladi (masalan, /24).

☆ 26. "TCP Three-Way Handshake" nima?

 Bu TCP ulanishni o'rnatish jarayoni bo'lib, 3 bosqichdan iborat:

1. **SYN:** Mijoz serverga "ulanishni boshlaylik" deydi.
2. **SYN-ACK:** Server javob beradi: "Kelishdik, men tayyorman".
3. **ACK:** Mijoz tasdiqlaydi: "Tushundim, ma'lumot yuboryapman".

? 27. "Latency" (Kechikish) nima?

📖 Ma'lumot paketining manbadan manzilga yetib borishi uchun ketgan vaqt (millisekundlarda o'lchanadi). "Ping" — bu kechikishni o'lchovchi asosiy ko'rsatkichdir.

☆ 28. "Bandwidth" (O'tkazuvchanlik qobiliyati) nima?

📖 Tarmoq kanali orqali ma'lum bir vaqt ichida o'tishi mumkin bo'lgan maksimal ma'lumot miqdori (Mbps yoki Gbps). Uni ko'pincha "Internet tezligi" deb ham atashadi.

? 29. "Packet Loss" (Paket yo'qolishi) nima?

📖 Ma'lumot paketlarining belgilangan manzilga yetib bormasligi. Bu tarmoqning yuklanishi, simlarning nosozligi yoki signallarning buzilishi natijasida yuz beradi.

☆ 30. Unicast, Broadcast va Multicast farqi?



- **Unicast:** Bitta qurilmadan bitta qurilmaga (1:1).
- **Broadcast:** Tarmoqdagi barcha qurilmalarga (1:hamma).
- **Multicast:** Faqat ma'lum bir guruh qurilmalarga (1:ko'pchilik).

? 31. FTP va SFTP farqi nimada?

📖 FTP fayllarni ochiq ko'rinishda uzatadi (xavfli). SFTP (Secure FTP) esa **SSH** protokoli orqali ma'lumotlarni shifrlab yuboradi.

☆ 32. SMTP, POP3 va IMAP nima uchun ishlatiladi?

📖 Bular elektron pochta protokollari:

- **SMTP:** Xat yuborish uchun.
- **POP3:** Xatni serverdan yuklab olib, serverdan o'chirib yuboradi.
- **IMAP:** Xatni serverning o'zida saqlaydi va bir nechta qurilmada sinxron ishlaydi (zamonaviy usul).

? 33. Telnet va SSH farqi nimada?

📖 Ikkalasi ham serverni masofadan boshqarish uchun. Telnet shifrlanmagan (xakerlar parolni ko'rishi mumkin), SSH (Secure Shell) esa barcha buyruqlarni shifrlaydi.

☆ 34. ARP (Address Resolution Protocol) vazifasi nima?

📖 IP-manzilni unga mos keladigan **MAC-manzilga** o‘g‘irib beradi. Qurilma IPni bilsa-da, paketni jo‘natish uchun fizik MAC-manzilni bilishi shart.

❓ 35. ICMP protokoli nima?

📖 Bu xatolarni xabar berish va diagnostika qilish protokoli. Mashhur Ping va Traceroute buyruqlari aynan ICMP ustida ishlaydi.

★ 36. VLAN (Virtual LAN) nima?

📖 Bitta jismoniy switch ichida tarmoqni mantiqiy ravishda bir nechta alohida tarmoqlarga bo‘lish. Bu xavfsizlikni oshiradi (masalan, buxgalteriya tarmog‘i mehmonlar tarmog‘idan ajratiladi).

❓ 37. "Hub" (Xab) va "Switch" (Switch) farqi nimada?

📖 Hub unga kelgan ma'lumotni barcha portlarga ko‘r-ko‘rona tarqatadi (tarmoqni yuklaydi). Switch esa faqat kerakli manzilga (MAC-manzil bo‘yicha) yuboradi. Hub’lar hozirda deyarli ishlatilmaydi.

★ 38. "Traceroute" buyrug‘i nima qiladi?

📖 Ma'lumot paketi sizdan servergacha qaysi routerlar (yo‘llar) orqali o‘tganini va har bir yo‘lda qancha vaqt ketganini ko‘rsatib beradi.

❓ 39. Proxy Server nima?

📖 Bu mijoz va internet orasidagi o‘rtachidir. U so‘rovlarni o‘z nomidan yuboradi, ma'lumotlarni keshlaydi (tezlashtiradi) va filtrlash imkonini beradi.

★ 40. "Load Balancer" (Yuklama taqsimlagichi) nima?

📖 Saytga bir vaqtning o‘zida millionlab odam kelsa, bitta server ko‘tara olmaydi. Load Balancer bu yuklamani bir nechta serverlarga teng taqsimlab beradi.

Tarmoq texnologiyalari bo‘yicha sanoqni 41-savoldan davom ettiramiz. Bu bosqichda Wi-Fi texnologiyalari, tarmoq xavfsizligi va murakkab ulanish protokollariga to‘xtalamiz.

❓ 41. Wi-Fi o‘zi nima?

📖 Wi-Fi — bu IEEE 802.11 standartlari asosida qurilmalarning radio to‘lqinlar orqali o‘zaro ma'lumot almashish texnologiyasi. U jismoniy kabel (Ethernet) o‘rnini bosadi.

★ 42. SSID (Service Set Identifier) nima?

📖 Bu simsiz tarmoqning (Wi-Fi) "nomi". Siz telefoningizda tarmoqlarni qidirganingizda ko'radigan nom aynan SSID hisoblanadi.

❓ 43. WPA2 va WPA3 protokollarining farqi nimada?

📖 Bular Wi-Fi xavfsizlik protokollaridir. **WPA3** zamonaviyroq va xavfsizroq bo'lib, u parollarni "brute-force" (taxmin qilib topish) hujumlaridan yaxshiroq himoya qiladi va ma'lumotlarni kuchliroq shifrlaydi.

★ 44. Wi-Fi'dagi 2.4 GHz va 5 GHz chastotalarining farqi?



- **2.4 GHz:** Masofasi uzoqroq, to'siqlardan (devorlardan) yaxshi o'tadi, lekin tezligi pastroq va shovqinlarga (mikroto'lqinli pech, Bluetooth) sezgir.
- **5 GHz:** Tezligi juda yuqori, shovqinlar kam, lekin masofasi qisqa va to'siqlardan yomon o'tadi.

❓ 45. "Access Point" (Kirish nuqtasi) nima?

📖 Bu qurilma simli tarmoqni simsiz tarmoqqa aylantiradi. U routerning Wi-Fi signalini yetib bormaydigan joylarga uzatish yoki katta binolarda yagona Wi-Fi tarmog'ini yaratish uchun ishlatiladi.

★ 46. Tarmoq topologiyasi nima?

📖 Bu tarmoqdagi qurilmalarning bir-biriga fizik yoki mantiqiy ulanish sxemasi.

❓ 47. "Star" (Yulduz) topologiyasining afzalligi nimada?

📖 Bunda barcha qurilmalar bitta markaziy tugunga (switch yoki hub) ulanadi. Afzalligi: bitta kompyuterning kabeli buzilsa, butun tarmoq to'xtab qolmaydi. Hozirgi ofis tarmoqlarining aksariyati shu turda.

★ 48. "Mesh" topologiyasi qayerda ishlatiladi?

📖 Har bir qurilma boshqa barcha qurilmalar bilan bevosita bog'langan bo'ladi. Bu o'ta ishonchli (redundant) tarmoq hisoblanadi. Agar bitta yo'l uzilsa, ma'lumot boshqa yo'l orqali ketadi. Harbiy va muhim infratuzilmalarda qo'llaniladi.

❓ 49. "Bus" (Shina) topologiyasi nima?

📖 Barcha qurilmalar bitta umumiy kabelga ketma-ket ulanadi. Bu eski va arzon usul, lekin asosiy kabel uzilsa, butun tarmoq ishdan chiqadi.

★ 50. DDoS (Distributed Denial of Service) hujumi nima?

 Xaker minglab "bot" (infeksiyalangan) kompyuterlar orqali bitta saytga bir vaqtda haddan tashqari ko'p so'rov yuboradi. Natijada server yuklamani ko'tara olmay "yiqiladi" va qonuniy foydalanuvchilar uchun yopilib qoladi.

51. "Man-in-the-Middle" (O'rtadagi odam) hujumi nima?

 Xaker foydalanuvchi va server orasidagi aloqani yashirincha kuzatadi yoki o'zgartiradi. Masalan, ochiq Wi-Fi orqali sizning parollaringizni tutib olishi mumkin.

52. "Port Scanning" nima uchun o'tkaziladi?

 Bu xaker yoki tarmoq administratori tomonidan serverdagi "ochiq eshiklar"ni (portlarni) topish uchun qilinadigan tekshiruv. Ochiq qolib ketgan va himoyalangan portlar hujum uchun yo'l ochishi mumkin.

53. "Intrusion Detection System" (IDS) nima?

 Bu tarmoq datchigi bo'lib, u shubhali harakatlarni (masalan, kiberhujum belgilarini) aniqlaydi va administratorga xabar beradi.

54. "SSL/TLS Handshake" nima?

 Brauzer va server o'rtasida xavfsiz (HTTPS) aloqa o'rnatilishidan oldin "salomlashish" jarayoni. Ular shifrlash kalitlarini almashib, bir-birining haqiqiyligini tekshiradilar.

55. "BGP" (Border Gateway Protocol) nima?

 Bu "internetning marshrutizatori". U yirik provayderlar (Autonomous Systems) o'rtasida ma'lumot almashish yo'llarini belgilaydi. Agar BGPda xato bo'lsa, butun bir mamlakat internetdan uzilib qolishi mumkin.

56. "VPN Tunneling" nima?

 Ma'lumotlar paketi boshqa bir paketning ichiga "yashirib" yuboriladi (encapsulation). Bu xuddi yopiq konvert ichida boshqa xat yuborishga o'xshaydi — yo'ldagilar ichida nima borligini ko'ra olmaydi.

57. "Quality of Service" (QoS) nima uchun kerak?

 Tarmoqdagi trafik turlariga ustuvorlik berish. Masalan, video qo'ng'iroq (Zoom) uzilib qolmasligi uchun unga ko'proq tezlik beriladi, oddiy fayl yuklash esa biroz kuttirilishi mumkin.

58. "Proxy" va "Reverse Proxy" farqi?



- **Proxy:** Foydalanuvchini internetdan yashiradi (anonimlik).
- **Reverse Proxy:** Serverni internetdan yashiradi. U kelayotgan so'rovlarni serverlar orasida taqsimlaydi (Load Balancing).

🔗 59. "MTU" (Maximum Transmission Unit) nima?

📖 Bitta tarmoq paketi ichida yuborilishi mumkin bo'lgan maksimal ma'lumot hajmi (odatda 1500 bayt). Agar ma'lumot kattaroq bo'lsa, u bo'laklarga bo'linadi (fragmentation).

★ 60. "VPC" (Virtual Private Cloud) nima?

📖 Bulutli servislar (AWS, Google Cloud) ichida foydalanuvchi uchun ajratilgan xususiy va izolyatsiya qilingan virtual tarmoq.

🔗 61. "Encapsulation" (Inkapsulyatsiya) nima?

📖 Bu ma'lumotning OSI qatlamlaridan o'tayotganda har bir bosqichda o'ziga xos sarlavha (header) bilan o'ralishi jarayonidir. Masalan, "Ma'lumot" transport qatlamida "Segment"ga, tarmoq qatlamida "Paket"ga, kanal qatlamida esa "Freym"ga (Frame) aylanadi.

★ 62. "Frame" (Freym) va "Packet" (Paket) o'rtasidagi farq nimada?



- **Packet:** OSI modelining 3-qatlamida (Network) ishlaydi va IP-manzillarni o'z ichiga oladi. Routerlar paketlar bilan ishlaydi.
- **Frame:** OSI modelining 2-qatlamida (Data Link) ishlaydi va MAC-manzillarni o'z ichiga oladi. Switchlar freymlar bilan ishlaydi.

🔗 63. "Transport Layer"da Portlar qanday vazifani bajaradi?

📖 Portlar kompyuterga kelgan ma'lumotlar aynan qaysi dasturga (masalan, brauzerga, telegramga yoki o'yinga) tegishli ekanini aniqlash uchun ishlatiladi. IP-manzil uyni topsa, Port aynan qaysi xonaga kirish kerakligini ko'rsatadi.

★ 64. "Network Layer"ning asosiy vazifasi nima?

📖 Bu qatlamning eng muhim vazifasi — **Marshrutizatsiya (Routing)**. U ma'lumot paketini manbadan manzilga yetkazish uchun eng maqbul va qisqa yo'lni tanlaydi.

🔗 65. "Physical Layer"da ma'lumotlar qanday ko'rinishda bo'ladi?

📖 Bu qatlamda ma'lumotlar elektr signallari, yorug'lik impulslari (optik tolada) yoki radioto'lqinlar (Wi-Fi) ko'rinishidagi **Bitlar (0 va 1)** holatida bo'ladi.

☆ 66. "AS" (Autonomous System - Avtonom Tizim) nima?

📖 Internet — bu kichik tarmoqlarning birlashmasi. Bitta tashkilot (masalan, Uztelecom yoki Google) boshqaruvidagi tarmoqlar guruhi Avtonom Tizim deb ataladi. Ular bir-biri bilan BGP protokoli orqali bog'lanadi.

? 67. "Anycast" manzillash usuli nima?

📖 Bunda bitta IP-manzil dunyoning turli nuqtalaridagi bir nechta serverlarga tegishli bo'ladi. Foydalanuvchi so'rov yuborganda, ma'lumot unga eng yaqin turgan serverga boradi. Bu asosan DNS va CDN xizmatlarida tezlikni oshirish uchun ishlatiladi.

☆ 68. "CDN" (Content Delivery Network) qanday ishlaydi?

📖 CDN — bu dunyo bo'ylab tarqalgan serverlar tarmog'i. U veb-sayt kontentini (rasm, video) foydalanuvchiga yaqin joyda keshlab saqlaydi. Masalan, Amerika saytidagi videoni Toshkentdan ko'rayotganingizda, u videoni Amerikadan emas, balki Toshkent yoki Moskvadagi eng yaqin CDN serverdan yuklab oladi.

? 69. "SSH Tunneling" nima?

📖 Bu xavfsiz bo'lmagan boshqa protokollarni (masalan, HTTP yoki ma'lumotlar bazasi ulanishlarini) shifrlangan SSH aloqasi ichidan o'tkazish usulidir. Bu xavfsizlikni oshirish uchun virtual "himoyalangan quvur" vazifasini o'taydi.

☆ 70. "VLAN Tagging" (802.1Q) nima?

📖 Bitta fizik kabel orqali bir nechta VLAN (virtual tarmoq) trafigi o'tayotganda, qaysi paket qaysi tarmoqqa tegishli ekanini ajratish uchun paketga maxsus "belgi" (tag) qo'shiladi.

? 71. IPv6 manzillari qanday ko'rinishda yoziladi?

📖 IPv4 dan farqli o'laroq (o'nlik sanoq tizimi), IPv6 16-lik (hexadecimal) sanoq tizimida yoziladi va 8 ta blokdan iborat bo'ladi. Masalan:
2001:0db8:85a3:0000:0000:8a2e:0370:7334.

☆ 72. IPv6 da "Dual Stack" nima?

📖 Bu qurilmaning bir vaqtning o'zida ham IPv4, ham IPv6 manzillari bilan ishlay olish qobiliyatidir. Bu IPv4 dan IPv6 ga to'liq o'tish davrida tarmoqlarning uzilishlarsiz ishlashini ta'minlaydi.

🔗 73. "SDN" (Software Defined Networking) nima?

📖 Bu tarmoqni boshqarishni apparat vositalaridan (router, switch) ajratib, dasturiy ta'minot darajasiga olib chiqishdir. Bunda markaziy kontroller orqali butun tarmoq trafiginı bitta dastur yordamida oson boshqarish mumkin.

★ 74. "NFV" (Network Function Virtualization) nima?

📖 Firewall, Load Balancer yoki Router kabi jismoniy qurilmalar o'rniga, ularning funksiyalarini oddiy serverlarda virtual mashina (dastur) sifatida ishlatish texnologiyasi.

🔗 75. "Zero Trust Network" (Nol ishonch tarmog'i) falsafasi nima?

📖 Bu xavfsizlik modeli bo'lib, unga ko'ra tarmoq ichidagi hech bir qurilma yoki foydalanuvchiga avtomatik ishonilmaydi. Har bir ulanish so'rovi qayerdan kelishidan qat'i nazar (hatto ofis ichidan bo'lsa ham) qaytadan autentifikatsiya va verifikatsiya qilinishi shart.

★ 76. "Default Gateway is not available" xatosi nimani bildiradi?

📖 Bu sizning kompyuteringiz router (darvoza) bilan aloqa o'rnatilganini bildiradi. Sababi: router o'chiq bo'lishi, kabel uzilishi yoki IP-manzillar noto'g'ri sozlangan bo'lishi mumkin.

🔗 77. "DNS Server not responding" muammosini qanday hal qilsa bo'ladi?

📖 Bu kompyuter domen nomlarini IPga o'gira olmayotganini bildiradi. Yechimi: tarmoq sozlamalarida DNSni qo'lda 8.8.8.8 (Google DNS) yoki 1.1.1.1 (Cloudflare)ga o'zgartirib ko'rish.

★ 78. "IP Conflict" (IP to'qnashuvi) nima?

📖 Bitta mahalliy tarmoqdagi ikkita qurilma bir xil IP-manzilni olishga harakat qilganda yuz beradi. Natijada ikkala qurilmada ham internet uzilib qoladi. Zamonaviy DHCP tizimlari buni avtomatik hal qiladi.

🔗 79. Tarmoq tezligi pastligini tekshirish uchun qaysi bosqichlardan o'tiladi?



1. Ping orqali kechikishni (latency) tekshirish.
2. Traceroute orqali yo'ldagi qaysi routerda kechikish borligini aniqlash.
3. Kabellar va Wi-Fi signal kuchini tekshirish.

☆ 80. "Broadcast Storm" nima?

📖 Tarmoqda (Layer 2) ma'lumotlar aylanib qolishi (Loop) natijasida broadcast paketlarining cheksiz ko'payib ketishi. Bu tarmoqni butunlay falaj qiladi. Buning oldini olish uchun switchlarda **STP (Spanning Tree Protocol)** ishlatiladi.

Tarmoq texnologiyalari bo'yicha yakuniy 20 ta (81–100) savol-javobga o'tamiz. Bu qismda zamonaviy veb-protokollar, dasturiy darajadagi ulanishlar va tarmoq administratorlarining kundalik vositalarini ko'rib chiqamiz.

? 81. HTTP/1.1 va HTTP/2 o'rtasidagi asosiy farq nimada?

📖 HTTP/1.1 har bir so'rov uchun alohida ulanish ochadi, bu esa sekinlashuvga olib keladi. **HTTP/2** esa "Multiplexing" texnologiyasidan foydalanadi — bitta TCP ulanish orqali bir vaqtning o'zida ko'plab rasm, matn va fayllarni yubora oladi. Shuningdek, u sarlavhalarni (headers) siqib yuboradi, bu esa yuklanishni tezlashtiradi.

☆ 82. HTTP/3 va QUIC protokoli nima?

📖 HTTP/3 ning inqilobiy tomoni shundaki, u TCP o'rniga **UDP** ustida ishlaydigan **QUIC** protokoliga tayanadi. TCPdagi uzoq davom etadigan "Handshake" (salomlashish) jarayoni olib tashlangan, natijada internet uzilishlari va mobil tarmoqlarda ulanish tezligi bir necha barobar oshgan.

? 83. WebSocket nima va u oddiy HTTPdan nimasi bilan farq qiladi?

📖 HTTP — bu "so'rov-javob" modeli (mijoz so'ramasa, server javob bermaydi). **WebSocket** esa "Full-duplex" aloqa o'rnatadi, ya'ni server va mijoz istalgan vaqtda bir-biriga ma'lumot yubora oladi. Bu onlayn chatlar, birja grafiklari va real vaqtdagi bildirishnomalar (notifications) uchun ishlatiladi.

☆ 84. "Socket" (Soket) tushunchasini qanday tushunish kerak?

📖 Soket — bu tarmoqdagi ulanishning yakuniy nuqtasi (endpoint). U **IP-manzil + Port** kombinatsiyasidan iborat. Masalan, 192.168.1.5:8080. Dasturchilar uchun soket — bu tarmoq orqali ma'lumot yuborish va qabul qilish uchun ochilgan "eshik"dir.

? 85. "Keep-Alive" funksiyasi nima uchun kerak?

📖 Bu funksiya bitta so'rovdan keyin TCP ulanishini darhol yopib yubormaslikni buyuradi. Bu keyingi so'rovlar uchun yangidan ulanish o'rnatishga vaqt sarflamaslikka yordam beradi va server yuklamasini kamaytiradi.

☆ 86. SNMP (Simple Network Management Protocol) nima?

📖 Bu tarmoqdagi qurilmalarni (routerlar, switchlar, printerlar) masofadan kuzatish va boshqarish protokoli. U orqali administrator qurilmaning protsessori qanchalik yuklangani, xotirasi to'lgani yoki portlari ochiqligini bitta markaziy panelda ko'rib turadi.

❓ 87. "Port Forwarding" (Portni yo'naltirish) nima?

📖 Uyingizdagi routerga internetdan kelgan so'rovni ichki tarmoqdagi ma'lum bir qurilmaga (masalan, uydagi kameraga yoki o'yin serveriga) yo'naltirish jarayoni. Bu NAT devorini aylanib o'tib, tashqi dunyodan ichki resursga ulanish imkonini beradi.

★ 88. "Loopback Interface" nima uchun "Virtual" deb ataladi?

📖 Chunki u jismoniy kabelga ega emas. Bu operatsion tizim ichidagi dasturiy interfeys bo'lib, u har doim "yuqorida" (up) bo'ladi. Routerlarda boshqaruv interfeysi sifatida ishlatiladi, chunki fizik portlar o'chib qolsa ham, Loopback manzili orqali routerga ulanish imkoni saqlanib qoladi.

❓ 89. "Bandwidth Throttling" nima?

📖 Provayder yoki administrator tomonidan foydalanuvchining internet tezligini ataylab cheklash jarayoni. Bu ko'pincha limit tugaganda yoki tarmoq haddan tashqari yuklanganda barcha foydalanuvchilarga teng sharoit yaratish uchun qilinadi.

★ 90. "Static Routing" va "Dynamic Routing" farqi?



- **Static:** Yo'nalishlarni (marshrutlarni) administrator qo'lda yozib chiqadi. Kichik tarmoqlar uchun mos.

- **Dynamic:** Routerlar maxsus protokollar (OSPF, EIGRP) orqali o'zaro gaplashib, eng yaxshi yo'lni avtomatik aniqlaydi. Katta va o'zgaruvchan tarmoqlar uchun shart.

❓ 91. `nslookup` yoki `dig` buyruqlari nima uchun ishlatiladi?

📖 Domen nomining DNS yozuvlarini tekshirish uchun. Masalan, `google.com` qaysi IPda turganini yoki u qaysi pochta serverlaridan (MX records) foydalanishini bilish uchun qo'llaniladi.

★ 92. `netstat` buyrug'i administratorga nima beradi?

📖 U kompyuterdagi barcha faol ulanishlarni, ochiq portlarni va qaysi dastur tarmoq orqali gaplashayotganini ko'rsatadi. Shubhali ulanishlarni (xakerlik hujumlarini) aniqlashda juda foydali.

❓ 93. **ipconfig (Windows) va ifconfig / ip addr (Linux) farqi?**

📖 Vazifasi bir xil: kompyuterning IP-manzili, tarmoq maskasi va shlyuzini ko'rsatish. Shuningdek, u orqali tarmoq interfeyslarini yoqish/o'chirish yoki DHCPdan yangi IP so'rash mumkin.

★ 94. **"Wireshark" dasturi nima vazifani bajaradi?**

📖 Bu "Paket analizatori" (Sniffer). U tarmoq simlari orqali o'tayotgan har bir bit va baytni tutib olib, inson tushunadigan tilda ko'rsatib beradi. U tarmoqdagi murakkab xatolarni topish va xavfsizlikni tekshirish uchun eng kuchli vositadir.

❓ 95. **"Collision Domain" nima?**

📖 Bu tarmoqning shunday qismiki, unda ikkita qurilma bir vaqtda ma'lumot yuborsa, signallar bir-biriga urilib, "kolliziya" (to'qnashuv) sodir bo'ladi. Switch har bir portini alohida kolliziya domeniga ajratadi, bu esa tarmoq samaradorligini oshiradi.

★ 96. **"Three-Tier Network Architecture" nima?**

📖 Yirik korporativ tarmoqlar 3 qatlamga bo'linadi:

1. **Core (Yadro):** O'ta yuqori tezlikda ma'lumotlarni markazda uzatish.
2. **Distribution (Taqsimot):** Siyosatlarni (VLAN, xavfsizlik) qo'llash.
3. **Access (Kirish):** Oxirgi foydalanuvchilar (kompyuterlar) ulanadigan joy.

❓ 97. **"Cloud Networking" an'anaviy tarmoqdan nimasi bilan farq qiladi?**

📖 Cloud tarmoqda siz fizik kabellar va routerlarni ko'rmaysiz. Hammasi dasturiy (virtual) tarzda boshqariladi. Siz bir necha sekund ichida butun dunyo bo'ylab tarqalgan virtual tarmoqni (VPC) yaratishingiz va sozlamalarni API orqali o'zgartirishingiz mumkin.

★ 98. **"PoE" (Power over Ethernet) nima?**

📖 Bitta Ethernet kabeli orqali ham ma'lumot, ham elektr quvvati yuborish texnologiyasi. Bu IP-kameralar, Wi-Fi nuqtalari va IP-telefonlar uchun alohida rozetka va adapter kerakmasligini ta'minlaydi.

❓ 99. **"Dark Fiber" nima?**

📖 Bular yotqizilgan, lekin hali foydalanilmayotgan (yorug‘lik signali yuborilmagan) optik tolali kabellar. Kompaniyalar kelajakda tarmoqni kengaytirish uchun zahira sifatida shunday kabellarni ijaraga oladilar.

☆ 100. Tarmoq muhandisi (**Network Engineer**) bo‘lish uchun qaysi sertifikatlar nufuzli?



- **CCNA (Cisco Certified Network Associate):** Poydevor bilimlar uchun eng mashhur sertifikat.

- **CompTIA Network+:** Umumiy tarmoq bilimlari.
- **CCNP / CCIE:** Yuqori darajadagi mutaxassislar uchun.

7. DevOps

Atamalar va izohlar

Atamalar	Izohlar
CI (Continuous Integration)	Kodni doimiy ravishda markaziy repozitoriyga qo'shish va avtomatik testlash.
CD (Continuous Deployment)	Testdan o'tgan kodni avtomat tarzda Production serverga yuklash.
Docker	Dasturni barcha kutubxonalari bilan izolyatsiya qilingan konteynerda ishlatish.
Kubernetes (K8s)	Konteynerlarni boshqarish (orkestratsiya) va avtomatlashtirish tizimi.
IaC (Infrastructure as Code)	Infratuzilmani (server, tarmoq) kod yozish orqali boshqarish.
Terraform	Bulutli infratuzilmani yaratish uchun ishlatiladigan IaC vositasi.
Ansible	Serverlar ichidagi sozlamalarni (config) boshqarish va dastur o'rnatish vositasi.
Jenkins	CI/CD poyp-laynlarini (pipelines) qurish uchun ochiq manbali vosita.
Pod	Kubernetesdagi eng kichik birlik (bir yoki bir nechta konteyner guruhi).
Node	Kubernetes klasteridagi jismoniy yoki virtual server.
Atama	Tavsif va Vazifasi
Microservices	Dasturni mustaqil bo'lgan kichik servislar to'plami sifatida qurish.
Orchestration	Konteynerlarning hayotiy siklini (ishga tushish, o'chish, kengayish) boshqarish.
Image	Konteyner yaratish uchun ishlatiladigan o'zgarmas shablon (template).
Container	Image-dan ishga tushirilgan jonli va izolyatsiya qilingan jarayon.
Pipeline	Kod yozilgandan to serverga yetib borguncha bo'lgan avtomatlashtirilgan qadamlar.
Artifact	Build jarayonida hosil bo'lgan tayyor mahsulot (Docker image, .jar fayl).
GitOps	Infratuzilma holatini Git orqali boshqarish falsafasi (masalan, ArgoCD).
YAML	Konfiguratsiya fayllarini yozish uchun ishlatiladigan o'qishga qulay format.
Monitoring	Tizim ko'rsatkichlarini (CPU, RAM, Traffic) raqamlarda kuzatish.
Logging	Tizimda sodir bo'lgan voqealar matnli tarixini yig'ish (ELK Stack).
Atama	Tavsif va Vazifasi
Prometheus	Metrikalarni yig'ish va saqlash uchun o'ta mashhur monitoring tizimi.
Grafana	Metrikalarni chiroyli dashboardlar ko'rinishida vizuallashtirish.
Blue-Green Deployment	Ikkita (eski va yangi) muhit orqali dasturni xavfsiz yangilash strategiyasi.
Canary Deployment	Yangilanishni avval foydalanuvchilarning kichik qismiga chiqarish usuli.
Helm	Kubernetes uchun paketlar menejeri (murakkab YAMLLarni oson o'rnatish).
Ingress	Klasterga kiruvchi tashqi trafikni boshqaruvchi darvoza (HTTP/HTTPS).
Load Balancer	Trafikni bir nechta serverlar orasida teng taqsimlovchi qurilma.

Sidecar Pattern	Asosiy pod ichida unga yordam beruvchi qo'shimcha konteynerni ishlatish.
Scaling	Yuklamaga qarab podlar sonini oshirish (HPA) yoki kamaytirish.
Self-healing	Ishdan chiqqan konteynerni Kubernetes tomonidan avtomatik qayta yoqish.
Atama	Tavsif va Vazifasi
DevSecOps	Xavfsizlikni DevOps jarayonining har bir bosqichiga qo'shish.
SAST	Kod yozilgan vaqtda uning matnini xavfsizlikka skanerlash.
SRE (Site Reliability Engineering)	Tizim barqarorligini ta'minlashga qaratilgan muhandislik yondashuvi.
Zero Downtime	Saytni foydalanuvchilar uchun o'chirmasdan yangilash.
Rollback	Xato aniqlanganda darhol avvalgi ishlagan barqaror versiyaga qaytish.
Volume	Konteyner o'chsa ham ma'lumotlarni saqlab qoluvchi tashqi xotira.
Secrets	Parollar va API kalitlarni shifrlangan holda saqlash mexanizmi.
Namespace	Kubernetes klasterni mantiqiy loyihalarga bo'lib ajratish.
ConfigMap	Dastur sozlamalarini koddagi o'zgaruvchilardan ajratib saqlash.
Cluster	Yagona tizim sifatida ishlovchi bir nechta serverlar to'plami.
Atama	Tavsif va Vazifasi
Service Mesh	Servislar orasidagi aloqa, shifrlash va kuzatuvni boshqarish (Istio).
Tracing	Murakkab tizimda so'rovning barcha servislar bo'ylab yo'lini kuzatish.
Bastion Host	Ichki yopiq serverlarga ulanish uchun ishlatiladigan o'ta himoyalangan server.
Serverless	Serverlarni boshqarmasdan faqat kodni (funksiyan) ishlatish.
VPC	Bulutli tizimlarda foydalanuvchi uchun ajratilgan xususiy tarmoq.
OOM Killer	Xotira (RAM) tugab qolganda Linux tomonidan jarayonlarni o'ldirilishi.
Liveness Probe	Konteyner tirikligini aniqlash uchun Kubernetes tekshiruvi.
Readiness Probe	Konteyner trafik qabul qilishga tayyorligini aniqlash tekshiruvi.
Health Check	Servisning ishchi holatini muntazam tekshirib turish jarayoni.
Observability	Tizim ichki holatini loglar, metrikalar va tracing orqali tushunish.

Savol-javoblar

❓ 1. DevOps o'zi nima?

📖 DevOps — bu dasturiy ta'minotni yaratuvchi dasturchilar (Dev) va tizim administratorlari (Ops) o'rtasidagi hamkorlikni yaxshilash, jarayonlarni avtomatlashtirish va mahsulotni mijozga tezroq yetkazib berishga qaratilgan madaniyat va amaliyotlar to'plamidir.

☆ 2. DevOps hayotiy sikli (Lifecycle) qanday bosqichlardan iborat?

📖 DevOps sikli cheksiz "Infinity" ramzi ko'rinishida tasvirlanadi:

1. **Plan** (Rejalashtirish)
2. **Code** (Kod yozish)
3. **Build** (Yig'ish)

4. **Test** (Tekshirish)
5. **Release** (Versiya chiqarish)
6. **Deploy** (Serverga joylash)
7. **Operate** (Boshqarish)
8. **Monitor** (Kuzatuv)

🔗 3. CI/CD tushunchasi nimani anglatadi?



- **CI (Continuous Integration):** Dasturchilar yozgan kodlarni markaziy repozitoriyga muntazam ravishda qo‘shib borish va har bir qo‘shilganda avtomatik test qilish.
- **CD (Continuous Delivery/Deployment):** Testdan o‘tgan kodni avtomatlashtirilgan tarzda serverga (Production) yetkazib berish.

★ 4. "Infrastructure as Code" (IaC) nima?



IaC — bu serverlar, tarmoqlar va bazalarni qo‘lda sozlash o‘rniga, ularni maxsus kod (konfiguratsiya fayllari) orqali boshqarishdir. Masalan, **Terraform** yordamida bitta buyruq bilan 100 ta serverni avtomatik yoqish mumkin.

🔗 5. Microservices arxitekturasini DevOps bilan qanday bog‘liq?



Microservices — bu dasturni kichik, mustaqil bo‘laklarga bo‘lib boshqarishdir. DevOps bu kichik bo‘laklarni bir-biridan mustaqil ravishda serverga yuklash va avtomatlashtirish uchun juda muhim hisoblanadi.

★ 6. Docker nima va u virtual mashinadan (VM) nimasi bilan farq qiladi?



Docker — dasturni barcha kutubxonalari bilan bitta paketga (konteynerga) o‘raydigan platforma.

- **VM:** Har bir dastur uchun alohida Operatsion Tizim (OS) talab qiladi (Og‘ir).
- **Docker:** OS yadrosini bo‘lishadi, shuning uchun u yengil va juda tez ishga tushadi.

🔗 7. Docker Image va Docker Container farqi nimada?



- **Image (Tasvir):** Bu dasturning o‘qish uchun mo‘ljallangan "shablone" (xuddi CD disk kabi).

- **Container:** Tasvirdan (image) ishga tushirilgan jonli jarayon (xuddi diskdan oʻrnatilgan dastur kabi).

☆ 8. Dockerfile nima vazifani bajaradi?

📖 Bu oddiy matnli fayl boʻlib, unda Docker tasvirini yaratish uchun kerakli barcha qadamlar yoziladi (masalan: Python oʻrnat, kodni nusxala, 8000-portni och).

? 9. Docker Compose nima uchun ishlatiladi?

📖 Agar dastur bir nechta konteynerdan (masalan: Django + PostgreSQL + Redis) iborat boʻlsa, **Docker Compose** ularni bitta buyruq (`docker-compose up`) bilan birgalikda ishga tushirish imkonini beradi.

☆ 10. Docker Volume nima uchun kerak?

📖 Konteyner oʻchirilsa, uning ichidagi maʼlumotlar ham yoʻqoladi. **Volume** — bu konteyner ichidagi maʼlumotlarni serverning doimiy xotirasida (diskda) saqlab qolish mexanizmi.

? 11. Kubernetes (K8s) oʻzi nima?

📖 Bu yuzlab yoki minglab Docker konteynerlarini avtomatlashtirilgan tarzda boshqarish (Orchestration) tizimidir. U serverlar oʻchib qolsa, konteynerlarni boshqa serverga oʻtkazadi va yuklamani taqsimlaydi.

☆ 12. Kubernetes "Pod"i nima?

📖 Pod — Kubernetesning eng kichik birligi. U bitta yoki bir nechta Docker konteynerini oʻz ichiga oladi. Bir pod ichidagi konteynerlar bitta IP-manzilga ega boʻladi.

? 13. Kubernetesda "Scaling" (Kengaytirish) qanday ishlaydi?

📖 Agar saytga kiruvchilar koʻpaysa, Kubernetes avtomatik ravishda konteynerlar (Podlar) sonini oshiradi (Horizontal Pod Autoscaler). Yuklama kamaysa, ularni oʻchirib resurslarni tejaydi.

☆ 14. "Self-healing" (Oʻz-oʻzini davolash) xususiyati nima?

📖 Agar Kubernetesdagi biror konteyner ishdan chiqsa, tizim buni sezadi va uni avtomatik ravishda oʻchirib, yangisini ishga tushiradi. Inson aralashuvi shart emas.

? 15. Helm nima?

📖 Helm — bu "Kubernetes uchun paketlar menejeri". U murakkab Kubernetes konfiguratsiyalarini (YAML fayllarni) bitta paket (Chart) ko‘rinishida oson o‘rnatish imkonini beradi.

★ 16. Jenkins nima?

📖 Jenkins — bu CI/CD jarayonlarini avtomatlashtirish uchun ishlatiladigan eng mashhur Open-Source vosita. U "Pipeline"lar yordamida kodni yig‘ish, testlash va serverga yuborishni amalga oshiradi.

? 17. Jenkins Pipeline nima?

📖 Bu dasturni servergacha yetib borish yo‘lini ifodalovchi kodli skriptlar to‘plami. Odatda "Stage" (Bosqich)larga bo‘linadi: Build -> Test -> Deploy.

★ 18. GitLab CI/CD va GitHub Actions o‘rtasidagi o‘xshashlik?

📖 Ikkalasi ham repozitoriyni ichida maxsus YAML fayli orqali CI/CD jarayonlarini boshqaradi. Ular Jenkins kabi alohida server talab qilmaydi (bulutli variantlari bor).

? 19. "Artifact" (Artifakt) nima?

📖 Build bosqichida hosil bo‘lgan tayyor fayllar to‘plami (masalan: .jar, .war, yoki Docker Image). Aynan shu artifakt keyinchalik serverga yuklanadi.

★ 20. "Blue-Green Deployment" strategiyasi nima?

📖 Bu xatolarni kamaytirish usuli. Ikki bir xil server bo‘ladi: **Blue** (eski versiya) va **Green** (yangi versiya). Agar Green versiya yaxshi ishlasa, foydalanuvchilar trafiki Green-ga yo‘naltiriladi. Muammo chiqsa, darhol Blue-ga qaytiladi.

DevOps bo‘yicha sanoqni 21-savoldan davom ettiramiz. Ushbu qismda infratuzilmani boshqarish (IaC), monitoring tizimlari va avtomatlashtirishning chuqur jihatlariga to‘xtalamiz.

? 21. Terraform nima va u qanday muammoni hal qiladi?

📖 Terraform — bu **Infratuzilmani kod sifatida (IaC)** boshqarish vositasi. U orqali siz AWS, Google Cloud yoki Azure kabi platformalarda serverlar, tarmoqlar va ma'lumotlar bazalarini qo‘lda (tugmalar bosib) emas, balki kod yozib yaratasisiz. Bu infratuzilmani versiyalash (Git-da saqlash), xatolarni kamaytirish va bir xil muhitni (staging/production) osongina nusxalash imkonini beradi.

★ 22. "Declarative" va "Imperative" yondashuv farqi nimada?



- **Declarative (Terraform, Kubernetes):** Siz yakuniy natijani belgilaysiz. Masalan: "Menga 3 ta server kerak". Tizim o'zi qanday yaratishni hal qiladi.

- **Imperative (Bash skriptlari, Ansible):** Siz har bir qadamni belgilaysiz. Masalan: "Serverni yoq", "OS o'rnat", "Kodni nusxala". DevOps-da Declarative yondashuv afzal ko'riladi, chunki u barqarorroqdir.

🔗 23. Ansible nima va u Terraformdan nimasi bilan farq qiladi?

📖 Terraform asosan infratuzilmani (serverning o'zini) **yaratish** uchun, Ansible esa yaratilgan server ichidagi dasturlarni **sozlash** (Configuration Management) uchun ishlatiladi. Terraform "bino quradi", Ansible esa "mebellarni joylashtiradi".

★ 24. "Immutable Infrastructure" (O'zgarmas infratuzilma) nima?

📖 Bu yondashuvda ishlayotgan server ichida hech qachon o'zgarish qilinmaydi. Agar dasturni yangilash kerak bo'lsa, eski server o'chiriladi va yangi versiya bilan yangi server yaratiladi. Bu "sozlamalar chalkashligi" (Configuration Drift) muammosini yo'qotadi.

🔗 25. Monitoring va Logging farqi nimada?



- **Monitoring (Prometheus, Zabbix):** Tizimning "sog'lig'ini" raqamlarda ko'rsatadi (CPU 80%, RAM 2GB qoldi). U "Nima sodir bo'lyapti?" degan savolga javob beradi.

- **Logging (ELK Stack):** Tizim ichidagi matnli voqealar tarixi. U "Nega bu xato sodir bo'ldi?" degan savolga javob berish uchun ishlatiladi.

★ 26. Prometheus va Grafana juftligi qanday ishlaydi?

📖 Prometheus — bu vaqtga bog'liq ma'lumotlarni (metrics) yig'uvchi va saqlovchi baza. Grafana esa bu raqamlarni chiroyli grafiklar va dashboardlar ko'rinishida vizuallashtiruvchi vositadir.

🔗 27. ELK Stack (EFK) nima va uning qismlari qaysilar?

📖 Bu loglarni markazlashtirilgan holda yig'ish tizimi:

1. **Elasticsearch:** Loglarni saqlash va tezkor qidirish bazasi.
2. **Logstash (yoki Fluentd):** Loglarni qabul qilish va ularga ishlov berish.
3. **Kibana:** Loglarni qidirish va tahlil qilish uchun veb-interfeys.

★ 28. "Alerting" (Ogohlantirish) nega DevOps-da muhim?

📖 Monitoring tizimi xatoni shunchaki ko'rsatib turishi yetarli emas. Agar server o'chib qolsa yoki disk to'lib borsa, tizim avtomatik ravishda Telegram, Slack yoki E-mail orqali muhandisga xabar (Alert) yuborishi shart.

❓ 29. Kubernetes "Deployment" va "StatefulSet" farqi?



- **Deployment:** Ma'lumot saqlamaydigan (stateless) dasturlar uchun (masalan: Frontend, Backend). Podlar istalgan vaqtda o'chib-yonishi mumkin.

- **StatefulSet:** Ma'lumot saqlaydigan (stateful) dasturlar uchun (masalan: Ma'lumotlar bazasi). Har bir podning o'z o'zgarmas ID-si va diski bo'ladi.

☆ 30. Kubernetes "Service" (Xizmat) turlari qaysilar?



1. **ClusterIP:** Podga faqat klaster ichidan ulanish imkonini beradi.
2. **NodePort:** Podni serverning maxsus porti orqali tashqi dunyoga ochadi.
3. **LoadBalancer:** Bulutli provayderning yuklama taqsimlagichi orqali tashqariga ulanish (eng ko'p ishlatiladigan).

❓ 31. Ingress nima vazifani bajaradi?

📖 Ingress — bu klasterga kirayotgan HTTP/HTTPS trafikni boshqaruvchi "aqlli darvoza". U bitta IP-manzil orqali turli domenlarni (`site.uz/api`, `site.uz/web`) tegishli podlarga yo'naltiradi.

☆ 32. "Liveness" va "Readiness" Probe nima?



Kubernetes podning holatini tekshirish usullari:

- **Liveness:** Pod tirikmi? Agar xato bo'lsa, K8s uni restart qiladi.
- **Readiness:** Pod mijozlarni qabul qilishga tayyormi? Agar tayyor bo'lmasa, unga trafik yuborilmaydi.

❓ 33. DevSecOps tushunchasi nimani anglatadi?

📖 Bu xavfsizlikni DevOps jarayonining har bir bosqichiga qo'shishdir. Ya'ni xavfsizlik dastur tayyor bo'lgandan keyin emas, balki kod yozish va build qilish jarayonida (avtomatik skanerlar yordamida) tekshiriladi.

☆ 34. Docker tasvirlarini (Images) skanerlash nega kerak?

📖 Siz ishlatayotgan Docker image ichida eski yoki xavfli kutubxonalar bo'lishi mumkin. **Trivy** yoki **Clair** kabi vositalar tasvirni skanerlab, undagi zaifliklar (vulnerabilities) haqida hisobot beradi.

❓ 35. "Secret Management" (Sirlarni boshqarish) qanday amalga oshiriladi?

📖 Ma'lumotlar bazasi parollari yoki API kalitlarni hech qachon Dockerfile yoki Git-da saqlash mumkin emas. Buning uchun **HashiCorp Vault** yoki Kubernetes **Secrets** kabi maxsus shifrlangan omborlardan foydalaniladi.

★ 36. YAML fayllari bilan ishlashda eng ko'p uchraydigan xato?

📖 YAML — bu probellar (indentation)ga o'ta sezgir format. Bitta ortiqcha yoki yetishmayotgan probel butun Kubernetes konfiguratsiyasini ishdan chiqarishi mumkin. YAML fayllarni doim "Linter" orqali tekshirish tavsiya etiladi.

❓ 37. "Canary Deployment" nima?

📖 Yangi versiyani birdaniga hamma foydalanuvchiga emas, balki faqat kichik bir qismga (masalan, 5%) chiqarish. Agar xato chiqmasa, asta-sekin 100% ga yetkaziladi. Bu xavfni minimal darajaga tushiradi.

★ 38. "Rolling Update" Kubernetesda qanday ishlaydi?

📖 Bu Kubernetes-ning standart yangilash usuli. Tizim avval bitta yangi podni yaratadi, keyin bitta eskini o'chiradi. Shu tariqa dastur bir soniya ham o'chib qolmasdan (Zero Downtime) yangilanadi.

❓ 39. "Rollback" jarayoni nima?

📖 Agar yangi versiya serverga yuklangach muammo chiqsa, tizimni darhol avvalgi ishlagan barqaror holatiga qaytarish jarayoni.

★ 40. "GitOps" falsafasi nima?

📖 GitOps — bu infratuzilma holatining "yagona haqiqat manbai" sifatida Git repozitoriyasini tanlashdir. Agar siz klasterda biror narsani o'zgartirmoqchi bo'lsangiz, terminalda buyruq yozmaysiz, balki Git-dagi faylni o'zgartirasiz. **ArgoCD** kabi vositalar Git-dagi o'zgarishni ko'rib, uni avtomatik klasterga qo'llaydi.

❓ 41. Docker tarmog'ining (Networking) qanday turlari bor?



- **Bridge (ko'prik):** Standart tarmoq. Konteynerlar o'zaro muloqot qilishi mumkin, lekin tashqi dunyodan portlarni yo'naltirish (mapping) orqali yopiq bo'ladi.

- **Host:** Konteyner alohida IP olmaydi, u to'g'ridan-to'g'ri serverning (host) tarmog'idan foydalanadi. Bu eng yuqori tezlikni beradi.
- **None:** Konteynerning hech qanday tarmog'i bo'lmaydi (to'liq izolyatsiya).
- **Overlay:** Bir nechta turli serverlarda joylashgan Docker konteynerlarini yagona virtual tarmoqqa birlashtirish (Docker Swarm uchun).

☆ 42. Docker'da "Bind Mount" va "Named Volume" farqi nimada?



- **Named Volume:** Ma'lumotlarni Docker boshqaradigan maxsus joyda saqlaydi. Bu xavfsizroq va boshqa serverga ko'chirish oson.
- **Bind Mount:** Serverning ixtiyoriy papkasini (masalan, /home/user/app) konteynerga bog'lash. Bu dastur kodini real vaqtda o'zgartirib turish uchun dasturchilarga qulay.

? 43. "Dangling Image" nima va uni qanday tozalash mumkin?



Bu hech qanday konteyner tomonidan ishlatilmayotgan va nomi yo'q (tag: <none>) eski tasvirlar. Ular server diskini to'ldirib yuboradi. `docker image prune` buyrug'i orqali ularni tozalash mumkin.

☆ 44. Kubernetes "Node Affinity" va "Taints/Tolerations" farqi?



- **Node Affinity:** Podning ma'lum bir serverga (Node) qarab "tortilishi". Masalan: "Ushbu pod faqat GPU-si bor serverda ishlasin".
- **Taints/Tolerations:** Serverning podlarni "itarishi". Serverga Taint qo'yiladi: "Men faqat maxsus ruxsati (Toleration) bor podlarni qabul qilaman". Bu serverni boshqa "begona" podlardan himoya qilish uchun kerak.

? 45. "ConfigMap" va "Secret" farqi nimada?



- **ConfigMap:** Shifrlanmagan sozlamalar (masalan, DB_HOST, LOG_LEVEL).
- **Secret:** Maxfiy ma'lumotlar (masalan, DB_PASSWORD). Kubernetesda Secret-lar base64 formatida saqlanadi, lekin yanada xavfsiz bo'lishi uchun ularni KMS (Key Management Service) bilan shifrlash tavsiya etiladi.

☆ 46. Kubernetes "Sidecar Container" patterni nima?



Bu asosiy pod ichida unga yordam beruvchi ikkinchi konteynerni ishlatish.

- **Misol:** Asosiy konteyner dasturni ishlatadi, Sidecar konteyner esa uning loglarini yig'ib Elasticsearch-ga yuboradi. Ular bitta pod ichida bo'lgani uchun resurslarni va tarmoqni bo'lishadi.

❓ 47. "Horizontal Pod Autoscaler" (HPA) qanday ishlaydi?

📖 HPA doimiy ravishda podlarning CPU va RAM sarfini kuzatib boradi. Agar yuklama belgilangan chegaradan (masalan, 70%) oshsa, u avtomatik ravishda yangi podlar (replikalar) yaratadi. Yuklama tushgach, ortiqchalarini o'chiradi.

★ 48. Linuxda "Zombie Process" nima?

📖 Bu o'z ishini tugatgan, lekin Operatsion Tizim jadvalidan o'chmagan jarayon. U RAM sarflamaydi, lekin jarayonlar soni (PID) limitini egallab qo'yishi mumkin. Agar zombilar ko'payib ketsa, serverda yangi dastur ochib bo'lmay qoladi.

❓ 49. "Standard Input", "Output" va "Error" (stdin, stdout, stderr) farqi?

📖 Har bir Linux buyrug'ida 3 ta oqim bor:

- **stdin (0):** Buyruqqa kiruvchi ma'lumot (klaviatura).
- **stdout (1):** Buyruqning muvaffaqiyatli natijasi (ekran).
- **stderr (2):** Xatolik haqidagi xabarlar. DevOps-da biz loglarni ajratish uchun `2> error.log` buyrug'i orqali xatolarni alohida faylga yo'naltiramiz.

★ 50. Nginx'da "Upstream" nima vazifani bajaradi?

📖 `upstream` blokida Nginx so'rovlarni yo'naltirishi kerak bo'lgan backend serverlar ro'yxati yoziladi. Bu Nginx-ni Load Balancer sifatida ishlatishning asosiy qismidir. U orqali yuklamani bir nechta IP-manzillar orasida taqsimlash mumkin.

❓ 51. "GitLab Runner" nima?

📖 GitLab Runner — bu GitLab CI/CD buyruqlarini (pipeline) jismoniy ravishda bajaruvchi agent. U alohida serverda turadi va yangi kod kelganida uni yig'adi (build), testlaydi va serverga yuklaydi.

★ 52. Docker Build-da "Layer Caching" nima va u nega muhim?

📖 Dockerfile-dagi har bir qator yangi "qatlam" (layer) hosil qiladi. Docker o'zgarmagan qatorlarni keshda saqlaydi.

- **Foydasi:** Agar siz faqat kodni o'zgartirib, kutubxonalarni (requirements.txt) o'zgartirmasangiz, Docker kutubxonalarni qayta yuklab o'tirmaydi va build jarayoni 10 soniyada tugaydi (noldan bo'lsa 5 daqiqa ketishi mumkin edi).

? 53. "Webhooks" CI/CD-da qanday rol o'ynaydi?

📖 Webhook — bu bir tizimdan (masalan, GitHub) ikkinchi tizimga (masalan, Jenkins) yuboriladigan avtomatik bildirishnoma. Dasturchi `git push` qilishi bilan GitHub Jenkins-ga "ish boshla" degan xabar (webhook) yuboradi.

☆ 54. "Serverless" (FaaS) tushunchasi nima?

📖 Siz serverni umuman boshqarmaysiz. Faqat bitta funksiyani (kod bo'lagini) yuklaysiz (masalan, AWS Lambda). U faqat kimdir chaqirganda ishlaydi va faqat ishlagan vaqti uchun pul to'laysiz. Qolgan vaqtda u nolga teng xarajat talab qiladi.

? 55. "VPC Peering" nima uchun kerak?

📖 Bu ikki xil virtual tarmoqni (VPC) xuddi bitta tarmoqdek xavfsiz bog'lash usuli. Masalan, sizning ma'lumotlar bazangiz bir tarmoqda, backend serveringiz boshqa tarmoqda bo'lsa, ular o'zaro ichki IP orqali gaplashishi uchun VPC Peering kerak.

DevOps bo'yicha sanoqni 56-savoldan davom ettiramiz. Ushbu qismda murakkab monitoring, xavfsizlik vositalari va Kubernetes infratuzilmasining chuqur jihatlarini ko'rib chiqamiz.

? 56. "Static Application Security Testing" (SAST) va DAST farqi nimada?



- **SAST (SonarQube, Snyk):** Kod hali ishga tushmasdan turib, uning ichidagi zaifliklarni (vulnerability) qidiradi. U kodning "matnini" tahlil qiladi.
- **DAST:** Dastur ishlayotgan vaqtda unga tashqaridan hujum qilib ko'rish orqali xavfsizlikni tekshiradi.

☆ 57. "SonarQube" DevOps poyp-laynida (pipeline) nima vazifani bajaradi?

📖 U kod sifatini nazorat qiluvchi "darvoza" (Gate). U koddagi xatolarni (bugs), "hidli kodlar"ni (code smells) va testlar bilan qamrab olinganlik darajasini (test coverage) tekshiradi. Agar kod sifati belgilangan me'yordan past bo'lsa, u CI/CD jarayonini to'xtatib qo'yadi.

? 58. "Git Hooks" nima va u nega foydali?

📖 Bu Git-dagi ma'lum bir harakat (masalan, `commit` yoki `push`) sodir bo'lganda avtomatik ishlaydigan skriptlar.

- **Foydasi:** Masalan, `pre-commit` hook orqali kodni Git-ga yuborishdan oldin uni avtomatik ravishda formatlash yoki sirlar (parollar) bor-yoʻqligini tekshirish mumkin.

☆ 59. "Persistent Volume" (PV) va "Persistent Volume Claim" (PVC) farqi?



- **PV:** Bu jismoniy disk (klasterdagi resurs). Uni administrator yaratadi.
- **PVC:** Bu foydalanuvchining (dasturchining) diskka boʻlgan "soʻrovi". Dastur "menga 10GB joy kerak" deb PVC yuboradi, Kubernetes esa unga mos PV-ni bogʻlab beradi.

? 60. "StorageClass" Kubernetes-da nima uchun ishlatiladi?



Bu "Dynamic Provisioning" uchun kerak. `StorageClass` yordamida administrator har safar qoʻlda PV yaratib oʻtirmaydi. PVC kelishi bilan Kubernetes bulutli provayderdan (AWS EBS yoki Google Disk) avtomatik ravishda disk sotib oladi va podga ulaydi.

☆ 61. "Service Mesh" (Istio, Linkerd) nima va u nega kerak?



Microservices soni yuzlab boʻlib ketganda, ularning oʻzaro aloqasini boshqarish qiyinlashadi. **Service Mesh** — bu servislar orasidagi trafikni boshqaruvchi infraquzilma qatlami. U avtomatik ravishda mTLS (shifrlash), Retry (qayta urinish) va servislar orasidagi bogʻliqlik grafiklarini (observability) taʼminlaydi.

? 62. "Distributed Tracing" (Jaeger, Zipkin) nima?



Microservices arxitekturasida bitta foydalanuvchi soʻrovi 10 ta servisdan oʻtishi mumkin. Agar xato chiqsa, u aynan qaysi servisdan sodir boʻlganini bilish qiyin. **Tracing** har bir soʻrovga ID beradi va uning barcha servislar boʻylab "sayohati" xaritasini chizib beradi.

☆ 63. "Chaos Engineering" tushunchasi nimani anglatadi?



Bu tizimning chidamliligini tekshirish uchun unga ataylab xatolik kiritish (masalan, kutilmaganda bitta serverni oʻchirib qoʻyish). **Netflix** tomonidan ommalashgan "Chaos Monkey" vositasi bunga misol boʻladi. Maqsad — real hayotda xato boʻlishidan oldin tizimning oʻzini tiklash qobiliyatini sinash.

? 64. "Golden Signals" monitoringda nima?



SRE (Site Reliability Engineering) tomonidan tavsiya etilgan 4 ta asosiy koʻrsatkich:

1. **Latency:** So‘rov qancha vaqt olyapti?
2. **Traffic:** Qancha so‘rov kelyapti?
3. **Errors:** Necha foiz so‘rov xato bilan tugayapti?
4. **Saturation:** Resurslar (CPU/RAM) qanchalik to‘lgan?

☆ 65. "Multi-stage Docker Build" nega kerak?

📖 Bu Docker tasviri hajmini kichraytirishning eng yaxshi usuli. Birinchi bosqichda barcha "build tool"lar yordamida dastur yig‘iladi, ikkinchi bosqichda esa faqat tayyor ishchi fayl (binary) olinadi. Natijada image hajmi 1GB-dan 50MB-ga tushishi mumkin.

❓ 66. "Artifact Repository" (Nexus, Artifactory) vazifasi nima?

📖 Bu tayyor yig‘ilgan dastur paketlarini (jar, docker image, helm chart) xavfsiz saqlash joyi. Build jarayonida har safar internetdan kutubxona yuklamaslik va tayyor versiyalarni arxivlash uchun ishlatiladi.

☆ 67. "GitOps" da ArgoCD qanday ishlaydi?

📖 ArgoCD klaster ichida o‘tiradi va doimiy ravishda Git repozitoriyasini kuzatadi. Agar dasturchi Git-dagi YAML faylni o‘zgartirsa, ArgoCD klasterdagi holat Git-ga mos emasligini ko‘radi va avtomatik ravishda klasteri yangilaydi (Synchronize).

❓ 68. Terraform-da "State File" nima?

📖 Bu Terraform yaratgan barcha resurslarning hozirgi holati haqidagi ma’lumotlar saqlanadigan JSON fayl. Agar bu fayl yo‘qolsa, Terraform qaysi serverlarni yaratganini "eslay olmaydi" va hamma narsani noldan yaratishga harakat qiladi. Uni odatda AWS S3 kabi xavfsiz joyda saqlash shart.

☆ 69. "Bastion Host" (Jump Server) nima?

📖 Xavfsizlik uchun ichki serverlar (baza, backend) tashqi internetga yopiq bo‘ladi. Ularga ulanish uchun faqat bitta, o‘ta himoyalangan server — **Bastion** ochiq qoldiriladi. Administrator avval Bastion-ga ulanadi, keyin u orqali ichki tarmoqqa o‘tadi.

❓ 70. "Blue-Green" va "Canary" deploymentning asosiy farqi nimada?



- **Blue-Green:** Trafikni birdaniga (100%) yangi versiyaga o‘tkazadi.

- **Canary:** Trafikni asta-sekin (5%, 20%, 50%, 100%) o'tkazadi. Canary xavfsizroq, lekin monitoring uchun ko'proq e'tibor talab qiladi.

? 71. "NetworkPolicy" nima va u Kubernetesda nega shart?

📖 Standart bo'yicha Kubernetes klasteridagi barcha podlar bir-biri bilan cheklovsiz gaplasha oladi. **NetworkPolicy** — bu podlar darajasidagi firewall (ekran).

- **Foydasi:** Masalan, xavfsizlik uchun "Frontend" podiga faqat "Backend" bilan gaplashishga ruxsat berib, "Ma'lumotlar bazasi"ga to'g'ridan-to'g'ri ulanishni taqiqlab qo'yish mumkin. Bu kiberhujumchi bitta podni buzib kirsam, butun klasterga yoyilib ketishini to'xtatadi.

☆ 72. "Cert-manager" nima vazifani bajaradi?

📖 Bu Kubernetes ichidagi SSL/TLS sertifikatlarini boshqarish vositasi. U **Let's Encrypt** kabi servislar bilan bog'lanib, saytlaringiz uchun sertifikatlarni avtomatik oladi, klasterga o'rnatadi va muddati tugashidan oldin avtomatik yangilaydi (renew). Buzib yuzlab domenlarning SSL muddatini kuzatish imkonsiz bo'lar edi.

? 73. "Helm Chart" ichidagi `values.yaml` faylining vazifasi nima?

📖 Helm Chart — bu dasturning shabloni. `values.yaml` esa o'sha shablondagi o'zgaruvchilar (variables) saqlanadigan joy.

- **Misol:** Siz bitta shablondan foydalanib, "Staging" uchun kichikroq CPU, "Production" uchun esa kattaroq CPU resurslarini aynan shu fayl orqali belgilaysiz. Bu bitta kodni turli muhitlarda qayta ishlatish imkonini beradi.

☆ 74. "Error Budget" (Xatolik budjeti) nima?

📖 Bu SRE falsafasining asosi. Masalan, agar sizning saytingiz 99.9% vaqt ishlashi kerak bo'lsa (SLA), sizga oyiga 43 daqiqa "to'xtab qolish" (downtime) huquqi beriladi.

- **Mantiq:** Agar siz bu 43 daqiqani ishlatib bo'lsangiz (budjet tugasa), yangi funksiyalar qo'shish to'xtatiladi va barcha kuch faqat barqarorlikni tiklashga qaratiladi.

? 75. "Post-mortem" hujjati nima uchun yoziladi?

📖 Tizimda jiddiy xato (incident) sodir bo'lib, tuzatilgandan keyin yoziladigan hisobot. Uning asosiy qoidasi — **aybdorni qidirmaslik** (Blameless). Maqsad: "Nega xato bo'ldi?", "Qanday tuzatdik?" va "Kelajakda qaytalanmasligi uchun nima qilish kerak?" degan savollarga javob topish.

☆ 76. "Vertical Pod Autoscaler" (VPA) va HPA farqi?



- **HPA (Horizontal):** Podlar sonini ko'paytiradi (1 tadan 5 tagacha).
- **VPA (Vertical):** Podning kuchini (CPU/RAM hajmini) oshiradi. VPA asosan ma'lumotlar bazasi kabi sonini ko'paytirish qiyin bo'lgan servislar uchun foydalidir.

? 77. "Load Average" ko'rsatkichi nimani bildiradi?

Linuxda `top` yoki `uptime` buyrug'i orqali ko'rinadigan bu raqam protsessor (CPU) navbatida turgan jarayonlar sonini ko'rsatadi. Agar 1 minutlik ko'rsatkich CPU yadrolari sonidan ko'p bo'lsa, demak server yuklamani ko'tara olmayapti va "tormozlanish" boshlangan.

☆ 78. Linuxda "OOM Killer" (Out of Memory) nima?

Serverda RAM tugab qolsa, operatsion tizim butunlay qotib qolmasligi uchun eng ko'p xotira yeyayotgan jarayonni (masalan, Java yoki Python dasturini) majburan o'ldiradi. DevOps muhandisi buni oldini olish uchun Kubernetesda "Resources Limits" qo'yishi shart.

? 79. "Swap space" nima va nega u Kubernetes serverlarida o'chirib qo'yiladi?

Swap — bu RAM to'lib qolganda diskning bir qismini vaqtincha xotira sifatida ishlatish. Kubernetesda bu o'chiriladi, chunki disk RAMdan ming barobar sekin ishlaydi va bu model Kubernetesning resurslarni aniq taqsimlash algoritmini (scheduler) adashtirib yuboradi.

☆ 80. "Multi-cloud" va "Hybrid Cloud" farqi?



- **Multi-cloud:** Bir vaqtning o'zida ham AWS, ham Google Cloud xizmatlaridan foydalanish (bitta provayderga bog'lanib qolmaslik uchun).
- **Hybrid Cloud:** Kompaniyaning o'z jismoniy serverlari (On-premise) va bulutli xizmatlarni (Public Cloud) birlashtirib ishlatishi.

? 81. "Infrastructure Drift" nima?

Bu Terraform orqali yaratilgan serverni kimdir qo'lda (AWS paneliga kirib) o'zgartirib qo'yishi. Natijada koddagi holat bilan real hayotdagi holat farq qilib qoladi. Buni terraform `plan` orqali aniqlash va `apply` orqali kod holatiga qaytarish mumkin.

☆ 82. "Serverless"da "Cold Start" muammosi nima?

📖 AWS Lambda kabi funksiyalar uzoq vaqt ishlatilmasa, "uyquga" ketadi. Kimdir unga murojaat qilganda, konteynerni noldan yoqish uchun 1-2 soniya vaqt ketadi. Bu birinchi foydalanuvchi uchun javob sekin kelishiga sabab bo'ladi.

❓ 83. "You build it, you run it" tamoyili nimani anglatadi?

📖 Bu Amazon kompaniyasi tomonidan ommalashgan shior. Ya'ni dasturni yozgan jamoa uning serverda ishlashi va xatolarini tuzatishga ham mas'ul. Bu dasturchilarni sifatliroq kod yozishga va operatsion jarayonlarni tushunishga majbur qiladi.

★ 84. Agar Production serverda disk 100% to'lib qolsa, birinchi yordam qanday bo'ladi?



1. `df -h` orqali qaysi disk to'lganini aniqlash.
2. `du -sh *` orqali eng katta fayllarni (ko'pincha eski loglarni) topish.
3. Keraksiz loglarni `truncate -s 0 file.log` orqali tozalash (o'chirib yuborgandan ko'ra xavfsizroq, chunki ochiq turgan faylni o'chirish diskni bo'shatmasligi mumkin).
4. Doimiy yechim sifatida "Log rotation"ni sozlash.

❓ 85. "Everything as Code" (EaC) tushunchasi nima?

📖 Nafaqat infratuzilmani (IaC), balki xavfsizlik qoidalarini (PaC - Policy as Code), hujjatlarni (DaC) va monitoring dashboardlarini ham kod ko'rinishida saqlash va avtomatlashtirish.

❓ 86. Nginx-da "Reverse Proxy" va "Load Balancer" o'rtasidagi farq nimada?



- **Reverse Proxy:** Bitta server oldida turib, so'rovlarni bitta ichki servisga yo'naltiradi. U asosan SSL-ni boshqarish va xavfsizlik uchun ishlatiladi.
- **Load Balancer:** So'rovlarni bir nechta backend serverlar (upstream) o'rtasida taqsimlaydi (Round Robin, Least Conn usullarida).

★ 87. "Shared Nothing Architecture" nima?

📖 Bu har bir tugun (node) butunlay mustaqil bo'lgan arxitektura. Markaziy bog'liqlik yo'qligi sababli, bitta server o'chsa, qolganlari hech qanday muammosiz ishlashda davom etadi. Bu tizimni cheksiz kengaytirish (scaling) imkonini beradi.

🔗 88. Git-da Merge va Rebase farqi nimada?



- **Merge:** Ikki tarmoqni birlashtiradi va yangi "merge commit" hosil qiladi. Tarixni saqlab qoladi, lekin tarmoqlar chalkashib ketishi mumkin.
- **Rebase:** Bir tarmoqdagi o'zgarishlarni ikkinchi tarmoqning eng uchiga ko'chirib qo'yadi. Natijada tarix bitta chiziqdek chiroyli ko'rinadi, lekin commit ID-lari o'zgarib ketadi.

★ 89. Docker-da "Non-root User" ishlatish nega shart?



Standart bo'yicha Docker konteynerlari `root` huquqi bilan ishlaydi. Agar xaker konteynerni buzib kirs, u asosiy serverni (host) ham boshqarish huquqiga ega bo'lib qolishi mumkin. Shuning uchun Dockerfile ichida doimo maxsus foydalanuvchi yaratish va `USER` buyrug'i bilan unga o'tish shart.

🔗 90. Kubernetesda "Resource Quota" va "LimitRange" farqi?



- **Resource Quota:** Butun bir **Namespace** (loyiha) uchun limit qo'yadi (masalan, bu loyiha jami 10 ta CPU-dan ko'p ishlata olmaydi).
- **LimitRange:** Har bir alohida **Pod** yoki konteyner uchun minimal/maximal chegaralarni belgilaydi.

★ 91. "Infrastructure as Code" da "Idempotency" nima?



Bu kodni necha marta ishga tushirishingizdan qat'i nazar, natija bir xil bo'lishi. Masalan, Terraform-ga "10 ta server yarat" desangiz va u yerda allaqachon 10 ta server bo'lsa, u hech narsani o'zgartirmaydi. Bu xatolarni oldini oluvchi eng muhim xususiyatdir.

🔗 92. "Micro-segmentation" xavfsizlikda nima?



Bu tarmoqni juda kichik, izolyatsiya qilingan bo'laklarga bo'lishdir. Agar xaker bitta servisni buzsa, u hatto yonidagi servisga ham o'ta olmaydi, chunki ularning orasida virtual "devor" (Network Policy) bor.

★ 93. Kubernetesda "StatefulSet" uchun "Headless Service" nega kerak?



Ma'lumotlar bazasi (masalan, MongoDB) klasterida har bir pod o'zining individual DNS nomiga ega bo'lishi kerak (masalan, `db-0.mongo`). Headless service yuklamani taqsimlamasdan, so'rovni aynan kerakli pod-ning IP-manziliga yo'naltirish imkonini beradi.

❓ 94. Ssenariy: Saytingiz dushanba kuni soat 9:00 da haddan tashqari sekinlashdi. Birinchi qadamingiz?



1. **Monitoring (Grafana/Prometheus):** Yuklama (CPU, Traffic, Memory) qayerda oshganini ko'rish.

2. **Logs (Kibana/ELK):** Xato xabarlarini yoki 5xx status kodlarini tekshirish.

3. **Database:** Bloklangan so'rovlar (long-running queries) bormi?

4. **Network:** DDoS hujumi belgilari bormi yoki tashqi API-lar sekin javob beryaptimi?

★ 95. "Blue-Green" deploymentda ma'lumotlar bazasi migratsiyasi qanday hal qilinadi?

📖 Bu eng qiyin qism. Odatda "**Backward Compatible**" (ortga moslashuvchan) o'zgarishlar qilinadi. Ya'ni baza yangi kodni ham, eski kodni ham bir vaqtda qo'llab-quvvatlaydigan holatga keltiriladi. Shundan keyingina trafik yangi versiyaga o'tkaziladi.

❓ 96. "Service Discovery" nima va u Kubernetesda qanday ishlaydi?

📖 Podlar o'chib-yonganda ularning IP-manzillari o'zgaradi. **Service Discovery** (Kubernetes DNS) orqali biz pod IP-sini bilishimiz shart emas, biz shunchaki servis nomiga (masalan, `http://backend-service`) murojaat qilamiz, K8s esa uni hozirgi tirik pod-ning IP-siga yo'naltiradi.

★ 97. "Container Runtime" (containerd, CRI-O) nima?

📖 Kubernetes konteynerlarni o'zi ishga tushirmaydi. U bu vazifani "Container Runtime"ga topshiradi. Ilgari bu Docker edi, hozirda esa yengilroq va tezroq bo'lgan containerd yoki CRI-O ishlatiladi.

❓ 98. "Ephemeral Storage" nima?

📖 Bu konteyner ichidagi vaqtinchalik xotira. Konteyner o'chishi bilan bu ma'lumotlar ham o'chib ketadi. U keshlar yoki vaqtinchalik fayllar uchun ishlatiladi. Muhim ma'lumotlar uchun doimo **Persistent Volume** ishlatish kerak.

★ 99. "Observability" va "Monitoring" farqi?



- **Monitoring:** "Tizim ishlayaptimi?" degan savolga javob beradi (Dashboardlar).

- **Observability:** "Nega u bunday ishlayapti?" degan savolga javob beradi (Logs + Metrics + Tracing). Bu tizimning ichki holatini tashqi ko'rsatkichlar orqali tushunish qobiliyatidir.

🔗 100. DevOps muhandisi uchun eng muhim "soft skill" nima?

📖 **Empatiya va hamkorlik.** DevOps — bu texnologiya emas, balki odamlar orasidagi to'siqlarni buzishdir. Dasturchining muammosini o'z muammosiday ko'rish va hamkorlikda yechim topish DevOps muhandisining asosiy kuchi hisoblanadi.

8. Web texnologiyalari

Atamalar va izohlar

Atamalar	Izohlar
HTML	Veb-sahifaning strukturasi va skeletini belgilovchi belgilash tili.
CSS	Sahifaning dizayni, ranglari va vizual ko'rinishini boshqaruvchi uslublar jadvali.
JavaScript	Sahifaga dinamika va interaktivlik bag'ishlovchi dasturlash tili.
DOM	HTML hujjatning brauzerdagi daraxtsimon ob'ekt modeli.
React / Vue	Frontend interfeyslarni komponentlar asosida qurish uchun kutubxona/freymvork.
Node.js	JavaScript-ni server tomonda ishlatish imkonini beruvchi muhit (runtime).
Express.js	Node.js uchun minimalist va tezkor veb-backend freymvorki.
REST API	Mijoz va server orasida HTTP metodlari orqali ma'lumot almashish standarti.
JSON	Ma'lumotlarni uzatish uchun ishlatiladigan yengil va o'qishga qulay format.
AJAX / Fetch	Sahifani yangilamasdan serverdan ma'lumotlarni yuklab olish texnologiyasi.
Atama	Tavsif va Vazifasi
SQL	Strukturaviy ma'lumotlar bazasi (PostgreSQL, MySQL) bilan ishlash tili.
NoSQL	Moslashuvchan, hujjatsiz ma'lumotlar bazasi (MongoDB, Redis).
MongoDB	Hujjatga yo'naltirilgan (JSON-like) eng mashhur NoSQL bazasi.
Middleware	So'rov va javob orasida ishlovchi oraliq dasturiy ta'minot funksiyasi.
Box Model	CSS-dagi elementlarning padding, border va margin qismlaridan iborat modeli.
Responsive	Saytning mobil, planshet va kompyuter ekranlariga moslashuvchanligi.
Virtual DOM	React-da tezlikni oshirish uchun ishlatiladigan DOM-ning xotiradagi nusxasi.
SPA	Faqat bitta sahifadan iborat, yuklanmasdan ishlovchi veb-ilova.
SSR	Sahifani serverda generatsiya qilib brauzerga yuborish usuli (SEO uchun muhim).
CSR	Sahifani foydalanuvchi brauzerida JavaScript orqali yig'ish usuli.
Atama	Tavsif va Vazifasi
JWT	Foydalanuvchini autentifikatsiya qilish uchun ishlatiladigan raqamli token.
CORS	Bir domendan boshqasiga ma'lumot so'rashni nazorat qiluvchi xavfsizlik mexanizmi.
GraphQL	Mijozga aynan qaysi ma'lumotlar kerakligini so'rash imkonini beruvchi til.
ORM / ODM	Kod orqali ma'lumotlar bazasini boshqarish vositasi (Sequelize, Mongoose).
WebSocket	Server va mijoz orasidagi real-vaqtda ikki tomonlama aloqa.
NPM / Yarn	JavaScript paketlari va kutubxonalarini boshqarish vositasi.
TypeScript	JavaScript-ning xatolar oldini oluvchi, tiplashtirilgan versiyasi.
Redux / Pinia	Katta loyihalarda ma'lumotlar holatini (state) markaziy boshqarish tizimi.
SASS / SCSS	CSS-ni funksiyalar va o'zgaruvchilar bilan kengaytiruvchi pre-protssessor.
Vite / Webpack	Kodni brauzer uchun kichraytirib va optimallashtirib yig'uvchi vositalar.
Atama	Tavsif va Vazifasi
PWA	Saytni mobil ilovaga o'xshatib ishlatish texnologiyasi.
Headless CMS	Frontend-ga bog'lanmagan, faqat ma'lumotlarni API orqali beruvchi boshqaruv tizimi.
Semantic HTML	Mazmunni anglatuvchi teglar (<header>, <main>) to'plami.
SEO	Saytni qidiruv tizimlarida (Google) yuqori o'rnlarga ko'tarish optimizatsiyasi.
Caching	Tezlikni oshirish uchun ma'lumotlarni vaqtincha saqlab turish (Redis, Browser Cache).

Bundling	Ko'plab fayllarni bitta JS/CSS fayliga birlashtirish jarayoni.
Hydration	SSR-dan kelgan HTML-ga JavaScript orqali funktsionallik qo'shish.
Lighthouse	Sayt sifatini (tezlik, SEO) tekshiruvchi Google vositasi.
Web Vitals	Sayt sifatini o'lchovchi Google metrikalari (LCP, FID, CLS).
Environment	Loyiha sozlamalari saqlanadigan .env fayli (API kalitlar va h.k.).
Atama	Tavsif va Vazifasi
BFF	Har bir frontend turi uchun alohida kichik backend yaratish patterni.
Event Loop	Node.js va JS-ning asinxron amallarni tartibga soluvchi mexanizmi.
Streams	Katta hajmdagi ma'lumotlarni bo'laklab o'qish/yozish usuli.
Jest / Cypress	Kodni unit va E2E testdan o'tkazish uchun kutubxonalar.
Refactoring	Kodning ishini o'zgartirmasdan, uning tuzilishini yaxshilash.
Middleware	So'rov serverga yetib borguncha o'rtada bajariladigan tekshiruvlar.
Rate Limiting	Bitta foydalanuvchi yuborishi mumkin bo'lgan so'rovlar sonini cheklash.
Authentication	Foydalanuvchining kimligini aniqlash (Login/Password).
Authorization	Foydalanuvchining nimaga ruxsati borligini aniqlash (Admin/User).
Deployment	Tayyor loyihani serverga (Vercel, AWS) yuklash jarayoni.

Savol-javoblar

? 1. **HTML nima?** 📖 **HTML (HyperText Markup Language)** — veb-sahifaning "skeleti" yoki strukturasi. U sahifadagi matn, rasm, havola va tugmalarning joylashuvini belgilaydi.

☆ 2. **HTML5 dagi semantik teglar nima va ular nega kerak?** 📖 Semantik teglar (masalan: <header>, <footer>, <article>, <section>) sahifaning mazmunini ham brauzerga, ham qidiruv tizimlariga (SEO) tushunarli qilib tushuntiradi.

? 3. **CSS nima?** 📖 **CSS (Cascading Style Sheets)** — veb-sahifaning "tashqi ko'rinishi" (dizayni). U ranglar, shriftlar, o'lchamlar va elementlarning sahifadagi joylashuv tartibini boshqaradi.

☆ 4. **"Box Model" tushunchasini tushuntiring.** 📖 Har bir HTML elementi CSS-da to'rtburchak quti sifatida ko'riladi. U quyidagilardan iborat:

- **Content:** Matn yoki rasm turadigan joy.
- **Padding:** Kontent va chegara orasidagi ichki bo'shliq.
- **Border:** Elementning tashqi chizig'i (chegarasi).
- **Margin:** Element va uning atrofida gilar orasidagi tashqi bo'shliq.

? 5. **"Responsive Web Design" nima?** 📖 Bu veb-saytning barcha qurilmalarda (telefon, planshet, kompyuter) ekran o'lchamiga qarab chiroyli va qulay moslashishidir. Buning uchun asosan CSS **Media Queries** ishlatiladi.

☆ **6. JavaScript (JS) nima?** 📖 JS — veb-sahifalarga "jon" bag'ishlovchi dasturlash tili. U sahifadagi harakatlarni (tugma bosilganda nima bo'lishi, ma'lumotlarni yuklash, animatsiyalar) boshqaradi.

? **7. DOM (Document Object Model) nima?** 📖 DOM — bu HTML sahifaning brauzer xotirasidagi daraxtsimon ko'rinishi. JavaScript DOM orqali HTML elementlarini o'zgartirishi, o'chirishi yoki yangisini qo'shishi mumkin.

☆ **8. "Closure" (Yopilish) tushunchasi JS-da nimani anglatadi?** 📖 Closure — bu ichki funksiyaning o'zidan tashqaridagi (tashqi funksiya) o'zgaruvchilarni, hatto tashqi funksiya ishini tugatgandan keyin ham eslab qolish qobiliyatidir.

? **9. ES6 (ECMAScript 2015) ning eng muhim yangiliklari nima?** 📖 `let` va `const`, **Arrow functions** (`=>`), **Template literals** (```), **Destructuring**, va **Promises** kabi zamonaviy qulayliklar.

☆ **10. AJAX va Fetch API nima uchun kerak?** 📖 Ular veb-sahifani qayta yuklamasdan (refresh qilmasdan), serverdan ma'lumotlarni orqa fonda yuklab olish va sahifani yangilash uchun ishlatiladi.

? **11. React nima?** 📖 Facebook tomonidan yaratilgan JavaScript kutubxonasi. U **Component-based** (qismlarga bo'lingan) yondashuv orqali murakkab interfeyslarni oson qurishga yordam beradi.

☆ **12. "Virtual DOM" React-da qanday ishlaydi?** 📖 React sahifani har safar to'liq yangilamaydi. U avval o'zgarishlarni "Virtual DOM" (xotiradagi nusxa)da hisoblaydi va faqat o'zgargan qismnigina real sahifada yangilaydi. Bu tezlikni bir necha barobar oshiradi.

? **13. React-da "State" va "Props" farqi nimada?** 📖

- **State:** Komponentning o'z ichidagi, o'zgarishi mumkin bo'lgan shaxsiy ma'lumotlari.
- **Props:** Yuqori komponentdan quyi komponentga uzatiladigan (faqat o'qish uchun) ma'lumotlar.

☆ **14. Vue.js ning React-dan asosiy farqi nimada?** 📖 Vue.js o'rganishga osonroq interfeysga ega va u HTML/CSS/JS-ni aniq ajratib ishlashni afzal ko'radi. Uning "Two-way data binding" xususiyati ma'lumotlarni real vaqtda sinxronlashni osonlashtiradi.

? **15. Single Page Application (SPA) nima?** 📖 SPA — bu sahifalar o'rtasida o'tganda brauzer to'liq yuklanmaydigan, faqat ichidagi ma'lumot almashadigan veb-ilo va (masalan: Gmail, Facebook).

☆ **16. Node.js nima?**  Bu JavaScript-ni brauzerdan tashqarida, ya'ni serverda ishlatish imkonini beruvchi muhit (runtime). U Google Chrome-ning **V8** dvigatelida ishlaydi.

? **17. Node.js nega "Single-threaded" bo'lishiga qaramay tez ishlaydi?**  U **Non-blocking I/O** va **Event Loop** mexanizmi orqali ishlaydi. Ya'ni, server ma'lumotlar bazasidan javob kutib turmaydi, balki boshqa so'rovlarni bajarishda davom etadi. Javob kelishi bilan "Event" orqali unga qaytadi.

☆ **18. Express.js nima?**  Bu Node.js uchun eng mashhur veb-freymvork bo'lib, u API yaratish, marshrutlash (routing) va server logikasini yozishni osonlashtiradi.

? **19. NPM (Node Package Manager) nima?**  Bu dunyodagi eng katta ochiq kodli kutubxonalar ombori. Dasturchilar tayyor kod bo'laklarini (paketlarni) loyihalariga qo'shish uchun ishlatishadi.

☆ **20. RESTful API nima?**  Bu mijoz (frontend) va server (backend) o'rtasida ma'lumot almashish standarti. U **GET** (olish), **POST** (yaratish), **PUT** (tahrirlash) va **DELETE** (o'chirish) kabi HTTP metodlariga asoslanadi.

? **21. SQL (Relational) ma'lumotlar bazasi nima?**  Ma'lumotlarni qat'iy jadvallar va ustunlar ko'rinishida saqlaydigan baza (masalan: PostgreSQL, MySQL). Ular orasidagi bog'liqliklar (Relational) juda muhim.

☆ **22. NoSQL (Non-relational) nima?**  Ma'lumotlarni jadvallar emas, balki moslashuvchan "Hujjat" (JSON) yoki "Kalit-qiyamat" shaklida saqlaydigan baza (masalan: **MongoDB**). U juda katta hajmdagi va tuzilishi o'zgaruvchan ma'lumotlar uchun mos.

? **23. MongoDB-da "Collection" va "Document" nima?**  SQL-dagi "Jadval" MongoDB-da **Collection**, "Qator" esa **Document** (JSON formatidagi ob'ekt) deb ataladi.

☆ **24. ORM (Object-Relational Mapping) nima uchun kerak?**  Bu dasturchiga SQL kod yozmasdan, o'zi ishlatayotgan dasturlash tili (masalan, JavaScript ob'ektlari) orqali ma'lumotlar bazasi bilan muloqot qilish imkonini beradi (masalan: Sequelize, Mongoose).

? **25. "Indexing" ma'lumotlar bazasida nima beradi?**  Indexing — bu bazada qidirish tezligini oshirish uchun ishlatiladigan texnologiya. Uni kitobning oxiridagi "mundarija" kabi tasavvur qiling: hamma sahifani varaqlamay, kerakli ma'lumotni darhol topasiz.

☆ **26. JWT (JSON Web Token) nima?** 📖 Bu foydalanuvchini autentifikatsiya qilish usuli. Foydalanuvchi login qilsa, server unga shifrlangan "token" (kalit) beradi. Foydalanuvchi keyingi so'rovlarida shu kalitni ko'rsatadi va server uni taniydi.

❓ **27. CORS (Cross-Origin Resource Sharing) xatosi nima?** 📖 Bu xavfsizlik cheklovi bo'lib, u bir domendagi sayt (masalan, `site.uz`) boshqa domendagi API-dan (`api.com`) ruxsatsiz ma'lumot olishini to'sib qo'yadi. Uni serverda ruxsat berish orqali hal qilinadi.

☆ **28. "Middleware" nima vazifani bajaradi?** 📖 Bu serverda so'rov kelganda, u yakuniy manzilga yetib borguncha o'rtada ishlovchi funksiya. Masalan, foydalanuvchi tizimga kirganmi yoki yo'qligini tekshirish uchun middleware ishlatiladi.

❓ **29. TypeScript nima va u nega JS-dan afzal?** 📖 TypeScript — bu JavaScript-ga "Tiplashtirish" (Type safety) xususiyatini qo'shadi. U dastur yozish vaqtida xatolarni aniqlab beradi va katta loyihalarni boshqarishni osonlashtiradi.

☆ **30. "Hydration" (Gidratsiya) nima?** 📖 Bu Server-Side Rendering (SSR) dan kelgan oddiy HTML-ga brauzerda JavaScript orqali "jon" bag'ishlash (uni interaktiv qilish) jarayonidir.

❓ **31. "Hoisting" (Ko'tarilish) nima?**

📖 Bu JavaScript-da o'zgaruvchi va funksiyalar e'lon qilinishidan oldin ularga murojaat qilish imkoniyatini beruvchi mexanizm. `var` bilan e'lon qilingan o'zgaruvchilar ko'tariladi (lekin qiymati `undefined` bo'ladi), `let` va `const` esa "Temporal Dead Zone" sababli xatolik beradi.

☆ **32. JavaScript-da Promise nima va u qanday holatlarga ega?**

📖 Promise — bu asinxron operatsiyaning yakuniy natijasini (muvaffaqiyat yoki xato) ifodalovchi ob'ektdir. U 3 xil holatda bo'ladi:

1. **Pending:** Kutilmoqda (bajarilmadi).
2. **Fulfilled:** Muvaffaqiyatli yakunlandi.
3. **Rejected:** Xatolik bilan yakunlandi.

❓ **33. Async/Await nima va u Promisedan nimasi bilan farq qiladi?**

📖 Bu asinxron kodni xuddi sinxron kod kabi o'qishga qulay qilib yozish usuli. `Await` — promise bajarilguncha kodning o'sha qismida kutishni buyuradi. Bu kodni "callback hell"dan saqlaydi.

☆ 34. "Event Bubbling" (Hodisaning ko'pirishi) nima?

📖 Agar siz ichma-ich joylashgan elementning (masalan, tugmaning) ustiga bossangiz, hodisa avval o'sha elementda ishlaydi, so'ngra zanjir bo'ylab uning ota-elementlariga (parent) yuqoriga qarab "ko'pirib" chiqadi.

❓ 35. JavaScript-da **this** kalit so'zi nimani anglatadi?

📖 **this** — bu hozirda kod bajarilayotgan kontekstga (ob'ektga) ishoradir. Uning qiymati funksiya qayerda va qanday chaqirilganiga qarab o'zgaradi. Masalan, ob'ekt ichidagi metodda **this** o'sha ob'ektning o'zini bildiradi.

☆ 36. "State Management" (Redux, Pinia) nega kerak?

📖 Katta loyihalarda ma'lumotlarni (state) bitta komponentdan 10 ta pastdagi komponentga uzatish qiyinlashadi. Redux yoki Pinia barcha ma'lumotlarni bitta markaziy "ombor"da saqlaydi va istalgan komponent undan ma'lumotni to'g'ridan-to'g'ri olishi mumkin.

❓ 37. "Next.js" yoki "Nuxt.js" kabi frameworklar nima beradi?

📖 Bular React va Vue ustiga qurilgan bo'lib, **Server-Side Rendering (SSR)** va **Static Site Generation (SSG)** imkoniyatini beradi. Bu saytning Google-da (SEO) yaxshi ko'rinishini va tez yuklanishini ta'minlaydi.

☆ 38. "SASS/SCSS" nima?

📖 Bu CSS-ning "kuchaytirilgan" versiyasi (Pre-processor). U o'zgaruvchilar, ichma-ich yozish (nesting) va funksiyalarni qo'llab-quvvatlaydi. Bu CSS kodini yozishni ancha tezlashtiradi.

❓ 39. Webpack yoki Vite nima vazifani bajaradi?

📖 Bular "Module Bundlers" (Yig'uvchilar). Ular siz yozgan yuzlab JS, CSS va rasm fayllarini bitta yoki bir nechta kichik "bundle" (paket) holatiga keltirib, brauzerga moslashtiradi.

☆ 40. "Progressive Web App" (PWA) nima?

📖 Bu oddiy veb-saytning o'zini xuddi mobil ilova kabi tutishidir. Uni telefonga o'rnatish, oflayn rejimda ishlatish va unga bildirishnomalar (push notifications) yuborish mumkin.

❓ 41. "GraphQL" nima va u REST API-dan nimasi bilan farq qiladi?

📖 REST-da server qanday ma'lumot bersa, shuni olasiz. **GraphQL**-da esa mijoz (frontend) aynan qaysi ustunlar kerakligini serverdan so'raydi. Bu tarmoq trafiginı tejaydi va ortiqcha ma'lumot yuklanishini oldini oladi.

★ 42. "Microservices" veb-dasturlashda qanday qo'llaniladi?

📖 Bitta ulkan backend dasturi o'rniga, u bir nechta kichik, mustaqil servislar (masalan: login servisi, to'lov servisi, qidiruv servisi)ga bo'linadi. Bu jamoalarga bir-biriga xalaqit bermay ishlash imkonini beradi.

❓ 43. "Socket.io" nima uchun ishlatiladi?

📖 U WebSocket protokoli orqali real vaqtdagi ikki tomonlama aloqani osonlashtiruvchi kutubxona. U yordamida "Online Chat"lar yoki "Live Tracking" tizimlari yaratiladi.

★ 44. "Rate Limiting" nima?

📖 Bu xavfsizlik chorasi bo'lib, bitta foydalanuvchining (IP-manzilning) ma'lum vaqt ichida API-ga necha marta so'rov yuborishini cheklaydi. Bu DDoS hujumlaridan himoya qiladi.

❓ 45. "Environment Variables" (.env) nima?

📖 Bu maxfiy sozlamalar (masalan, Ma'lumotlar bazasi paroli yoki API kaliti) saqlanadigan fayl. U xavfsizlik uchun hech qachon Git-ga yuklanmaydi.

★ 46. "Redis" nima va u nega kerak?

📖 Redis — bu ma'lumotlarni operativ xotirada (RAM) saqlaydigan o'ta tezkor baza. U asosan keshlar (tez-tez so'raladigan ma'lumotlarni saqlash) va foydalanuvchi seanslarini (session) saqlash uchun ishlatiladi.

❓ 47. "Database Migration" nima?

📖 Bu ma'lumotlar bazasining strukturasini (ustunlar, jadvallar) versiyalash usuli. U orqali bir necha dasturchilar bazada o'zgarish qilganda, hamma bir xil strukturada qolishini ta'minlaydi.

★ 48. "Normalizatsiya" va "Denormalizatsiya" (SQL) nima?



- **Normalizatsiya:** Ma'lumotlarni takrorlanmasligi uchun jadvallarga bo'lish.
- **Denormalizatsiya:** Tezlikni oshirish uchun ma'lumotlarni ataylab bitta jadvalga birlashtirish.

❓ 49. "Transactions" (Tranzaksiyalar) nima?

📖 Bu bazadagi bir nechta amallar guruhi. Agar bittasi xato bo'lsa, hammasi bekor qilinadi (Rollback). Masalan, bankda pul o'tkazishda: biri hisobidan pul kamayishi va ikkinchisiga qo'shilishi bitta tranzaksiya bo'lishi shart.

★ 50. "Search Engines" (Elasticsearch) nima uchun kerak?

📖 Oddiy SQL bazalar matn ichidan (masalan, millionlab tovarlar orasidan) juda sekin qidiradi. Elasticsearch esa matnlarni maxsus index-lab saqlaydi va millisekundlarda qidirib topadi.

Veb dasturlash bo'yicha sanoqni 51-savoldan davom ettiramiz. Ushbu qismda veb xavfsizlik, autentifikatsiya tizimlari va zamonaviy veb-arxitektura usullarini ko'rib chiqamiz.

❓ 51. "XSS" (Cross-Site Scripting) hujumi nima?

📖 XSS — bu xaker tomonidan veb-saytga zararli JavaScript kodini kiritish hujumi. Bu kod boshqa foydalanuvchilarning brauzerida ishga tushadi va ularning seanslarini yoki maxfiy ma'lumotlarini o'g'irlashi mumkin.

★ 52. "SQL Injection" hujumi nima va uni qanday oldini olish mumkin?

📖 Bu hujumda xaker input maydonlariga (masalan, login formasi) maxsus SQL buyruqlarini yozib, ma'lumotlar bazasiga ruxsatsiz kirishga urinadi. **Yechim:** "Prepared Statements" va "Parameterized Queries"dan foydalanish. Hech qachon foydalanuvchi yozgan matnni to'g'ridan-to'g'ri SQL kodiga qo'shib yubormaslik kerak.

❓ 53. "CSRF" (Cross-Site Request Forgery) nima?

📖 Bu foydalanuvchini o'zi bilmagan holda, u login qilgan boshqa saytda biror amal bajarishga majburlash (masalan, pul o'tkazish). **Yechim:** "CSRF Token"laridan foydalanish. Har bir so'rov bilan birga faqat serverga ma'lum bo'lgan maxfiy kalit yuboriladi.

★ 54. "OAuth 2.0" protokoli nima?

📖 Bu foydalanuvchiga o'z parolini bermasdan, uchinchi tomon ilovalariga (masalan, Google orqali saytga kirish) ruxsat berish standarti. Sayt parolni ko'rmaydi, faqat Google-dan "bu foydalanuvchi haqiqiy" degan tasdiqni oladi.

❓ 55. "Salting" va "Hashing" parollar bilan ishlashda nega shart?

📖 Parollarni bazada ochiq matn ko'rinishida saqlash xavfli.

- **Hashing:** Parolni qayta tiklab bo'lmaydigan kodga o'giradi.
- **Salting:** Hash qilishdan oldin parolga tasodifiy belgilar qo'shish. Bu xakerlarga "Rainbow Tables" yordamida parollarni topishni qiyinlashtiradi.

☆ 56. "Server-Side Rendering" (SSR) va "Client-Side Rendering" (CSR) farqi?



- **SSR (Next.js):** Sahifa serverda tayyorlanib, brauzerga tayyor HTML ko'rinishida keladi. SEO uchun juda foydali.
- **CSR (React):** Brauzerga bo'sh HTML va katta JS fayl keladi. Sahifa foydalanuvchi qurilmasida yig'iladi. Dastlabki yuklanish sekin bo'lishi mumkin.

? 57. "Micro-frontends" nima?



Bu microservices g'oyasini frontendga ko'chirish. Bitta katta sayt bir nechta mustaqil jamoalar tomonidan yaratilgan bo'laklarga bo'linadi (masalan, savat qismi Vue-da, qidiruv qismi React-da bo'lishi mumkin).

☆ 58. "WebAssembly" (Wasm) nima?



Bu brauzerda JavaScript-dan ko'ra tezroq ishlaydigan kodlarni (C++, Rust kabi tillarda yozilgan) ishga tushirish imkonini beruvchi texnologiya. U o'ta murakkab video tahrirlash yoki 3D o'yinlarni vebda ishlatish uchun kerak.

? 59. "Serverless Functions" (Lambda) qanday ishlaydi?



Dasturchi serverni sozlashi shart emas. U faqat bitta funksiyani (kodni) yuklaydi. Bu kod faqat biror voqea (masalan, rasm yuklanganda) sodir bo'lganda ishlaydi va bajarilgach o'chadi.

☆ 60. "BFF" (Backend for Frontend) patterni nima?



Har bir platforma (mobil ilova, veb-sayt) uchun alohida kichik backend yaratish. Bu frontendga aynan o'sha qurilma uchun kerakli bo'lgan formatda ma'lumot berishga yordam beradi.

? 61. "Lighthouse" vositasi nima?



Bu Google Chrome ichidagi vosita bo'lib, u saytning tezligi, SEO sifati va qulayligini (Accessibility) 100 ballik tizimda tekshirib beradi.

☆ 62. "Lazy Loading" (Tembel yuklash) nima?

📖 Saytdagi rasm yoki videolarni foydalanuvchi sahifaning o'sha qismiga tushgandagina yuklash usuli. Bu sahifaning birinchi yuklanish vaqtini sezilarli qisqartiradi.

❓ 63. "Tree Shaking" nima?

📖 Dasturni yig'ish (build) vaqtida ishlatilmaydigan kod bo'laklarini (kutubxonalardan olingan ortiqcha funksiyalar) avtomatik ravishda olib tashlash jarayoni.

★ 64. "Critical CSS" nima?

📖 Foydalanuvchi saytga kirganda ekranda ko'rinadigan birinchi qism (Above the fold) uchun kerakli CSS-ni alohida ajratib, uni birinchi bo'lib yuklash.

❓ 65. "HTTP Caching" (ETag, Cache-Control) qanday ishlaydi?

📖 Server brauzerga: "Bu fayl o'zgarmadi, uni xotirangdan (diskdan) ishlat" deb buyruq beradi. Bu serverga qayta-qayta so'rov yuborishni kamaytiradi.

★ 66. "Atomic Design" nima?

📖 Interfeysni kichikdan kattaga qarab qurish: Atomlar (tugma) -> Molekulalar (qidiruv paneli) -> Organizmlar (shapka) -> Shablonlar -> Sahifalar.

❓ 67. "Storybook" vositasi nega kerak?

📖 Bu UI komponentlarni asosiy dasturdan alohida holda vizuallashtirish va testlash imkonini beruvchi muhit. Dizaynerlar va dasturchilar hamkorligi uchun juda qulay.

★ 68. "Shadow DOM" nima?

📖 Bu asosiy sahifa CSS-idan mustaqil bo'lgan, element ichidagi maxfiy DOM. U asosan "Web Components" yaratishda stil va skriptlar bir-biriga xalaqit bermasligi uchun ishlatiladi.

❓ 69. "Web Vitals" nima?

📖 Google tomonidan belgilangan sayt sifatini o'lchovchi 3 ta asosiy metrika:

1. **LCP (Largest Contentful Paint):** Asosiy rasm/matn yuklanish tezligi.
2. **FID (First Input Delay):** Sayt tugmalari qanchalik tez javob berishi.
3. **CLS (Cumulative Layout Shift):** Sahifa elementlari yuklanganda qanchalik qimirlab ketishi (barqarorlik).

★ 70. "Headless CMS" (Strapi, Contentful) nima?

📖 Bu faqat ma'lumotlarni boshqarish paneli bo'lib, uning "yuzi" (frontend) yo'q. Ma'lumotlar API orqali uzatiladi, dasturchi esa frontendni xohlagan tilda (React, Vue) mustaqil yozadi.

Veb dasturlash bo'yicha sanoqni 71-savoldan davom ettiramiz va 100 tagacha yetkazamiz. Ushbu yakuniy qismda asosan Node.js nozikliklari, testlash, arxitektura va deployment jarayonlariga e'tibor qaratamiz.

❓ 71. Node.js-da "Streams" (Oqimlar) nima va ular nega kerak?

📖 Streams — bu ulkan hajmdagi ma'lumotlarni (masalan, 5GB video fayl) bo'laklab o'qish va yozish usuli. U butun faylni RAMga yuklamaydi, balki kichik paketlarda qayta ishlaydi. Bu server xotirasini tejash uchun juda muhimdir.

☆ 72. "Event Emitter" Node.js-da qanday vazifani bajaradi?

📖 Bu Node.js-dagi "Kuzatuvchi" (Observer) patternidir. Biror voqea (event) sodir bo'lganda (masalan, fayl yuklab bo'linganda), Event Emitter unga bog'langan funksiyalarni (listeners) avtomatik ishga tushiradi.

❓ 73. "Middleware" funksiyalari Express.js-da qanday tartibda ishlaydi?

📖 Middleware-lar kodda yozilgan tartibda, yuqoridan pastga qarab ishlaydi. Har bir middleware `next()` funksiyasini chaqirishi shart, aks holda so'rov (request) keyingi bosqichga o'tmay "osilib" qoladi.

☆ 74. Node.js-da "Worker Threads" nima uchun ishlatiladi?

📖 Node.js asosan bitta oqimda (single-thread) ishlaydi. Lekin og'ir matematik hisob-kitoblar (masalan, videoni kodlash) oqimni band qilib qo'yishi mumkin. Worker Threads — bu og'ir ishlarni parallel ravishda boshqa oqimlarga topshirish imkonini beradi.

❓ 75. "Nginx" veb-server sifatida Node.js bilan qanday bog'lanadi?

📖 Odatda Nginx "Reverse Proxy" vazifasini bajaradi. U internetdan kelgan so'rovlarni qabul qiladi va ichki tarmoqda ishlayotgan Node.js dasturiga yo'naltiradi. Bu xavfsizlik va yuklamani taqsimlash uchun qulaydir.

☆ 76. Unit Test, Integration Test va End-to-End (E2E) Test farqi?



- **Unit Test (Jest):** Dasturning eng kichik bo'lagini (bitta funksiyani) alohida tekshiradi.

- **Integration Test:** Bir nechta qismlarning (masalan, API va Ma'lumotlar bazasi) birgalikda to'g'ri ishlashini tekshiradi.

- **E2E Test (Cypress):** Haqiqiy foydalanuvchi kabi brauzerni ochib, login qilishdan to natija olishgacha bo'lgan butun jarayonni tekshiradi.

🔗 77. "TDD" (Test Driven Development) nima?

📖 Bu yondashuvda dasturchi avval kod yozmaydi, balki test yozadi. Test albatta xato beradi (chunki kod yo'q). Keyin testdan o'tish uchun minimal kod yoziladi va keyinchalik kod optimallashtiriladi (Refactor).

★ 78. "Mocking" testlashda nima uchun kerak?

📖 Agar testingiz tashqi API (masalan, bank to'lovi)ga bog'liq bo'lsa, har safar real pul o'tkazib ko'rib bo'lmaydi. Mocking — bu tashqi API-ning soxta nusxasini yaratib, uning "to'g'ri javob berishini" imitatsiya qilishdir.

🔗 79. "gRPC" nima va u qachon RESTdan yaxshiroq?

📖 gRPC — bu Google tomonidan yaratilgan yuqori tezlikdagi aloqa tizimi. U JSON o'rniga "Protocol Buffers" (binar format) ishlatadi. Bu microservices o'rtasidagi aloqani RESTdan ko'ra bir necha barobar tezlashtiradi.

★ 80. "Webhooks" qanday ishlaydi?

📖 Bu "Teskari API"dir. Masalan, to'lov tizimi (Payme) to'lov muvaffaqiyatli bo'lganini sizning serveringizga aytishi uchun siz bergan URL-ga so'rov yuboradi. Ya'ni, siz serverdan "pul tushdimi?" deb so'ramaysiz, serverning o'zi sizga aytadi.

🔗 81. "Rate Limiting" va "Throttling" farqi?



- **Rate Limiting:** Foydalanuvchi ma'lum vaqtda (masalan, 1 minutda) ko'pi bilan 100 ta so'rov yubora olishi.

- **Throttling:** So'rovlar juda ko'payib ketsa, server ularni sekinlashtirishi yoki navbatga qo'yishi.

★ 82. "Web Workers" brauzerda nima qiladi?

📖 Bu brauzerning asosiy oqimini band qilmagan holda, orqa fonda (background) og'ir JavaScript kodlarini ishga tushirish imkonini beradi. Bu sahifaning "qotib" qolishini oldini oladi.

🔗 83. "Service Workers" nima?

📖 Bu brauzer va tarmoq o'rtasida turadigan skript. U ma'lumotlarni keshlaydi va internet yo'q bo'lganda ham saytning ishlashini (PWA) ta'minlaydi.

★ 84. "Preload", "Prefetch" va "Preconnect" farqi?

📖 Bular brauzerga resurslarni oldindan yuklash buyruqlaridir:

- **Preload:** Hozirgi sahifa uchun juda muhim faylni (masalan, asosiy font) darhol yuklash.
- **Prefetch:** Foydalanuvchi keyingi sahifaga o'tishi ehtimoli bo'lgan resursni bo'sh vaqtda yuklab qo'yish.
- **Preconnect:** Tashqi server bilan (masalan, Google Fonts) ulanishni oldindan o'rnatib qo'yish.

❓ 85. "Dockerize" qilish nima?

📖 Bu veb-dasturni (Frontend yoki Backend) barcha sozlamalari bilan Docker konteyneriga o'rash jarayoni. Bu dasturning dasturchi kompyuterida ham, serverda ham bir xil ishlashini kafolatlaydi.

★ 86. "Blue-Green Deployment" vebda qanday amalga oshiriladi?

📖 Serverda dasturning ikki versiyasi turadi: Blue (eski) va Green (yangi). Agar yangi versiya (Green) muvaffaqiyatli ishlasa, Load Balancer orqali barcha foydalanuvchilar o'sha tomonga yo'naltiriladi. Muammo bo'lsa, darhol Blue versiyaga qaytiladi.

❓ 87. "CI/CD Pipeline" veb loyiha uchun qanday bosqichlardan iborat?



1. **Push:** Kod Git-ga yuboriladi.
2. **Build:** NPM kutubxonalar yuklanadi va kod yig'iladi.
3. **Test:** Unit va Integration testlar ishlaydi.
4. **Deploy:** Tayyor kod serverga (masalan, Vercel yoki AWS) yuklanadi.

★ 88. "Micro-frontends" da "Module Federation" nima?

📖 Bu Webpack 5 texnologiyasi bo'lib, bir-biridan mustaqil bo'lgan frontend loyihalarga (masalan, React va Vue loyihalari) o'z kodlarini real vaqtda bir-biri bilan baham ko'rish imkonini beradi.

❓ 89. "Edge Computing" (Vercel Edge Functions) nima?

📖 Kodni markaziy serverda emas, balki foydalanuvchiga eng yaqin bo'lgan nuqtada (Edge nuqtalarda) ishlatish. Bu javob qaytarish vaqtini (latency) deyarli nolga tushiradi.

★ 90. "Headless CMS" (Strapi, Contentful) an'anaviy CMSdan (WordPress) nimasi bilan farq qiladi?

📖 WordPressda ma'lumot ham, uning ko'rinishi ham bitta tizimda bo'ladi. Headless CMSda faqat ma'lumotlar (API) bo'ladi. Dasturchi frontendni o'zi xohlagan texnologiyada (masalan, Next.js) noldan yaratadi.

❓ 91. "Server Actions" (Next.js) nima?

📖 Bu frontenddan turib, hech qanday API yozmasdan, to'g'ridan-to'g'ri serverdagi funktsiyani chaqirish imkoniyati. Bu "Client" va "Server" orasidagi chegarani yanada yaqinlashtiradi.

★ 92. "Web Sockets" vs "Server-Sent Events" (SSE)?



- **Web Sockets:** Ikki tomonlama ochiq kanal (chatlar uchun).
- **SSE:** Faqat serverdan mijozga bir tomonlama doimiy ma'lumot oqimi (masalan, yangiliklar lentasi yoki birja narxlari).

❓ 93. "BFF" (Backend for Frontend) patterni nima?

📖 Bu mobil ilova va veb-sayt uchun alohida-alohida oraliq backend yaratish. Mobil ilova kichikroq ma'lumot so'rasa, veb-sayt to'liqroq ma'lumot olishi mumkin.

★ 94. "Semantic HTML" SEO uchun nega o'ta muhim?

📖 Google botlari sahifani inson kabi ko'rmaydi, ular kodni o'qiydi. Agar siz `<div>` o'rniga `<article>` yoki `<nav>` ishlatsangiz, bot saytning qayeri asosiy mazmun ekanini oson tushunadi va reytingni ko'taradi.

❓ 95. "CORS" xatosi faqat brauzerdami?

📖 Ha. CORS — bu brauzer xavfsizlik mexanizmi. Agar siz serverdan-serverga (masalan, Python-dan Node.js-ga) so'rov yuborsangiz, CORS xatosi hech qachon chiqmaydi.

★ 96. "Denormalization" NoSQLda qanday qo'llaniladi?

📖 MongoDB-da tezlik uchun biz ma'lumotlarni jadvallarga bo'lmaymiz. Masalan, "Post" ichiga uning "Muallif"i haqidagi ma'lumotni ham qo'shib saqlaymiz. Bu bitta so'rov bilan hamma narsani olish imkonini beradi (Joins-dan qochish).

❓ 97. "Hydration" xatosi (Mismatch) nega chiqadi?

📖 Agar serverda generatsiya qilingan HTML bilan brauzerda JavaScript yig'gan HTML farq qilib qolsa (masalan, vaqtni ko'rsatuvchi kod serverda boshqa, mijozda boshqa bo'lsa), React xatolik beradi.

☆ 98. "Zod" yoki "Joi" kabi kutubxonalar nima uchun kerak?

📖 Bu API-ga kelayotgan ma'lumotlar formatini (shemasini) tekshirish (Validation) uchun. Masalan, "Email haqiqatda email formatidami?" yoki "Yosh manfiy son emasmi?" kabi tekshiruvlar uchun.

❓ 99. "Sticky Sessions" yuklama taqsimlagichda nima uchun kerak?

📖 Agar foydalanuvchi ma'lumoti faqat birinchi serverning RAM-ida saqlangan bo'lsa, keyingi so'rovda Load Balancer uni ikkinchi serverga yubormasligi uchun u aynan birinchi serverga "yopishib" (sticky) qolishi kerak.

☆ 100. Veb dasturchi bo'lish uchun eng muhim ko'nikma nima?

📖 **Doimiy o'rganish.** Veb olamida har 6 oyda yangi texnologiya chiqadi. Eng yaxshi dasturchi — bu yangi narsalarni tezda o'rganib, amaliyotda qo'llay oladigan mutaxassisdir.

9. Videokonferensiya aloqa tizimlari

Atamalar va izohlar

Atamalar	Izohlar
WebRTC	Brauzerlararo ishlovchi pluginlarsiz real-vaqt aloqa texnologiyasi.
SIP	Aloqa seanslarini o'rnatish va boshqarish protokoli.
H.323	Professional apparatli videokonferensiya aloqa tizimlari standarti.
VoIP	Internet protokoli orqali ovoz uzatish.
Latency	Signalning bir nuqtadan ikkinchi nuqtaga yetib borish vaqti.
Jitter	Paketlar kelish vaqtidagi tebranishlar (notekislik).
Packet Loss	Tarmoqda ma'lumot paketlarining yo'qolishi.
Codec	Videoni siquvchi va qayta ochuvchi algoritm (H.264, VP9).
MCU	Videolarni serverda birlashtiruvchi markaziy qurilma.
SFU	Videolarni birlashtirmay, yo'naltirib beruvchi zamonaviy server.
STUN	Qurilmaning tashqi IP-manzilini aniqlash xizmati.
TURN	Ma'lumotlarni o'zi orqali tranzit o'tkazuvchi rele serveri.
ICE	NAT/Tarmoqlararo ekranni aylanib o'tish yo'lini qidiruvchi mexanizm.
SDP	Qurilma imkoniyatlarini tavsiflovchi protokol.
RTP	Real vaqtda ma'lumot uzatishning asosiy protokoli.
RTCP	RTP oqimi sifatini nazorat qiluvchi protokol.
AEC	Akustik aks-sadoni (echo) so'ndirish texnologiyasi.
AGC	Mikrofon ovozini avtomatik tenglashtirish.
PTZ	Masofadan boshqariladigan kamera (Pan, Tilt, Zoom).
Full-Duplex	Bir vaqtning o'zida ham gapirish, ham eshitish imkoniyati.
QoS	Video trafikka tarmoqda ustuvorlik berish (xizmati) sozlamasi.
Bandwidth	Internet kanalining o'tkazuvchanlik qobiliyati.
E2EE	Faqat suhbatdoshlar tushunadigan o'ta xavfsiz shifrlash.
Simulcast	Bir vaqtning o'zida bir necha sifatda video yuborish.
SVC	Videoni qatlamlarga bo'lib siqish texnologiyasi.
H.239	Ikki tomonlama video (taqdimot + ma'ruzachi) protokoli.
SIP Trunk	Videokonferensiya aloqa tizimini shahar telefon liniyasiga bog'lash kanali.
Gatekeeper	H.323 tarmog'idagi dispatcher (administrator).
Transcoding	Videoni bir formatdan ikkinchisiga real vaqtda o'girish.
Spatial Audio	Ovozni 3D hajmda (ekrandagi o'ringa qarab) eshitish.
Frame Rate	Bir soniyada namoyish eriladigan kadrlar soni (FPS).
Resolution	Tasvirning aniqligi (720p, 1080p, 4K).
Lip Sync	Ovoz va tasvirning bir-biriga mos kelishi.
Echo Cancellation	Dinamikdan chiqqan ovozni mikrofondan o'chirib tashlash.
Beamforming	Mikrofonning faqat gapirayotgan odamga fokuslanishi.
Noise Floor	Mikrofonning o'zidan chiqadigan minimal shovqin.
Screen Sharing	Kompyuter ekranini boshqalarga ko'rsatish.
Whiteboard	Virtual dskada birgalikda chizish imkoniyati.
Meeting ID	Konferensiyaga kirish uchun takrorlanmas raqam.
Waiting Room	Moderator ruxsat berguncha ishtirokchilar turadigan joy.
DTLS	WebRTC-da kalitlarni xavfsiz almashish protokoli.
SRTP	Shifrlangan real vaqtli ma'lumot uzatish protokoli.
Chroma Key	Yashil fonni o'chirib, virtual rasm qo'yish.
Burstiness	Tarmoqda ma'lumotlarning to'satdan ko'payib ketishi.
FEC	Yo'qolgan paketlarni matematik tiklash usuli.

Interoperability	Turli tizimlarning (masalan, Zoom va Cisco) o‘zaro ishlashi.
LMS	Ta’limni boshqarish tizimi (Videokonferensiya aloqa bilan integratsiya qilinadi).
SIP Proxy	SIP so‘rovlarini yo‘naltiruvchi oraliq server.
Bitrate	Bir soniyada uzatiladigan ma’lumot hajmi (kbps/Mbps).
Latency Spike	Kechikishning kutilmaganda keskin ortib ketishi.
PAL	Fazasi o‘zgaruvchan qator (Phase Alternating Line)
NTSC	Milliy televideniye tizimi qo‘mitasi (National Television System Committee)
Full HD	To‘liq yuqori aniqlik (Full High Definition) hisoblanib unda piksellar soni: 1920x1080 (ya’ni, eniga 1920 ta, bo‘yiga 1080 ta nuqta). Ko‘pincha 1080p deb ham ataladi.

Savol-javoblar

❓ 1. Videokonferensiya aloqasi nima?

📖 Bu ikki yoki undan ortiq uzoqdagi abonentlar o‘rtasida real vaqt rejimida audio, video va ma’lumotlar almashish imkonini beruvchi interaktiv texnologiya hisoblanadi.

★ 2. VoIP (Voice over IP) nima?

📖 **VoIP** - bu ovozli signallarni raqamlashtirib, internet protokoli (IP) tarmog‘i orqali paketlar ko‘rinishida uzatish texnologiyasi. Bu an’anaviy telefon liniyalariga ehtiyojni yo‘qotadi.

❓ 3. Videokonferensiyada “Latency” (Kechikish) nima?

📖 Bu signalning bir nuqtadan ikkinchi nuqtaga yetib borishi uchun ketgan vaqt. Agar kechikish 150-200 ms dan oshsa, muloqotda uzilishlar sezila boshlaydi va odamlar bir-birining gapini bo‘lib qo‘yadi.

★ 4. “Jitter” tushunchasi nimani anglatadi?

📖 **Jitter** - bu ma’lumot paketlarining yetib kelish vaqtidagi tebranishlar (o‘zgaruvchanlik). Agar paketlar bir tekis kelmasa, video “qotib-qotib” qoladi yoki ovozda robotlashgan aks-sado paydo bo‘ladi.

❓ 5. “Echo Cancellation” (Aks-sadoni so‘ndirish) nega kerak?

📖 Bu karnaydan chiqayotgan ovoz mikrofon tomonidan qayta tutib olinishini to‘stuvchi algoritm. Busiz foydalanuvchi o‘z ovozini bir necha soniya kechikish bilan qayta eshitadi, bu esa gapirishga xalaqit beradi.

★ 6. SIP (Session Initiation Protocol) nima?

📖 **SIP** - bu IP-tarmoqlarida aloqa seanslarini o‘rnatish, boshqarish va tugatish uchun ishlatiladigan eng keng tarqalgan signalizatsiya protokoli. U faqat “aloqa

oʻrnatish” bilan shugʻullanadi, maʼlumotni oʻzini (video va audio) boshqa protokollar uzatadi.

? 7. H.323 standarti nima?

📖 Bu ITU-T tomonidan ishlab chiqilgan eskiroq, ammo oʻta ishonchli protokol. U koʻproq professional apparatli (qurilmaga asoslangan) videokonferensiya aloqa tizimlarida (Polycom, Cisco apparat yechimlarida) ishlatiladi.

☆ 8. WebRTC (Web Real-Time Communication) nima?

📖 **WebRTC** - bu brauzerlar (Chrome, Firefox) oʻrtasida hech qanday qoʻshimcha plaginlarsiz toʻgʻridan-toʻgʻri video va audio aloqa oʻrnatish texnologiyasi. Google Meet va Telegram Web aynan shu texnologiya asosida ishlaydi.

? 9. RTP (Real-time Transport Protocol) vazifasi nima?

📖 RTP - bu audio va video maʼlumotlarni real vaqtda uzatuvchi asosiy protokol. U har bir paketga vaqt belgisi (timestamp) va tartib raqami qoʻshadi, bu esa brauzerga paketlarni toʻgʻri tartibda yigʻishga yordam beradi.

☆ 10. SDP (Session Description Protocol) nima uchun kerak?

📖 Bu WebRTC yoki SIP muloqoti boshlanishidan oldin qurilmalarning imkoniyatlari haqidagi maʼlumot (masalan: “men qaysi kodeklarni tushunaman?”, “ekranim oʻlchami qanday?”) almashish imkonini beradi.

? 11. MCU (Multipoint Control Unit) nima?

📖 Bu markazlashtirilgan arxitektura hisoblanadi. Server barcha foydalanuvchilarning videosini qabul qiladi, ularni bitta virtual ishchi oynaga birlashtiradi va har bir kishiga tayyor bitta oqim qilib yuboradi. Bu foydalanuvchi kompyuterini ortiqcha yuklamadan ozod etadi, ammo server uchun juda ogʻir yuklanishni yuzaga kelishiga olib keladi.

☆ 12. SFU (Selective Forwarding Unit) nima?

📖 Bu zamonaviyroq arxitektura (Zoom va Jitsi shunday ishlaydi). Server videolarni birlashtirmaydi, shunchaki ularni boshqalarga yoʻnaltirib beradi. Foydalanuvchi bir vaqtning oʻzida bir nechta kichik oqimlarni qabul qiladi.

? 13. P2P (Peer-to-Peer) ulanish qachon ishlatiladi?

📖 Agar videokonferensiya jarayonida faqat ikki kishi boʻlsa, maʼlumot serverga bormasdan, toʻgʻridan-toʻgʻri brauzerdan brauzerga oʻtadi. Bu xavfsizlik va tezlikni taʼminlaydi.

☆ 14. Video Kodek nima? (H.264, VP8, VP9)

📖 Kodek - bu videoni (kodlab) siquvchi va qayta (dekodlab) ochuvchi algoritim. Masalan, **H.264** kodegi o'ta ommabop, **VP9** kodegi esa Google tomonidan yaratilgan bo'lib, u internet tezligi past bo'lganda ham yuqori sifatli videotasvir beradi.

❓ 15. Audio kodeklar (Opus, G.711) farqi?

📖 **Opus** - zamonaviy va eng yaxshi audio kodek bo'lib, u inson ovozi ham, musiqani ham birdek sifatli uzata oladi. **G.711** esa eski raqamli telefoniya standarti hisoblanadi.

☆ 16. Bandwidth Adaptation nima?

📖 Bu tizimning internet tezligiga moslashishi. Agar internetingiz tezligi sekinlashsa, tizim videoni o'chirmaydi, balki uning sifatini (resolution) pasaytiradi, natijada aloqa uzilmaydi.

❓ 17. Frame Rate (Kadrlar chastotasi) nima?

📖 Monitorda bir soniya ichida necha marta rasm almashishi (FPS). Videokonferensiya aloqa uchun 15-30 FPS yetarli, ammo o'yinlar yoki dinamik videolar uchun 60 FPS kerak bo'ladi.

☆ 18. ICE (Interactive Connectivity Establishment) nima?

📖 Bu ikki qurilma bir-birini internetda qanday topishini hal qiluvchi mexanizm. U NAT va tarmoqlararo ekranni aylanib o'tish uchun yo'l qidiradi.

❓ 19. STUN va TURN serverlari nima uchun kerak?



- **STUN:** Qurilmaga o'zining tashqi IP-manzilini bilishga yordam beradi.
- **TURN:** Agar qurilmalar to'g'ridan-to'g'ri bog'lana olmasa, barcha ma'lumotlarni o'zi orqali o'tkazuvchi "rele" (relay) server hisoblanadi.

☆ 20. Videokonferensiya tizimlarida End-to-End Encryption (E2EE) nima va u qanday ishlaydi.

📖 Bu shifrlash usuli bo'lib, bu jarayonda faqat gaplashayotgan odamlardagina ushbu kalit bo'ladi. Hatto konferensiya o'tayotgan server (masalan, Zoom serveri) ham videoni ko'ra olmaydi.

❓ 21. Jitsi Meet nima?

📖 Bu ochiq manbali (Open Source) videokonferensiya aloqa platformasi hisiblanadi. Jitsi Meetni kompaniyalar o'zlarining lokal serverlariga o'rnatib, mutlaqo maxfiy va xavfsiz videokonferensiya aloqa tizimini sozlashlari mumkin.

☆ 22. Screen Sharing jarayoni (Ekran almashish) qanday ishlaydi?

📖 Brauzer videokamera oqimi o'rniga, kompyuter ekranidan kelayotgan piksellar oqimini (MediaStream) raqamlashtiradi va tarmoqqa uzatishni amalga oshiradi

❓ 23. Foydalanuvchi internet tezligi yuqori bo'lsa ham, video tasviri nega to'liqlanib yoki piksellarida kamchiliklar yuzaga keladi?

📖 Bu holat ko'pincha internetning umumiy tezligiga (bandwidth) emas, balki *Packet Loss* (Paket yo'qolishi) yoki *Jitter* muammosiga bog'liq bo'ladi. Videokonferensiya aloqa tizimlari UDP protokoli orqali ishlaydi. Agar tarmoqda 2-3% dan ko'p paket yo'qolsa, video kodek (masalan, H.264) rasmdagi o'zgarishlarni tiklay olmay qoladi. Natijada ekranda turli rangli kvadratchalar yoki tasvirning qotib qolishi kuzatiladi. *Yechim:* Wi-Fi o'rniga simli (Ethernet) ulanishdan foydalanish yoki routerda *QoS* (*Quality of Service*) sozlamalarini yoqib, video trafikka yuqori ustuvorlik berish kerak.

☆ 24. Packet Loss Concealment (PLC) texnologiyasi yo'qolgan paketlar o'rnini qanday (tiklaydi) to'ldiradi?

📖 Bu xatoliklarni tuzatuvchi aqlli algoritm bo'lib, u internetda yo'qolgan kichik audio yoki video bo'laklarini "bashorat qilish" orqali yaratadi. Audio holatida: Algoritm oldingi tovush to'liqlinini biroz cho'zadi yoki silliqalaydi, natijada inson qulog'i ovozda kichik uzilishni sezmaydi. Video holatida: Oldingi kadrning o'zgarmagan qismlari takroran ko'rsatiladi. Bu texnologiya bo'lmasa, har bir yo'qolgan paket audio va videoni keskin uzilib qolishiga sabab bo'lar edi.

❓ 25. Asymmetric Bandwidth (Asimmetrik tezlik) konferensiya sifatiga qanday ta'sir qiladi?

📖 Ko'pchilik foydalanuvchilar *yuklash* (*Download*) tezligiga e'tibor berishadi, ammo videokonferensiya aloqa uchun *yuborish* (*Upload*) tezligi birdek muhim. Agar sizning Upload tezligingiz past bo'lsa, siz boshqalarni yaxshi ko'rasiz (chunki Download yuqori), ammo boshqalar sizni juda sifatsiz va uzilishlar bilan ko'rishadi. **Tavsiya:** HD sifatli video yuborish uchun kamida 3-4 Mbit barqaror Upload tarmoq tezligi talab qilinadi.

☆ 26. Professional videokonferens-zallarda nega PTZ kameralar o'rnatiladi?

 **PTZ (Pan-Tilt-Zoom)** kameralar masofadan boshqariladigan mexanizmga ega bo'lib, ular chapga-o'ngga buriladi, yuqoriga-pastga egiladi va tasvirni sifatni yo'qotmasdan yaqinlashtira oladigan mexanizmga ega hisoblanadi. Bu katta videokonferens-zallarda ma'ruzachining yuzini yoki taqdimot jarayonidagi byumlarni yaqindan ko'rsatish uchun muhim sanaladi. Shuningdek, zamonaviy PTZ kameralar "*Auto-framing*" funksiyasi orqali gapirayotgan odamni avtomatik topib, kamerani unga qaratish imkoniyatiga ega.

27. **Boundary Microphone (Chegara mikrofoni) va Ceiling Microphone Array farqi nimada?**



- **Boundary Microphone:** Stol ustiga qo'yiladi. U stol yuzasidan qaytgan tovushlarni ham yig'adi va 3-5 metr radiusdagi odamlarni yaxshi eshitadi.

- **Ceiling Microphone Array:** Shiftga o'rnatiladi va u yuzlab kichik mikrofonlardan iborat. U "*Beamforming*" texnologiyasi orqali gapirayotgan odam turgan nuqtaga fokusini qaratadi va atrofidagi shovqinlarni (masalan, qog'oz shitirlashi yoki stul g'ijirlashi) filtrlaydi. Katta videokonferens-zallar uchun shiftga o'rnatilgan mikrofon eng toza ovozni ta'minlaydi.

28. **Full-Duplex audio aloqa nima va u nega Half-Duplexdan afzal?**



- **Half-Duplex audio:** Xuddi ratsiya kabi - bir vaqtning o'zida faqat bir kishi gapira oladi, ikkinchisi eshitishi kerak bo'ladi.

- **Full-Duplex audio:** Ishtirokchilarning ikkalasi ham bir vaqtda bir-birlariga gapirishi va eshitishi mumkin. Videokonferensiya aloqa tizimi Full-Duplex rejimida bo'lishi muloqotni tabiiy qilishga yordam beradi. Agar tizim Half-Duplex bo'lsa, kimdir gapirayotganda sizning "Ha, tushundim" degan so'zingiz uning ovozi uzib qo'ygan bo'lar edi.

29. **WebRTC ulanish o'rnatishda "Offer" va "Answer" (SDP almashinuvi) jarayoni qanday kechadi?**

 Muloqot boshlanishidan oldin ikki brauzer bir-birining tili (kodeklar, tarmoq manzillari)ni tushunishi kerak.

1. Birinchi brauzer "*Offer*" (Taklif) yaratadi: uning ichida u qo'llaydigan video formatlar va tarmoq yo'llari yozilgan bo'ladi.

2. Bu taklif server orqali ikkinchi brauzerga yuboriladi.

3. Ikkinchi brauzer buni qabul qilib, o‘zining imkoniyatlari bilan solishtiradi va “Answer” (Javob) qaytaradi. Faqat ushbu “muloqat” muvaffaqiyatli o‘tsagina, video oqimni almashish boshlanadi.

☆ 30. NAT va tarmoqlararo ekran orqali video signalni o‘tkazishda “STUN” va “TURN” qanday yordam beradi?

📖 Aksariyat kompyuterlar xususiy “yopiq” (Private) IP-manzilga ega.

- **STUN:** Dron uyingizni topishi uchun tashqi ko‘cha nomini aytgandek, STUN serveri qurilmaga uning tashqi Global IP-manzilini aytib beradi.

- **TURN:** Agar tarmoqlararo ekran oq ro‘yxat rejimida ishlayotgan bo‘lsa va hech qanday signal o‘tmasa, TURN serveri ma’lumotni o‘zidan “transit” sifatida o‘tkazadi. Videokonferensiya aloqa tizimlarida 90% ulanishlar STUN orqali bo‘ladi, ammo eng qiyin holatlarda TURN yordamga keladi.

❓ 31. Simulcast texnologiyasi turli internet tezligiga ega foydalanuvchilar muammosini qanday hal qiladi?

📖 Tasavvur qiling, konferensiyada 10 kishi bor. Bittasining interneti o‘ta sekin. Simulcast texnologiyasi bo‘lmaganda, server hamma uchun sifatni tushirishga majbur bo‘lar edi. Simulcast texnologiyasi bilan esa ma’ruzachining brauzeri bir vaqtning o‘zida uch xil sifatda (4K, HD, 360p) video tasvirni yuboradi. Server esa interneti tez odamga 4K video tasvirni, interneti sekin odamga esa faqat 360p video tasvirni yo‘naltiradi. Bu har bir ishtirokchi uchun individual qulaylik yaratadi.

☆ 32. Meeting Bombing (masalan, Zoom-bombing) hujumlaridan qanday himoyalaniş mumkin?

📖 Bu ruxsat etilmagan foydalanuvchining konferensiyaga kirib, xalaqit berish jarayoni hisoblanadi. Himoya choralar:

1. Har bir uchrashuv uchun individual (statik bo‘lmagan) ID ishlatish.
2. Waiting Room (Kutish xonasi) funksiyasini yoqish - moderator har bir kishini tanib, keyin ulanishga ruxsat beradi.
3. Konferensiya boshlangach, “Lock Meeting” tugmasi orqali yangi ishtirokchilar kirishini butunlay yopishi ham mumkin.

❓ 33. End-to-End Encryption (E2EE) yoqilganda qaysi funksiyalar ishlamay qoladi?

📖 E2EE o‘ta xavfsiz, ammo uning salbiy tomonlari ham bor. Ma’lumotlar faqat foydalanuvchi qurilmasida shifrlangani uchun, server ularni o‘qiy olmaydi. Natijada quyidagilar ishlamay qolishi mumkin:

- Serverda videoni yozib olish (Cloud Recording).
- Telefonda qo'ng'iroq qilib ulanish (PSTN dial-in).
- Jonli subtitrlar (Closed Captioning) va avtomatik tarjima. Chunki bu funksiyalar uchun server videoni ko'rishi va tahlil qilishi shart hisoblanadi.

❓ 34. H.264 (AVC) va H.265 (HEVC) kodeklari o'rtasidagi real amaliy farq nimada?

📖 H.265 kodeki H.264 ga qaraganda videoni 50% samaraliroq siqadi. Ya'ni, bir xil sifatdagi tasvirni uzatish uchun H.265 ikki barobar kamroq internet trafigini talab qiladi. Amaliyotda 4K formatda konferensiya o'tkazmoqchi bo'lsangiz, H.265 kodegi kerak bo'ladi. Biroq, H.265 kodegini siqish va qayta ochish uchun kompyuterdan juda katta protsessor quvvati talab qilinadi. Shu sababli, eskiroq noutbuklarda H.265 ishlatilsa, kompyuter qizib ketishi yoki video qotishi mumkin. Ko'p tizimlar (masalan, Zoom) hali ham moslashuvchanlik uchun H.264 dan foydalanadi.

☆ 35. SVC (Scalable Video Coding) texnologiyasi an'anaviy video uzatishdan nimasi bilan ustun?

📖 An'anaviy tizimlarda bitta video oqimi yuboriladi. Agar internet biroz sekinlashsa, butun kadr buziladi. SVC tizimida esa video bir nechta qatlamlarga (Base layer va Enhancement layers) bo'lingan holda uzatiladi.

- *Base Layer*: Tasvirning eng minimal, past sifati.
- *Enhancement Layers*: Tasvirga aniqlik va rang qo'shuvchi qo'shimcha qatlamlar. Agar internetingiz yomonlashsa, tizim shunchaki yuqori qatlamlarni tashlab yuboradi va sizga faqat "Base Layer"ni ko'rsatadi. Natijada aloqa uzilmaydi, faqat sifat biroz pasayadi.

❓ 36. Video tasvirdagi Jello Effect nima va u nega paydo bo'ladi?

📖 Bu asosan arzonroq veb-kameralarda yoki smartfonlarda uchraydi. Kamera datchigi (sensor) butun tasvirni bir vaqtda emas, balki qatorma-qator (Rolling Shutter) o'qiydi. Agar kamera qaltirasa yoki harakat tez bo'lsa, tasvir "gelatin" kabi qiyshayib ko'rinadi. Professional videokonferensiya aloqa kameralarida (Global Shutter) bu muammo bo'lmaydi, chunki ular butun kadrni bir millisekundda suratga oladi.

☆ 37. Dual Streaming (H.239 yoki BFCP) rejimi qanday ishlaydi?

📖 Bu professional konferensiyalarda bir vaqtning o'zida ham ma'ruzachining yuzini, ham uning taqdimotini (slaydlarini) alohida-alohida va yuqori sifatda ko'rish imkonini beradi. Tizim ikkita mustaqil video oqimini ochadi. Bu juda muhim, chunki

ma'ruzachining yuzi 15 FPS (past tezlikda) bo'lishi mumkin, lekin slaydlar ichidagi mayda yozuvlar o'ta aniq (High Resolution) bo'lishi shart.

❓ 38. “SIP Trunking” videokonferensiya tizimini tashqi dunyo bilan qanday bog'laydi?

📖 Tasavvur qiling, sizda kompaniya ichki Zoom/Teams tizimi bor. SIP Trunking orqali siz bu tizimni an'anaviy telefon liniyalariga bog'laysiz. Maliy misol sifatida Interneti yo'q bo'lgan hamkoringiz oddiy uy telefoni orqali sizning videokonferensiyangizga faqat audio formatida ulanishi mumkin bo'ladi. U maxsus raqamni teradi, SIP server esa uning analog ovozini raqamli IP paketlarga aylantirib, sizning konferensiyangizga qo'shadi.

★ 39. Gatekeeper (H.323 tarmog'ida) va SIP Proxy o'rtasidagi farq nimada?

📖 Ikkalasi ham tarmoq dispatcheri vazifasini bajaradi:

- **Gatekeeper:** H.323 tizimida ishlaydi. U qurilmalarning IP manzillarini boshqaradi, ruxsatlarni tekshiradi va tarmoq yuklamasini (bandwidth management) nazorat qiladi.

- **SIP Proxy:** SIP tizimida so'rovlarni kerakli manzilga yo'naltiradi. Professional darajadagi videokonferensiya klasterida 1000 ta terminal bo'lsa, busiz aloqa o'rnatish tartibsiz bo'lib ketadi (kim qaysi IP manzilda ekanini bilib bo'lmaydi).

❓ 40. Signaling Server WebRTC arxitekturasida nega o'zi ma'lumot uzatmaydi?

📖 WebRTC standartida signaling (Signal almashish) qanday amalga oshishi aniq belgilanmagan. Signaling server faqat tanishishtiruvchi vazifasini bajaradi. U ikki kishiga bir-birining IP-manzilini beradi xolos. Video va audio ma'lumotlar esa server orqali emas, balki to'g'ridan-to'g'ri foydalanuvchilar o'rtasida (P2P) trafik o'tadi. Bu serverga bo'lgan yuklamani kamaytiradi va maxfiylikni oshiradi.

★ 41. TURN serveridan foydalanish nega qimmatga tushadi?

📖 STUN serveri bepul bo'lishi mumkin, chunki u faqat kichik bir IP-manzilni aytadi. TURN serveri esa butun video-trafikni (gigabaytlab ma'lumotni) o'zidan o'tkazadi. Agar sizning ilovangizda 1000 kishi gaplashsa va ularning tarmoqlari kuchli himoyalangan bo'lsa (ya'ni TURN orqali o'tishga majbur bo'lishsa), sizga o'ta kuchli va qimmat server infratuzilmasi kerak bo'ladi.

❓ 42. Data Channels WebRTC-da video va audiodan tashqari nima uchun kerak?

📖 WebRTC nafaqat video va audio, balki ixtiyoriy ma'lumotlarni ham o'ta tezkorda uzata oladi. Amaliy misol sifatida videokonferensiya vaqtida fayl almashish, matnli chat, yoki ma'ruzachining sichqonchasi qayerda turganini ko'rsatuvchi koordinatalar aynan Data Channel orqali yuboriladi.

☆ 43. Differentiated Services (DiffServ/DSCP) tarmoqda videoni qanday himoya qiladi?

📖 Router orqali o'tayotgan har bir paketning yorlig'i (tag) bo'ladi. DSCP sozlamasi orqali video paketlarga "VIP" maqomi beriladi. Agar tarmoqda yuklama oshsa (masalan, kimdir katta fayl yuklay boshlasa), router birinchi bo'lib fayl paketlarini sekinlashtiradi, lekin video paketlarini navbatsiz o'tkazib yuboradi. Bu konferensiya vaqtida ovoz uzilmasligini kafolatlaydi.

❓ 44. Adaptive Bitrate (ABR) mexanizmi qanday ishlaydi?

📖 Tizim har 2-3 soniyada internet tezligini teslab ko'radi. Agar u paketlar kechikayotganini sezsa, kodekka sifatni tushir degan buyruq beradi. Sifat tushishi - kadrlar sonini kamaytirish yoki tasvir o'lchamini (masalan, 720p dan 360p ga) o'zgartirish orqali amalga oshadi. Bu aloqani saqlab qolishning eng samarali yo'li hisoblanadi.

❓ 45. Sun'iy intellekt yordamida Background Noise Suppression (Shovqinni so'ndirish) an'anaviy usullardan nimasi bilan farq qiladi?

📖 An'anaviy usullar ma'lum bir chastotadagi doimiy shovqinlarni (masalan, konditsioner g'uvullashi) filtrlaydi. Sun'iy intellekt (Deep Learning) texnologiyasi esa maxsus neyron tarmoqlar inson ovozi va shovqin (it vovillashi, klaviatura taqillashi, chaqaloq yig'isi) farqini o'rganadi. Sun'iy intellekt ovoz oqimidan faqat inson nutqiga tegishli to'lqinlarni ajratib oladi va qolganlarini 99% gacha o'chirib tashlaydi. Bu hatto qurilish maydonchasida turib ham xuddi studiyadagidek gaplashish imkonini beradi.

☆ 46. Eye Contact Correction (Nigohni to'g'rilash) texnologiyasi qanday muammoni hal qiladi?

📖 Videokonferensiya jarayonida biz ekrandagi suhbatdoshga qaraymiz, lekin kamera odatda ekranning tepasida joylashgan bo'ladi. Natijada suhbatdoshga biz pastga qarab turgandek ko'rinamiz. Sun'iy intellekt texnologiyasi yechimi esa neyron tarmoqqa real vaqtda ko'z qorachiqslari holatini o'zgartirib, ularni to'g'ridan-to'g'ri kameraga (ya'ni suhbatdoshning ko'ziga) qarab turgandek qilib vizual ravishda to'g'rilaydi. Bu muloqotda ishonch va psixologik yaqinlikni oshiradi.

❓ 47. Virtual fon texnologiyasida Chroma Key va AI Segmentationning farqi nimada?



• **Chroma Key:** Orqada bir xil rangli (odatda yashil) mato bo'lishini talab qiladi. Dastur shu rangni o'chirib, o'rniga rasm qo'yadi.

• **AI Segmentation:** Neyron tarmoq har bir kadrda inson tanasi konturini (shaklini) qidiradi va uni orqa fondan ajratadi. Bunda yashil fon shart emas, sun'iy intellekt hatto chalkash xonada ham insonni aniq qirqib oladi.

☆ 48. Cascading MCU (Kaskadli serverlar) arxitekturasini nega yirik tashkilotlar uchun muhim?

📖 Agar konferensiya bir vaqtning o'zida turli mamlakatlarda o'tayotgan bo'lsa, barcha foydalanuvchilarni bitta markaziy serverga (masalan, AQSHga) ulash katta kechikishlarga olib keladi. Kaskadlash esa har bir hududda (Toshkent, London, Nyu-York) o'zining mahalliy MCU serveri bo'ladi. Mahalliy foydalanuvchilar o'z serveriga ulanadi, serverlar esa o'zaro bitta magistral kanal orqali bog'lanadi. Bu xalqaro trafikni tejaydi va kechikishni kamaytiradi.

❓ 49. Cloud-Bursting videokonferensiya servislarida qanday ishlaydi?

📖 Tasavvur qiling, kompaniyaning o'z serveri bor va u bir vaqtda 500 kishini ko'tara oladi. Kutilmaganda 600 kishi ulanmoqchi bo'lsa, tizim qolgan 100 kishini avtomatik ravishda bulutli serverlarga (masalan, AWS yoki Azure) yo'naltiradi. Bu foydalanuvchilar uchun sezilsiz o'tadi va tizimni qotib qolishi va xizmat ko'rsatishdan bosh tortishini oldini oladi.

☆ 50. Spatial Audio (Fazoiy audio) konferensiyalarda nima uchun kerak?

📖 Oddiy konferensiyalarda hamma ovoz markazdan keladi, bu esa bir necha kishi baravar gapirganda tushunarsiz shovqin hosil qiladi. Spatial Audio texnologiyasi esa har bir ishtirokchining ovozi uning ekrandagi o'rniga qarab joylashtiriladi. Agar odam ekranning chap burchagida bo'lsa, uning ovozi sizning chap qulog'ingizga ko'proq eshitiladi. Bu inson miyasiga kim gapirayotganini tezroq ajratishga va charchoqni kamaytirishga yordam beradi.

❓ 51. Acoustic Fencing (Akustik to'siq) texnologiyasi ochiq ofislarda qanday yordam beradi?

📖 Bu professional mikrofonlar tizimi bo'lib, ular kamera ko'rib turgan ma'lum bir burchak (masalan, 90 daraja) ichidagi ovozlarni yig'adi. Agar o'sha burchakdan tashqarida (masalan, yoningizdagi stolda) kimdir baland ovozda gapirsa, mikrofon uni

umuman eshitmaydi. Bu xuddi ko‘rinmas shisha devor ichida o‘tirgandek effekt beradi.

☆ **52. Dual-Stack (SIP va H.323) qo‘llab-quvvatlaydigan terminallar nega qimmat turadi?**

📖 SIP - bu zamonaviy, veb-tizimlarga yo‘naltirilgan protocol hisoblanadi. H.323 - bu murakkab, klassik telekommunikatsiya protokoli. Ushbu ikki texnologiya (til) biri-biri bilan bevosita gaplasha olmaydi. Dual-stack qurilmalar ichida o‘ta murakkab transcoding va translatsiya tizimlari mavjud bo‘ladi, ular real vaqtda bir protokolni ikkinchisiga o‘giradi. Bu esa o‘ta kuchli apparatli resurslarni talab qiladi.

❓ **53. Keep-Alive paketlari videokonferensiya aloqa tizimlarida ulanishni qanday saqlab turadi?**

📖 Agar tarmoqda ma‘lum vaqt ma‘lumot uzatilmasa, ko‘pgina routerlar xavfsizlik uchun ulanishni yopib qo‘yishi mumkin. Keep-Alive paketlari esa tizim har 15-30 soniyada men shu yerdaman degan kichik, ko‘rinmas paketlarni yuborib turadi. Bu aloqa kanalini “qaynoq” holatda ushlab turadi va kutilmagan uzilishlarning oldini oladi.

☆ **54. Annotation (Belgilash) funksiyasi ekran almashish vaqtida qanday amalga oshiriladi?**

📖 Bu murakkab sinxronizatsiya jarayoni. Birinchi foydalanuvchi ekranda chiziq chizganda, video o‘zgarmaydi. Buning o‘rniga, chiziqning koordinatalari (X, Y nuqtalari) WebRTC Data Channel orqali barcha ishtirokchilarga yuboriladi. Ishtirokchilarning brauzeri bu koordinatalarni qabul qilib, o‘z ekranlari ustidagi shaffof qatlamda (Canvas) o‘sha chiziqni darhol chizib beradi.

❓ **55. Videokonferensiyada bobotlashgan ovoz (Metallic sound) nega paydo bo‘ladi va uni qanday tuzatish mumkin?**

📖 Bu muammo odatda **Jitter Buffer** bilan bog‘liq. Internet paketlari tartibsiz kelganda (ba‘zilari tez, ba‘zilari kechikib), audio-pleyerni boshqaruvchi tizim paketlarni kutishga majbur bo‘ladi. Agar kutish vaqti cho‘zilib ketsa, tizim tovush to‘lqinini kesib yoki cho‘zib yuboradi, bu esa inson qulog‘iga metall yoki robot ovozidek eshitiladi. Bunga amaliy misol qilib routerda Bufferbloatni kamaytirish va internet provayderdan ulanishning barqarorligini (Pingni) tekshirishni so‘rash kerak. Shuningdek, fonda ishlayotgan barcha katta internet yuklamali dasturlarni to‘xtatish shart.

☆ **56. Packet Loss (Paket yo‘qolishi) 5% dan oshsa, nima uchun faqat video qotadi, ovoz esa eshitilaveradi?**

📖 Bu videokonferensiya aloqa tizimlarining ustuvorlik algoritmi bilan bog'liq. Audio ma'lumotlar hajmi juda kichik (masalan, 32-64 kbps), video esa juda katta (3-4 Mbps). Tizim birinchi navbatda audio paketlarini himoya qiladi. Agar tarmoq yomonlashsa, tizim video o'chsa ham, muloqot to'xtamasin degan tamoyil asosida videoni to'xtatadi (chunki yo'qolgan video paketlarini tiklash qiyin), ammo audio uchun yetarli bo'lgan tezligi past tarqmoqdan foydalanishda davom etadi.

❓ 57. Wi-Fi orqali videokonferensiya tashkil qilganda mikroto'lqinli xalaqit qanday ta'sir qiladi?

📖 Bu real hayotda ko'p uchraydi. Eski 2.4 GHz Wi-Fi chastotasi va oddiy oshxonadagi mikroto'lqinli pechlar bitta to'lqin uzunligida ishlaydi. Agar siz routerga yaqin joyda oshxonadagi pechni yoqsangiz, u Wi-Fi signalini butunlay bosib qo'yishi mumkin. Natijada sizda kutilmagan uzilish sodir bo'ladi. Professional videokonferensiya aloqa tizimlari uchun doimo 5 GHz Wi-Fi yoki eng yaxshisi Ethernet kabelga asoslangan ulanishidan foydalanish tavsiya etiladi.

★ 58. Automatic Gain Control (AGC) funksiyasi konferensiyada qatnashchilarni nega asabiylashtirishi mumkin?

📖 Automatic Gain Control mikrofonga kelayotgan ovoz balandligini avtomatik tenglashtirishga harakat qiladi. Agar xonada sukunat bo'lsa, AGC mikrofoni sezgiriligini maksimalga ko'tarib yuboradi. Natijada uzoqdagi mashina ovozi, konditsioner g'uvullashi yoki kimdir qog'oz shitirlatishi juda baland eshitiladi. Professional studiya sharoitida Automatic Gain Controlni o'chirib, ovoz balandligini (Gain) qo'lda bir marta ideal darajaga sozlash maqsadga muvofiq hisoblanadi.

❓ 59. Oq rang balansi noto'g'ri bo'lsa, tasvir qanday o'zgaradi?

📖 Agar xonadagi chiroqlar sariq bo'lsa, kamera esa ko'k nurga sozlangan bo'lsa, foydalanuvchining yuzi g'ayritabiiy ko'rinadi. Ko'p veb-kameralar buni avtomatik bajaradi, ammo yorug'lik o'zgarganda (masalan, derazadan quyosh tushganda) kamera adashib qolishi mumkin. Shuning uchun professional kameralarda oq rang balansini qo'lda bitta standartga qotirib qo'yish tasvir barqarorligini ta'minlaydi.

★ 60. Symmetric NAT WebRTC uchun nega eng katta to'siq hisoblanadi?

📖 Bu o'ta kuchli himoyalangan korporativ tarmoqlarda uchraydi. Oddiy NATda siz bir marta tashqi portni bilsangiz, u o'zgarmaydi. Symmetric NAT esa har bir yangi ulanish uchun yangi port ochadi. Natijada STUN serveri bergan manzil ishlamay qoladi. Bunday holatda WebRTC ulanishining yagona yo'li - TURN serveridan foydalanish kerak bo'ladi. Bu tarmoq ma'muri uchun qo'shimcha xarajat va server yuklamasini bildiradi.

❓ 61. DTLS-SRTP shifrlashi WebRTCda qanday xavfsizlikni kafolatlaydi?

📖 WebRTCda video va audio ochiq holda uzatilmaydi.

- DTLS: Ulanish oʻrnatilayotgan vaqtda ikki foydalanuvchi oʻrtasida shifrlash kalitlarini almashishni nazorat qiladi.
- SRTP (Secure Real-time Transport Protocol): Maʼlumotning oʻzini (video piksellari va audio bitlarini) shifrlab uzatadi. Bu jarayon bitta paket darajasida amalga oshiriladi, yaʼni xaker paketni tutib olsa ham, uning ichidan tasvirni tiklay olmaydi.

☆ 62. Virtual Breakout Rooms (Kichik guruhlar boʻlinish) texnik jihatdan qanday amalga oshiriladi?

📖 Bu server darajasidagi mantiqiy boʻlinish hisoblanadi. SFU serveri asosiy konferensiya identifikatorini (ID) vaqtinchalik bir nechta sub-identifikatorlarga ajratadi. Foydalanuvchi asosiy xonadan chiqib, kichik xonaga oʻtganda, uning brauzeri yangi oqimni (stream) qabul qila boshlaydi, ammo avvalgi ulanish uzilmaydi (yashirin qoladi). Bu moderatorga barcha xonalarni bir vaqtda kuzatish va xabarlarni hamma uchun birdaniga yuborish imkonini beradi.

❓ 63. Videokonferensiyada Latency vs Quality (Kechikish va Sifat) balansini qanday saqlanadi?

📖 Bu videokonferensiya jarayonida doimiy kechadigan kurash hisoblanadi. Sifatni oshirish uchun kodek koʻproq maʼlumotni siqishi kerak, bu esa vaqt oladi. Agar videokonferensiya interaktiv boʻlsa (chat, savol-javob), tizim kechikishni kamaytirish uchun sifatni biroz tushiradi. Agar tizim shunchaki vebinar (bir tomonlama translyatsiya) boʻlsa, u sifatni maksimal qiladi va videoni 5-10 soniya kechikish (Buffer ushlab turish) bilan koʻrsatadi, chunki bu holatda foydalanuvchi uchun real vaqtda javob qaytarish muhim emas.

☆ 64. Professional videokonferensiya aloqa terminallari (Cisco, Poly) nega oddiy kuchli kompyuterdan yaxshiroq?

📖 Ularning ichida maxsus DSP (Digital Signal Processor) chiplari mavjud boʻladi. Oddiy kompyuter protsessori (CPU) koʻp vazifali boʻlsa, DSP chiplari faqat videoni siqish va audioni qayta ishlash uchun sozlangan. Natijada apparatli terminal kompyuterga qaraganda videoni tezroq siqadi, aks-sadoni yaxshiroq soʻndiradi va videokonferensiya arayoni tasvirlar yuqotishlarsiz barqaror ishlay oladi (kompyuter kabi qotib qolmaydi).

❓ 65. Lip Sync (Ovoz va lab harakati mos kelmasligi) muammosi nima uchun paydo boʻladi?

📖 Video va audio ma'lumotlar tarmoq orqali alohida oqimlarda uzatiladi. Videoni siqish va ochish (decoding) audionikiga qaraganda ko'proq vaqt va protsessor quvvatini talab qiladi. Natijada ovoz tasvirdan o'zib ketishi mumkin. RTP protokoli ichidagi RTCP (RTP Control Protocol) orqali har bir paketga vaqt belgisi (timestamp) qo'yiladi. Qabul qiluvchi qurilma ushbu belgilar yordamida videoni kutib turadi va ovozni tasvir bilan sinxiron namoyish etadi. Agar noutbukungiz o'ta kuchsiz bo'lsa, u videoni qayta ishlashga ulgurmaydi va Lip Sync buziladi.

★ 66. Soatlar tafovuti (Clock Drift) uzoq davom etgan konferensiyalarga qanday ta'sir qiladi?

📖 Har bir qurilmaning o'z ichki kristalli generatori (soati) bor va ular millisekund darajasida farq qilishi mumkin. Agar konferensiya 3-4 soat davom etsa, jo'natuvchi va qabul qiluvchi soatlari o'rtasidagi farq yig'ilib, ovozning uzilishiga yoki tasvirning sekinlashishiga olib keladi. Zamonaviy videokonferensiya aloqa tizimlari vaqtinchalik xotira (buffer) hajmini dinamik o'zgartirish orqali ushbu tafovutni real vaqtda kompensatsiya qilib boradi.

❓ 67. 5G tarmog'i videokonferensiyalarda qanday inqilobni yuzaga keltiradi?

📖 5G nafaqat tezlik, balki o'ta past kechikish (Ultra-Low Latency) va ko'p sonli qurilmalarni bir vaqtda ulash imkonini beradi. 5G yordamida mobil qurilmalarda 4K va hatto 8K formatdagi videolarni hech qanday kechikishlarsiz uzatish mumkin bo'ladi. Bu ayniqsa masofaviy jarrohlik amaliyoti yoki dronlarni video orqali real vaqtda boshqarishda hal qiluvchi rol o'ynaydi.

★ 68. Golografik videokonferensiya oddiy videodan nimasi bilan farq qiladi?

📖 Golografik aloqada suhbatdoshning 2D tasviri emas, balki uning 3D hajmdagi nusxasi (avatar yoki gologrammasi) uzatiladi. Buning uchun foydalanuvchi atrofida o'nlab kameralar (Volumetric Capture) o'rnatiladi. Qabul qiluvchi esa maxsus ko'zoynaklar (AR/VR) yoki lazerli gologramma proyektorlari orqali suhbatdoshni o'z xonasida turgandek his qiladi.

❓ 69. WebRTC Leak (IP-manzil sizib chiqishi) nima va u nega xavfli?

📖 WebRTC ulanishni o'rnatish uchun qurilmaning barcha IP-manzillarini (shu jumladan VPN orqasidagi haqiqiy IP-ni ham) STUN serveriga yuboradi. Agar xavfsizlik choralari ko'rilmasa, veb-sayt foydalanuvchining yashirin joylashuvini bilib olishi mumkin. Brauzer sozlamalarida "mDNS" texnologiyasini yoqish kerak, shunda WebRTC haqiqiy IP o'rniga shifrlangan identifikatorni uzatadi.

☆ 70. Nega Jitsi Meet kabi tizimlar Docker konteynerlarida o'rnatilishi tavsiya etiladi?

📖 Videokonferensiya aloqa tizimi faqat bitta dastur emas, u bir nechta komponentlardan (Videobridge, Jicofo, Prosody) iborat. Docker har bir komponentni alohida muhitda saqlaydi. Agar bitta komponent (masalan, chat servisi) buzilsa, u butun video aloqani to'xtatib qo'ymaydi. Shuningdek, Docker orqali tizimni bir nechta serverlarga masshtablashtirish osonroq.

? 71 Ekranni almashish (Screen Sharing) vaqtida matnlar nega xira ko'rinadi?

📖 Bu video kodekning (Harakat) ustuvorligiga sozlanganidan bo'ladi. Videoda harakat muhim, matnda esa aniqlik. Dastur sozlamalaridan "Optimize for Text" yoki "Optimize for Static Images" rejimini tanlash kerak. Bunda tizim kadrlar sonini (FPS) 5 tagacha tushiradi, lekin har bir kadrning aniqligini maksimal darajaga ko'taradi.

☆ 72. Majlislar zalida Acoustic Feedback (G'uvullash) qanday to'xtatiladi?

📖 Bu mikrofon va karnay bir-biriga yaqin turganda ovoz aylanishidan paydo bo'ladi. Professional videokonferensiya aloqa tizimlarida AEC (Acoustic Echo Cancellation) bloklari bo'ladi. Ular karnayga borayotgan raqamli signalni mikrofondan kelayotgan signal bilan solishtiradi va bir xil to'lqinlarni olib tashlaydi.

? 73. Nega videokonferensiya yozuvlari ba'zan o'zi to'g'ridan-to'g'ri emas, balki serverda yozib olinadi?

📖 Mahalliy yozib olish (Local Recording) sizning kompyuteringiz resurslarini (CPU va Disk) ishlatadi. Agar kompyuteringiz kuchsiz bo'lsa, yozuv sifati yomon bo'ladi. Bulutli yozib olish serverga kelayotgan barcha oqimlarni serverning o'zida birlashtiradi. Bu foydalanuvchiga hech qanday og'irlikni yuklamaydi va video yozuvni istalgan qurilmadan havola orqali ko'rish imkonini beradi. Shuningdek, server video yozuv vaqtida ishtirokchilarning ismlarini va chat xabarlarini ham metadata sifatida yozib keta oladi.

☆ 74. Video yozuvlar uchun MP4 formati MKV yoki AVI dan ko'ra ko'proq ishlatiladi?

📖 MP4 (H.264) - bu universal konteyner hisoblanadi. U barcha brauzerlar, smartfonlar va Smart TV-lar tomonidan qo'shimcha kodeklarsiz oq'llab-quvvatlanadi. MKV ko'proq funksiyalarga ega bo'lsada (ko'p tilli audio yo'laklar), uni veb-brauzerda to'g'ridan-to'g'ri ko'rsatish qiyinroq kechadi. Videokonferensiya aloqa

tizimlari uchun ochilish tezligi va moslashuvchanlik hamma narsadan ustun hisoblanadi.

? 75. Shovqin darvozasi (Gating) texnologiyasi audio sifatini qanday yaxshilaydi?

📖 Bu texnologiya mikrofonni faqat ma'lum bir balandlikdagi ovoz (inson nutqi) kelgandagina ochadi. Agar siz gapirmayotgan bo'lsangiz va xonada faqat past darajadagi fon shovqini bo'lsa, mikrofon o'zini-o'zi Mute (o'chirib qo'yadi) qiladi. Bu konferensiyada o'nlab odamlar bo'lganda, ularning uyidagi fon shovqlari birikib, umumiy shovqin hosil qilmasligi uchun juda muhim.

☆ 76. O'z ovozini eshitish (Side-tone) funksiyasi quloqchilarda nega kerak?

📖 Yopiq turdagi professional quloqchilarda odam o'z ovozini eshitmaydi va beixtiyor baqirib gapira boshlaydi. Side-tone funksiyasi sizning ovozingizni mikrofondan olib, juda past darajada quloqchiningizga qaytaradi. Bu insonning o'z ovoz balandligini tabiiy nazorat qilishiga yordam beradi va uzoq muloqotda charchoqni kamaytiradi.

? 77. Traceroute buyrug'i videokonferensiyadagi uzilishlarni topishda qanday foydalaniladi?

📖 Agar aloqa yomon bo'lsa, muammo sizda yoki serverda ekanini bilish kerak. **Traceroute** sizdan servergacha bo'lgan barcha sakrashlarni (hops) ko'rsatadi. Agar 3-4 chi sakrashda kechikish (ms) keskin ortsa, demak muammo sizning provayderingizning magistral kanallarida. Agar faqat oxirgi nuqtada kechikish bo'lsa, demak server yuklamani ko'tara olmayapti.

☆ 78. Videokonferensiya aloqa tizimlarida Port Triggering va Static Port Mappingning farqi nimada?



- **Static Mapping:** Routerda ma'lum bir portni (masalan, UDP 5060) doimiy bitta kompyuterga bog'lab qo'yish.

- **Port Triggering:** Dinamikroq usul bo'lib, u portni faqat dastur ishga tushganda va so'rov yuborgandagina ochadi. Bu xavfsizlik nuqtai nazaridan yaxshiroq, chunki portlar doimiy ochiq turmaydi.

? 79. Teletibbiyot uchun videokonferensiya aloqa tizimlarida qaysi ko'rsatkich eng muhim?

📖 Bu yerda rang aniqligi (**Color Accuracy**) va tasvirning yuqori ruxsati (**High Resolution**) birinchi o'rinda turadi. Masalan, shifokor bemorning terisidagi rang o'zgarishini ko'rishi uchun video kodek ranglarni buzmasdan (4:4:4 chroma subsampling) uzatishi shart. Oddiy konferensiya tizimlari trafikni tejash uchun ranglarni siqib yuboradi, bu esa tibbiy diagnostika uchun yaroqsiz bo'lishi mumkin.

★ **80. Masofaviy ta'limda LMS Integration (Moodle, Canvas) qanday ishlaydi?**

📖 Videokonferensiya aloqa tizimi (masalan, BigBlueButton yoki Zoom) LTI (Learning Tools Interoperability) protokoli orqali dars jadvali bilan bog'lanadi. Talaba o'quv portaliga kirishi bilan "Darsga kirish" tugmasini bosadi va tizim uni avtomatik taniydi (SSO - Single Sign On), uning ism-familiyasi va roli (talaba yoki o'qituvchi) avtomatik o'tadi.

❓ **81. Apparatli tezlatish (Hardware Acceleration) brauzer sozlamalarida nega yoqilgan bo'lishi kerak?**

📖 Agar bu funksiya o'chiq bo'lsa, videoni ochish (decoding) vazifasi markaziy protsessorga (CPU) yuklama sifatida tushadi. Agar u yoqilgan bo'lsa, bu ishni videokarta (GPU) bajaradi. GPU videoni qayta ishlash uchun maxsus yaratilgani sababli, u buni tezroq va kamroq energiya sarflab bajaradi. Natijada noutbukungiz qizimaydi va video qotmaydi.

★ **82. Fixed Focus va Auto Focus veb-kameralari o'rtasidagi qanday farq mavjud?**



- **Fixed Focus:** Fokus masofasi o'zgarmas (odatda 50 sm dan cheksizlikkacha). Bu arzonroq va doimo fokusda turadi, ammo juda yaqin kelgan narsani xira ko'rsatadi.

- **Auto Focus:** Obyektivni harakatlantirib, fokusni sizning yuzingizga to'g'rilaydi. Agar siz doim qimirlab tursangiz yoki qo'lingizda biror narsani yaqindan ko'rsatmoqchi bo'lsangiz, bu juda qulay.

❓ **83. H.460 protokoli professional videokonferensiya aloqa qurilmalari uchun nima beradi?**

📖 Professional terminallar (Poly, Cisco) korporativ tarmoqlararo ekranlar ortida turadi. H.460 - bu maxsus standart bo'lib, u H.323 paketlarini tarmoqlararo ekran orqali hech qanday portlarni qo'lda ochmasdan "o'tishi" imkonini beradi.

★ **84. Whitelisting IP xavfsizlik uchun qanchalik muhim?**

📖 Agar kompaniya ichida maxfiy konferensiya o'tkazilsa, tizimni faqat ma'lum bir IP-manzillar (masalan, faqat ofis IP-lari) orqali kirishga sozlash mumkin. Bu dunyoning boshqa nuqtasidan kimdir havola va parolni o'g'irlasa ham, konferensiyaga kira olmasligini kafolatlaydi.

❓ 85. SIP-TLS nima uchun kerak?

📖 SIP protokoli standart holatda barcha buyruqlarni (kim kimga qo'ng'iroq qilyapti, telefon raqamlar) ochiq matn ko'rinishida yuboradi. SIP-TLS bu ma'lumotlarni shifrlaydi, natijada xaker qo'ng'iroqlarni boshqarish buyruqlarini tutib ololmaydi.

★ 86. Nega ba'zan videokonferensiya jarayonida ekran almashish (screen share) ishlaydi, ammo o'zining kamerasi ishlamaydi?

📖 Buning sababi - brauzer yoki operatsion tizim darajasidagi ruxsatlar hisoblanadi. Ba'zan antivirus yoki "Xususiy" sozlamalar kameradan foydalanishni taqiqlab qo'yadi, ammo ekranni yozib olishga (screen capture) ruxsat beradi. Yana bir sabab - kamerani boshqa bir dastur (masalan, Skype) band qilib turgan bo'lishi mumkin.

❓ 87. Ovozda aks-sado (Echo) paydo bo'lsa, birinchi bo'lib kimni Mute (audiosini o'chirib) qilish kerak?

📖 Birinchi bo'lib o'z aks-sadongizni eshitsangiz, muammo sizda emas, balki suhbatdoshingizda. Uning karnayidan chiqqan ovoz uning mikrofoniga qayta kiryapti. Shuning uchun aks-sado eshtilishi bilan hamma ishtirokchilarni navbat bilan o'chirib ko'rish orqali aybdorni topish mumkin.

★ 88. Sun'iy intellekt konspekti (AI Meeting Summary) qanday ishlaydi?

📖 Tizim avval audioni matnga o'giradi (Speech-to-Text). So'ngra LLM (Large Language Model) ushbu matnni tahlil qilib, uchrashuvning eng muhim qismlarini ajratadi, kim qanday vazifa olganini belgilaydi va qisqacha hisobot tayyorlaydi.

❓ 89. Subsecond Latency nima va u kimga kerak?

📖 Bu video oqimni 1 soniyadan kam bo'lgan kechikish hisoblanadi. Bu asosan kimoshdi savdolari (Online Auction) yoki real vaqtdagi sport translyatsiyalari uchun kerak, bu yerda 1 soniyalik farq katta pul yoki natijani hal qilishi mumkin.

★ 90. Avatar-based Calling nima?

📖 Agar siz uyda videokonferensiyada qatnashish uchun kiyinib tayyor bo'lmagan bo'lsangiz (kiyinmagan yoki sochlar tartibsiz), sun'iy intellekt sizning yuz

harakatlaringizni kuzatib, ekranda sizning 3D avataringizni ko'rsatadi. Bu Applening Memoji yoki Metaning Avatars tizimiga o'xshaydi.

? 91. Dual-homed server videokonferensiya aloqa tizimidan nega foydalaniladi?

 Bunday server ikkita tarmoq kartasiga ega: biri ichki tarmoq (LAN), ikkinchisi tashqi internet (WAN) uchun. Bu tashqi foydalanuvchilarni ichki tarmoqqa to'g'ridan-to'g'ri kiritmasdan, xavfsiz aloqa o'rnatish imkonini beradi.

☆ 92. Multipoint Layouts (Gallery view vs Speaker view) serverga qanday yuklama beradi?



- Speaker view: Server faqat bitta katta oqimni yuboradi.
- Gallery view: SFU serveri foydalanuvchiga 25-49 ta kichik oqimlarni baravar yuboradi. Bu foydalanuvchi noutbukining CPU va tezkor xotirasini keskin yuklanishiga olib keladi.

? 93. Noise Floor tushunchasi mikrofon tanlashda nimani anglatadi?

 Bu mikrofonning o'zidan chiqadigan shovqin darajasini bildiradi. Sifatli mikrofonlarda bu raqam juda past bo'ladi, natijada jimlik vaqtida siz hech qanday "ts-ts-ts, shirt-shitr" kabi fon shovqinlarini eshitmaysiz.

☆ 94. Jitter Buffer hajmini oshirishning salbiy tomoni bormi?

 Jitter bufferni qanchalik katta qilsangiz, audio shunchalik barqaror bo'ladi, ammo kechikish ortadi. Shuning uchun zamonaviy tizimlar tarmoq holatiga qarab bu buffer hajmini avtomatik o'zgartirib borad..

? 95. Forward Error Correction (FEC) paketlar yo'qolishini qanday bartaraf etadi?

 Tizim har bir paket bilan birga qo'shimcha tiklovchi ma'lumotlar kodlarini ham qo'shib yuboradi. Agar yo'lda 1 yoki 2 ta paket yo'qolishi yuzaga kelsa, qabul qiluvchi qurilma o'sha qo'shimcha ma'lumotlar yordamida yo'qolgan paketni matematik hisob kitoblar orqali qayta tiklaydi.

☆ 96. Packet Pacing nima?

 Bu paketlarni tarmoqqa bir vaqtda emas, balki ma'lum bir vaqt oralig'ida tekis tartibda yuborish hisoblanadi. Bu routerlarda tiqilib qolish hosil bo'lishini oldini oladi.

? 97. VBR (Variable Bitrate) va CBR (Constant Bitrate) farqi nimadan iborat?



- **VBR:** Tasvir harakatiga qarab tezlikni o'zgartiradi (harakat yo'q joyda tejaydi).
- **CBR:** Doimo bir xil tezlikda yuboradi. Videokonferensiya aloqa uchun VBR yaxshiroq, chunki u internetni tejaydi.

☆ **98. TURN-over-TCP nima uchun oxirgi chora hisoblanadi?**

📖 Agar tarmoqda UDP portlari butunlay yopiq bo'lsa, videokonferensiya aloqa tizimi videoni sekinroq bo'lsada, TCP protokoli orqali yuborishga harakat qiladi. Bu ulanish ishonchliligini kafolatlaydi, ammo paketlar kechikishni oshiradi.

? **99. SD-WAN texnologiyasi korporativ videokonferensiya aloqa sifatini qanday yaxshilaydi?**

📖 U ikkita internet provayderni birlashtiradi. Agar bitta liniyada paketlar yo'qola boshlasa, u real vaqtda video oqimini ikkinchi provayder liniyasiga o'tkazib yuboradi.

☆ **100. Videokonferensiya aloqa mutaxassisi bo'lish uchun eng muhim ko'nikma nima?**

📖 Tarmoq, audio-vizual texnologiyalar va protokollarni (SIP/WebRTC) birdek tushunish. Ushbu uchlikning kesishmasida eng murakkab muammolar hal qilinadi.

10. Kompyuter grafikasi va videomontaj

Atamalar va izohlar

Atamalar	Izohlar
Pixel	Rastir tasvirning eng kichik bo'lagi, rangli kvadrat.
Vector	Matematik chiziqlar va nuqtalardan iborat, sifati buzilmaydigan grafika.
Raster	Piksellardan tashkil topgan rasm (fotosuratlar).
Rendering	Kompyuter tomonidan 3D modelni yoki effektlarni tayyor tasvirga aylantirish jarayoni.
Resolution	Tasvirning bo'yi va enidagi piksellar soni (masalan, 1920x1080).
Frame Rate (FPS)	Bir soniyada ko'rsatiladigan kadrlar soni.
Keyframe	Animatsiyadagi harakatning boshlanish va tugash nuqtasi (tayanch kadr).
Timeline	Videomontaj dasturidagi barcha kadrlar va audio joylashgan vaqt chizig'i.
Codec	Video ma'lumotlarni siquvchi va qayta ochuvchi algoritm.
Color Grading	Videoga badiiy rang berish va kayfiyat yaratish jarayoni.
Color Correction	Ranglar va yorug'likdagi texnik xatolarni to'g'rilash.
LUT	Ranglarni o'zgartirish uchun tayyor matematik filtr (Look-Up Table).
Masking	Tasvirning ma'lum bir qismini kesib olish yoki berkitish.
Tracking	Videodagi ob'ektning harakatini kompyuter tomonidan kuzatish.
Compositing	Bir nechta turli vizual elementlarni bitta kadrda birlashtirish.
Chroma Key	Yashil yoki ko'k fonni o'chirib tashlash texnologiyasi.
Motion Blur	Tez harakatlanayotgan ob'ektning kinematografik xiralashuvi.
Aspect Ratio	Ekran o'lchami nisbati (masalan, 16:9 yoki 9:16).
Bitrate	Bir soniyada uzatiladigan video ma'lumot hajmi (Mbps).
Opacity	Ob'ektning shaffoflik darajasi.
Lower Third	Ekranning pastki qismida chiqadigan yozuv (ism, lavozim).
B-Roll	Asosiy voqeani to'ldiruvchi yordamchi kadrlar.
Transition	Ikki kadr o'rtasidagi o'tish effekti.
Proxy	Montajni tezlashtirish uchun ishlatiladigan videoning yengil nusxasi.
VFX	Kompyuterda yaratilgan vizual effektlar (Visual Effects).
3D Mesh	3D modelning nuqta va chiziqlardan iborat "panjarasi".
Texture	3D model ustiga yopishiriladigan rasm (material).
UV Mapping	3D modelni 2D tekislikka yoyish jarayoni.
Rigging	3D model ichiga virtual "suyaklar" o'rnatish.
Polygons	3D modelni tashkil etuvchi ko'pburchaklar.
Global Illumination	3D-da nurlarning devorlardan qaytishini hisoblaydigan yoritish.
Ray Tracing	Yorug'lik nurlarini fizik aniqlikda hisoblaydigan renderlash.
Exposure	Tasvirning yorug'lik darajasi (ekspozitsiya).
Saturation	Ranglarning to'yinganlik darajasi.
Contrast	Tasvirdagi eng yorug' va eng qorong'u joylar o'rtasidagi farq.
Vignette	Kadrning chetlarini qorong'ulashtirish effekti.
Grain / Noise	Tasvirdagi raqamli "shovqin" yoki kinosimon donadorlik.
Luma	Tasvirning faqat yorug'lik (oq-qora) qismi.
Chroma	Tasvirning rang qismi.
Waveform	Yorug'likni o'lchovchi diagnostika grafigi.
Vectorscope	Rang tusini va to'yinganligini o'lchovchi asbob.
Keying	Rang bo'yicha qirqib olish (masalan, yashil fonni).
Adjustment Layer	O'zidan pastdagi barcha qatlamlarga effekt beruvchi shaffof qatlam.
Null Object	Ko'rinmas, lekin boshqa elementlarni boshqaruvchi "dasta".

Parenting	Bir qatlamni ikkinchisiga bog'lab qo'yish (ergashtirish).
Parallax	Turli masofadagi ob'ektlarning turlicha tezlikda siljishi.
Raw Video	Kamera sensoridan olingan, ishlov berilmagan xom video.
MoGraph	Motion Graphics so'zining qisqartmasi (harakatlanuvchi grafika).
Topology	3D model yuzasidagi poligonlarning tuzilishi.
Post-production	Suratga olishdan keyingi barcha jarayonlar (montaj, rang, ovoz).

Savol-javoblar

? 1. Rastrli va Vektorli grafikaning real hayotdagi eng katta farqi nimada?

 **Rastrli grafika (Photoshop, JPEG):** Bu piksellar (kichik rangli kvadratlar) to'plamidir. Agar rasmni juda yaqinlashtirsangiz, u "piksellashib" ketadi va sifati buziladi. U murakkab detallarga ega fotosuratlar uchun ideal.

Vektorli grafika (Illustrator, SVG): Bu matematik formulalar (nuqtalar va chiziqlar) asosida quriladi. Siz uni xohlagancha kattalashtirishingiz mumkin, sifati zarracha ham o'zgarmaydi. U logotiplar va ikonkalarni yaratish uchun ishlatiladi.

? 2. "Bit Depth" (Rang chuqurligi) videomontajda nega hal qiluvchi rol o'ynaydi?

 Bu har bir piksel necha xil rangni ifodalay olishini bildiradi.

- **8-bit:** Jami 16.7 million rang (standart).
- **10-bit:** 1 milliarddan ortiq rang. **Amaliyotda:** Agar siz 8-bitli videoda osmonning rangini o'zgartirmoqchi bo'lsangiz, ranglar o'rtasida chiziqlar (banding) paydo bo'ladi. Professional rang berish (Color Grading) uchun doimo 10-bit yoki 12-bitli xomashyo (raw footage) kerak.

? 3. "Resolution" (Aniqilik) va "Aspect Ratio" (Nisbat) tushunchalari qanday bog'langan?

 **Resolution** — bu ekrandagi piksellar soni (masalan, 1920x1080). **Aspect Ratio** — bu ekran kengligining balandligiga nisbati. **Real ssenariy:** YouTube uchun standart 16:9 (keng ekran), Instagram Reels yoki TikTok uchun esa 9:16 (vertikal) nisbat ishlatiladi. Agar siz 16:9 videoni 9:16 ga qo'ysangiz, tasvirning chetlari kesilib ketadi yoki qora joylar qoladi.

? 4. "Proxy Video" montaj vaqtida nega ishlatiladi?

 Agar kompyuteringiz 4K videoni tahrirlashda "qotib" qolsa, Premiere Pro ushbu videoning o'ta kichik va yengil nusxasini (**Proxy**) yaratadi. Siz montajni shu yengil faylda qilasiz (kompyuter uchib ishlaydi), render qilish vaqtida esa dastur avtomatik ravishda barcha effektlarni asl 4K faylga qo'llaydi. Bu o'rta darajadagi kompyuterda ham professional montaj qilish imkonini beradi.

❓ 5. "Frame Rate" (FPS) tanlashda "24fps" va "60fps" o'rtasidagi estetik farq nimada?



- **24 FPS:** Kinematografik standart. Inson ko'zi harakatni biroz xira (motion blur) ko'rishga o'rgangan, bu kino ko'rish hissini beradi.
- **60 FPS:** O'ta silliq harakat. Bu asosan sport translyatsiyalari, o'yinlar yoki videoni keyinchalik "Slow-motion" (sekinlashtirish) qilish uchun ishlatiladi. Agar 24fps videoni sekinlashtirsangiz, u "sakrab-sakrab" qoladi.

❓ 6. "Color Correction" va "Color Grading" farqi nimada?



- **Color Correction:** Bu texnik bosqich. Tasvirdagi oq rang balansini to'g'rilash, yorug'likni me'yoriga keltirish va hamma kadrlarni bir xil ko'rinishga keltirish.
- **Color Grading:** Bu badiiy bosqich. Videoga ma'lum bir kayfiyat berish (masalan, qo'rqinchli kino uchun ko'k/sovuq tus, baxtli xotiralar uchun issiq/sariq tus berish). Buning uchun ko'pincha **LUT (Look-Up Tables)** ishlatiladi.

❓ 7. "Keyframes" (Tayanch kadrlar) animatsiya yaratishda qanday ishlaydi?

Siz dasturga "Vaqtning 1-soniyasida bu rasm chapda bo'lsin" (1-keyframe) va "5-soniyasida o'ngda bo'lsin" (2-keyframe) degan buyruqni berasiz. Dastur esa bu ikki nuqta orasidagi barcha harakatni matematik hisoblab, silliq siljishni o'zi yaratib beradi.

❓ 8. "Masking" va "Rotoscoping" nima uchun kerak?

Masking — tasvirning ma'lum bir qismini kesib olish yoki berkitish (masalan, ekrandagi mashina raqamini yopish). **Rotoscoping** — bu o'ta murakkab jarayon bo'lib, harakatlanayotgan odamni orqa fondan qatorma-qator (kadrbay-kadr) qirqib olish. Bu yashil fonsiz (green screen) odamni boshqa joyga ko'chirish uchun kerak.

❓ 9. "Compositing" jarayoni After Effects-da qanday kechadi?

Bu bir nechta turli manbalardan olingan vizual elementlarni (masalan, kamerada olingan odam, 3D yaratilgan ajdarho va kompyuterda chizilgan yomg'ir) bitta realistik tasvirga birlashtirish san'atidir. Bunda soyalar, yorug'lik va ranglar hamma elementda bir xil bo'lishi shart.

❓ 10. "Polygonal Modeling" va "Sculpting" farqi nimada?



- **Polygonal:** Nuqtalar, chiziqlar va yuzalar (polygon) bilan ishlash. Bu asosan bino, mashina kabi aniq geometrik shakllar uchun qulay.
- **Sculpting:** Xuddi loydan haykal yasagandek modellashtirish. Bu inson yuzi, hayvonlar va murakkab organik shakllar yaratishda ishlatiladi.

? 11. "Rendering" jarayonida CPU va GPU renderning farqi nimada?



- **CPU Render (V-Ray, Arnold):** O'ta aniq va realistik tasvir beradi, lekin juda uzoq vaqt oladi (soatlab/kunlab). Murakkab fizik hisob-kitoblar uchun mos.
- **GPU Render (Redshift, Octane, Cycles):** Videokarta yordamida ishlaydi. U CPU-dan 10-50 barobar tezroq, lekin o'ta murakkab sahnalarda xotira (VRAM) yetishmasligi mumkin. Zamonaviy animatsiyalarda asosan GPU render ishlatiladi.

? 12. "Texture Mapping" (Tekstura berish) modellashtirishda qanday rol o'ynaydi?

 3D model — bu shunchaki kulrang shakl. Unga "hayot" berish uchun ustiga tekstura yopishiriladi.

- **Diffuse Map:** Rang beradi.
- **Normal Map:** Yuzada g'adir-budirlik (masalan, g'ishtning yoriqlari) effekti beradi.
- **Roughness Map:** Qayeri yaltirashi va qayeri xira bo'lishini belgilaydi.

? 13. "Lossy" va "Lossless" siqish turlarining farqi?



- **Lossy (H.264, MP4):** Fayl hajmini kichraytirish uchun inson ko'zi sezmaydigan ba'zi ma'lumotlarni o'chirib yuboradi. Arxivlash va internet uchun ideal.
- **Lossless (ProRes 4444, Animation):** Hech qanday ma'lumot yo'qolmaydi, sifat 100% saqlanadi, lekin fayl hajmi o'ta ulkan bo'ladi. Bu faqat montaj jarayonining o'rta bosqichlarida ishlatiladi.

? 14. "Bitrate" (VBR vs CBR) video eksportida sifatga qanday ta'sir qiladi?



- **CBR (Constant):** Butun video davomida bir xil tezlikda ma'lumot uzatadi.

- **VBR (Variable):** Aqlli usul. Harakat ko'p joyda (masalan, portlashda) bitrate-ni oshiradi, harakat kam joyda (osmon) pasaytiradi. Bu sifatni saqlab, fayl hajmini tejaydi.

🔗 15. "Chroma Key" (Yashil fon) bilan ishlashda nega aynan yashil yoki ko'k rang tanlanadi?

📖 Inson tanasi va terisining rangida yashil va ko'k pigmentlari eng kam uchraydi. Bu dasturga (masalan, After Effects) insonni fondan qirqib olishda adashmaslik imkonini beradi. Yashil rang raqamli kameralar sensori tomonidan yaxshiroq qabul qilinadi (shovqin kamroq bo'ladi), ko'k rang esa kiyimda yashil elementlar bo'lsa yoki sochlarning cheti tabiiyroq ko'rinishi kerak bo'lganda tanlanadi.

★ 16. "Motion Tracking" (Harakatni kuzatish) amaliyotda qayerda kerak bo'ladi?

📖 Tasavvur qiling, harakatlanayotgan mashinaning yoniga o'zingizning logotipingizni "yopishtirib" qo'yishingiz kerak. Dastur videodagi mashinaning biror nuqtasini tanlab, uning harakat koordinatalarini (X, Y) hisoblaydi. Keyin siz qo'shmoqchi bo'lgan logotip o'sha koordinatalarga bog'lanadi va u xuddi mashinaning bir bo'lagidek u bilan birga harakatlanadi.

🔗 17. "Camera Tracking" (3D Tracking) oddiy trackingdan nimasi bilan farq qiladi?

📖 Agar kamera o'zi ham harakatlanayotgan bo'lsa (masalan, dron bilan olingan video), bizga 3D tracking kerak. Dastur videodagi barcha piksellarni tahlil qilib, kameraning fazodagi virtual nusxasini yaratadi. Bu sizga videoga 3D modellarni (masalan, ko'cha o'rtasiga ulkan haykalni) xuddi u yerda turgandek joylashtirish imkonini beradi.

★ 18. "PBR" (Physically Based Rendering) teksturalar nima uchun standartga aylandi?

📖 PBR — bu materiallarning real dunyodagi fizik qonunlar asosida yorug'likka bo'lgan munosabatini ifodalash usuli.

- **Albedo:** Ob'ektning asosiy rangi.
- **Metallic:** U metalmi yoki yo'qmi?
- **Roughness:** U silliqmi (yaltiroq) yoki g'adir-budirmi (xira)? Ushbu xaritalar birlashganda, 3D model har qanday yorug'lik sharoitida (quyoshda ham, lampochka ostida ham) o'ta realistik ko'rinadi.

❓ 19. "Global Illumination" (GI) renderlashda qanday effekt beradi?

📖 An'anaviy yoritishda yorug'lik faqat manbadan chiqqan joyni yoritadi. GI esa "bilvosita yoritish"ni hisoblaydi. Ya'ni, yorug'lik devordan qaytib, xonaning qorong'u burchaklarini ham biroz yoritadi. Bu tasvirga fotorealistik ko'rinish beruvchi eng asosiy texnologiyalardan biridir.

☆ 20. "UV Unwrapping" (UV yoyilmasi) jarayoni nima uchun kerak?

📖 3D modelni 2D formatdagi rasm (tekstura) bilan qoplash uchun uni tekislikka "yoyish" kerak. Xuddi konfet qog'ozini ochib yoygandek, 3D ob'ektning yuzalari tekis qilib yoyiladi. Shundan keyingina Photoshop-da chizilgan tekstura modelning qaysi qismiga tushishini aniq belgilash mumkin bo'ladi.

❓ 21. "J-Cut" va "L-Cut" montaj usullari nima uchun ishlatiladi?

📖 Bu usullar videoning silliq (tabiiy) o'tishini ta'minlaydi:

- **J-Cut:** Siz hali yangi kadrni ko'rmasangiz ham, uning ovozi eshitishni boshlaysiz (ovoz kadrni "yetaklaydi").
- **L-Cut:** Kadr almashadi, lekin eski kadrning ovozi hali biroz davom etib turadi. Bular tomoshabinga montajdagi "kesish"larni sezmaslikka va hikoyaga sho'ng'ishga yordam beradi.

☆ 22. "Sound Design" (Foley) videoning sifatiga qanday ta'sir qiladi?

📖 Sifatli video — bu 50% tasvir va 50% ovozdir. Foley — bu videodagi har bir harakat uchun alohida tovush qo'shish (qadam tovushi, kiyim shishirlashi, eshik g'irchillashi). Agar bu ovozlari bo'lmasa, video "o'lik" va sun'iy ko'rinadi.

❓ 23. "Rule of Thirds" (Uchdan bir qoidasi) kadrlashda nega muhim?

📖 Ekranni vertikal va gorizontal 3 qismga bo'luvchi chiziqlar o'tkaziladi. Eng muhim ob'ektni (masalan, inson ko'zini) ushbu chiziqlar kesishgan nuqtalarga qo'yish kerak. Bu kompozitsiyani dinamik va tomoshabin ko'zi uchun yoqimli qiladi (ob'ektni shunchaki markazga qo'ygandan ko'ra effektliroq).

☆ 24. "Safe Margins" (Xavfsiz hududlar) montajda nima uchun kerak?

📖 Eski televizorlar yoki ba'zi ijtimoiy tarmoq interfeyslari videoning chetlarini to'sib qo'yishi mumkin. **Action Safe** va **Title Safe** chiziqlari yordamida biz muhim harakatlar va yozuvlarni (titrlarni) ekranning o'ta chetiga qo'ymaymiz. Shunda har qanday qurilmada yozuvlar to'liq ko'rinadi.

❓ 25. Render vaqtida "Artifacts" (Tasvirdagi nuqsonlar) nega paydo bo'ladi?

📖 Buning sababi ko'pincha juda past **Bitrate** yoki noto'g'ri kodek sozlamasidir. Agar video o'ta qattiq siqilsa (Compression), dastur o'xshash rangli piksellarni birlashtirib yuboradi va natijada "bloklar" yoki g'alati rangli dog'lar paydo bo'ladi. **Yechim:** Eksport vaqtida "Target Bitrate"ni oshirish yoki "2-pass encoding"ni tanlash.

❓ 26. "Log" (Logarithmic) formatda video olishning foydasi nimada? Nega u dastlab xira ko'rinadi?

📖 **Log** — bu kameraning sensoridan maksimal darajadagi dinamik diapazonni (dynamic range) saqlab qolish uchun mo'ljallangan format. U tasvirdagi eng yorug' va eng qorong'u nuqtalardagi detallarni o'chirib yubormasdan saqlaydi. Tasvirning xira (past kontrastli) ko'rinishiga sabab — ma'lumotlarning "siqilgani"dir. Montaj jarayonida rang ustasi (colorist) ushbu xira tasvirga kontrast va rang qo'shganda, tasvir "sinib" ketmaydi (banding paydo bo'lmaydi) va professional kino ko'rinishiga keladi.

☆ 27. "Color Wheels" (Rang g'ildiraklari) yordamida videoni tahrirlashda Lift, Gamma va Gain farqi?

📖 Bu vositalar tasvirning turli yorug'lik qismlarini boshqaradi:

- **Lift (Shadows):** Tasvirdagi eng qorong'u (qora) qismlarning rangini o'zgartiradi.
- **Gamma (Midtones):** O'rtacha yorug'likdagi qismlar, masalan, inson terisi rangini boshqaradi.
- **Gain (Highlights):** Tasvirdagi eng yorug' (oq) qismlar, masalan, osmon yoki chiroq nurini tahrirlaydi. Professional montajda avval ranglar muvozanati (Balance) to'g'rilanadi, keyin ushbu g'ildiraklar orqali estetik ko'rinish beriladi.

❓ 28. "Depth of Field" (DOF - Maydon chuqurligi) videoda qanday kayfiyat yaratadi?

📖 Bu fokusdagi ob'ekt va fonning qanchalik xiralashganligi (blur) nisbatidir.

- **Shallow DOF (Sayoz):** Orqa fon o'ta xira, faqat asosiy ob'ekt aniq ko'rinadi. Bu tomoshabin e'tiborini bir nuqtaga qaratish va hissiyotlarni kuchaytirish uchun ishlatiladi.
- **Deep DOF (Chuqur):** Ham oldingi, ham orqa fon aniq ko'rinadi. Bu asosan manzara (peyzaj) suratlarida qo'llaniladi.

☆ 29. "Parallax Effect" (Parallaks effekti) 2D rasmni qanday qilib 3D ko'rinishga keltirishi mumkin?

📖 Bu harakatlanish vaqtidagi vizual aldamchilik. Oldindagi ob'ektlar tezroq, uzoqdagi ob'ektlar esa sekinroq harakatlanadi. **Amaliyotda:** After Effects dasturida bitta rasmni qatlamlarga (odam, daraxtlar, tog'lar) bo'lib, ularni 3D fazoga turli masofalarga joylashtiramiz. Kamera harakatlanganda, qatlamlar turlicha siljiydi va natijada oddiy 2D rasmda chuqurlik (hajm) paydo bo'ladi.

❓ 30. "Ray Tracing" (Nurlar kuzatuvi) texnologiyasi renderlashni nega inqilobiy darajaga chiqardi?

📖 An'anaviy renderlashda yorug'lik taxminiy hisoblangan. **Ray Tracing** esa yorug'lik nurlarining har birini fizik qonunlar asosida kuzatib boradi: nur ob'ektdan qanday qaytadi, oynada qanday akslanadi va suvda qanday sinadi. Bu texnologiya tufayli 3D grafikada ko'zgu effektlari va soyalar real dunyodan farq qilmaydigan darajaga yetdi.

☆ 31. "Subsurface Scattering" (SSS) materiali nima va u nega inson terisini yaratishda shart?

📖 Ba'zi ob'ektlar (mum, marmar, inson terisi, uzum) yorug'likni o'zidan qisman o'tkazadi. Yorug'lik tana ichiga kiradi, u yerda tarqaladi va orqaga qaytib chiqadi. **Misol:** Agar qulog'ingiz orqasidan fonar yoqsangiz, qulog'ingiz qizil bo'lib nurlanadi. 3D-da bu effektni (SSS) ishlatmasangiz, inson terisi plastik yoki toshdek sun'iy ko'rinadi.

❓ 32. "Match Cut" nima va u hikoyani qanday bog'laydi?

📖 Bu ikki xil kadrni ularning shakli yoki harakati o'xshashligi orqali birlashtirish. **Misol:** Bir kadrda dumaloq kofeli finjonni ko'rsatib, keyingi kadrda xuddi o'sha o'rinda turgan avtomobil g'ildiragiga o'tish. Bu usul tomoshabin uchun vizual "ko'prik" vazifasini o'taydi va o'tishni silliq hamda aqlli qiladi.

☆ 33. "Keying" (Yashil fonni tozalash) vaqtida "Spill Suppression" nima uchun kerak?

📖 Yashil fonda suratga olinganda, yashil rang nuri aks etib odamning yuziga yoki kiyimiga "yuqib" (spill) qoladi. Agar siz shunchaki yashilni o'chirib, fonni o'zgartirsangiz, odamning chetlari yashil bo'lib qolaveradi. **Spill Suppression** algoritmi shu yashil nur qoldiqlarini neytrallashtiradi va ob'ekt yangi fonga tabiiy ravishda "singishi"ni ta'minlaydi.

❓ 34. "Interlaced" (1080i) va "Progressive" (1080p) videoning farqi nimada?



- **Progressive (p):** Har bir kadr to'liq tasvirlanadi. Bu zamonaviy monitorlar va internet uchun standartdir.

- **Interlaced (i):** Kadr qatorlarga bo'linib, navbat bilan (toq va juft qatorlar) ko'rsatiladi. Bu eski televideniye standarti. Agar 1080i videoni kompyuterda ko'rsangiz, harakatlanayotgan ob'ektlar chetida "taroqsimon" chiziqlar (combing) ko'rasiz.

★ 35. "HDR" (High Dynamic Range) video montajida nima beradi?



HDR video oddiy (SDR) videoga qaraganda yuz barobar ko'proq yorug'lik va rang ma'lumotlarini o'z ichiga oladi. HDR-da siz qorong'u soyadagi detallarni ham, quyoshning o'ta yorqin nurini ham bir vaqtda aniq ko'ra olasiz. Bu tasvirga o'ta chuqur realizm va "tiriklik" bag'ishlaydi.

❓ 36. Animatsiyadagi "Ease-in" va "Ease-out" (Silliqlash) qonuniyati fizik nuqtai nazardan nimani anglatadi?



Real hayotda hech qanday jism bir soniyada maksimal tezlikka chiqmaydi va bir soniyada taqa-taq to'xtamaydi. **Ease-in:** Harakat sekin boshlanib, tezlashadi (masalan, mashina yurishi). **Ease-out:** Harakat tez boshlanib, oxirida sekinlashib to'xtaydi. Agar siz After Effects-da bu funktsiyani (Easy Ease) ishlatmasangiz, harakatlar "robotlashgan" va sun'iy ko'rinadi. Silliqlash grafiklari yordamida harakatga "vazn" va "inersiya" hissini berish mumkin.

★ 37. "Motion Blur" (Harakat xiralashuvi) videoda nega shart? Uni After Effects-da qanday boshqariladi?



Agar kadrda ob'ekt o'ta tez harakatlansa, biz uni biroz xira ko'rishimiz kerak (bu ko'zimizning ishlash printsipli). After Effects-da "Motion Blur" yoqilsa, dastur harakat tezligiga qarab ob'ektni xiralashtiradi. **Amaliyotda:** Agar siz 3D modelni videoga qo'shsangiz va unda motion blur bo'lmasa, u kadrda "o'ynab" turgandek ko'rinadi. Shuningdek, "Shutter Angle" sozlamasi orqali bu xiralik darajasini kino standartiga (odatda 180 daraja) moslash mumkin.

❓ 38. "Null Object" nima uchun ishlatiladi va u "Parenting" bilan qanday bog'langan?



Null Object — bu videoda ko'rinmaydigan, lekin boshqa elementlarni boshqarish uchun xizmat qiladigan "ko'rinmas dasta". **Ssenariy:** Sizda 10 ta matnli

qatlam (layers) bor va ularning hammasini bir vaqtda aylantirishingiz kerak. Har biriga alohida keyframe qo'yish o'rniga, bitta Null Object yaratasiz va barcha qatlamlarni unga **"Parent"** (bog'lash) qilasiz. Endi Null Object-ni qimirlatsangiz, barcha 10 ta qatlam unga ergashadi. Bu murakkab sahnalarni boshqarishning eng samarali usuli.

☆ 39. "Adjustment Layer" (Sozlovchi qatlam) qanday vazifani bajaradi?

📖 Bu shaffof qatlam bo'lib, unga qo'yilgan barcha effektlar uning ostida turgan barcha videolarga baravar ta'sir qiladi. **Amaliyotda:** Agar siz butun montajingizga rang bermoqchi bo'lsangiz (Color Grading), har bir kadrda alohida effekt qo'yib o'tirmaysiz. Eng yuqoriga Adjustment Layer qo'yasiz va rangni shunga berasiz — natijada barcha kadrlar bir xil rangga kiradi.

❓ 40. "Volumetric Lighting" (Hajmli yorug'lik) videoga qanday realizm qo'shadi?

📖 Bu havoda chang yoki tuman borligida nur nurlarining ko'rinishi (masalan, qorong'u xonadagi derazadan tushayotgan quyosh nuri). 3D grafikada bu effektni yaratish uchun "God Rays" texnologiyasi ishlatiladi. Bu shunchaki yorug'lik emas, balki nurning havodagi zarrachalarga urilib qaytishini simulyatsiya qilishdir. Bu sahnalarga sirli va kinematografik muhit beradi.

☆ 41. "Topology" (Topologiya) 3D modelni animatsiya qilishda nega hal qiluvchi rol o'ynaydi?

📖 Topologiya — bu modelni tashkil qiluvchi to'rtburchaklar (quads) va uchburchaklar (tris) tarmog'i. **Muammo:** Agar inson yuzining topologiyasi noto'g'ri bo'lsa (yuzlar tartibsiz bo'lsa), modelni kuldirish yoki gapirtirish vaqtida yuzda g'alati burmalar va "siniqlar" paydo bo'ladi. Professional 3D modelerlar har doim modelning "toza" (ko'pincha to'rtburchakli) bo'lishiga harakat qilishadi, chunki bu silliq animatsiya uchun shart.

❓ 42. "Baking" (Pishirish) jarayoni 3D renderlashda va o'yinlarda nega kerak?

📖 Murakkab soyalar va yorug'liklarni har bir kadrda hisoblash (render) juda uzoq vaqt oladi. **Baking:** Bu barcha yorug'lik va soyalarni modelning teksturasiga "yopishtirib" yuborishdir. Natijada kompyuter yorug'likni qayta hisoblamaydi, shunchaki tayyor rasmni ko'rsatadi. Bu texnologiya tufayli mobil o'yinlarda ham o'ta realistik grafikalar bo'lishi mumkin.

☆ 43. "Anamorphic Lenses" (Anamorfik ob'ektivlar) videoga qanday o'ziga xos ko'rinish beradi?

📖 Bu linzalar tasvirni gorizontol ravishda qisib oladi. Natijada:

1. Ekran o'ta keng (Wide) bo'ladi.

2. Chiroqlar nur sochganda ko'k gorizontol chiziqlar (flares) hosil bo'ladi.

3. Orqa fonning xiralashishi (bokeh) dumaloq emas, balki tuxumsimon (oval) bo'ladi. Ko'plab Gollivud kinolari aynan shu "estetika" uchun anamorfik linzalarda suratga olinadi.

🔗 44. "Field of View" (FOV) va "Focal Length" o'rtasidagi bog'liqlik nimada?



- **Focal Length (Fokus masofasi):** O'lchami kichik bo'lsa (masalan, 14mm) — bu keng burchak, ko'p narsa kadrda sig'adi, lekin chetlar biroz qiyshayadi.

- **Telephoto (masalan, 85mm):** Burchak torayadi, uzoqdagi narsa yaqinlashadi va fon o'ta chiroyli xiralashadi. Professional montajda "Yaqin plan" (Close-up) uchun 50-85mm, "Manzara" uchun 14-24mm fokus masofasi tanlanadi.

☆ 45. "Bit Depth" 8-bit va 10-bit o'rtasidagi farqni real hayotda qachon sezasiz?

📖 Tasavvur qiling, videoda moviy osmon bor.

- **8-bitda:** Osmonning quyuq ko'kdan och ko'kka o'tish joyida "zinapoyasimon" chiziqlar (banding) ko'rinadi.

- **10-bitda:** Ranglar o'tishi o'ta silliq bo'ladi. Agar siz ranglar bilan professional ishlamoqchi bo'lsangiz (Color Grading), kadrda detallar saqlanib qolishi uchun 10-bitli video shart.

🔗 46. "Expressions" (Ifodalar) After Effects-da qanday muammoni hal qiladi?

📖 Tasavvur qiling, sizga ob'ektning to'xtovsiz titrab turishi (masalan, zilzila effekti) kerak. Buning uchun minglab tayanch kadrlarni (keyframes) qo'lda qo'yish o'rniga, kichik JavaScript kodi yoziladi (masalan: `wiggle(5, 20)`). Bu kod dasturga "soniyasiga 5 marta 20 piksel radiusda tasodifiy harakatlan" degan buyruqni beradi. Ifodalar murakkab animatsiyalarni matematik aniqlikda va juda tez yaratish imkonini beradi.

☆ 47. "Null Object" va "Slider Control" birgalikda qanday ishlatiladi?

📖 Bu "Rigging" (boshqaruv tizimi) yaratishning asosiy usuli. Siz bitta Null Object-ga bir nechta "Slider" (surgich) effektini qo'shasiz va ularni boshqa

qatlamlarning xususiyatlariga (masalan, rang, shaffoflik, o'lcham) bog'laysiz. Natijada, siz 50 ta qatlamning ichiga kirmasdan, bitta surish orqali butun sahnaning kayfiyatini yoki dinamikasini o'zgartira olasiz.

❓ 48. Blender-dagi "Geometry Nodes" an'anaviy modellashtirishdan nimasi bilan ustun?

📖 An'anaviy usulda siz ob'ektni shakllantirasiz va uni o'zgartirish qiyin bo'ladi. **Geometry Nodes** esa "protsedurali" yondashuvdir. **Misol:** Siz bitta "tugunlar tarmog'i" (node tree) yaratasiz va u avtomatik ravishda chizig'ingiz bo'ylab devor quradi. Agar siz chiziqni egsangiz, devor ham o'z-o'zidan egiladi. Bu usul orqali o'ta murakkab landshaftlar, o'rmonlar yoki shaharlarni soniyalar ichida generatsiya qilish mumkin.

★ 49. "Real-time Rendering" (Unreal Engine) nega kino sanoatini o'zgartirmoqda?

📖 Ilgari bitta kadrni render qilish uchun soatlab vaqt ketardi. **Unreal Engine** (o'yin dvigateli) buni soniyaning mingdan bir bo'lagida bajaradi. **Virtual Production:** Hozirda yashil fon o'rniga ulkan LED ekranlar ishlatilmoqda. Aktyor ortidagi 3D fon real vaqtda kamera harakatiga moslashadi. Bu aktyorlarga muhitni his qilishga va rejissyorga yakuniy natijani suratga olish jarayonidayoq ko'rishga imkon beradi (masalan, "The Mandalorian" seriali).

❓ 50. "LUT" (Look-Up Table) nima va u nega "sehrli tayoqcha" emas?

📖 LUT — bu ranglarni matematik o'zgartirish jadvali. U o'ziga xos "filtr" kabi ishlaydi. **Xato:** Ko'pchilik yangi boshlovchilar LUT-ni xira (Log) videoga tashlasa, darhol kino bo'ladi deb o'ylaydi. **Amaliyot:** LUT-dan oldin doimo "Primary Correction" (oq rang balansi va ekspozitsiya) qilinishi shart. Agar kadr noto'g'ri yoritilgan bo'lsa, hech qanday LUT uni qutqara olmaydi. LUT — bu binoning o'zi emas, balki uning oxirgi pardoqlash bo'yog'idir.

★ 51. "ACES" (Academy Color Encoding System) nima uchun professional montajda standartga aylandi?

📖 Agar loyihada turli kameralar (Sony, RED, iPhone) ishlatilgan bo'lsa, ularning ranglari turlicha bo'ladi. **ACES** barcha bu ranglarni yagona, o'ta keng "rang maydoni"ga o'tkazadi. Bu barcha kadrlarni bir xil rang sxemasiga keltirishni va turli monitorlarda (TV, Telefon, Kino) bir xil sifatni ta'minlashni osonlashtiradi.

❓ 52. "Pacing" (Tezlik) va "Rhythm" (Ritm) montajda tomoshabinga qanday ta'sir qiladi?



- **Pacing:** Bu sahnalarning uzunligi. Jangovar sahnalarda kadrlar juda qisqa bo'ladi (tez-tez almashadi), bu hayajonni oshiradi. Dramatik sahnalarda esa kadrlar uzoqroq ushlab turiladi, bu tomoshabinga hissiyotni his qilishga vaqt beradi.

- **Rhythm:** Bu kadrlar almashinuvining musiqa yoki nutq bilan uyg'unligi. Agar kadr musiqaning "bit"iga (zarbiga) tushsa, tomoshabin buni ongsiz ravishda o'ta sifatli va yoqimli deb qabul qiladi.

☆ 53. "Invisible Cut" (Ko'rinmas kesish) qanday sirlarga ega?

 Bu montajdagi uzilishni foydalanuvchiga sezdirmaslik san'ati. **Usullar:**

1. **Match on Action:** Ob'ekt harakatni boshlaganda kesish (masalan, eshikni ochishni boshlaganda yaqin plandan uzoq planga o'tish).

2. **Eye-line Match:** Aktyor bir tomonga qarashi bilan, u ko'rayotgan narsaga kesish. Inson miyasi mantiqiy bog'liqlikni qidiradi, agar bog'liqlik bo'lsa, montajdagi "kesish"ni sezmaydi.

❓ 54. "Particle Systems" (Zarrachalar tizimi) nima uchun ishlatiladi?

 Bu minglab kichik ob'ektlarning harakatini simulyatsiya qilish: yomg'ir, qor, olov, tutun yoki portlashdan uchayotgan parchalar. Har bir zarrachaga gravitatsiya, shamol va bir-biri bilan to'qnashish (collision) kabi fizik qoidalar beriladi. Bu orqali real dunyo hodisalarini raqamli muhitda yaratish mumkin.

☆ 55. "Rigging" (Skelet yaratish) 3D personajlarni qanday "tiriltiradi"?

 3D model shunchaki "teri" (mesh). Uni harakatlantirish uchun ichiga virtual "suyaklar" va "bo'g'inlar" (bones and joints) qo'yiladi. **IK (Inverse Kinematics):** Bu oson boshqaruv usuli. Siz personajning qo'lini pastga tortsangiz, tirsak va yelka o'z-o'zidan tabiiy ravishda egiladi. Sifatli rig bo'lmasa, hatto eng chiroyli model ham xunuk harakatlanadi.

❓ 56. Cinema 4D dasturidagi "MoGraph" moduli nima uchun dizaynerlar orasida o'ta mashhur?

 MoGraph — bu minglab ob'ektlarni bitta buyruq bilan nusxalash va ularni turli effektlar (shamol, tovush, gravitatsiya) orqali boshqarish tizimidir. **Amaliyotda:** Agar sizga ekranda minglab kubiklar musiqa ritmiga qarab raqsga tushishi kerak bo'lsa, MoGraph buni soniyalar ichida amalga oshiradi. U "Cloner" va "Effectors" yordamida murakkab matematik harakatlarni oson vizuallashtiradi, bu esa After Effects-da soatlab vaqt olishi mumkin edi.

☆ 57. "Fields" texnologiyasi Cinema 4D-da animatsiyani qanday boshqaradi?

📖 Fields (Maydonlar) — bu animatsiya qayerda va qachon sodir bo'lishini belgilaydigan "ko'rinmas shakllar". **Ssenariy:** Sizda tekis yotgan 100 ta kitob bor. Siz ekran bo'ylab "Sferik maydon"ni (Spherical Field) harakatlantirasiz. Kitoblar faqat ushbu sfera tegib o'tgan joydagina ochiladi. Bu usul animatsiyani o'ta aniq va interaktiv boshqarish imkonini beradi.

🔗 58. ".MOGRT" (Motion Graphics Templates) fayllari Premiere Pro va After Effects hamkorligini qanday yaxshilaydi?

📖 Bu fayllar After Effects-da yaratilgan o'ta murakkab animatsiyalarni (masalan, chiroyli titrlar) Premiere Pro ichida ishlatish imkonini beradi. **Foydasi:** Montajchi After Effects-ni bilishi shart emas. U shunchaki MOGRT-ni yuklaydi va Premiere Pro panelida matnni, rangni yoki logotipni o'zgartiradi, animatsiya esa o'z-o'zidan saqlanib qoladi. Bu professional studiyalarda ishni 10 barobar tezlashtiradi.

☆ 59. "Tracking Data"ni 2D-dan 3D-ga eksport qilish nega kerak?

📖 Tasavvur qiling, siz kamerada olingan videoga 3D-da yasalgan robotni qo'shmoqchisiz. Buning uchun siz avval After Effects yoki Mocha dasturida videodagi nuqtalarni track qilasiz. So'ngra ushbu harakat koordinatalarini Cinema 4D yoki Blender-ga eksport qilasiz. Endi 3D dastur ichidagi virtual kamera xuddi real hayotdagi kameradek harakatlanadi va sizning robotingiz videoga "mixlangandek" tabiiy o'tiradi.

🔗 60. "Sub-sampling" (4:2:2, 4:2:0) tushunchasi rang sifatiga qanday ta'sir qiladi?

📖 Bu videoni siqish paytida rang ma'lumotlarini tejash usulidir.

- **4:4:4:** Hech qanday ma'lumot yo'qolmaydi (o'ta yuqori sifat).
- **4:2:2:** Professional standart, rang berish (grading) uchun yetarli.
- **4:2:0:** Internet va telefonlar uchun standart (rang ma'lumotlarining 75% i tejaladi). Agar siz 4:2:0 formatdagi videoda "Green Screen"ni tozalashga harakat qilsangiz, ob'ekt chetlari g'adir-budir va "shovqinli" chiqadi, chunki rang chegaralari aniq emas.

☆ 61. "HDR Mastering" jarayonida "Nit" tushunchasi nimani anglatadi?

📖 Nit — bu yorug'lik yorqinligi birligi. Oddiy monitorlar 250-300 nit yorqinlikka ega. Professional HDR monitorlar (masalan, Apple Pro Display XDR) 1000-1600 nitgacha chiqadi. HDR montajida siz "Oq" rangni shunchaki oq emas,

balki "1000 nitli quyosh nuri" deb belgilaysiz. Bu tomoshabin ko'ziga xuddi derazadan real quyoshga qaragandek yorqin effekt beradi.

? 62. "Camera Projection" (Kamera proyeksiyasi) texnikasi nima?

📖 Bu 2D rasmni 3D ob'ekt ustiga "proyektor" kabi tushirishdir. **Amaliyotda:** Sizda bitta tog'ning rasmi bor. Siz 3D-da tog'ning oddiy shaklini yasaysiz va rasmni uning ustiga proyeksiya qilasiz. Endi kamerani biroz qimirlatsangiz, tog' xuddi haqiqiy 3D ob'ektdagidek hajmga ega bo'ladi. Bu usul kino sanoatida murakkab 3D landshaftlarni noldan yasamasdan, tezkor yaratish uchun ishlatiladi.

☆ 63. "Deep Compositing" nima va u oddiy qatlamlar (layers) bilan ishlashdan nimasi bilan farq qiladi?

📖 Oddiy kompozitingda har bir piksel faqat rang va shaffoflik (RGBA) ma'lumotiga ega. **Deep Compositing**-da esa har bir piksel o'zining "chuqurlik" (Z-depth) ma'lumotiga ham ega bo'ladi. **Foydasi:** Agar sizda 3D render va tuman (fog) bo'lsa, siz ularni qaysi biri oldinda, qaysi biri orqada ekanini maskasiz (masks) aniq boshqara olasiz. Bu o'ta murakkab VFX sahnalarida (masalan, chang-to'zon ichidagi janglar) shart.

? 64. "Pipeline" (Quvur) tushunchasi studiyalarda nimani anglatadi?

📖 Bu loyihani boshidan oxirigacha o'tadigan qat'iy tartibi.

1. **Pre-viz:** Loyihaning xomaki 3D ko'rinishi.
2. **Modeling/Texturing:** Ob'ektlarni yaratish.
3. **Animation:** Harakat berish.
4. **Rendering:** Tasvirni hisoblash.

5. **Compositing:** Hamma narsani birlashtirish. Agar biror qadamda xato bo'lsa, "quvur" tiqilib qoladi va ish to'xtaydi. Shuning uchun har bir bosqichning o'z o'rnini va texnik talablari bor.

☆ 65. "Render Farm" (Render fermasi) nima?

📖 Agar bitta kompyuter 10 daqiqalik 3D filmni 1 yilda render qilsa, **Render Farm** (yuzlab kuchli serverlar birlashmasi) buni 1 kunda bajaradi. Hozirda ko'pchilik dizaynerlar o'z kompyuterini qiynamasdan, bulutli render fermalardan foydalanishadi.

Kompyuter grafikasi va videomontaj bo'yicha sanoqni 66-savoldan davom ettiramiz va 100 tagacha yetkazib, yakunida 50 ta eng muhim atamalarni jadvalda ko'rib chiqamiz.

❓ 66. Professional montajda "Video Scopes" (Waveform, Vectorscope) nega ko'zga qaraganda ishonchliroq?

📖 Inson ko'zi va monitorlar aldashi mumkin (masalan, xonadagi yorug'likka qarab rang har xil ko'rinadi).

- **Waveform:** Tasvirdagi yorug'lik (Luma) darajasini ko'rsatadi. U orqali ranglar "to'yib" ketganini (clipping) aniq ko'rish mumkin.
- **Vectorscope:** Ranglarning to'yinganligi va tusini (hue) ko'rsatadi. Ayniqsa, inson terisi rangini (Skin Tone Line) to'g'ri sozlashda bu yagona aniq o'lchov asbobidir.

☆ 67. "Secondary Color Correction" nima va u "Primary" dan nimasi bilan farq qiladi?



- **Primary:** Butun kadrning rangini va yorug'ligini muvozanatlash.
- **Secondary:** Kadrning faqat ma'lum bir qismini (masalan, faqat qizil ko'ylakni yoki faqat osmonni) tanlab olib, uning rangini o'zgartirish. Buning uchun **HSL Secondary** (Hue, Saturation, Luma) maskalari ishlatiladi.

❓ 68. "Deepfake" texnologiyasi qanday fizik asosga ega?

📖 Bu **GAN (Generative Adversarial Networks)** neyron tarmoqlariga asoslanadi. Bitta tarmoq tasvirni yaratadi, ikkinchisi uni asliga qanchalik o'xshashligini tekshiradi. Millionlab takrorlashlardan so'ng, AI bir insonning yuz harakatlarini ikkinchi insonga o'ta realistik tarzda ko'chirib o'tkazadi. Bu videomontajda "Face replacement" (yuzni almashtirish) jarayonini avtomatlashtirdi.

☆ 69. After Effects-dagi "Roto Brush 3.0" nega sehrli vosita hisoblanadi?

📖 Ilgari odamni fondan qirqib olish uchun har bir kadrda qo'lda chizish (rotoscoping) kerak edi. AI yordamida ishlaydigan **Roto Brush** ob'ektni birinchi kadrda belgilab berishingiz bilan, keyingi kadrlar bo'ylab uning konturini avtomatik kuzatib boradi. Bu haftalab davom etadigan ishni bir necha soatga qisqartirdi.

❓ 70. "Blender Simulation Nodes" bilan nimalar qilish mumkin?

📖 Bu zarrachalar va fizik hodisalarni (suv oqishi, matoning yirtilishi, olov) protsedurali usulda yaratish imkonini beradi. Siz matematik qoidalar yozasiz va tizim minglab ob'ektlarni o'zaro to'qnashishini (collision) hisoblaydi. Bu orqali, masalan, bitta tugun (node) yordamida butun bir binoning qulashini simulyatsiya qilish mumkin.

☆ 71. "Audio Ducking" nima va u montajda qanday qulaylik beradi?

📖 Bu avtomatik jarayon bo‘lib, kadrda kimdir gapira boshlashi bilan orqadagi fon musiqasi ovozi pasaytiradi. Gap tugashi bilan musiqa yana balandlashadi. Zamonaviy NLE tizimlari (Premiere Pro) buni AI yordamida bir soniyada bajaradi, bu esa qo‘lda keyframe qo‘yish azobidan xalos qiladi.

❓ 72. Videoda "Phase Cancellation" (Ovoz o‘chib qolishi) nega sodir bo‘ladi?

📖 Agar ikkita mikrofon bitta manbani yozsa va ulardan biri biroz kechiksa, tovush to‘lqinlari bir-birini "yeb" yuboradi. Natijada ovoz o‘ta "yassi" va xunuk chiqadi. Montajda buni to‘lqinlarni millisekund darajasida surish (align) orqali to‘g‘rilash mumkin.

☆ 73. "Motion Capture" (MoCap) texnologiyasi qanday ishlaydi?

📖 Aktyor ustiga maxsus datchiklar o‘rnatiladi. Uning barcha harakatlari kompyuterga nuqtalar ko‘rinishida uzatiladi. So‘ngra bu harakatlar 3D modelga (masalan, Gollum yoki King Kong) bog‘lanadi. Bu insoniy hissiyotlar va harakatlarni 3D-ga ko‘chirishning eng mukammal yo‘lidir.

❓ 74. "Blend Shapes" (yoki Shape Keys) yuz animatsiyasida nima uchun kerak?

📖 "Suyaklar" (rigging) tana uchun yaxshi, lekin lab yoki qosh harakatlari uchun qiyinlik qiladi. **Blend Shapes** — bu yuzning turli holatlari (tabassum, g‘azab, "o" harfini aytish) oldindan tayyorlanadi va dastur ular orasida silliq o‘tishni amalga oshiradi.

☆ 75. "Proxies" (Proksi fayllar) bilan ishlashda "Attach Proxies" nega muhim qadam?

📖 Siz proksi (yengil) faylda montaj qildingiz. Eksport qilishdan oldin dastur asl (original) 4K/8K fayllarni qayta bog‘lashi (link) shart. Aks holda, sizning yakuniy videongiz xira va sifatsiz bo‘lib chiqadi.

❓ 76. "Constant Bitrate" (CBR) qaysi turdagi videolar uchun mos?

📖 CBR asosan jonli translyatsiyalar (Live Streaming) uchun ishlatiladi. Bu tarmoqqa bir tekis yuklama berishni ta’minlaydi. Arxivlash yoki sifatli video uchun esa doimo **VBR 2-pass** yaxshiroq.

❓ 77. "Parallax Scrolling" texnikasi 2D animatsiyada qanday qilib 3D chuqurlik hissini yaratadi?

📖 Bu inson ko'zining ko'rish fizikasiga asoslangan. Oldindagi ob'ektlar (masalan, daraxtlar) kamera harakatlanganda tezroq siljiydi, uzoqdagi ob'ektlar (masalan, tog'lar yoki oy) esa juda sekin harakatlanadi. **Amaliyotda:** After Effects dasturida qatlamlarni turli "Z-depth" (chuqurlik) masofalariga joylashtirib, bitta virtual kamerani harakatlantirish orqali o'ta realistik makon hissini yaratish mumkin.

☆ 78. "Anamorphic Desqueeze" jarayoni montajda nima uchun bajariladi?

📖 Anamorfik linzalar tasvirni sensorga "siqilgan" (toraygan) holda tushiradi. Agar siz bunday videoni shunchaki ochsangiz, odamlar o'ta oriqlik va baland ko'rinadi. **Montajda:** Dasturga tasvirni gorizontall ravishda 1.33x yoki 2x marta "yozish" (desqueeze) buyrug'i beriladi. Natijada biz Gollivud filmlariga xos bo'lgan o'ta keng (Ultrawide) tasvir va o'ziga xos optik xususiyatlarni (masalan, oval ko'rinishdagi bokeh) olamiz.

🔗 79. 3D grafikada "Normal Map" va "Displacement Map" o'rtasidagi real farq nimada?



- **Normal Map:** Bu illyuziya. U yuzadagi yorug'lik qaytishini o'zgartirib, g'adir-budirlik (masalan, g'isht chiziqlari) ko'rinishini beradi, lekin ob'ektning cheti tekisligicha qoladi.
- **Displacement Map:** Bu haqiqiy geometriya. U modelning nuqtalarini (vertices) rasmga qarab yuqoriga yoki pastga suradi. Agar siz ob'ektga yondan qarasangiz, u haqiqatda g'adir-budir ekanini ko'rasiz. U render vaqtida ko'proq resurs talab qiladi.

☆ 80. "Fluid Simulation" (Suyuqlik simulyatsiyasi) nima uchun renderlashda eng qiyin jarayon hisoblanadi?

📖 Chunki kompyuter millionlab kichik "zarrachalar"ning (particles) har birini bir-biri bilan to'qnashishini, yopishqoqligini (viscosity) va gravitatsiyani hisoblashi kerak. Suvning har bir tomchisi yuzasidan yorug'lik sinishi (refraction) hisoblangani uchun, 1 soniyalik suv simulyatsiyasi bir necha kunlab render qilinishi mumkin.

🔗 81. "Pre-composing" (Pre-comp) After Effects-da qanday tartib o'rnatadi?

📖 Bu o'nlab qatlamlarni (layers) bitta "konvert"ga (kompozitsiyaga) solib qo'yishdir. **Foydasi:** Agar siz 20 ta elementdan iborat murakkab logotip yasagan bo'lsangiz, ularning hammasiga bitta effekt (masalan, soya) bermochi bo'lsangiz,

ularni Pre-comp qilasiz va effektini o'sha bitta "konvert"ga berasiz. Bu "Timeline"ni toza saqlash va loyihani boshqarish uchun shart.

☆ 82. "Alpha Channel" va "Luma Matte" farqi?



- **Alpha Channel:** Tasvirning shaffoflik ma'lumoti (masalan, PNG rasm orqasi ko'rinib turishi).

- **Luma Matte:** Bir qatlamning oq va qora ranglariga qarab, ikkinchi qatlamni ko'rsatish yoki berkitish. Oq joylar ko'rinadi, qora joylar esa shaffof bo'ladi. Bu o'ta murakkab o'tish effektlarini (transitions) qo'lda yaratishda ishlatiladi.

🔗 83. "Color Clipping" (Rang sinishi) nima va u skoplarda qanday ko'rinadi?



Agar siz videoning yorug'ligini (Gain) o'ta ko'paytirib yuborsangiz, oq nuqtadagi barcha detallar yo'qolib, shunchaki "o'lik oq dog"ga aylanadi. **Waveform** skopida bu signalning eng yuqori chiziqqa (100 IRE) urilib, "yassilanib" qolganida ko'rinadi. Bu ma'lumotni qayta tiklab bo'lmaydi.

☆ 84. "Complementary Colors" (To'ldiruvchi ranglar) montajda qanday psixologik effekt beradi?



Bu ranglar g'ildiragida bir-biriga qarama-qarshi turgan ranglar (masalan, To'q sariq va Moviy - Teal & Orange). **Natija:** Ushbu ranglar kombinatsiyasi eng yuqori kontrastni beradi va inson ko'zi uchun juda yoqimli hisoblanadi. Gollivud rang ustalari inson terisini (issiq rang) fondan (sovuq rang) ajratish uchun aynan shu usulni qo'llashadi.

🔗 85. "VRAM" (Video RAM) montajda nima uchun oddiy RAMdan muhimroq bo'lishi mumkin?



VRAM — bu videokartaning shaxsiy xotirasi. 4K montajda, ayniqsa 3D teksturalar va GPU render (masalan, Octane yoki Redshift) bilan ishlaganda, agar VRAM yetishmasa, dastur kutilmaganda yopilib ketadi (crash). 12GB+ VRAM zamonaviy grafik dizayner va montajchi uchun minimal standartdir.

☆ 86. "Hardware Encoding" (NVENC/QuickSync) va "Software Encoding" farqi?



- **Software:** Videoni protsessor (CPU) yordamida siqadi. Sifat o'ta yuqori bo'ladi, lekin uzoq vaqt oladi.

- **Hardware:** Videokartadagi maxsus blok yordamida siqadi. O'ta tez (5-10 barobar), lekin o'ta yuqori siqish darajalarida sifat biroz pastroq bo'lishi mumkin.

❓ 87. "Cut on Action" (Harakat ustida kesish) montajni nega silliq qiladi?

📖 Agar aktyor qo'lini ko'tara boshlaganda kadrni kessangiz va keyingi kadrda u qo'lini ko'tarib bo'lganini ko'rsatsangiz, inson miyasi harakatning davomiyligiga chalg'ib, "kesish"ni (cut) sezmay qoladi. Bu "Invisible Editing"ning eng asosiyligiga qoidasidir.

★ 88. "Kuleshov Effect" montajda inson psixologiyasini qanday boshqaradi?

📖 Bu shuni ko'rsatadiki, bitta kadrning ma'nosi uning yonidagi kadrqa qarab o'zgaradi. **Misol:** Aktyorning hissiyotsiz yuzi + Ovqat rasmi = Ochlik. Aktyorning aynan o'sha yuzi + Janoza rasmi = G'am. Montajchi kadrlar tartibi orqali hikoyaning butun hissiyotini o'zgartirishi mumkin.

❓ 89. "Lumen" (Unreal Engine 5) yoritish tizimi nega inqilobiy?

📖 Ilgari realistik yoritish (Global Illumination) soatlab render qilinishi kerak edi. **Lumen** buni real vaqtda (sekundiga 60 kadr) bajaradi. Siz chiroqni o'chirsangiz, butun xona nurlari devorlardan qaytishi bilan birga zumda o'chadi. Bu kinoni real vaqtda yaratish imkonini beradi.

★ 90. "Nanite" texnologiyasi 3D modellardagi poligonlar soni cheklovini qanday yo'qotdi?

📖 **Nanite** millionlab va hatto milliardlab poligonlardan iborat modellarni (masalan, o'ta detallashgan haykallar) kompyuter qotmasdan ko'rsatish imkonini beradi. U faqat ekrandagi piksellar soniga teng miqdordagi poligonlarni render qiladi. Endi 3D modelerlar modelni "yengillashtirish" (retopology) ustida bosh qotirishi shart emas.

❓ 91. "Foley Sound" nima uchun sun'iy yozilishi kerak? Kamera mikrofonini yetarli emasmi?

📖 Kamera mikrofonini hamma shovqinni (fon shovqini, shamol) yozadi. **Foley** ustalari esa studiyada har bir harakatni (masalan, kiyimning shitirlashi, qadam tovushi) alohida va o'ta tiniq yozib olishadi. Bu tovushlar videoga qo'shilganda, tasvir "yaqinroq" va "haqiqiyroq" ko'rinadi.

★ 92. "Mastering" audioda oxirgi nuqta sifatida nima beradi?

📖 Bu butun videodagi tovush darajalarini standartga keltirish (Loudness Normalization). Masalan, YouTube yoki TV-da ovoz "sakrab" ketmasligi, juda past yoki juda baland bo'lmisligi uchun **LUFS** (Loudness Units relative to Full Scale) ko'rsatkichi bo'yicha sozlanadi.

❓ 93. "Container" va "Codec" farqini tushuntiring.



- **Codec (H.264, ProRes):** Ma'lumotni siqish va ochish usuli (ichidagi narsa).
- **Container (.MP4, .MOV, .MKV):** Bu "quti". Uning ichida video oqimi, bir nechta audio yo'laklar va subtitrlar birga saqlanadi.

★ 94. "Interlaced" video (1080i) nega hozirgi kunda "yomon odat" hisoblanadi?

📖 Chunki u kadrni qatorlarga bo'lib ko'rsatadi. Zamonaviy LCD/OLED monitorlar esa "Progressive" (1080p) ishlaydi. Interlaced videoni kompyuterda ko'rsangiz, tez harakatlarda "taroqsimon" chiziqlar (combing) paydo bo'ladi, bu sifatni keskin tushiradi.

❓ 95. "Inpainting" texnologiyasi videoni tahrirlashda qanday ishlaydi?

📖 Siz videodan biror keraksiz ob'ektni (masalan, kadrda kirib qolgan odamni) belgilaysiz. AI esa uning orqasida nima bo'lishi mumkinligini (devor yoki o'tloq) tahlil qilib, ob'ektni o'chiradi va joyini o'sha fon bilan to'ldiradi.

★ 96. "Neural Radiance Fields" (NeRF) nima?

📖 Bu kelajak texnologiyasi. U bir nechta fotosuratlar yordamida butun bir 3D makonni yaratadi. Siz u yerga 3D modeler kabi bino qurib o'tirmaysiz, AI shunchaki rasmlar orqali makonning nur va hajm ma'lumotlarini o'rganadi.

❓ 97. "Deepfake" dan ijobiy maqsadlarda qanday foydalaniladi?

📖 Masalan, vafot etgan aktyorning obrazini tiklash (kino sanoatida) yoki aktyorning og'iz harakatlarini boshqa tilga (dublyajga) moslab o'zgartirish (AI dubbing).

★ 98. "Export Settings" da "Target Bitrate" ni qanday hisoblash kerak?

📖 1080p 30fps video uchun YouTube standarti 8-10 Mbps. 4K uchun 35-45 Mbps. Agar bitrate-ni me'yordan oshirib yuborsangiz, fayl o'ta og'ir bo'ladi, lekin sifat deyarli o'zgarmaydi.

❓ 99. "Metadata" (XMP) fayllari loyihani saqlashda nega muhim?

📖 U yerda videoning qachon, qaysi kamerada va qaysi sozlamalar bilan olingani saqlanadi. Bu katta jamoalarda loyihani boshqarishda (archive) yordam beradi.

☆ **100. Grafik dizayner va videomontajchi uchun eng muhim ko'nikma nima?**

📖 **"Visual Library" (Vizual kutubxona).** Faqat texnikani o'rganish yetarli emas. Eng yaxshi kinolarni, dizaynlarni va san'at asarlarini ko'p ko'rish (observation) orqali didni shakllantirish — bu sohada muvaffaqiyatning 80% idir.

Aloqa tayyorgarligi bo'yicha savollar va javoblar

1. Savol. Antenna bu nima?

Javob. Antenna – bu radioto'lqinlarni uzatish va qabul qilish uchun mo'ljalangan qurilma.

2. Savol. Radiostansiya bu nima?

Javob. Radiostansiya – bu radioaloqani tashkil etish uchun yagona kompleksda birlashgan texnik vositalar majmui.

3. Savol. Radioaloqa bu nima?

Javob. Radioaloqa bu mobil aloqa turi bo'lib, cheklanmagan masofalarga oz vaqt, kuch va vositalar sarf qilgan holda aloqani tashkil qilish.

4. Savol. Radioaloqa intizomi deb nima deyiladi?

Javob. Aloqa vositalari uchun o'rnatilgan ish rejimiga, ularni qo'llash tartibiga, aloqa kanalida so'zlashuv qoidalariga qat'iy va to'g'ri rioya qilishga radioaloqa intizomi deb aytiladi.

5. Savol. R-159 radiostansiyasida qanday ish turlari qo'llaniladi?

Javob. TLF- ChM(chastota modulyatsiyali), TLG– AT, tovush chaqiriq.

6. Savol. R-159 radiostansiyasining ishchi chastotalar ko'lami qancha?

Javob. 30-75,999 MGts.

7. Savol. R-159 radiostansiyasining ishchi chastotalar oralig'idagi interval qancha?

Javob. 1 kGts.

8. Vopros. R-159 radiostansiyasining radiouzatgich quvvati qancha?

Javob. 5 Vt.

9. Savol. R-159 radiostansiyasida da qanday antenna turlari ishlatiladi?

Javob. ASh(shtir antenasi)-1,5 m, ASh-2,7m, yuguruvchi to'lqinli antenna (ABV) – 40 m.

10. Savol. R-159 radiostansiyasining aloqa uzoqligi qancha?

Javob. - ASh(shtir antenasi) antenasi uchun ASh - 1,5 m TLF rejimida - 12 km, TLG rejimda- 10 km;

- ASh(shtir antenasi) antenasi uchun - 2,7 m TLF rejimda - 18 km, TLG rejimda - 35 km;

- yuguruvchi to'liqinli antenna (ABV) antenasi uchun TLF rejimda - 35 km, TLG rejimda - 50 km.

11. Savol. R-159 radiostansiyasining uzluksiz ish vaqti qancha?

Javob. R-159 radiostansiyasi qabul qilish vaqtini uzatish vaqtiga 5/1 nisbatda 9 soat.

12. Savol. R-159 radiostansiyasida qanday akkumulyator batareya turlari qo'llaniladi?

Javob. R-159 radiostansiyasida 10NKP-10, 10NKP-8, 10ANKS(s) –7,0 yoki 10 NKBN-3,5s akkumulyator batareya turlari qo'llaniladi.

13. Savol. R-159 radiostansiyasining vazni qancha?

Javob. 11,7 kg

14. Savol. R-157 radiostansiyasida qanday ish turlari qo'llaniladi?

Javob. Tonal chaqiriqli TLF- ChM(chastota modulyatsiyali).

15. Savol. R-157 radiostansiyasining ishchi chastotalar ko'lami qancha?

Javob. 44-53,9 MGts.

16. Savol. R-157 radiostansiyasining radiouzatgich quvvati qancha?

Javob. 0,15 Vt.

17. Savol. R-157 radiostansiyasida qanday antenna turi ishlatiladi?

Javob. ASh-1,5 m.

18. Savol. R-157 radiostansiyasining bir turdagi radiostansiyalar bilan aloqa uzoqligi qancha?

Javob. 1 km gacha.

19. Savol. R-157 radiostansiyasining 1 AKB dan uzluksiz ishlash vaqti qancha?

Javob. Qabul qilish vaqti uzatish vaqtiga 5/1 nisbatda 9 soat.

20. Savol. R-157 radiostansiyasida qanday akkumulyator batareya turlari qo'llaniladi?

Javob. R-157 radiostansiyasida 10 SNK-0,45-12,6 V akkumulyator batareya turlari qo'llaniladi.

21. Savol. R-157 radiostansiyasining vazni qancha?

Javob. 2,2 kg.

22. Savol. R-158 radiostansiyasida qanday ish turlari qo'llaniladi?

Javob. Tonal chaqiriqli TLF – ChM(chastota modulyatsiyali).

23. Savol. R-158 radiostansiyasining ishchi chastotalar ko'lami qancha?

Javob. 30-79,795 MGts.

24. Savol. R-158 radiostansiyasining quvvati qancha?

Javob. 1 Vt.

25. Savol. R-158 radiostansiyasida qanday antenna turi ishlatiladi?

Javob. ASh-1,45 m.

26. Savol. R-158 radiostansiyasining bir turdagi radiostansiyalar bilan aloqa uzoqligi qancha?

Javob. 5-8 km.

27. Savol. R-158 radiostansiyasining 1 AKB dan uzluksiz ishlash vaqti qancha?

Otvet. Qabul qilish vaqti uzatish vaqtiga 5/1 nisbatda 12 soat.

28. Savol. R-158 radiostansiyasida qanday akkumulyator batareya turlari qo'llaniladi?

Javob. R-158 radiostansiyasida 10NKGS-1D – 12,6 V akkumulyator batareya turlari qo'llaniladi.

29. Savol. R-158 radiostansiyasining vazni qancha?

Javob. 3,6 kg.

30. Savol. R-148 radiostansiyasining ishchi chastotalar ko'lami qancha?

Javob. 37-51,95 MGts.

31. Savol. R-148 radiostansiyasining aloqa uzoqligi qancha?

Javob. 4-6 km.

32. Savol. R-148 radiostansiyasida qanday ish turlari qo'llaniladi?

Javob. TLF – ChM(chastota modulyatsiyali).

33. Savol. R-148 radiostansiyasining quvvati qancha?

Javob. 1 Vt.

34. Savol. R-148 radiostansiyasida qanday antenna turi ishlatiladi?

Javob. ASh-1,5 M.

35. Savol. R-148 radiostansiyasining 1 AKB dan uzluksiz ishlash vaqti qancha?

Javob. R-148 radiostansiyasi qabul qilish vaqti uzatish vaqtiga 5/1 nisbatda 10 soat vaqt davomida uzluksiz ishlaydi.

36. Savol. R-148 radiostansiyasida qanday akkumulyator batareya turlari qoʻllaniladi?

Javob. R-148 radiostansiyasida 10NKGS-1D – 12,6 V akkumulyator batareya turlari qoʻllaniladi.

37. Savol. R-148 radiostansiyasining vazni qancha?

Javob. 3 kg dan oshmaydi.

38. Savol. R-123 radiostansiyasida qanday ish turlari qoʻllaniladi?

Javob. Tonal chaqiriqli, TLF – ChM(chastota modulyatsiyali).

39. Savol. R-123 radiostansiyasining ishchi chastotalar koʻlami qanday?

Javob. 20-51,5 MGts.

40. Savol. R-123 radiostansiyasining ishchi chastotalari oraligʻidagi interval qancha?

Javob. 25 kGts.

41. Savol. R-123 radiostansiyasining quvvati qancha?

Javob. 20 Vt.

42. Savol. R-123 radiostansiyasida qanday antenna turlari ishlatiladi?

Javob. ASh-4 m, ASh –2,7 m, ShDA 11 m-lik machtada.

43. Savol. R-123 radiostansiyasining bir turdagi radiostansiya bilan aloqa uzoqligi qancha?

Javob. 13-20 km harakatda, 20 km – toʻxtab turgan joyda.

44. Savol. R-123 radiostansiyasining vazni qancha?

Javob. 43 kg.

45. Savol. R-123 radiostansiyasining uzluksiz ishlash vaqti qancha?

Javob. 27 V li bort tarmogʻidan kecha-kunduz davomida ishlaydi.

46. Savol. Radio orqali ochiq matn bilan qanday maʼlumotlarni uzatish taqiqlanadi?

Javob. Radio orqali davlat yoki harbiy sirni tashkil etuvchi maʼlumotlarni uzatish, hamda quyidagilarni ochiq nomlash taqiqlanadi:

- harbiy unvonlar, mansabdor shaxslarning familiyalari va nomlarini aytish;
- aloqa tugunlarining shartli nomlari, harbiy qismlar va dala pochталari tartib raqamlarini aytish;
- harbiy qismlarning joylashish punktlari nomlarini aytish;
- radioxujjatlar mazmuni yoritish;
- ishlatilayotgan radio apparaturaning texnik-taktik ko'rsatkichlari va ishlash tartibini aytish;
- radioaloqa seanslarining navbatdagi vaqtlarini bildirish;
- radiostansiyalarning qaysi qo'shin turlariga, boshqaruv qimslariga tegishlilikini va bajarilayotgan vazifalarning fe'l-atvorini aniqlash mumkin bo'lgan joylashish rayonidagi ob-havo holati haqidagi va boshqa ma'lumotlar uzatish.

47. Savol. TA-57 telefon apparatining vazifasi qanday?

Javob. TA-57 telefon apparati dala sharoitida telefon aloqani ta'minlab berish uchun mo'ljallangan va induktor chaqiriqli MB sistemasida ishlaydi, hamda SB sistemasidagi stansiyalarga ham ulanishi mumkin.

48. Savol. TA-57 telefon apparatining aloqa uzoqligi qancha?

Javob. TA-57 telefon apparati P-274M dala kabeli orqali 44 km gacha, 3mm diametrli doimiy havo liniyalari orqali – 150-250 km ga mustahkam aloqani ta'minlaydi.

49. Savol. P-274M yengil dala aloqa kabellari nima uchun mo'ljallangan?

Javob. P-274M yengil dala aloqa kabellari taktik bo'g'inda telefon aloqani, ichki aloqa tarmog'i abonent liniyalarini boshqaruv punktlariga moslashtirish uchun mo'ljallangan. 2 simli aloqa liniyalarini masofaviy boshqarish sxemasi radiostansiyalarni masofaviy boshqarish liniyalari orqali boshqaruv imkoniyatini beradi.

50. Savol. P-193M telefon kommutatorining vazifasi.

Javob: P-193M induktorli dala telefon kommutatori, dala sharoitlaridagi kichik sig'imdagi telefon stansiyalarni jihozlash uchun mo'ljallangan.

51. Savol. P-193M telefon kommutatorining aloqa uzoqligi qancha?

Javob. Abonent liniyalari orqali chaqiriq signallarini qabul qilish P-275 kabeli uchun 10-12 km, P-274 kabeli uchun 20-25 km ni tashkil etadi.

52. Savol. P-193M telefon kommutatorida qanday batareya turlari qo'llanadi?

Javob. P-193 telefon kommutatorining kuchaytirgichi AZU-10 komplektiga kiruvchi 9V kuchlanishga ega bo'lgan GB-10-U-1,3 yoki 7D-0.55S turdagi batareyadan quvvatlanadi.

53. Savol. P-193M telefon kommutatorining vazni qancha?

Javob. Kommutatorning chiziqli to'sqich va kiritish kabelisiz og'irligi - 13 kg; chiziqli to'sqich bilan kiritish kabelining og'irligi - 9 kg; komplektning umumiy og'irligi - 22 kg.

54. Savol. Telefon kommutatorlarining ulanish bo'yicha klassifikatsiyasi qanday?

Javob. Telefon kommutatorlari ulanishi bo'yicha avtomatik va qo'lda xizmat ko'rsatish turlariga bo'linadi.

55. Savol. Telefon kommutatorlarining tuzilishi bo'yicha klassifikatsiyasi qanday?

Javob. Telefon kommutatorlari tuzilishi bo'yicha qo'zg'almas va dalada ishlatiladigan turlariga bo'linadi.

56. Savol. Telefon kommutatorlarining sig'imi bo'yicha klassifikatsiyasi qanday?

Javob. Telefon kommutatorlari sig'imi bo'yicha kichik, o'rta va katta turlarga bo'linadi.

57. Savol. Simli aloqa vositalari nima uchun mo'ljallangan?

Javob: Simli aloqa vositalari, ma'lumotlarni elektr signallari ko'rinishida simli liniyalar orqali uzatish uchun mo'ljallangan.

58. Savol. Simli aloqa vositalarining afzalliklari qanday?

Javob: Simli aloqa vositalarining afzalliklari quyidagicha:

- so'zlashuv olib borish qulay va radioaloqaga nisbatan uzatishning yashirinligi katta;
- bitta xalkadan bir nechta aloqa kanallarini olish imkoniyati mavjud;
- dushman tomonidan shovqin hosil qilishdan mustasno;
- aloqa sifati yilning qaysi oyi, kuni va havo shovqiniga bog'liq emas;
- ancha katta o'tkazish xususiyatiga ega.

59. Savol. Simli aloqaning kamchiliklari qanday?

Javob: Simli aloqaning asosiy kamchiliklariga quyidagilar kiradi:

aloqa kabel liniyalari dushmanning o't ochishidan zaiflanishi va ularning transport vositalari ta'sirida ishdan chiqishi;

moddiy qismlarini qo‘polligi va kabel liniyalarini yotkizish va yig‘ib olish ishlari tezligi kamligi;

zaharlangan va qiyin kesishuvchi joylarda kabel liniyalarini yotkizish va yig‘ib olish qiyinligi, ba’zi xollarda imkoniyat yuqligi;

kabel liniyalarini yotqizish, xizmat ko‘rsatish va yig‘ib olish uchun ko‘p miqdorda shaxsiy tarkibga zarurat tug‘ilishi.

60. Savol. Aloqa vositalariga qanay texnik xizmat ko‘rsatiladi?

Javob: Aloqa vositalariga quyidagi texnik xizmat turlari ko‘rsatiladi:

-nazorat ko‘rigi;

-kunlik texnik xizmat ko‘rsatish;

-oylik texnik xizmat ko‘rsatish (TO-1);

-yillik texnik xizmat ko‘rsatish (TO-2);

-mavsumiy texnik xizmat ko‘rsatish;

-reglament bo‘yicha texnik xizmat ko‘rsatish.

61. Savol. R-142N radiostansiya tarkibi qanday aloqa vositalaridan iborat?

Javob. R-142N radiostansiya tarkibiga quyidagi aloqa vositalari kiradi: R-130-1, R-111-2, R-123MT-1, TA-57-2, R-012-1, P-274-1.

62. Savol. R-111 radiostansiyasining ishchi chastotalar ko‘lami qancha?

Javob. 20-52 MGts.

63. Savol. R-111 radiostansiyasining quvvati qancha?

Javob. 75 Vt.

64. Savol. R-111 radiostansiyasida da qanday antenna turlari ishlatiladi?

Javob. ASh-3,4 m, keng ko‘lamli antenna (ShDA) 11 m-lik machtada.

Savol. R-111 radiostansiyasining bir turdagi radiostansiya bilan aloqa uzoqligi qancha?

Javob. ASh-3,4 harakatda 25-35 km, ASh-3,4 to‘xtab turgan joyda 35-45 km, ASh-3,4 11m machtada to‘xtab turgan joyda 50-60 km, keng ko‘lamli antenna (ShDA) 16m-lik machtada turgan joyda 70-80 km.

65. Savol. R-111 radiostansiyasining uzluksiz ishlash vaqti qancha?

Javob. 23-29 V li bort tarmog‘idan kecha-kunduz davomida ishlaydi.

66. Savol. R-111 radiostansiyasining vazni qancha?

Javob. 100 kgdan oshmaydi.

67. Savol. R-130 radiostansiyasining ishchi chastotalar ko'lam qancha?

Javob. 1,5 –10,99 MGts.

68. Savol. R-130 radiostansiyasida qanday antenna turlari ishlatiladi?

Javob. ASh-4 m, qiyali nur, zenit nurlanishli antenna (ZNA), 10,5 m machtada joylashgan simmetrik dipol.

69. Savol. R-130 radiostansiyasining aloqa uzoqligi qancha?

Javob. ASh-4m antennadan foydalangan holda harakatda va to'xtab turgan joyda 20-50 km, qiyali nur to'xtab turgan joyda 30-75 km, simmetrik dipol turgan joyda 350 km gacha, zenit nurlanishli antennada (ZNA) turgan joyda va harakatda 350 km gacha.

70. Savol. R-130 radiostansiyasini quvvat manbai qancha?

Javob. Bort tarmog'idan 27 V.

71. Savol. R-130 radiostansiyasining vazni qancha?

Javob. 100 kg dan oz emas.

72. Savol. UKV diapazonining afzalliklari qanday?

Javob. UKV diapazonining asosiy afzalliklari quyidagicha:

- katta chastotalik sig'imga ega;
- har xil turdagi shovqin beruvchi to'liqlari past darajada;
- kichik gabaritli antennalarni qo'llash imkoniyati bor;
- kechayu-kunduz aloqani ta'minlash sharoitlari bir xil;
- dushmanning razvedka va ataylab qilingan shovqin beruvchi to'liqlaridan chegaralangan yer to'liqlarining shovqin berish doirasi himoyalanganligi oshirilgan.

73. Savol. KV radioaloqaning sifatiga qanday omillar ta'sir ko'rsatadi?

Javob. KV radioaloqaning sifatiga quyidagi omillar ta'sir ko'rsatadi:

- sutka payti;
- yil vaqti;
- quyosh nurining faolligi;
- radiostansiyaning geografik o'rni.

74. Savol. Radiotarmoq qanday tashkil qilinadi?

Javob: Radiotarmoq – (komandirlar, shtablar tomonidan) uch yoki undan ortiq bo'lgan boshqaruv punktlari orasida tashkil qilinadi. Radiotarmoq komanda va signallarni sirkulyar

ravishda o'tkazadi, hamda axborotni ko'p sonli muhbirlarga yetkazishni bir vaqtda (sirkulyar) ta'minlaydi.

75. Savol. Radioyo'nalish qanday tashkil qilinadi?

Javob: Radioyo'nalish – ikki boshqaruv punktlari orasida (komandirlar, shtablar tomonidan) tashkil qilinadi. Radioyo'nalish radiotarmoqqa qaraganda o'tkazish qobiliyati ko'proqligi, ishonchliligi va yashirinligi bilan ajralib turadi, lekin radiovositalar va radiochastotalar sarfini ko'payishni talab qiladi.

76. Savol. Radiostansiyalarni uzatishda ishlashini umumiy taqiqlash rejimi nima?

Javob. Radiostansiyalarni uzatishda ishlashini umumiy taqiqlash rejimi bu radiostansiyalarning diapazoni va quvvatidan qat'iy nazar uzatishda ishlashini taqiqlash.

77. Savol. Radioma'lumot tarkibiga nimalar kiradi?

Javob. Radioma'lumot tarkibiga chastota, radiostansiyalar shartli nomlari, mas'ul shaxslarning shartli nomlari, sirkulyar shartli nomlar, navbatchi radistning so'zlashuv jadvallarining kalitlari, radioparol va radiostansiyalar ishlash tartibi xaqidagi ko'rsatmalar kiradi.

78. Savol. Ixtiyoriy radioma'lumotlardan foydalanish mumkinmi?

Javob. Ixtiyoriy radioma'lumotlardan foydalanish qat'iy taqiqlanadi.

79. Savol. Signalli aloqa vositalari nima uchun ko'llaniladi?

Javob: Signalli aloqa vositalari, xabar berish signallarini uzatish uchun, hamda qo'shnlarni boshqarishda va o'zaro xarakatida qo'llaniladi. Signalli aloqa vositalari sifatida ko'rishga oid, tovushli, infraqizil va radiotexnik vositalar ishlatiladi.

80. Savol. Aloqaning asosiy vazifalarini ta'riflab bering.

Javob. Aloqaning asosiy vazifalari bu:

jangovar boshqaruv signallarini o'z vaqtida qabul qilish va yuqori shtab bilan barqaror aloqani ta'minlash;

xar xil sharoitlarda bo'ysinuvchi bo'linmalarni boshqaruvini ta'minlash;

dushman tomonidan bakteriologik (biologik) va kimyoviy, radioaktiv zaxarlanish, hamda ommaviy qirg'in qurollari qo'llanishining bevosita havfi, havo dushmani haqida qo'shnlarni ogohlantirish va o'z vaqtida xabardor qilish signallarini uzatishni ta'minlash;

o‘zaro xarakatlanuvchi birlashmalar, qismlar, bo‘linmalar orasida axborot almashuvini ta’minlash;

qo‘shinlarning jangovar xarakatini ta’minlash turlari bo‘yicha ma’lumotlarni qabul qilish hamda farmoyishlarni jo‘natish.

81. Savol. Aloqaga qo‘yiladigan talablar qanday?

Javob. O‘z vaqtida, ishonchli, xavfsiz.

82. Savol. Aloqa sistemasi deb nimaga aytiladi?

Javob. Qo‘shinlarni boshqaruvini ta’minlash masalalarini yechish uchun yagona rejaga asosan yaratilayotgan (yoyilayotgan) turli vazifalarni bajaruvchi aloqa liniyalari va stansiyalari, aloqa tugunlarining faoliyat qilish vaqti va joyi, muvofiqlashgan va o‘zaro bog‘langan masalalar majmuiga aloqa sistemasi deyiladi.

83. Savol. Brigada aloqa rejasi nimalarni o‘z ichiga oladi?

Javob: Karta izohlash xati bilan birga va 1 varaqda aloqani tashkil qilish sxemasini o‘z ichiga oladi.

84. Savol. Motoo‘qchi bataloni aloqa boshlig‘i aloqani tashkil qilish bo‘yicha qanday hujjatlar ishlab chiqadi?

Javob: Motoo‘qchi bataloni aloqa boshlig‘i aloqani tashkil qilish sxemasini ishlab chiqadi va ishchi xarita yuritadi.

85. Savol. Aloqani tashkil qilish sxemasi qanday ishlab chiqiladi?

Javob: Aloqani tashkil qilish sxemasi aloqa bo‘linmalarining shtatini, aloqa kuchlari va vositalari aniq borligini, hamda radiotarmoqlarning va radioyo‘nalishlarning aloqa tashkil qilish turini ko‘rsatgan xolda mas’ul shaxslarga biriktirilganligini hisobga olgan holda to‘liq sxema bo‘yicha ishlab chiqiladi.

86. Savol. Frontda aloqa tashkil qilish bo‘yicha javobgarlik kimga yuklanadi?

Javob. Frontda aloqa tashkil qilish bo‘yicha javobgarlik o‘ng qo‘shniga yuklanadi.

87. Savol. Front ortidan to frontgacha aloqa tashkil qilish bo‘yicha javobgarlik kimga yuklanadi?

Javob. Front ortidan to frontgacha aloqa tashkil qilish bo‘yicha javobgarlik ikkinchi eshelonda (zaxira) joylashgan birlashma (qism) shtabiga yuklanadi.

88. Savol. Qo‘shin turlari va umumqo‘shin birlashma (qism) hamda O‘R QK turlari birlashmalari (qismlari) bilan aloqa tashkil qilish bo‘yicha javobgarlik kimga yuklanadi?

Javob. Qo'shin turlari va umumqo'shin birlashma (qism) hamda O'R QK turlari birlashmalari (qismlari) bilan aloqa tashkil qilish bo'yicha javobgarlik umumqo'shin birlashma (qism) hamda O'R QK turlari birlashmalari (qismlari) shtabiga yuklanadi.

89. Savol. Maxsus qo'shinlar qismlari (bo'linmalari) bilan umumqo'shin birlashmasi (qismi) da aloqa tashkil etish bo'yicha javobgarlik kimga yuklanadi?

Javob. Maxsus qo'shinlar qismlari (bo'linmalari) bilan umumqo'shin birlashmasi (kismi) da aloqa tashkil etish bo'yicha javobgarlik umumqo'shin birlashmasi (qismi) shtabiga yuklanadi.

90. Savol. Umumqo'shin komandiri bo'linmalarni qanday usullar bilan boshqaradi?

Javob. Umumqo'shin komandiri bo'linmalarni yakka muloqot, shtab ofitserlari va xar xil aloqa vositalari yordamida boshqaradi.

91. Savol. Motoo'qchi va tankchi bo'linmalarda boshqaruv uchun qanday aloqa vositalari qo'llaniladi?

Javob. Motoo'qchi va tankchi bo'linmalarda boshqaruv uchun radio, simli, ko'chma va signalli aloqa vositalari qo'llaniladi.

92. Savol. Motoo'qchi (tankchi) batalonda radioaloqani tashkil qilishning asosiy usullari qanday?

Javob. Motoo'qchi (tankchi) batalonda radioaloqa asosan radiotarmoq orqali tashkil qilinadi, asosiy aloqa turi bu telefon aloqa.

93. Savol. Xarakatlanuvchi aloqa vositalarining qo'llanilishi va vazifalari qanday?

Javob: Xarakatlanuvchi aloqa vositalari qo'shinlarning har xil turdagi jangovar faoliyatida yuqori shtab bilan, birlashma boshqaruv punktlari orasida, bo'ysinuvchi va o'zaro xarakatlanuvchi birlashma va qismlar boshqaruv punktlari bilan fel'deger pochta aloqasini xizmatini tashkil qilish uchun mo'ljallangan.

94. Savol. Ish rejimi bo'yicha taktik bo'g'in radiostansiyalari qanday turlanadi?

Javob: Ish rejimi bo'yicha taktik bo'g'in radiostansiyalari simpleksli va dupleksli turlarga bo'linadi.

95. Savol. QShM nima uchun mo'ljallangan?

Javob: QShMga - aloqa vositalari bilan birgalikda ishlash o'rinlari joylashtirilgan bo'lib, u komandirga va birlashma (qism) shtabi ofitserlariga bo'ysunuvchilar, bo'sunuviga berilgan va o'zaro xarakatlanuvchi birlashmalar (qismlar,

bo‘linmalar) bilan aloqani joyida va harakat olib borilayotganda ta‘minlash uchun mo‘ljallangan.

96. Savol. Motoo‘kchi batalonida radiostansiyalarning hujum vaqtidagi ishlash rejimi qanday?

Javob. Motoo‘kchi batalonida radiostansiyalarning hujum vaqtidagi ishlash rejimi chegaralanmagan.

97. Savol. Motoo‘kchi bataloni hujum uyushtirgan vaqtda aloqani tashkil qilishga ta‘sir etuvchi omillar:

Javob. Hujumda aloqani tashkil qilishga quyidagi omillar ta‘sir etadi:

hujumga o‘tish usuli;

motoo‘qchi batalonining motoo‘qchi brigadasi jangovar tartibidagi tutgan o‘rni va joyi;

motoo‘qchi batalonining jangovar tartibi va vazifasi;

qabul qilingan boshqaruv sistemasi;

dushman tomonidan qarshi radioelektron ta‘sir qo‘llash darajasi;

aloqa kuchlari va vositalari, xolati va mavjudligi.

98. Savol. Birlashma boshqaruv punkti aloqa tuguni (BP AT) tarkibiga nimalar kiradi?

Javob. KShM guruxi aloqa vositalari bilan birga;

telefon stansiyasi;

telegraf stansiyasi,

o‘rta quvvatli radiostansiyalar (alohida radiostansiyalar) guruxi;

radiorele stansiyasi;

aloqa boshqaruv punkti;

ekspeditsiya;

elektr quvvati beruvchi stansiyalar.

99. Savol. Front orti boshqaruv punkti (TPU) aloqa tuguni nima uchun mo‘ljallangan?

Javob: Front orti boshqaruv punkti (TPU) aloqa tuguni komandirning front orti va qurol aslaxa bo‘yicha o‘rinbosarlariga yuqori qo‘mondonlik front orti boshqaruv punkti birlashmasi, front orti va texnik xizmat ko‘rsatish bo‘linmalari va o‘zining birlashmasi boshqaruv punkti bilan aloqani ta‘minlab berish uchun mo‘ljallangan.

1) Hardware va Software asoslari

1. Tanenbaum A.S., Austin T. **Structured Computer Organization**. — Pearson.
2. Patterson D.A., Hennessy J.L. **Computer Organization and Design: The Hardware/Software Interface**. — Morgan Kaufmann.
3. Stallings W. **Computer Organization and Architecture**. — Pearson.

2) Python asoslari

4. Lutz M. **Learning Python**. — O'Reilly Media.
5. Python Software Foundation. **Python Documentation (Python 3.x)** ElektronmanbaElektronmanbaElektronmanba. [Python documentation+1](#)

3) Sun'iy intellekt / Machine Learning / Deep Learning

6. Russell S., Norvig P. **Artificial Intelligence: A Modern Approach**. — Pearson.
7. Goodfellow I., Bengio Y., Courville A. **Deep Learning**. — MIT Press.
8. Géron A. **Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow**. — O'Reilly Media.
9. Scikit-learn Developers. **scikit-learn: User Guide / API Reference** ElektronmanbaElektronmanbaElektronmanba. [scikit-learn.org+2scikit-learn.org+2](#)

4) Kompyuter tarmoqlari (Networking)

10. Kurose J.F., Ross K.W. **Computer Networking: A Top-Down Approach**. — Pearson.
11. Tanenbaum A.S., Wetherall D.J. **Computer Networks**. — Pearson.
12. Postel J. **RFC 793: Transmission Control Protocol (TCP)** ElektronmanbaElektronmanbaElektronmanba. [datatracker.ietf.org+1](#)
13. Fielding R. va boshq. **RFC 9110: HTTP Semantics** ElektronmanbaElektronmanbaElektronmanba. [datatracker.ietf.org+1](#)

5) Axborot xavfsizligi va Kiberxavfsizlik

14. Stallings W. **Cryptography and Network Security: Principles and Practice**. — Pearson.
15. NIST. **Cybersecurity Framework (CSF) 2.0 (NIST CSWP 29, 26 Feb 2024)** ElektronmanbaElektronmanbaElektronmanba. [nvlpubs.nist.gov+1](#)
16. OWASP. **OWASP Top 10:2021** ElektronmanbaElektronmanbaElektronmanba. [owasp.org+1](#)

6) DevOps (CI/CD, konteynerlash, orkestratsiya, versiya nazorati)

17. Humble J., Farley D. **Continuous Delivery**. — Addison-Wesley.
18. Kim G., Debois P., Willis J., Humble J. **The DevOps Handbook**. — IT Revolution.
19. Docker. **Docker Documentation** ElektronmanbaElektronmanbaElektronmanba. [Docker Documentation](#)

20. Kubernetes. **Kubernetes Documentation** ElektronmanbaElektron manbaElektronmanba. [Kubernetes+1](#)
21. Chacon S., Straub B. **Pro Git** ElektronmanbaElektron manbaElektronmanba. [git-scm.com+1](#)

7) Web texnologiyalar (HTML/CSS/JS, HTTP, Web API)

22. MDN. **HTML / CSS / JavaScript Documentation** ElektronmanbaElektron manbaElektronmanba. [developer.mozilla.org+3developer.mozilla.org+3developer.mozilla.org+3](#)
23. IETF. **RFC 9110: HTTP Semantics** ElektronmanbaElektron manbaElektronmanba. [datatracker.ietf.org](#)

8) Videokonferensiya aloqa tizimlari

24. Yusupov B.K. “Videokonferensiya aloqa tizimlari” fanidan o‘quv uslubiy majmua. — Toshkent, 2022.
25. W3C. **WebRTC 1.0 / WebRTC: Real-Time Communication in Browsers** ElektronmanbaElektron manbaElektronmanba. [W3C+1](#)
26. WebRTC Project. **webrtc.org** ElektronmanbaElektron manbaElektronmanba. [WebRTC](#)

9) Kompyuter grafikasi (2D/3D, tasvirni qayta ishlash, CV)

27. Gonzalez R.C., Woods R.E. **Digital Image Processing**. — Pearson.
28. OpenCV. **OpenCV Documentation** ElektronmanbaElektron manbaElektronmanba. [docs.opencv.org+1](#)
29. Blender Foundation. **Blender Manual** ElektronmanbaElektron manbaElektronmanba. [docs.blender.org+1](#)

10) Videomontaj (formatlar, transcoding, amaliy muharrirlar)

30. FFmpeg. **FFmpeg Documentation (ffmpeg, ffprobe, filters, formats)** ElektronmanbaElektron manbaElektronmanba. [ffmpeg.org+1](#)
31. Adobe. **Premiere Pro User Guide / Help** ElektronmanbaElektron manbaElektronmanba. [helpx.adobe.com+2helpx.adobe.com+2](#)

FOYDALANILGAN INTERNET SAYTLAR RO‘YXATI

1. Python rasmiy hujjatlari — [Python documentation+1](#)
2. scikit-learn rasmiy hujjatlari — [scikit-learn.org+1](#)
3. NIST CSF 2.0 (PDF) — [nvlpubs.nist.gov](#)
4. OWASP Top 10 — [owasp.org+1](#)
5. Docker Docs — [Docker Documentation](#)
6. Kubernetes Docs — [Kubernetes+1](#)
7. Git (Pro Git, reference) — [git-scm.com+1](#)
8. MDN Web Docs (HTML/CSS/JS) — [developer.mozilla.org+2developer.mozilla.org+2](#)
9. IETF Datatracker (RFC 793, RFC 9110) — [datatracker.ietf.org+1](#)

10. W3C WebRTC — [W3C](#)+1
11. WebRTC.org — [WebRTC](#)
12. OpenCV Docs — [docs.opencv.org](#)
13. Blender Manual — [docs.blender.org](#)
14. FFmpeg Documentation — [ffmpeg.org](#)+1
15. Adobe Premiere Pro User Guide — [helpx.adobe.com](#)+1